STUDY TEXTBOOK

Discrete Mathematics and Proofs

Logic, proofs and olympiad

First edition · 10 October 2026

Contents

Start here: from zero **ix**

1 Logic and quantifiers **1**

1.1 Statements and connectives . . . 2
1.2 Truth tables . . . 2
1.3 Implication and its family . . . 3
1.4 Quantifiers . . . 5
1.5 Negating statements mechanically . . . 7
1.6 English and symbols . . . 8
1.7 Parity and primes: a first look . . . 10
1.8 Chapter summary . . . 11
1.9 Common mistakes . . . 12
1.10 Additional exercises . . . 12
Notes . . . 14

2 How to write a proof **15**

2.1 What a proof is . . . 16
2.2 Direct proof and the template . . . 16
2.3 Proof by cases . . . 17
2.4 Contrapositive . . . 18
2.5 Contradiction . . . 18
2.6 If and only if . . . 19
2.7 Existence and uniqueness . . . 20
2.8 Disproof and counterexamples . . . 21
2.9 Mathematical writing . . . 22
2.10 Strategy before proof . . . 22
2.11 Chapter summary . . . 23
2.12 Common mistakes . . . 23

2.13 Additional exercises . . . 23
Notes . . . 26

3 Induction **27**
3.1 The domino idea and the template . . . 28
3.2 Sums and formulas by induction . . . 28
3.3 Induction for inequalities . . . 29
3.4 Strong induction . . . 30
3.5 Well-ordering: induction in disguise . . . 30
3.6 Structural examples: a tiling, a recurrence, a fake proof . . . 31
3.7 Choosing what to induct on . . . 33
3.8 Chapter summary . . . 34
3.9 Common mistakes . . . 34
3.10 Additional exercises . . . 34
Notes . . . 36

4 Sets, relations and functions **37**
4.1 Sets, subsets and operations . . . 38
4.2 Power set and Cartesian product . . . 38
4.3 Functions: definition, domain, image . . . 39
4.4 Injective, surjective, bijective — with templates . . . 39
4.5 Relations, equivalence relations, partitions . . . 40
4.6 Cardinality: bijections count, diagonalisation escapes . . . 41
4.7 The set $[n]$ and indexing conventions . . . 43
4.8 Chapter summary . . . 43
4.9 Common mistakes . . . 44
4.10 Additional exercises . . . 44
Notes . . . 46

5 Divisibility and primes **47**
5.1 Divisibility and the division algorithm . . . 47
5.2 Gcd and the Euclidean algorithm . . . 48
5.3 Bézout's identity — and why the proof is an algorithm . . . 49
5.4 Primes and the fundamental theorem . . . 50
5.5 Euclid's infinitude of primes . . . 51

5.6 Digit criteria and base representations . . . 52
5.7 Linear Diophantine equations . . . 53
5.8 Chapter summary . . . 53
5.9 Common mistakes . . . 53
5.10 Additional exercises . . . 54
Notes . . . 56

6 Modular arithmetic **57**
6.1 Congruence as an equivalence . . . 58
6.2 The residue system $\mathbb{Z}/m\mathbb{Z}$ . . . 58
6.3 Cancellation and multiplicative inverses . . . 59
6.4 Linear congruences . . . 60
6.5 Modular patterns: last digits, power cycles, casting out nines . . . 61
6.6 Two moduli at once: a CRT preview . . . 63
6.7 Chapter summary . . . 63
6.8 Common mistakes . . . 64
6.9 Additional exercises . . . 64
Notes . . . 65

7 The Chinese remainder theorem and Fermat–Euler **67**
7.1 The Chinese remainder theorem . . . 68
7.2 Three ways to hold the theorem . . . 69
7.3 Systems in practice . . . 70
7.4 Fermat's little theorem, two ways . . . 70
7.5 Euler's totient and Euler's theorem . . . 71
7.6 Orders modulo p . . . 72
7.7 Wilson's theorem (enrichment) . . . 72
7.8 Applications: last digits again, check digits . . . 73
7.9 Chapter summary . . . 73
7.10 Common mistakes . . . 73
7.11 Additional exercises . . . 74
Notes . . . 75

8 Counting principles **76**
8.1 Sum and product rules . . . 76

8.2 Permutations and factorials . . . 77
8.3 Binomial coefficients and Pascal's triangle . . . 78
8.4 The binomial theorem, proved two ways . . . 78
8.5 Bijection is counting . . . 79
8.6 Stars and bars . . . 79
8.7 The pigeonhole principle . . . 80
8.8 Committee proofs . . . 81
8.9 Chapter summary . . . 81
8.10 Common mistakes . . . 81
8.11 Additional exercises . . . 82
Notes . . . 84

9 Inclusion–exclusion and generating functions 85
9.1 Two sets, three sets, then the formula . . . 86
9.2 Derangements . . . 86
9.3 Surjections . . . 87
9.4 Euler's totient by PIE . . . 88
9.5 Ordinary generating functions . . . 88
9.6 The catalog and the convolution rule . . . 88
9.7 GFs for counting . . . 88
9.8 GFs meet recurrences . . . 89
9.9 Chapter summary . . . 90
9.10 Common mistakes . . . 90
9.11 Additional exercises . . . 91
Notes . . . 92

10 Recurrences 93
10.1 Recurrence vs closed form . . . 93
10.2 Guessing, then proving . . . 94
10.3 Characteristic equations . . . 95
10.4 Fibonacci, tilings, and derangements . . . 95
10.5 Solving by generating functions . . . 97
10.6 A feel for growth orders . . . 97
10.7 Chapter summary . . . 98
10.8 Common mistakes . . . 98

10.9 Additional exercises . . . 98
Notes . . . 100

11 Graphs, trees and planarity **101**
11.1 Graphs and the handshake lemma . . . 101
11.2 Walks, paths, cycles, connectivity . . . 102
11.3 Bipartite graphs . . . 102
11.4 Trees . . . 103
11.5 Eulerian trails . . . 103
11.6 Planar graphs and Euler's formula . . . 104
11.7 Graph colouring basics . . . 105
11.8 Chapter summary . . . 106
11.9 Common mistakes . . . 106
11.10 Additional exercises . . . 106
Notes . . . 108

12 Probability on finite spaces **109**
12.1 Sample spaces and uniform models . . . 110
12.2 Events, complements, "at least one" . . . 110
12.3 Conditional probability . . . 110
12.4 Independence, and when it fails . . . 111
12.5 Random variables and expectation . . . 112
12.6 Linearity of expectation . . . 112
12.7 Simpson's paradox . . . 113
12.8 Geometric probability preview . . . 113
12.9 Chapter summary . . . 114
12.10 Common mistakes . . . 114
12.11 Additional exercises . . . 114
Notes . . . 116

13 Inequalities **117**
13.1 What's allowed: the algebra of $<$ . . . 117
13.2 Squares are nonnegative . . . 118
13.3 AM–GM . . . 118
13.4 The power-mean chain . . . 119

13.5 Cauchy–Schwarz . . . 119
13.6 Equality conditions . . . 120
13.7 Muirhead and Schur . . . 120
13.8 Strategy selection . . . 121
13.9 Summary . . . 122
13.10 Code: numerical sanity checks . . . 122
13.11 Common mistakes . . . 122
Notes . . . 126

14 Functional equations **127**
14.1 What "solve over K" means . . . 127
14.2 Guessing candidates . . . 128
14.3 Substitution tactics . . . 128
14.4 Milestones: injectivity, surjectivity, involutions . . . 129
14.5 Cauchy's equation over $\mathbb{Q}$ . . . 130
14.6 The pointwise trap . . . 130
14.7 Feeding back: verify the answer . . . 131
14.8 Summary . . . 131
14.9 Code: hunting solutions by brute force . . . 132
14.10 Common mistakes . . . 132
Notes . . . 135

15 Olympiad geometry **136**
15.1 Angle chasing systematically . . . 136
15.2 Cyclic quadrilaterals . . . 137
15.3 The incenter/excenter lemma . . . 138
15.4 Power of a point . . . 138
15.5 Ceva, Menelaus, extended sine law . . . 139
15.6 Homothety and spiral similarity . . . 139
15.7 Inversion in a circle . . . 140
15.8 Analytic backup plans . . . 140
15.9 Summary . . . 141
15.10 Code: two numerical verifications . . . 141
15.11 Common mistakes . . . 141
Notes . . . 145

16 Invariants, monovariants and extremal arguments **146**

16.1 Invariants . . . 146
16.2 Monovariants . . . 147
16.3 The extremal principle . . . 147
16.4 Advanced pigeonhole . . . 148
16.5 Coloring and parity arguments . . . 148
16.6 Games . . . 149
16.7 Summary . . . 150
16.8 Code: watching the invariant . . . 150
16.9 Common mistakes . . . 150
Notes . . . 154

17 Writing a four-hour exam and the proof portfolio **155**

17.1 What a timed contest tests . . . 156
17.2 The four-hour budget . . . 156
17.3 Write-up standards . . . 157
17.4 Partial credit and fake-solving . . . 157
17.5 Checking your work . . . 158
17.6 The 20-proof portfolio . . . 159
17.7 Code: a linter, not a grader . . . 159
17.8 Summary . . . 159
17.9 Common mistakes . . . 160
Notes . . . 163

Hints for the hard exercises **164**

Selected solutions **168**

Watch list **171**

Start here: from zero

You do not need to have seen a proof before. This book assumes you can add, multiply, and tell an even number from an odd one, and it builds everything else. This short chapter explains *why* a mathematician is not satisfied by checking examples, lists the few school facts we use, and shows how the symbols in Chapter 1 read as ordinary English.

Why prove anything?

Try this. Compute $n^2 + n + 41$ for $n = 0, 1, 2, \ldots$:

$$n = 0 :\ 41, \qquad n = 1 :\ 43, \qquad n = 2 :\ 47, \qquad n = 3 :\ 53, \qquad n = 4 :\ 61, \ \ldots$$

Every value is a prime number. It stays prime for $n = 0, 1, \ldots, 39$, forty cases in a row. Surely the formula always gives a prime? It does not: at $n = 40$,

$$40^2 + 40 + 41 = 40(40 + 1) + 41 = 40 \cdot 41 + 41 = 41 \cdot 41 = 1681,$$

which is 41^2, not prime. Forty correct examples were not a proof, and the forty-first case broke it. A *proof* is an argument that covers *every* case at once, so that no later case can surprise you. Contest mathematics and computer science run on that guarantee: an algorithm proved correct is correct on inputs you never tested.

School facts we use

Integers. The whole numbers $\ldots, -2, -1, 0, 1, 2, \ldots$; the set of all of them is written $\mathbb{Z}$. The positive ones, $1, 2, 3, \ldots$, are the *natural numbers*, $\mathbb{N}$.

Even and odd. An integer is even if it can be written $2k$ for an integer k, and odd if it can be written $2k + 1$. Every integer is exactly one of the two.

Divides. $a \mid b$ means $b = ak$ for some integer k: $3 \mid 12$ because $12 = 3 \cdot 4$; $5 \nmid 12$.

Prime. An integer greater than 1 whose only positive divisors are 1 and itself: $2, 3, 5, 7, 11, \ldots$. The number 1 is not prime.

Fractions and real numbers. $\mathbb{Q}$ is the set of fractions a/b with a, b integers and $b \neq 0$; $\mathbb{R}$ is the set of all numbers on the number line.

Reading the symbols

Every symbol below is shorthand for an English phrase. Chapter 1 introduces them one by one; this table is for reference.

Symbol	Read it as
$x \in S$	"x is an element of S" ($3 \in \{1, 2, 3\}$)
$\forall x$	"for every x"
$\exists x$	"there is at least one x such that"
$P \Rightarrow Q$	"if P then Q"
$P \Leftrightarrow Q$	"P if and only if Q": each implies the other
$\neg P$	"not P"
$P \wedge Q$, $P \vee Q$	"P and Q", "P or Q (or both)"

How the books of this series fit together

This is Book 2 of eight. It is the proof-and-counting spine for the others, and it is a good first book if you plan to continue.

Book	Connection
1 Competitive programming	Implements the number theory (Chapters 5–7 here) and counting (Chapters 8–9) as fast code, and uses induction and exchange arguments to justify greedy algorithms.
3 Linear algebra	Uses the proof habits of Chapters 2–4: every theorem there is stated, then proved, in the same layout.
6 Quant research	Needs the finite probability of Chapter 12.

How to use the boxes in each chapter

Each chapter begins with *After this chapter you can*, naming what you will be able to *do*. From Chapter 2, a *Before you start* box lists what earlier chapters (or other books) the chapter relies on, with a one-sentence recap, so that you can tell whether to reread something first.

Work the exercises with a pencil. A proof you only read is a proof you have agreed with; a proof you wrote is one you can use.

CHAPTER 1

Logic and quantifiers

Every claim in this book is either a statement you will prove or a statement you will refute. Before any technique, you need the grammar those statements are written in: connectives that combine simple claims, and quantifiers that say *how many* objects a claim covers. After this chapter you will be able to write any contest-level claim in symbols, negate it mechanically without thinking, and say precisely what a proof or a counterexample of it would have to deliver. Nothing here is deep; everything here gets used weekly. Read it once carefully and it will save you from the most common proof errors in the rest of the book.

Why this exists. Computers do exactly what they are told, so every claim about a program ("this sort always finishes") needs a justification stronger than "I tried it". Logic is the language in which such claims are stated unambiguously, and proof is how they are settled. Olympiad problems ask for the same thing: not an answer, but an argument that holds in every case. Learn the language first; the proof techniques of the next chapters are written in it.

BEFORE YOU START

Nothing. If you can add, multiply and tell an even number from an odd one, you can start. The *Start here* chapter lists the few school-level facts we use (what an integer is, what "divides" means, what a prime is) and takes two minutes to skim.

AFTER THIS CHAPTER YOU CAN

- decide whether a sentence is a statement with a definite truth value;
- build a truth table and use it to check that two statements say the same thing;
- write "every" and "some" claims with quantifiers and negate them correctly;
- say exactly what a proof or a counterexample of a claim has to show.

HOW TO THINK ABOUT THIS

Mathematical language is just English with the ambiguity removed. When you read a symbolic statement, say it aloud in plain words; when you meet an English claim, ask "for *every* what? for *some* what?" Almost every proof mistake starts with a quantifier that was quietly misread.

1.1 Statements and connectives

Definition 1.1 (Statement)**.** A *statement* (also called a *proposition*) is a sentence that is either true or false. It need not be provable, known, or interesting — it only needs a definite truth value.

"7 is prime" is a statement, and a true one. "9 is prime" is a statement, and a false one. "Every even integer greater than 2 is a sum of two primes" is a statement — nobody knows which of the two truth values it has, but it has one. On the other hand "$x + 2 = 7$" is not a statement, because it is true for some x and false for others; it is an *open formula*, and Section 1.4 shows how quantifiers close it. Questions and commands ("is it raining?", "solve this") are not statements at all.

Statements combine into larger ones using *connectives*.

Definition 1.2 (Basic connectives)**.** Let P and Q be statements.

- $P \wedge Q$ (*conjunction*, "P and Q") is true exactly when both are true.
- $P \vee Q$ (*disjunction*, "P or Q") is true exactly when at least one is true — **including when both are true**.
- $\neg P$ (*negation*, "not P") is true exactly when P is false.

P	Q	$P \wedge Q$	$P \vee Q$	$\neg P$
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Remark. Mathematics uses the *inclusive* or: "P or Q" allows both. English does not always. "You may take the exam in June or in August" usually means one or the other. "A student who studied or who is lucky will pass" allows both. When a contest statement wants the exclusive version it will say so: "exactly one of".

1.2 Truth tables

A connective is defined by what it does to truth values, so the whole behaviour of a compound statement is computable line by line. Writing out all 2^n rows for n atomic statements is called a *truth table*. It is the brute-force verifier of propositional logic: two formulas are *equivalent* — interchangeable inside any larger statement — exactly when their truth tables agree on every row.

Here is the most useful equivalence there is, in action:

Proposition 1.3 (De Morgan's laws for statements)**.** *$\neg(P \wedge Q)$ is equivalent to $\neg P \vee \neg Q$, and $\neg(P \vee Q)$ is equivalent to $\neg P \wedge \neg Q$.*

Proof. Compare the two sides of the first law on all four cases:

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$	$\neg P \vee \neg Q$	$\neg P$	$\neg Q$
T	T	T	F	F	F	F
T	F	F	T	T	F	T
F	T	F	T	T	T	F
F	F	F	T	T	T	T

Columns four and five agree on every row, so the formulas are equivalent. The second law is checked the same way. ■

Read De Morgan in English: "not both" means "at least one fails"; "neither" means "both fail". You will use it every time you negate a compound statement in Section 1.5.

Exercise 1.1 (easy)**.** Write the truth table of $\neg P \vee Q$ for all four rows of (P, Q). Keep it — in Section 1.3 this formula gets a name.

1.3 Implication and its family

Most theorems have the shape "if P then Q".

Definition 1.4 (Implication)**.** $P \Rightarrow Q$ (*implication*) is false in exactly one case: P true and Q false. P is the *hypothesis*, Q the *conclusion*. English renders it as "if P then Q", "P implies Q", "Q if P", "P only if Q", and "Q whenever P".

The last two rows of the table are the part everyone trips on:

P	Q	$P \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Remark. If P is false, $P \Rightarrow Q$ is automatically true — *vacuously* true. "If $2 + 2 = 5$ then this textbook has a million pages" is a true statement, and proves nothing about the book. An implication only ever makes a promise about the case where the hypothesis holds.

Exercise 1.1 now pays off: the table you computed shows that $P \Rightarrow Q$ is equivalent to $\neg P \vee Q$. This one fact explains the whole implication family and, later, how to negate an implication.

Definition 1.5 (The implication family)**.** For $P \Rightarrow Q$:

- the *converse* is $Q \Rightarrow P$;
- the *inverse* is $\neg P \Rightarrow \neg Q$;
- the *contrapositive* is $\neg Q \Rightarrow \neg P$.

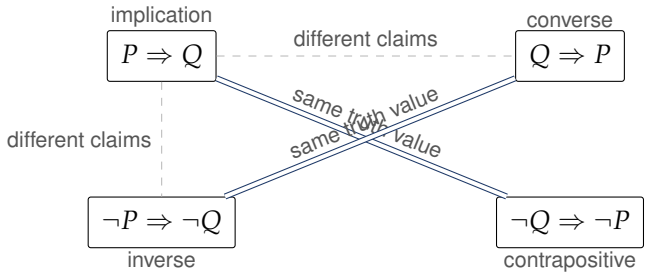


Figure 1.1: The implication family. Notice that equivalence lives on the *diagonals*: an implication and its contrapositive are the same claim rephrased, and so are the converse and the inverse. Neighbours are different claims.

Theorem 1.6. *$P \Rightarrow Q$ is equivalent to its contrapositive $\neg Q \Rightarrow \neg P$, and to $\neg P \vee Q$. It is* not *equivalent to its converse $Q \Rightarrow P$.*

Proof. Apply the equivalence $X \Rightarrow Y \equiv \neg X \vee Y$ twice, with commutativity of $\vee$ and double negation between them: $P \Rightarrow Q \equiv \neg P \vee Q \equiv Q \vee \neg P \equiv \neg(\neg Q) \vee \neg P \equiv \neg Q \Rightarrow \neg P$. For the converse, the rows $(P, Q) = (T, F)$ and (F, T) of the truth table already disagree between $P \Rightarrow Q$ and $Q \Rightarrow P$, so no equivalence is possible. ■

The contrapositive is the single most useful free move in proof-writing: whenever a statement is awkward in the form given, its contrapositive is the same statement in a different shape. Chapter 2 exploits this constantly.

Definition 1.7 (Biconditional)**.** $P \iff Q$ ("P if and only if Q", written *iff*) is $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$: true when P and Q have the same truth value. A claim stated as iff is *two* obligations, not one.

Example 1.8. Evaluate $\neg(P \wedge Q) \Rightarrow R$ when P and Q are true and R is false.

Strategy. Evaluate inside out: leaves first, then each connective, never reasoning about the whole formula at once. A tree of subformulas tracks the order.

Solution. $P \wedge Q$ is T since both inputs are T. Then $\neg(P \wedge Q)$ is F, so the statement is $F \Rightarrow F$, which is T. Figure 1.2 shows the same computation for the companion formula $(P \wedge Q) \Rightarrow R$, where this assignment is the *only* row that fails.

Exercise 1.2 (medium)**.** For the statement "if n is divisible by 6 then n is even", write the converse, the inverse and the contrapositive, and decide which of the four statements are true over the integers.

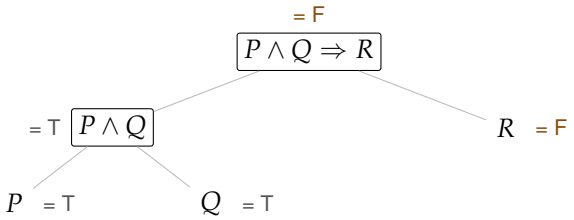


Figure 1.2: Evaluation tree of $(P \wedge Q) \Rightarrow R$ on the assignment P=T, Q=T, R=F. Notice that the implication is false exactly where the left child evaluates to T and the right to F — one row out of eight decides it.

1.4 Quantifiers

"$x + 2 = 7$" was not a statement because x was floating free. Quantifiers bind such variables.

Definition 1.9 (Universal and existential quantifiers)**.** Fix a *universe of discourse* U — the set of objects x may range over. For an open formula $P(x)$:

- $\forall x\, P(x)$ (*for all*): true when $P(x)$ holds for every $x \in U$.
- $\exists x\, P(x)$ (*there exists*): true when $P(x)$ holds for at least one $x \in U$.
- $\exists! x\, P(x)$ (*there exists a unique*): exactly one $x \in U$ satisfies $P(x)$.

Write $\forall x \in U$ or $\exists x \in U$ when the universe needs naming.

Read the two quantifiers as jobs. $\forall$ says *be ready for anything*: to verify $\forall x\, P(x)$ you must handle every single x, and to refute it one *counterexample* suffices. $\exists$ says *produce a witness*: to verify it, hand over one x that works; to refute it you must exclude *every* candidate. Asymmetry all the way down.

Example 1.10 ($\forall\exists$ versus $\exists\forall$)**.** On the set of contestants {Ana, Budi, Citra, Dewi} and problems $\{P1, P2, P3, P4\}$, let $S(c, p)$ mean "contestant c solved problem p", with solved problems:

	$P1$	$P2$	$P3$	$P4$
Ana	✓			
Budi		✓		
Citra			✓	
Dewi				✓

Compare $\forall c\, \exists p\, S(c, p)$ and $\exists p\, \forall c\, S(c, p)$.

Strategy. Read the first quantifier as the *job*, the second as the *answer*. In $\forall c\, \exists p$, the answer p may depend on c — each contestant may bring a different solved problem. In $\exists p\, \forall c$, one single p must serve everyone.

Solution. Every row of the table has a checkmark, so $\forall c\, \exists p\, S(c,p)$ is *true*. No column is full of checkmarks, so $\exists p\, \forall c\, S(c,p)$ is *false*. The two claims differ: order of quantifiers is not decoration.

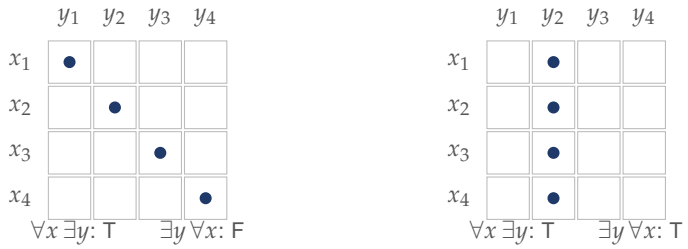


Figure 1.3: Two finite worlds. Left: marks on the diagonal — every row has a mark ($\forall x\, \exists y$ true) but no column covers all rows ($\exists y\, \forall x$ false). Right: one column fully marked, so both claims hold. Notice that $\exists y\, \forall x$ is the *stronger* claim: when it holds, $\forall x\, \exists y$ follows.

The same swap runs under the hood of definitions you will meet later. $\forall \varepsilon > 0\, \exists \delta > 0 \ldots$ in analysis means "however small the tolerance, there is a response *that may depend on it*". Confusing the two orders is the most common way definitions get misread; Listing 1.1 lets you check both verdicts by brute force on any small world.

Checkpoint 1.1 *Predict before you run.* In Listing 1.1, world 1 is the table of Example 1.10. Before running it, predict the four printed verdicts: two for world 1, then world 2 (everyone solved $P1$ only) and world 3 (Dewi solved nothing).

```
def verdicts(passed):
    fe = all(any(p in passed[c] for p in problems)
             for c in contestants)
    ef = any(all(p in passed[c] for c in contestants)
             for p in problems)
    return fe, ef
```

Listing 1.1: `verify_quantifiers.py`: checking $\forall\exists$ vs $\exists\forall$ by brute force on finite worlds.

The full script (in `code/verify_quantifiers.py`) builds three worlds and prints both verdicts for each:

```
world 1:
  every contestant solved some problem : True
  some problem was solved by everyone : False
world 2:
  every contestant solved some problem : True
  some problem was solved by everyone : True
world 3:
  every contestant solved some problem : False
  some problem was solved by everyone : False
```

Notice that the first claim can hold without the second (world 1), both can hold (world 2), and both can fail (world 3) — the claims are strictly different, and $\exists p\, \forall c$ implies $\forall c\, \exists p$ but not conversely.

Example 1.11 (A finite world)**.** In the 4×4 grid of Figure 1.4, each cell holds one symbol from $\{\triangle, \circ, \square\}$. Evaluate: (a) every row contains a $\triangle$; (b) some column consists entirely of $\circ$; (c) every $\circ$ shares its row with a $\square$; (d) some column contains both a $\triangle$ and a $\square$.

Strategy. When the universe is small and finite, work on the model, not on the symbols. $\forall$ becomes a checklist over rows or cells; $\exists$ becomes a search that stops at the first hit.

Solution. (a) Rows 1–4 have $\triangle$ in columns $1, 2, 3, 4$ respectively: true. (b) Scanning columns 1–4, each contains a non-$\circ$ cell: false. (c) Every row contains a $\square$ (columns $3, 4, 1, 2$), and every row contains $\circ$ cells, so each $\circ$ shares its row with a $\square$: true. (d) Column 2 contains $\triangle$ (row 2) and $\square$ (row 4): true.

	col 1	col 2	col 3	col 4
row 1	$\triangle$	$\circ$	$\square$	$\circ$
row 2	$\circ$	$\triangle$	$\circ$	$\square$
row 3	$\square$	$\circ$	$\triangle$	$\circ$
row 4	$\circ$	$\square$	$\circ$	$\triangle$

Figure 1.4: The finite world of Example 1.11. Notice that the $\triangle$ marks form a diagonal — that is why (a) is true while every column fails the "all $\circ$" test in (b). Small worlds in this style go back to the Tarski's World exercises used in logic courses [5].

Exercise 1.3 (easy)**.** Negate "every contestant solved some problem" from Example 1.10, first in English, then in symbols. Check against world 3 of the listing.

1.5 Negating statements mechanically

Negation is the highest-frequency operation in proof work: every contradiction, every contrapositive, every disproof starts by negating something. Do it by mechanical rules, not by feel.

NEGATE FROM THE OUTSIDE IN

To negate a statement, push $\neg$ inward past each operator, flipping as you go:

$$\neg\forall \to \exists\neg, \qquad \neg\exists \to \forall\neg, \qquad \neg(P \wedge Q) \to \neg P \vee \neg Q,$$

$$\neg(P \vee Q) \to \neg P \wedge \neg Q, \qquad \neg(P \Rightarrow Q) \to P \wedge \neg Q.$$

When $\neg$ reaches an atomic predicate it stops and negates it directly.

The last rule follows from $P \Rightarrow Q \equiv \neg P \vee Q$: negating gives $P \wedge \neg Q$. In English: the only way "if P then Q" fails is a case where the hypothesis happens and the conclusion does not.

Example 1.12. Negate $\forall x\, \exists y\, R(x, y)$.

Strategy. Work left to right. Each quantifier flips once, the innermost predicate negates.

Solution. $\neg\big(\forall x\,\exists y\,R(x,y)\big) \equiv \exists x\,\neg\big(\exists y\,R(x,y)\big) \equiv \exists x\,\forall y\,\neg R(x,y)$. One witness x for which *every* y fails is what refutes the original.

TRIPWIRE

A very common slip negates $\forall x\,\exists y$ as $\exists x\,\exists y$: both quantifiers flipped, but the order was left untouched. The correct negation is $\exists x\,\forall y$. The $\exists$-witness comes *first* in the negation, and it must defeat *all* y.

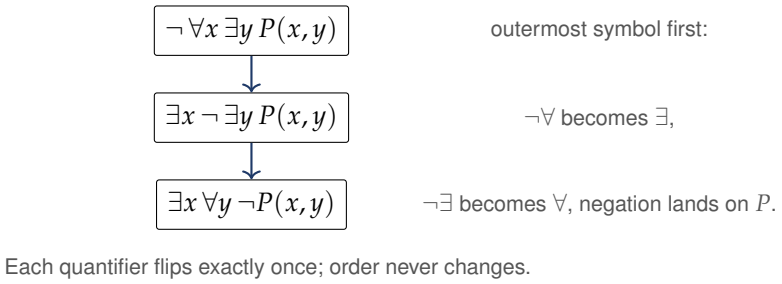


Figure 1.5: Negation pushed inward through a two-quantifier chain. Notice: each quantifier flips exactly once, the order is untouched, and the negation lands on the innermost predicate.

Example 1.13 (First attempt: negating by flipping every word)**.** A teacher says: "Every student passed the exam." You want to say this is false. The natural first attempt is "Every student failed the exam."

What goes wrong. Suppose 20 students sat the exam and 19 passed. The teacher's sentence is false (one student did not pass), yet "every student failed" is also false (19 passed). The attempted negation is false *in a case where the original is false too*, so it cannot be the negation. In symbols, with $P(s)$ meaning "s passed", the original is $\forall s\,P(s)$. The correct negation is $\exists s\,\neg P(s)$: "there is a student who did not pass." Flipping the quantifier *and* the statement inside it is the rule of Section 1.5; flipping only the inside is the commonest error in this book.

1.6 English and symbols

Contest statements arrive in English; proofs need symbols. The translation step is where most wrong readings happen, so we make it explicit. The usual hiding places: "a", "an" and "the" often mean $\exists$; "every", "each", "any" and "no matter which" mean $\forall$; "some" and "at least one" mean $\exists$; "no" means $\neg\exists$ (equivalently $\forall\ldots\neg$); "only" reverses arrows — "P only if Q" is $P \Rightarrow Q$, not $Q \Rightarrow P$.

Example 1.14 (Goldbach's claim)**.** Write in symbols, then negate: *every even integer greater than* 2 *is a sum of two primes*.

Strategy. Outer structure first. The sentence is a "for all" over even integers with a condition (> 2, even), so it is $\forall n\big((n \text{ even} \wedge n > 2) \Rightarrow \ldots\big)$ — a universal over a restricted domain, which always means a $\forall$ wrapped around an implication. The inner claim "is a sum of two primes" is existential: $\exists p\,\exists q$ with p, q prime and $p + q = n$. Then negate from the outside in.

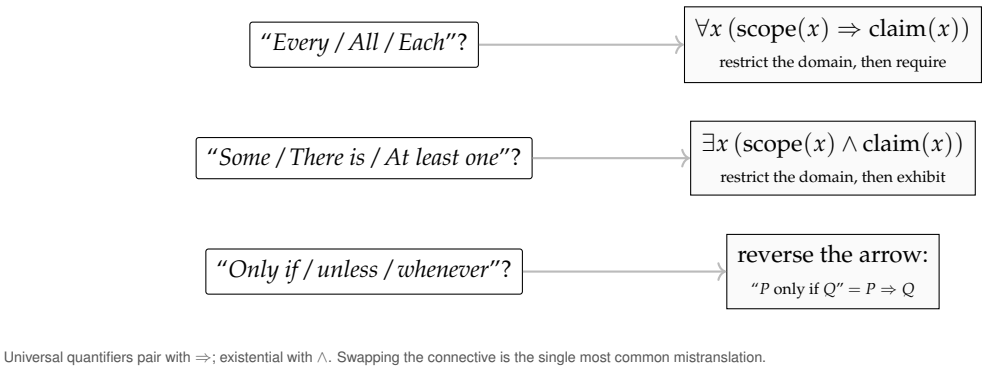


Figure 1.6: The three recurring English patterns and their connectives. Notice the left column: universals take $\Rightarrow$ because you restrict the domain and then *require* the claim; existentials take $\wedge$ because you restrict the domain and then *exhibit* a member.

Solution. Let $E(n)$, $P(n)$ abbreviate "n is even" and "n is prime". The claim is

$$\forall n \in \mathbb{Z}\ \Big(\big(E(n) \wedge n > 2\big) \Rightarrow \exists p\, \exists q\ \big(P(p) \wedge P(q) \wedge p + q = n\big)\Big).$$

Negating: $\exists n \in \mathbb{Z}$ such that $E(n) \wedge n > 2$ and $\forall p\, \forall q\, \neg\big(P(p) \wedge P(q) \wedge p + q = n\big)$ — *some even $n > 2$ that is not a sum of two primes*. One counterexample kills the claim; no amount of examples proves it. This is *Goldbach's conjecture*: stated 1742, still unproved as of 2026 — yet mechanically, its negation took two lines.

A TRANSLATION IS DONE WHEN…

every English word has become a named object, a quantifier, or a connective — and the direction of every "only if" and every "unless" has been checked on a case where it matters. If a quantifier's universe is left implicit in English, name it in symbols.

Remark. Competition problems are dense quantified statements wearing prose clothes. AMC 12 2024 A P22 is casework counting organised around a closed-loop (universal) constraint; AMC 12 2024 B P5 is a sign-flip problem whose engine is magnitude bookkeeping — each flip changes the total by twice the flipped term, and the parity of the sum never moves. Both are worth reading on the archive [7] once you can spot the quantified claims inside them.

Example 1.15 ("There is a unique…")**.** Translate: *there is a unique prime p such that $p+2$ is prime*. Then evaluate the statement.

Strategy. "Unique" is two claims AND-ed: existence ($\exists p$) and at-most-one (any two candidates coincide). Write both halves, then hunt for the witness — every prime except 2 is odd, and an odd p makes $p+2$ odd only in the right way, so check small candidates directly.

Solution. With $P(n)$ = "n is prime", uniqueness is

$$\exists p\ \Big(P(p) \wedge P(p{+}2)\Big)\ \wedge\ \forall q\, \forall r\ \big(\big(P(q) \wedge P(q{+}2) \wedge P(r) \wedge P(r{+}2)\big) \Rightarrow q = r\big).$$

Existence: $p = 3$ works ($3, 5$ both prime). At-most-one fails immediately: $q = 3$ and $r = 5$ both satisfy the property but $3 \neq 5$ — $(5, 7)$ is another prime pair. The English statement

is *false*: there are (at least) two such primes, so the "unique" claim dies on the second half, not the first. The English "unique" is exactly the kind of word that silently adds a second quantified clause — always expand it.

Exercise 1.4 (medium)**.** Write in symbols over $\mathbb{Z}$: "exactly one of the numbers a, b, c is even". Then decide: is the resulting statement, quantified over all $a, b, c \in \mathbb{Z}$, true, false, or an open formula? Justify.

1.7 Parity and primes: a first look

The grammar of this chapter is now complete; here is some content to practise it on. Two definitions run through the whole book.

Definition 1.16 (Even, odd, divides)**.** An integer n is *even* when $\exists k \in \mathbb{Z}\ (n = 2k)$, and *odd* when $\exists k \in \mathbb{Z}\ (n = 2k+1)$. For integers d, m we say d *divides* m when $\exists k \in \mathbb{Z}\ (m = d \cdot k)$.

Definition 1.17 (Prime)**.** An integer $n > 1$ is *prime* when its only positive divisors are 1 and n: in symbols, $\forall d \in \mathbb{Z}^+\ \big(d \text{ divides } n \Rightarrow (d = 1 \vee d = n)\big)$. An $n > 1$ that is not prime is *composite*.

Both definitions are quantified statements. "n is prime" is a $\forall \Rightarrow$ statement — the expensive kind, which is why proving primality means excluding every candidate divisor, while disproving it needs one witness. Already you can classify the work: "every prime > 2 is odd" is $\forall p\ \big((P(p) \wedge p > 2) \Rightarrow \text{odd}(p)\big)$; a proof (Chapter 2) must take an arbitrary such p and derive oddness, while one even prime > 2 would kill it. The claims and their forms:

Claim	Shape	To refute it you need
every prime > 2 is odd	$\forall \Rightarrow$	one even prime > 2
every even $n > 2$ is composite	$\forall \Rightarrow$	one even prime > 2
there are infinitely many primes	$\forall n\, \exists p\ (p > n \wedge P(p))$	a largest prime
some even n is a sum of two primes	$\exists$	check all — impossible

Notice the last row: a bare $\exists$ claim can almost never be *refuted* by search, only verified. That asymmetry is why existence claims and universal claims play different roles in problems.

Parity is governed by a small arithmetic. You may verify all four rows yourself — each is a two-line calculation once "even" is read as $2k$:

x	y	$x+y$	xy
even	even	even	even
even	odd	odd	even
odd	even	odd	even
odd	odd	even	odd

This table is the entire engine behind parity arguments — and it is also worth knowing where it does *not* apply. AMC 12 2024 B P5 asks for the fewest $+$-to-$-$ sign flips in $1 + 3 + \cdots + 99$ that make the sum negative; flipping a term t changes the total by $2t$, so the sum's parity is an invariant and the real engine is magnitude — how few of the largest terms sum past half of 2500. Proofs that this table is correct come in Chapter 2; the shape to notice is $\forall$ *pairs of integers, a finite check of four cases* — a universal statement with a finite universe in disguise.

Exercise 1.5 (medium)**.** Find all positive integers n for which the statement "n divides $n + 6$" is true. (Use the division definition above: d divides m when $m = d \cdot k$ for some integer k.)

1.8 Chapter summary

Symbol	Name	English forms	Negation
$P \land Q$	conjunction	and; both; but	$\neg P \lor \neg Q$
$P \lor Q$	disjunction	or (inclusive); at least one of	$\neg P \land \neg Q$
$\neg P$	negation	not; it is false that	P
$P \Rightarrow Q$	implication	if P then Q; P only if Q; Q whenever P	$P \land \neg Q$
$P \iff Q$	biconditional	iff; exactly when; equivalent	$(P \land \neg Q) \lor (\neg P \land Q)$
$\forall x\, P(x)$	universal	every; each; any; no matter which	$\exists x\, \neg P(x)$
$\exists x\, P(x)$	existential	some; there is; at least one	$\forall x\, \neg P(x)$
$\exists! x\, P(x)$	unique existence	exactly one	$\forall x\, \neg P(x) \lor$ two distinct witnesses

Table 1.1: The grammar of the book on one page. The last column is the mechanical negation rule — the most-used column.

Claim shape	To establish it	To refute it
$\forall x\, P(x)$	handle every x (proof, Chapter 2)	one counterexample
$\exists x\, P(x)$	one witness	rule out every x
$P \Rightarrow Q$	assume P, derive Q; or assume $\neg Q$, derive $\neg P$	one case with P true, Q false
$P \iff Q$	both directions, separately	one direction fails
$\forall x\, \exists y$	give each x its own y	one x defeating all y
$\exists y\, \forall x$	one y serving all x	for each y, some x it fails

Table 1.2: What a proof or a refutation owes you, per claim shape.

1.9 Common mistakes

- **Asserting the converse.** From "if n is divisible by 6 then n is even" and "n is even" you get nothing — $n = 10$ satisfies the conclusion and not the hypothesis. The converse $Q \Rightarrow P$ is a *different* claim (Figure 1.1); proving it is a separate job.
- **Negating $\forall\exists$ as $\exists\exists$.** The negation of "every row has a mark" is "*some* row has *no* marks" ($\exists x\, \forall y$), not "some row has some mark". Flip each quantifier once and keep the order.
- **Reading "or" as exclusive.** "x is even or x is prime" is true of $x = 2$ twice over; disjunction does not promise exactly one branch. When a statement means exactly one, it says so — translate that as two claims ($\vee$ plus $\neg\wedge$), as in Exercise 1.4.
- **Negating an implication as another implication.** $\neg(P \Rightarrow Q)$ is $P \wedge \neg Q$, not $P \Rightarrow \neg Q$. A counterexample to "if n is prime then n is odd" is a prime that is *not* odd ($n = 2$) — an actual object, not a reversed claim.
- **Marking vacuously true claims false.** "Every even prime greater than 2 is divisible by 5" is *true*: there are no even primes above 2, so the implication never gets a test case. Vacuous truth feels like a cheat; it is the correct reading.
- **Assuming one witness serves two existential claims.** From $\exists x\, P(x)$ and $\exists x\, Q(x)$ you cannot conclude $\exists x\, \big(P(x) \wedge Q(x)\big)$ — the x's may be different objects (some number is even; some number is odd; no number is both). Existentials only merge when one statement produces the witness the other needs.

1.10 Additional exercises

Exercise 1.6 (easy)**.** Write the truth table of $(P \vee Q) \wedge \neg(P \wedge Q)$ and compare it with "exactly one of P, Q". This is the *exclusive or*.

Exercise 1.7 (easy)**.** Negate mechanically: (a) $\exists x\, \forall y\, R(x, y)$; (b) $\forall x\, \big(P(x) \Rightarrow Q(x)\big)$; (c) "there is no largest prime" after writing it in symbols.

Exercise 1.8 (easy)**.** Over $\mathbb{Z}$, is $\exists x\, \forall y\ (x \cdot y = y)$ true or false? Give a witness or a refutation, and state the claim in English.

Exercise 1.9 (easy)**.** Classify each as statement, open formula, or neither: (a) n^2 is even; (b) n is even for every n; (c) every multiple of 4 is even; (d) prove that n is even.

Exercise 1.10 (medium)**.** Write in symbols and determine truth over $\mathbb{Z}$: (a) "for every x there is a y with $x + y = 0$"; (b) "there is a y such that for every x, $x + y = 0$". Explain the difference in one sentence.

Exercise 1.11 (medium)**.** Prove or disprove: if x^2 is divisible by 4, then x is divisible by 4. (If disproving, exhibit the smallest counterexample.)

Exercise 1.12 (medium)**.** Negate the ε–δ statement you will meet in analysis: $\forall \varepsilon > 0\, \exists \delta > 0\, \forall x\, (|x - a| < \delta \Rightarrow |f(x) - f(a)| < \varepsilon)$. Your answer should begin "$\exists \varepsilon > 0$ such that…".

Exercise 1.13 (medium)**.** On the set $\{1, 2, 3\}$, define a relation $R(x, y)$ such that $\forall x\, \exists y\, R(x, y)$ is true and $\exists y\, \forall x\, R(x, y)$ is false; then one where both are true. Draw both as grids of marks like Figure 1.3.

Exercise 1.14 (hard)**.** Define $\exists! x\, P(x)$ two ways: (i) $\exists x \big(P(x) \wedge \forall y\, (P(y) \Rightarrow y = x)\big)$ and (ii) $\exists x\, P(x) \;\wedge\; \forall x \forall y \big(P(x) \wedge P(y) \Rightarrow x = y\big)$. Prove the two readings agree: each implies the other. (This is the shape of every "unique solution" claim in the book.)

Exercise 1.15 (hard)**.** Extend `code/verify_quantifiers.py` into a tiny verifier: input a finite grid of marks and a claim of the form $\forall x\, \exists y$, $\exists y\, \forall x$, or their negations, and have it print the verdict. Test it on the three worlds of the listing and on Figure 1.3's two grids.

Exercise 1.16 (easy)**.** Sort these into equivalent pairs: $P \Rightarrow Q$; $\neg Q \Rightarrow \neg P$; $Q \Rightarrow P$; $\neg P \Rightarrow \neg Q$; $\neg P \vee Q$; $\neg(P \wedge \neg Q)$. Which pairs are provably equal by truth table?

Exercise 1.17 (medium)**.** Decide the truth value of each claim over $\mathbb{Z}$, and say whether any is vacuously true: (i) every negative perfect square is divisible by 4; (ii) every integer $n > 1$ has a divisor d with $1 < d \leq \sqrt{n}$ or n is prime; (iii) some composite n has no prime factor at most $\sqrt{n}$.

Exercise 1.18 (medium)**.** Write "there is a unique even prime" in symbols using $\exists! x\, P(x)$ expanded as in the earlier hard exercise, then prove the statement true by exhibiting the witness and showing uniqueness.

Exercise 1.19 (medium)**.** A classmate translated "no even number above 2 is prime" as $\forall n \big(\text{even}(n) \wedge n > 2 \Rightarrow \neg\text{prime}(n)\big)$ — correct — but then negated it as $\exists n\, (\text{even}(n) \wedge n > 2 \Rightarrow \text{prime}(n))$. Find the two errors and write the real negation in English.

Exercise 1.20 (hard)**.** Prove by truth table that $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$, then use it to negate, in one line each: (i) "if n is even then n^2 is divisible by 4"; (ii) "whenever the algorithm halts, its output is sorted". Give the English reading of each negation.

Exercise 1.21 (hard)**.** "No" hides a negated existential. Write "no prime above 10 ends in 5" three ways: as $\neg\exists$, as $\forall \ldots \neg$, and as a single implication over $\mathbb{Z}$. Verify all three agree on a truth-table-style check over $n \in \{2, \ldots, 20\}$ (by hand or code).

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 2 — `https://richardhammack.github.io/BookOfProof/` — free PDF; the friendliest full treatment of connectives and quantifiers, with a large exercise bank pitched exactly at this chapter.
- MIT OpenCourseWare, *6.042J Mathematics for Computer Science* (Fall 2010), Ch. 1 — `https://ocw.mit.edu/6-042JF10` — the course this book's proof-first order follows; Ch. 1 covers propositions, implication and proof patterns.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science* (the 6.042J text), §1 — `https://courses.csail.mit.edu/6.042/spring18/mcs.pdf` — free PDF; denser than Hammack, worth it for the "logic in programs" angle.
- Stanford CS103, *Mathematical Foundations of Computing* course reader — `https://web.stanford.edu/class/cs103/` — free reader + problem sets; its logic units are the best drills on translation and negation.
- The Open Logic Project, *forall x: Calgary* — `https://openlogicproject.org/` — free intro-logic text; go here when you want the formal (Fitch-style) counterpart of what this chapter teaches informally.

- CMU 15-151, *Mathematical Foundations for Computer Science* (Great Ideas in Theoretical CS) — https://www.cs.cmu.edu/~15281/ — lecture notes by Anil Ada et al.; the early chapters put propositional logic to work on real problems fast.

WATCH INSTEAD

- **MIT OpenCourseWare — "Lec 1 | MIT 6.042J"**, Tom Leighton, 44:09, https://www.youtube.com/watch?v=L3LMbpZIKhQ — covers §1.1–1.3: propositions, truth tables, implication, and the first proofs. (Verified 2026-10-11.)
- **Neso Academy — "Logical Operators: Negation, Conjunction & Disjunction"**, 6:09, https://www.youtube.com/watch?v=6kYngPvoGxU — covers §1.1–1.2: the connectives and their truth tables, the quickest possible refresher. (Verified 2026-10-11.)
- **Mathematical Visual Proofs — "Predicates and Quantifiers [Discrete Math Class]"**, ∼10:00, https://www.youtube.com/watch?v=0rvKhma-3f4 — covers §1.4–1.5: predicates, quantifier order, and the negation laws that match Figure 1.5. (Verified 2026-10-11.)

Notes

The order — connectives, truth tables, implication family, then quantifiers and mechanical negation — follows Velleman's first two chapters, the clearest prose treatment of this material [2], and Hammack's logic chapter, whose exercise mix (translate, negate, truth-table, find-the-error) shaped ours [3]. The proof-first ordering across the whole book follows MIT 6.042J [4]. The finite-world evaluation of Example 1.11 is in the tradition of Barwise and Etchemendy's *Tarski's World* [5]; Cut-the-Knot's logic and fallacy pages [6] supply many of the standard traps catalogued in Common mistakes. Problem citations (AMC 12 2024 A P22, AMC 12 2024 B P5) point to the AoPS archive [7]; read the statements there — they are referenced, never reproduced.

CHAPTER 2

How to write a proof

Chapter 1 gave you the grammar claims are written in. This chapter gives you the machinery that establishes them: a small inventory of proof techniques — direct, cases, contrapositive, contradiction, two directions for an iff, witnesses for existence, counterexamples for disproof — and, just as important, the writing habits that make a proof readable and checkable. After this chapter you will be able to look at a claim, name the technique its shape asks for, and produce a proof a grader can verify line by line. Every later chapter is practice on this one; the techniques here never change, only the raw material does.

BEFORE YOU START

Chapter 1: connectives, "if…then", and the quantifiers $\forall$ and $\exists$. *Recap:* to prove "for every x, $P(x)$" you must handle an *arbitrary* x; to disprove it, one counterexample is enough.

AFTER THIS CHAPTER YOU CAN

- write a short direct proof in the standard layout: claim, assumptions, steps, conclusion;
- choose between direct proof, cases, contrapositive and contradiction for a given claim;
- prove an "if and only if" in both directions and an existence claim by construction;
- read your own proof the way a grader would and mark its gaps.

HOW TO THINK ABOUT THIS

A proof is a conversation with a sceptical friend who accepts only what you can justify. Do not start by writing; start by asking "what do I know, and what do I want?" and by trying a small example to see *why* the claim is true. The proof is then you writing that reason down so clearly that the friend has nothing left to object to.

2.1 What a proof is

Definition 2.1 (Proof)**.** A *proof* of a statement is a finite chain of assertions ending at that statement, where each assertion is either an axiom, a definition, a hypothesis, or a consequence of earlier lines by a rule of inference.

Three kinds of raw material can appear in the chain. *Axioms* are the ground rules you do not prove: for this book, the arithmetic laws of $\mathbb{Z}$, $\mathbb{Q}$ and $\mathbb{R}$ (addition, multiplication, order), plus the definitions Chapter 1 fixed. *Definitions* unfold on demand: "even" always expands to "$= 2k$ for some integer k". *Theorems* are claims already proved; once proved, a theorem becomes a tool you may cite by name instead of re-proving it.

The legal moves between lines are the rules of *inference*, which are just Chapter 1 applied: from P and $P \Rightarrow Q$ you may write Q (*modus ponens*); from $\forall x\, P(x)$ you may write $P(a)$ for a specific a; from $P \wedge Q$ you may write P. If a line of your proof cannot name its licence — "by definition of even", "by modus ponens on lines 2 and 4" — the line does not belong yet.

Remark. "Axioms vs. proven tools" is worth keeping straight for one practical reason: in an exam you may cite theorems you have proved or been given, but not the claim you are currently proving. Citing your own goal is the circularity trap of Section 2.12.

2.2 Direct proof and the template

The workhorse. To prove $P \Rightarrow Q$: assume P, derive Q. Almost every proof you write this year will carry this skeleton, often with other techniques nested inside.

THE ASSUME–DERIVE–CONCLUDE TEMPLATE

Write the first and last lines before the middle: *Assume* the hypothesis (unfolded to definitions), *conclude* the goal (restated), then *derive* the gap. Figure 2.1 shows the shape; the middle is where the problem lives, but the two ends are free.

Theorem 2.2. *If m and n are even integers, then $m + n$ is even.*

Proof. Let m and n be even integers. By definition of even, $m = 2j$ and $n = 2k$ for some integers j and k. Then $m + n = 2j + 2k = 2(j + k)$, and $j + k$ is an integer, so $m + n$ is even. ∎

Read the proof again against the template: the assume line unfolds "even" on both inputs at once; the derivation is a single algebraic rewrite; the conclude line checks the definition's existential ("for some integer") by exhibiting $j + k$. That last detail — the new object $j + k$ must be shown to be an integer — is what separates a proof from a gesture.

Remark. Two even numbers need two names: $m = 2j$, $n = 2k$. Writing $m = 2k$ and $n = 2k$ forces $m = n$, a silent extra assumption. Fresh objects get fresh names — one of the cheapest correctness rules in this book.

Exercise 2.1 (easy)**.** Prove that the sum of two odd integers is even, using the template. (Odd means $2k + 1$ for some integer k.)

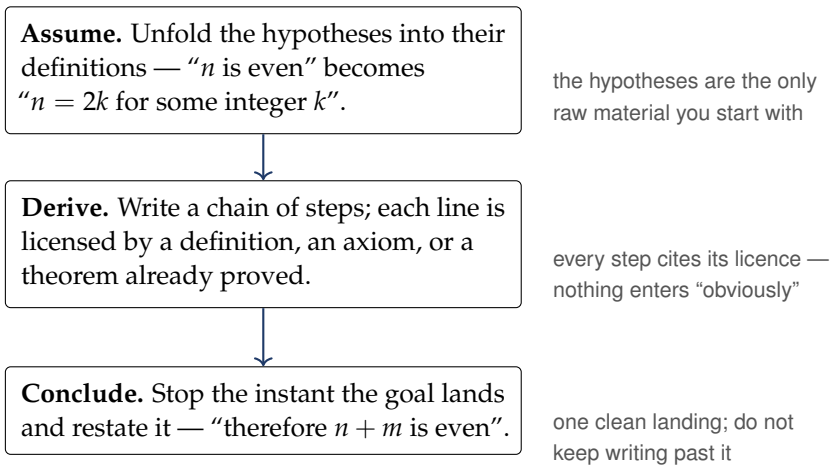


Figure 2.1: The direct-proof template. Notice that the creative part is confined to the middle stage — the assume line and the conclude line are determined by the claim itself, so writing them down first costs nothing and fixes the target.

2.3 Proof by cases

Sometimes the hypothesis does not unfold into one usable form, but the universe splits into a few shapes that each do. *Proof by cases* handles every shape separately; it is valid exactly when the cases are *exhaustive* — every object falls into at least one case.

Theorem 2.3. *For every integer n, the number $n^2 + n$ is even.*

Proof. Every integer is even or odd (Section 1.7), so the two cases cover all n. *Case 1: n even.* Write $n = 2k$. Then $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$, and $2k^2 + k$ is an integer, so $n^2 + n$ is even. *Case 2: n odd.* Write $n = 2k + 1$. Then $n^2 + n = 4k^2 + 6k + 2 = 2(2k^2 + 3k + 1)$, again even. ∎

CASES DONE RIGHT

Before the cases, **say why they are exhaustive** ("every integer is even or odd"). A third case you forgot is not a gap in style — it is a hole in the proof. When the split is by residues ($n \bmod 4 \in \{0, 1, 2, 3\}$), exhaustiveness is automatic; when it is by geometry or sign, state it explicitly.

Exercise 2.2 (medium)**.** Prove that for every integer n, the product $n(n+1)(n+2)$ is divisible by 6. (Cases on n mod 6? Or: is one factor always divisible by 2, and one by 3?) For the contest version of divisibility-by-casework, see AMC 12 2023 A P3 [7].

2.4 Contrapositive

Chapter 1 showed $P \Rightarrow Q$ and $\neg Q \Rightarrow \neg P$ carry the same truth value (Figure 1.1). So they are the same job: prove whichever unfolds more usefully. The contrapositive earns its keep exactly when the *hypothesis* has no handle but the *negated conclusion* does.

Example 2.4. Prove: for every integer n, if n^2 is even then n is even.

Strategy. Try it directly first and watch where it stalls: n^2 even gives $n^2 = 2k$, and now you must say something about n — but n is $\sqrt{2k}$, and square roots of integers need not be integers. The hypothesis talks about n^2; nothing in it unfolds n. Flip to the contrapositive, where the hypothesis is "n is odd" — a form of n itself, which is exactly the handle we lacked.

Solution. We prove the contrapositive: if n is odd, then n^2 is odd. Let n be odd, so $n = 2k + 1$ for some integer k. Then

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

and $2k^2 + 2k$ is an integer, so n^2 is odd. By the contrapositive, if n^2 is even then n is even.

Two habits to copy from Example 2.4: the proof *says out loud* that it is proving the contrapositive (a reader cannot guess your route), and the strategy block shows the stalled direct attempt — knowing why the obvious route fails is how you learn to pick the right one.

Exercise 2.3 (easy)**.** Prove by contrapositive: for real numbers x, y, if $x + y \geq 2$ then $x \geq 1$ or $y \geq 1$.

2.5 Contradiction

To prove P by contradiction: assume $\neg P$ and derive an impossibility — something colliding with a definition, an axiom, or arithmetic itself. It is the technique of last resort and of necessity: for claims of the form "no object satisfies…" it is often the only door, since a universal negative gives you nothing to construct.

CONTRADICTION TEMPLATE

"Assume, for contradiction, that *the claim fails*." Then treat the negated claim — usually $\exists$ of something forbidden — as a new object in hand and squeeze it until it breaks.

Example 2.5 ($\sqrt{2}$ is irrational)**.** Prove: there are no integers p, q with $q \neq 0$ and $(p/q)^2 = 2$.

Strategy. The claim is a non-existence claim — the natural habitat of contradiction. Suppose a fraction p/q in lowest terms squares to 2 ("lowest terms" is the extra lever: it makes the witness *minimal*, and minimal objects can be forced below themselves). Unfolding $p^2 = 2q^2$ shows p is even; that forces q even too; and two even numbers were never in lowest terms.

Solution. Assume, for contradiction, that $\sqrt{2} = p/q$ for integers p, q with $q \neq 0$, written in lowest terms, so p and q share no common factor. Then $p^2 = 2q^2$, so p^2 is even, so by

Example 2.4 p is even: write $p = 2a$. Substituting, $4a^2 = 2q^2$, hence $q^2 = 2a^2$ is even, so q is even too. Then p and q share the factor 2, contradicting lowest terms. Therefore no such p, q exist: $\sqrt{2}$ is irrational.

Example 2.6 (Working modulo 4)**.** Prove: there are no integers a, b with $a^2 - 4b = 2$.

Strategy. Again a non-existence claim, so contradiction. But squeeze *where*? The term $4b$ is invisible modulo 4: reducing the equation mod 4 erases b entirely and leaves the finite question "can a^2 be 2 mod 4?" — checkable by cases on a. Modular arithmetic is a lens: it discards the parts of a statement that cannot obstruct you and keeps the part that can.

Solution. Assume integers a, b satisfy $a^2 - 4b = 2$. Reducing mod 4 gives $a^2 \equiv 2 \pmod 4$. But a is even or odd: if $a = 2k$ then $a^2 = 4k^2 \equiv 0$; if $a = 2k + 1$ then $a^2 = 4k^2 + 4k + 1 \equiv 1$. So $a^2 \bmod 4 \in \{0, 1\}$, never 2 — contradiction. No such a, b exist.

Notice the proof inside Example 2.6 is a two-case argument nested in a contradiction: techniques compose. The mod-4 lens will return throughout the number theory chapters; remainders problems in this style appear constantly on contests (see AMC 12 2024 B P14 on the AoPS archive [7]).

Exercise 2.4 (medium)**.** Adapt Example 2.5 to prove $\sqrt{3}$ is irrational. (Where does the argument use that 2 is prime? What goes wrong for $\sqrt{4}$?)

2.6 If and only if

A biconditional $P \iff Q$ is two jobs stapled together, and you must do both: the forward direction $P \Rightarrow Q$ and the reverse $Q \Rightarrow P$ — each by whichever technique suits it. Label them ($\Rightarrow$) and ($\Leftarrow$) so the reader can audit both.

Theorem 2.7. *For every integer n: n is even if and only if n^2 is even.*

Proof. ($\Rightarrow$) If $n = 2k$, then $n^2 = 4k^2 = 2(2k^2)$, which is even. ($\Leftarrow$) This is Example 2.4, proved there by contrapositive. ∎

TRIPWIRE

Proving one direction and stopping is the single most common broken biconditional. "n even iff n^2 even" needs *both* halves; a proof of ($\Rightarrow$) alone leaves the claim unproved — and in contests, an unproved direction usually hides the interesting half. See Common mistakes.

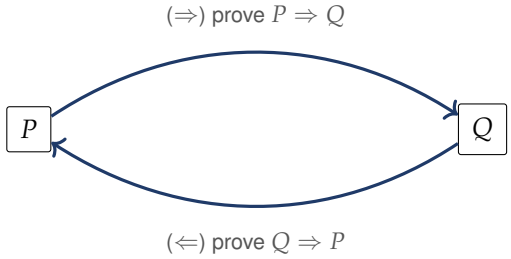


Figure 2.2: A biconditional is two one-way streets. Notice: the two arrows are *separate* proofs — each may need a different technique, and finishing one leaves the claim half-proved.

2.7 Existence and uniqueness

An existential claim $\exists x\, P(x)$ has the cheapest proof of all: exhibit one x and check it. This is a *constructive* proof — you hand over the object. To prove *unique* existence $\exists! x\, P(x)$, add a second act: any two candidates x_1, x_2 with $P(x_1), P(x_2)$ must satisfy $x_1 = x_2$ (existence + uniqueness, the two halves from Chapter 1).

Not every existence proof constructs. The classic counterexample: there exist irrational numbers a, b such that a^b is rational. Consider $\sqrt{2}^{\sqrt{2}}$: if it is rational, take $a = b = \sqrt{2}$. If it is irrational, take $a = \sqrt{2}^{\sqrt{2}}$ and $b = \sqrt{2}$ — then $a^b = (\sqrt{2}^{\sqrt{2}})^{\sqrt{2}} = \sqrt{2}^2 = 2$, rational. Either way some pair works; the proof never says which, and it is still a complete proof. *Non-constructive* existence is legal — just know which kind you are writing, since only constructive proofs produce objects you can compute with.

Example 2.8 (A unique solution)**.** Prove there is a unique positive integer n with $n^2 + n = 156$.

Strategy. Two acts again: find one n (search small values — $f(n) = n^2 + n$ grows fast), then show no other positive integer works. For uniqueness, monotonicity is the cheapest tool: if f is strictly increasing on positive integers, two different inputs can never land on 156.

Solution. Existence. $n = 12$: $12^2 + 12 = 144 + 12 = 156$. *Uniqueness.* Suppose m is a positive integer with $m^2 + m = 156$. If $m > 12$ then $m^2 + m \geq 13^2 + 13 = 182 > 156$; if $m < 12$ then $m^2 + m \leq 11^2 + 11 = 132 < 156$. So $m = 12$. ■

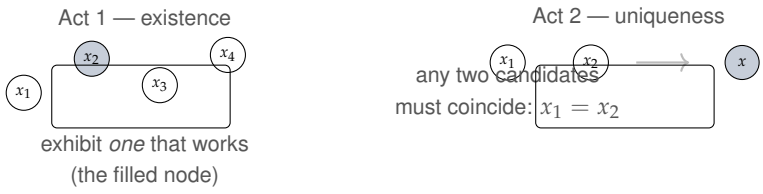


Figure 2.3: The two acts of $\exists!$. Notice: existence needs only one working object, but uniqueness needs an argument covering *all* candidates — here, squeezing m between 11 and 13.

Exercise 2.5 (easy)**.** Prove $\exists! x \in \mathbb{Z}$ with $3x - 7 = 2$: exhibit the witness, then prove any two witnesses are equal.

2.8 Disproof and counterexamples

To refute $\forall x\, P(x)$ you owe one thing only: a single x where P fails. The negation is $\exists x\, \neg P(x)$, and existentials cost one witness. Where proofs get long, disproofs get short — *if* the counterexample is real. Convention: hunt the *smallest* candidate where the claim could break, and check it honestly.

Example 2.9 (Euler's polynomial)**.** For which integers $x \geq 0$ is $f(x) = x^2 + x + 41$ prime?

Strategy. Evaluate the first values and you get a famous run: $f(0) = 41$, $f(1) = 43$, $f(2) = 47$, all prime — and the run continues far past where most people stop checking. The trap is to call the claim proved by accumulated evidence; the discipline is to ask where it *must* break. Since $f(x) = x(x+1) + 41$, the value is a multiple of 41 whenever 41 divides $x(x+1)$ — first at $x = 40$ (and $x = 41$). That is the smallest forced failure. Check it before believing the streak.

Solution. $f(40) = 40 \cdot 41 + 41 = 41^2 = 1681$, composite. So the claim "$f(x)$ is prime for all $x \geq 0$" is false; the counterexample is $x = 40$. Figure 2.4 and Listing 11.1 show how far the run actually goes.

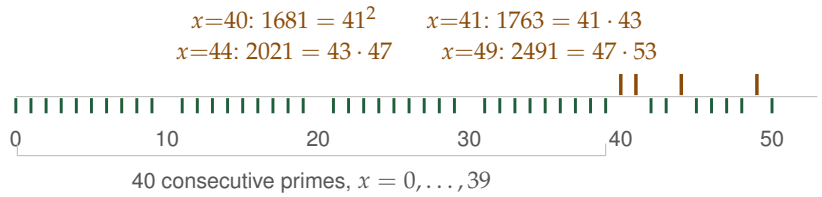


Figure 2.4: The streak that is not a theorem. Notice the first composite lands at the first x with $41 \mid x(x+1)$, exactly where the algebra says it must — forty consecutive primes proved nothing on their own.

Checkpoint 2.1 *Predict before you run.* Listing 11.1 tests $x = 0$ through 50. Before running: predict the first composite x, predict how many composites appear in 0..50, and predict the smallest factor at $x = 40$ and $x = 41$ without dividing anything — use $f(x) = x(x+1) + 41$.

```
f = lambda x: x*x + x + 41
composites = [(x, f(x)) for x in range(51) if not is_prime(f(x))]
```

Listing 2.1: `check_prime_run.py`: where does the $x^2 + x + 41$ prime streak break?

The full script (in `code/check_prime_run.py`) defines `is_prime` and `least_factor` and prints every composite. Its real output:

```
f(x) = x^2 + x + 41
x = 0..39: all prime (40 consecutive values)
x = 40: f(40) = 1681 = 41 * 41 <-- first composite
x = 41: f(41) = 1763 = 41 * 43
x = 44: f(44) = 2021 = 43 * 47
x = 49: f(49) = 2491 = 47 * 53
4 composites in x = 0..50
```

The machine confirms the algebra: $x \equiv 0$ or $-1 \pmod{41}$ forces $41 \mid f(x)$ (that is $x = 40, 41$; the $x = 44$ and $x = 49$ failures arrive through other prime factors). Forty examples built the conjecture; one number killed it. That is the division of labour between examples and proofs for the rest of this book: **examples inform, proofs establish, counterexamples demolish.**

2.9 Mathematical writing

A proof is read by a person, and graded by one too. The rules that make it checkable are few and mechanical:

WRITING RULES

1. **Complete sentences.** A proof is English with symbols inside the sentences, not a pile of symbols. Every line parses as a sentence.
2. **State the claim and the technique first.** "We prove the contrapositive." "Assume, for contradiction, that…" One sentence of orientation replaces a paragraph of guessing.
3. **Name every object.** "Let $n = 2k$ for an integer k." An unnamed letter is an uncontrolled assumption.
4. **Displayed math for anything long.** Multi-line algebra goes

   on its own line

   so the reader can verify it out of the prose flow.
5. **Ban unjustified "clearly/obviously".** If it is obvious, proving it takes one line — write the line. If it is not, you were about to lose the reader (or the point).

Figure 2.5 applies the rules to a real flawed paragraph — the kind that loses marks not because the idea is wrong but because nothing in it is licensed.

2.10 Strategy before proof

One convention of this book deserves its own section. Every worked example here opens with a **Strategy** paragraph — what to try, why this technique fits the claim's shape — and only then the **Solution**, the cleaned proof. The strategy is scaffolding: it is how the solution was found, and it is the part no polished textbook ever shows you. In your own write-ups (contests, the portfolio of Chapter 17) you submit only the proof — but you should still be writing the strategy somewhere, on scratch paper or in your head, because choosing the technique *is* most of the problem. Figure 2.6 is the book's choosing procedure in one diagram.

"**Claim:** if n^2 is even then n is even. **Proof:** n^2 is even, so $n^2 = 2k$. [1] Then $n = \sqrt{2k}$. [2] Since n^2 is even, n is even. [3] So the statement holds for all n. [4] Check $n = 4$: $4^2 = 16$ is even and 4 is even, so it works."

[1] No licence for this step: $\sqrt{2k}$ need not be an integer, and the detour never touches the parity of n — the thing the claim is about.

[2] Circular: the conclusion is restated as its own justification. "Since n^2 is even, n is even" is exactly the claim under proof.

[3] n was never fixed. The proof must open with "let n be an integer"; an unnamed variable quantifies nothing.

[4] Proof by example. One check verifies $n = 4$ and says nothing about the other infinitely many integers.

Figure 2.5: A flawed "proof" of the claim of Example 2.4, with four defects marked and explained. Notice that three of the four defects are about *licence* — steps nobody gave the writer permission to take — and the fourth is evidence posing as argument. The corrected proof is Example 2.4.

2.11 Chapter summary

2.12 Common mistakes

- **Half a biconditional.** "P iff Q" proved only in the direction that was convenient. The other direction is a separate claim — and usually the one that carries the content. Label ($\Rightarrow$) and ($\Leftarrow$) so a missing half cannot hide (Section 2.6).
- **Circular argument.** Using the goal — or a statement that secretly contains it — as a step. Marked [2] in Figure 2.5: "since n^2 is even, n is even" asserts exactly what was to be shown. Every line needs a licence from *earlier* lines.
- **Proof by example.** Checking $x = 0, 1, 2, \ldots, 39$ proves $x = 40$ nothing — Figure 2.4's streak died at exactly $x = 40$. Examples inform conjectures; they never close a universal claim.
- **Unfounded "clearly".** "Clearly n is even" is where proofs go to hide the one step they cannot justify. Replace each "clearly" with its one-line justification, or admit the step is not there.

2.13 Additional exercises

Exercise 2.6 (easy)**.** Prove directly: for every integer n, if n is odd then $3n + 1$ is even.

Exercise 2.7 (easy)**.** Prove directly: the product of two odd integers is odd.

Exercise 2.8 (easy)**.** Disprove: for every $n \geq 0$, $n^2 - n + 11$ is prime. (Find the smallest counterexample — it arrives sooner than Euler's did.)

Exercise 2.9 (easy)**.** Prove $\exists\, x, y \in \mathbb{Z}$ with $x^2 - y^2 = 9$; then prove the solution with $x > y > 0$ and $x + y = 9$ is unique.

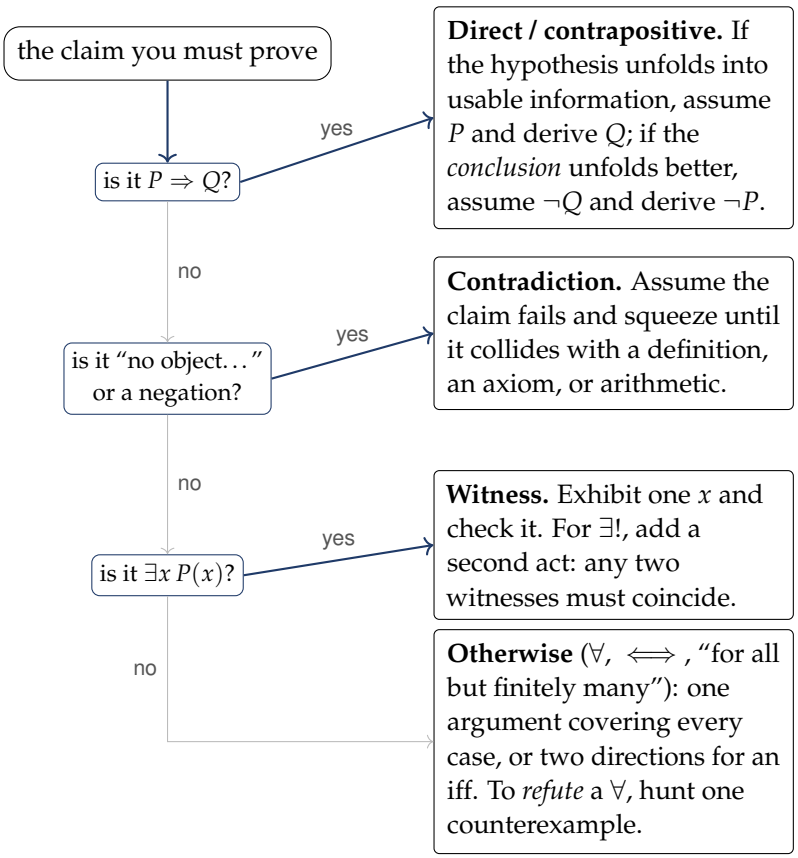


Figure 2.6: Choosing a technique by the goal's shape. Notice what is *not* on the chart: "proof by induction" and friends arrive in Chapter 3, and every shape still lands on a technique — there is no "I don't know where to start" branch. When stuck, write the negation: the shape you see there often names the route.

Exercise 2.10 (medium)**.** Prove by cases: for all real x, y, $|x + y| \leq |x| + |y|$. (Split on the signs of $x + y$, or of x and y; AMC 12 2024 A P6 is a contest instance of sign-casework [7].)

Exercise 2.11 (medium)**.** Prove by contrapositive: for integers a, b, if $a + b$ is odd then exactly one of a, b is even. (Careful — "exactly one" is two claims.)

Exercise 2.12 (medium)**.** Prove there are no integers x, y with $6x + 9y = 1$. Which single lens does all the work? (AIME 2010 I P5 applies the same one [7].)

Exercise 2.13 (medium)**.** Prove: for every integer n, n is odd if and only if n^2 is odd. State the technique used in each direction.

Exercise 2.14 (hard)**.** Prove, by two different methods, that no integers x, y satisfy $x^2 - 4y = 2$: (i) modulo 4 as in Example 2.6, and (ii) a parity argument that never mentions residues. Then state in one sentence which method generalises to $x^2 - 9y = 3$ and why.

Exercise 2.15 (hard)**.** Extend `code/check_prime_run.py` to test other prime-producing polynomials: $g(x) = x^2 + x + 17$ on $x = 0..40$ and $h(x) = 2x^2 + 29$ on $x = 0..60$. Report each first composite and explain — by factoring like $f(40) = 41^2$ — why it was forced.

Exercise 2.16 (hard)**.** Euclid's argument. Assume there are only finitely many primes $p_1, p_2, \ldots, p_k$, and let $N = p_1 p_2 \cdots p_k + 1$. Prove that N has a prime factor not among

Goal shape	Technique	First line to write	Killer detail
$P \Rightarrow Q$, hypothesis unfolds	direct	"Assume P."	unfold definitions
$P \Rightarrow Q$, hypothesis has no handle	contrapositive	"We prove the contrapositive: assume $\neg Q$."	say the route aloud
"no object satisfies…"; $\neg P$	contradiction	"Assume the claim fails."	find the collision
hypothesis splits into shapes	cases	"Every x is … or …; cases cover all."	prove exhaustiveness
$P \iff Q$	two directions	"($\Rightarrow$) …" then "($\Leftarrow$) …"	both halves, each labelled
$\exists x\, P(x)$	witness	"Take $x = \ldots$"	check it works
$\exists! x\, P(x)$	witness + uniqueness	"Take $x = \ldots$; now let x_1, x_2 both work."	force $x_1 = x_2$
refute $\forall x\, P(x)$	counterexample	"$x = \ldots$ fails."	smallest candidate first

Table 2.1: The technique menu on one page: the shape of the goal names the proof. The last column is where each technique most often breaks in student work.

$p_1, \ldots, p_k$, conclude infinitely many primes exist, and explain why "N is prime" is *not* the right claim to make.

Exercise 2.17 (easy)**.** For each claim, name the proof technique the shape suggests (no proof needed): (i) "if $3n + 2$ is odd then n is odd"; (ii) "no integer is both even and odd"; (iii) "n is even iff $n{+}2$ is even"; (iv) "for every nonzero digit d, d^2's last digit is in $\{1, 4, 5, 6, 9\}$".

Exercise 2.18 (medium)**.** Prove by cases on n mod 3: for every integer n, $n^3 - n$ is divisible by 3. Write the case-exhaustiveness line explicitly.

Exercise 2.19 (medium)**.** Disprove or prove, choosing the right tool: "for every prime p, $2^p - 1$ is prime." If false, give the smallest counterexample (check $p = 2, 3, 5, 7, 11$).

Exercise 2.20 (hard)**.** Prove: for every integer n, n is odd iff $n^2 \equiv 1 \pmod 8$. Label both directions and say which one needs cases on $n = 4k{+}1$ vs $4k{+}3$.

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 4–10 — https://richardhammack.github.io/BookOfProof/ — free PDF; the technique-by-technique spine this chapter mirrors, with many more worked proofs.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 1–2 — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; "patterns of proof" done CS-style, fast and rigorous.
- E. Chen, *Remarks on English* — https://web.evanchen.cc/handouts/english/english.pdf — the write-up standards handout; read it once per section of this chapter.
- Stanford CS103, *Mathematical Foundations of Computing* — https://web.stanford.edu/class/cs103/ — free reader; its proof-writing guides cover exactly this chapter's ground.
- MIT OpenCourseWare, *6.042J Mathematics for Computer Science* (Fall 2010) — https://ocw.mit.edu/6-042JF10 — lecture notes and problem sets for the proofs-first half of the course.
- AoPS, *How to Write a Solution* — contest write-up advice; pairs with §2.9. https://artofproblemsolving.com/articles/how-to-write-solution

WATCH INSTEAD

- **TrevTutor — "[Discrete Mathematics] Direct Proofs Examples"**, 6:59, https://www.youtube.com/watch?v=uDJfx4bK3Jc — covers §2.2: two worked direct proofs including divisibility. (Verified 2026-10-11.)
- **TrevTutor — "Can You Prove? Proofs in Discrete Math and Real Analysis"** (playlist), https://www.youtube.com/playlist?list=PLDDGPdw7e6AjLVCdoobmqvyZZpthHGBJ6 — covers §2.3–2.7: contraposition, contradiction, cases and existence proofs in short segments. (Verified 2026-10-11.)

Notes

The technique-per-chapter architecture (direct, contrapositive, contradiction, then the non-conditional shapes) follows Hammack's proof chapters, whose "Mathematical Writing" section inspired Section 2.9 and whose disproof chapter put counterexamples on equal footing with proofs [3]. Velleman's third chapter organises the same material by the logical form of the goal — our Figure 2.6 is a compressed version of his summary tables [2]. MIT 6.042J's opening chapters supplied the "axioms vs. proven tools" framing and the squares-mod-4 lens [4]. The writing rules of Section 2.9 and the strategy/solution split of Section 2.10 follow Chen's *Remarks on English*, whose side-by-side marked-up solutions modelled Figure 2.5 [8], and the mistakes list in OTIS Excerpts ch. 1 [9]; Cut-the-Knot's fallacy pages supply the gallery this section samples [6]. Euler's polynomial example is standard lore (Euler, 1772); problem citations (AMC 12 2024 B P14, AMC 12 2023 A P3, AIME 2010 I P5, AMC 12 2024 A P6) point to the AoPS archive [7].

CHAPTER 3

Induction

A claim about every $n \geq 1$ cannot be proved one n at a time — there are infinitely many. Induction is the machine that handles all of them in one argument: prove the first case by hand, then prove that each case carries the next. After this chapter you will be able to run that machine in its two standard gears (ordinary and strong induction), recognise its disguise (the minimal-counterexample argument), audit other people's uses of it — and construct a geometric induction like the tromino tiling, not just an algebraic one. Chapters 5, 7 and 9 run on it constantly.

BEFORE YOU START

Chapters 1–2. *Recap:* a proof is a chain of steps, each justified by a definition or an earlier result. Induction is a proof template for statements about $n = 1, 2, 3, \ldots$ that you will use constantly from here on.

AFTER THIS CHAPTER YOU CAN

- prove "for all $n \geq n_0$" statements by induction, writing base case, hypothesis and step explicitly;
- prove closed formulas for sums and inequalities by induction;
- choose between ordinary and strong induction;
- spot why a plausible-looking induction proof is wrong.

HOW TO THINK ABOUT THIS

Induction works like a line of dominoes: if the first falls, and each one knocks over the next, they all fall. The skill is finding what "the next one" means. Ask: *if I already knew the claim for n, what extra step would give me $n+1$?* If you can answer that, the proof nearly writes itself.

3.1 The domino idea and the template

Suppose $P(n)$ is a statement about an integer n. To establish "$P(n)$ for all $n \geq b$" by induction, you prove exactly two things:

Definition 3.1 (Principle of mathematical induction)**.** If (i) *base case*: $P(b)$ is true, and (ii) *inductive step*: for every $n \geq b$, $P(n) \Rightarrow P(n+1)$, then $P(n)$ holds for all $n \geq b$. Inside the step, the assumed $P(n)$ is called the *inductive hypothesis* (I.H.).

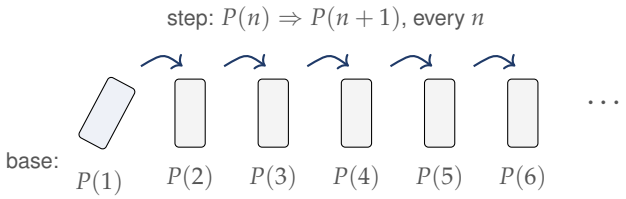


Figure 3.1: The machine's two parts. Notice that neither part alone does anything: the fallen first domino without working links knocks over nothing, and working links without a first domino have nothing to transmit.

The template below is compulsory in this book, on exams, and in the portfolio: graders grade the parts, and the parts are where induction proofs break.

THE INDUCTION TEMPLATE

1. **State** $P(n)$ — write the claim as a named predicate on n.
2. **Base case** — prove $P(b)$ directly.
3. **Inductive step** — "Let $n \geq b$. Assume $P(n)$." Derive $P(n+1)$, marking the exact line where the I.H. was used.
4. **Conclusion** — "By induction, $P(n)$ holds for all $n \geq b$."

Why does the template work? Informally: $P(b)$ starts the chain and the step transmits it; formally it is equivalent to the well-ordering principle of Section 3.5. Until then, treat the machinery as a licence granted by Chapter 2's rules — the step is an ordinary conditional proof ($P(n) \Rightarrow P(n+1)$) nested inside a bigger claim.

3.2 Sums and formulas by induction

Sum formulas are induction's home turf, and they teach the single most-used move: *peel the last term* so the I.H. has something to bite.

Example 3.2. Prove: for every $n \geq 1$, $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Strategy. Template first: name $P(n)$, prove $P(1)$, then in the step split the sum at $k = n$ so $\sum_{k=1}^{n} k$ appears — that subexpression *is* the I.H., and the rest is algebra.

Solution. Let $P(n)$ be the claim $\sum_{k=1}^{n} k = n(n+1)/2$. *Base.* $P(1)$: $\sum_{k=1}^{1} k = 1 = 1 \cdot 2/2$. Check. *Step.* Let $n \geq 1$ and assume $P(n)$: $\sum_{k=1}^{n} k = n(n+1)/2$. Then

$$\sum_{k=1}^{n+1} k = \left(\sum_{k=1}^{n} k\right) + (n+1) \overset{\text{I.H.}}{=} \frac{n(n+1)}{2} + (n+1) = \frac{(n+1)(n+2)}{2},$$

which is exactly $P(n+1)$. *Conclusion.* By induction, $P(n)$ holds for all $n \geq 1$.

Two things to copy: the I.H. is marked where it enters (never let a grader hunt for it), and the goal $P(n+1)$ was kept in sight — the whole middle of the step is traffic between those two endpoints.

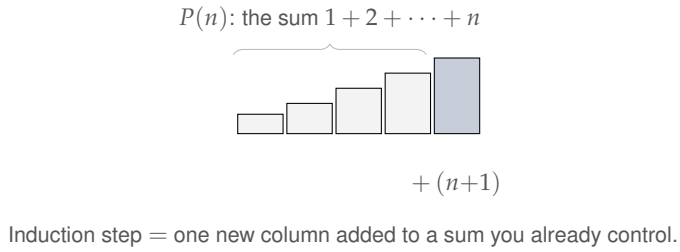


Figure 3.2: The induction step on a sum is one new column. Notice: the induction hypothesis owns the whole shaded block — the step only has to absorb the last column.

Exercise 3.1 (easy)**.** Prove by induction: $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$. (Same engine: peel the last term.)

3.3 Induction for inequalities

Induction proves inequalities too, but the step is an algebraic squeeze, not a substitution: you chain $\leq$ and $\geq$ signs from the I.H. to the target, and each link needs its own reason.

Example 3.3. Prove: $2^n > n^2$ for every integer $n \geq 5$.

Strategy. The claim is false below 5 (check $n = 2, 3, 4$ — the base window matters), so induct from $n = 5$. In the step, the I.H. gives $2 \cdot 2^n > 2n^2$; the remaining job is the auxiliary inequality $2n^2 \geq (n+1)^2$, which holds once $n \geq 3$. Every $\geq$ gets a name.

Solution. Let $P(n)$ be "$2^n > n^2$". *Base.* $P(5)$: $32 > 25$. *Step.* Let $n \geq 5$ and assume $2^n > n^2$. Then $2^{n+1} = 2 \cdot 2^n \overset{\text{I.H.}}{>} 2n^2$. And for $n \geq 3$, $2n^2 - (n+1)^2 = n^2 - 2n - 1 = (n-1)^2 - 2 \geq 2 > 0$, so $2n^2 > (n+1)^2$ and $2^{n+1} > (n+1)^2$. *Conclusion.* By induction, $P(n)$ holds for all $n \geq 5$.

TRIPWIRE

An inequality that is true "eventually" is only proved from where it starts. $2^n > n^2$ fails at $n = 2, 3, 4$; proving $P(5)$ and the step for $n \geq 5$ is not optional bookkeeping — it is the claim.

Exercise 3.2 (medium)**.** Prove by induction: $n! > 2^n$ for all $n \geq 4$. (Find the smallest working base first.)

3.4 Strong induction

Ordinary induction lets the step lean on $P(n)$ only. *Strong induction* lets it lean on *every* earlier case: to prove $\forall n \geq b\ P(n)$, prove base cases $P(b), P(b{+}1), \ldots, P(b{+}s{-}1)$ and the step $\big(P(b) \wedge P(b{+}1) \wedge \cdots \wedge P(n)\big) \Rightarrow P(n+1)$ for all $n \geq b{+}s{-}1$. The number of base cases s equals how far back the step reaches — Figure 3.3 draws the difference.

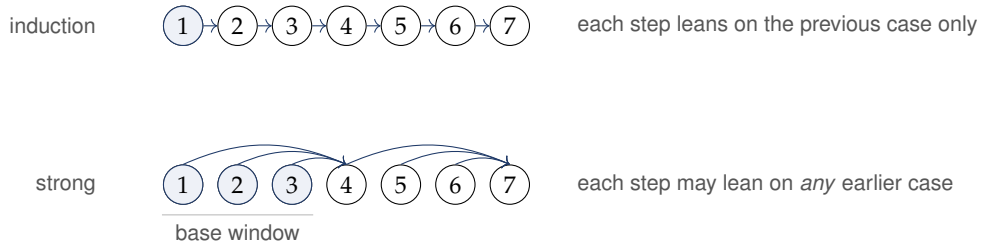


Figure 3.3: What each gear is allowed to lean on. Notice the strong row needs a *window* of base cases: if the step reaches back s positions, the dominoes inside reach of no step must be toppled by hand.

Example 3.4 (Postage)**.** Prove: every integer $n \geq 12$ can be written as $n = 3a + 7b$ for some nonnegative integers a, b.

Strategy. Adding 3 moves forward in steps of 3, so the step will reach back three positions: $P(n-2) \Rightarrow P(n+1)$. A reach of 3 needs a base window of 3 consecutive cases — check $12, 13, 14$ by hand.

Solution. Let $P(n)$ be "$n = 3a + 7b$ for some $a, b \in \mathbb{Z}_{\geq 0}$". *Base.* $12 = 3 \cdot 4 + 7 \cdot 0$, $13 = 3 \cdot 2 + 7 \cdot 1$, $14 = 3 \cdot 0 + 7 \cdot 2$. *Step.* Let $n \geq 14$ and assume $P(m)$ for all $12 \leq m \leq n$. Then $n + 1 \geq 15$, so $n - 2 \geq 12$ and $n - 2 \leq n$; by the I.H., $n - 2 = 3a + 7b$ for some $a, b \geq 0$. Then $n + 1 = 3(a + 1) + 7b$. *Conclusion.* By strong induction, $P(n)$ holds for all $n \geq 12$.

Note $n = 11$ has no such representation — 12 is the honest base, not a convenience. This is a baby Frobenius problem; the general theory waits for Chapter 5's gcd machinery.

Exercise 3.3 (medium)**.** Prove every $n \geq 4$ is $2a + 5b$ for nonnegative a, b. How wide is your base window?

3.5 Well-ordering: induction in disguise

Definition 3.5 (Well-ordering principle)**.** Every nonempty subset of $\mathbb{N}$ has a least element.

WOP and induction are the same engine in different clothes. The translation is the *minimal counterexample* template: suppose some n fails $P(n)$; take the *smallest* failure n_0 (WOP supplies it); then every smaller case holds, and the inductive step applied at $n_0 - 1$ produces $P(n_0)$ — contradiction. If you can prove the step but induction bookkeeping annoys you, WOP is the same argument with the accounting hidden.

Theorem 3.6. *Every integer $n \geq 2$ has a prime factor.*

Proof. Suppose not, and let $n_0 \geq 2$ be the smallest counterexample (well-ordering). Then n_0 is not prime — otherwise it is its own prime factor — so $n_0 = ab$ with $2 \leq a, b < n_0$. By minimality of n_0, the integer a has a prime factor p, and $p \mid a \mid n_0$ makes p a prime factor of n_0: contradiction. ■

Compare with strong induction on the same claim: the "smallest failure" plays the role of $n+1$, and "every smaller case" is exactly the strong hypothesis. Same proof, two costumes — keep both, because some arguments (descent, Section 3.6's fake proof) read more naturally as minimal counterexamples.

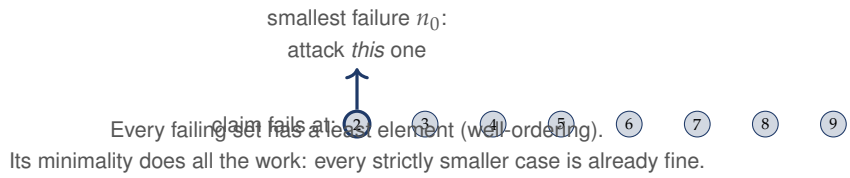


Figure 3.4: Well-ordering finds the smallest failure. Notice: minimality is a *strong* induction hypothesis in disguise — every smaller case is already fine, which is exactly the ammunition the contradiction needs.

3.6 Structural examples: a tiling, a recurrence, a fake proof

Induction is not only algebra. The object being inducted on can be a board, a tree, a formula — anything with a *size* that decomposes into smaller instances of itself.

Example 3.7 (The tromino tiling)**.** An L-tromino covers three cells in an L shape. Prove: for every $n \geq 1$, a $2^n \times 2^n$ board with any one cell removed can be tiled by L-trominos.

Strategy. Induct on n: the base 2×2 minus a cell *is* one tromino. For the step, the board must be made to look like four smaller instances — cut it into four $2^{n-1} \times 2^{n-1}$ quadrants. The hole lives in one quadrant; place a single L-tromino on the three centre cells of the other quadrants. Now *every* quadrant is a 2^{n-1} board with one cell spoken for — the I.H. finishes each.

Solution. Let $P(n)$ be the claim for 2^n boards. *Base.* $P(1)$: a 2×2 board missing one cell is exactly one L-tromino. *Step.* Let $n \geq 2$ and assume $P(n-1)$: every $2^{n-1} \times 2^{n-1}$ board with one cell removed is tileable. Given a $2^n \times 2^n$ board with a hole, quarter it. The quadrant containing the hole is a 2^{n-1} board missing a cell; the other three quadrants each lose one cell to the single centre tromino we place (Figure 3.5, second panel). Apply the I.H. to all four. *Conclusion.* By induction, $P(n)$ holds for all $n \geq 1$.

The proof is not just existence — it is a construction recipe, and it runs:

Checkpoint 3.1 *Predict before you run.* Listing 3.1 tiles an 8×8 board with hole $(5, 3)$. Before running: predict the number of trominos ($(4^3 - 1)/3$ — why is it an integer?), and predict whether any cell can be covered twice.

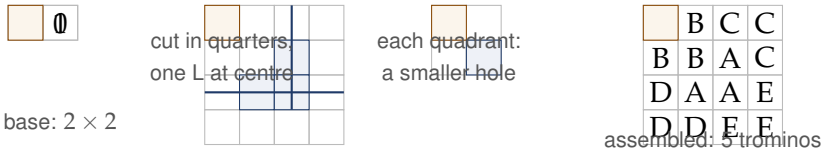


Figure 3.5: The induction at work. Notice what the step actually needs: one L placed at the centre converts "one quadrant has a hole" into "every quadrant has a hole" — after which four copies of $P(n-1)$ finish the job. The last panel is a real tiling produced by Listing 3.1's algorithm.

```python
def tile(board, top, left, size, hr, hc, tid):
    if size == 2: # base: one L fills the non-hole cells
        ...
    h = size // 2
    q = (hr >= top+h)*2 + (hc >= left+h) # quadrant holding the hole
    ... # centre tromino covers one cell of each other quadrant
    for i, (qt, ql) in enumerate(origins): # recurse into all four
        nhr, nhc = (hr, hc) if i == q else centers[i]
        tid = tile(board, qt, ql, h, nhr, nhc, tid)
```

Listing 3.1: `verify_tiling.py`: the proof, compiled. Recursion *is* the induction.

The full script (in `code/verify_tiling.py`) runs the construction on 8×8 and prints each tromino as a letter. Its real output:

```
board 8x8, hole at (5,3)
C C D D H H I I
C B B D H G G I
E B F F J J G K
E E F A A J K K
M M N N A R S S
M L N # R R Q S
O L L P T Q Q U
O O P P T T U U
trominos used: 21, cells covered: 63, expected: 21
```

Each letter is one tromino — read the grid and you can *see* the quartering: the centre L is `A`, and each 4×4 quadrant repeats the construction inside itself.

Induction is also how closed forms for recurrences get proved — guess a formula, then verify the step preserves it (Chapter 10 owns this; contest sequences like Fibonacci identities are already fair game, e.g. AMC 12 2024 B P18 [7]).

Finally, the autopsy every student owes themselves once — the most famous broken induction ever written:

Example 3.8 (Fake-proof autopsy: all horses one colour)**.** "Theorem": all horses are the same colour. "Proof": induct on the number of horses. *Base*: one horse has one colour. *Step*: given $n+1$ horses, remove one — the remaining n share a colour by the I.H.; swap it back and remove a different one — again one colour; since both groups share horses, all $n+1$ agree. Where does it break?

Strategy. Do not hunt for a subtle logical flaw — hunt for the smallest n where the step's *ingredients* stop existing. The argument needs the two (n)-horse groups to *share* a horse to transfer colour through. When do they not?

Solution. At $n+1=2$: the groups $\{h_1\}$ and $\{h_2\}$ share nothing — Figure 3.6 marks the dead link. The step itself is genuinely valid for every $n \geq 3$ (the shared middle horse transfers colour), but validity alone never fires it: with no proof of $P(2)$, the link from $P(1)$ to $P(2)$ is missing, so $P(2), P(3), P(4), \ldots$ are all left unestablished — only $P(1)$ was ever proved. A single missing link kills the entire chain from the break onward.

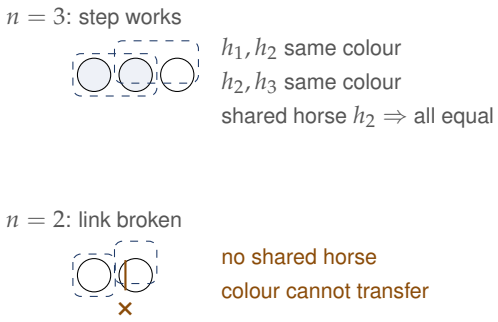


Figure 3.6: Why the horses proof dies at $n = 2$. Notice the colour-transfer mechanism is real for $n \geq 3$ — the fake proof fails on the *reach* of the step, not on its logic. This is the commonest way inductions break in the wild.

Exercise 3.4 (medium)**.** A "proof" claims every $n \geq 0$ satisfies $n = 0$, because $P(0)$ holds and $P(n) \Rightarrow P(n+1)$ is proved by "adding nothing to both sides of $n = 0$". Identify the actual failure.

3.7 Choosing what to induct on

The template asks one question most students skip: *what is n?* The statement's variable is the default, but any quantity that measures the problem's size works — board dimension, set cardinality, string length, number of operators. Rule of thumb: induct on the quantity the problem *decomposes along*. A 2^n board splits along n; a graph along $|V|$; a sum along its term count.

A subtler choice: the *claim itself*. Sometimes $P(n)$ is too weak to carry the step, and the fix is *inductive loading* — prove a *stronger* statement so the I.H. gives you more. Classic instance: bound $\sum_{k=1}^{n} 1/k^2 < 2$ directly and the step stalls (the surplus shrinks); prove the stronger $\sum_{k=1}^{n} 1/k^2 \leq 2 - 1/n$ and the step goes through cleanly. Chapter 13 uses this move for inequalities.

Exercise 3.5 (easy)**.** For each claim, name what you would induct on: (a) every full binary tree with ℓ leaves has $2\ell - 1$ vertices; (b) every $n \geq 60$ is a sum of 6's, 9's and 20's; (c) a formula with m operators has at most $m + 1$ variables.

3.8 Chapter summary

Situation	Gear	Base obligation	Step obligation
standard $\forall n \geq b$	induction	$P(b)$	$P(n) \Rightarrow P(n+1)$
step reaches back s	strong induction	s consecutive bases	all of $P(b..n) \Rightarrow P(n+1)$
"no smallest bad case" style	well-ordering	—	smallest failure contradicts itself
algebraic sum/formula	peel last term	check by hand	split so the I.H. appears
inequality (eventual)	squeeze	the *first* true n	name every $\geq$ link
structure that decomposes	structural	smallest instance	reassemble smaller copies
claim too weak for the step	strengthen $P(n)$	stronger base	stronger I.H. carries it

Table 3.1: The induction menu on one page. The right column is where each variant actually breaks in student work — check it before submitting.

3.9 Common mistakes

- **No base case — or the wrong window.** The horses proof dies exactly here: a step valid only from $n \geq 3$ needs $P(1)$ *and* $P(2)$ checked by hand. Always ask: how far back does my step reach? That distance is your base window.
- **Using the conclusion inside the step.** "Assume $P(n+1)\ldots$" or quietly writing $\sum_{k=1}^{n+1} = \frac{(n+1)(n+2)}{2}$ mid-derivation is assuming what you must prove. The I.H. covers $P(n)$ (or earlier), never $P(n+1)$.
- **Inducting on the wrong variable.** On a two-parameter claim ($2^n \times 2^n$ boards, $n \times m$ grids), inducting on the wrong coordinate gives a step that cannot decompose the object. Induct on the quantity the structure splits along (Section 3.7).

3.10 Additional exercises

Exercise 3.6 (easy)**.** Prove by induction: $\sum_{k=1}^{n}(2k-1) = n^2$. (What does the picture of n^2 as nested Ls say?)

Exercise 3.7 (easy)**.** Prove by induction: $\sum_{k=1}^{n} k \cdot k! = (n+1)! - 1$.

Exercise 3.8 (easy)**.** Prove: $3 \mid (4^n - 1)$ for all $n \geq 0$. (This is why $21 = (4^3-1)/3$ trominos was guaranteed to be an integer.)

Exercise 3.9 (easy)**.** Each of these inductions is broken — name the failure type from Common mistakes: (a) a proof that all integers are equal ("true for n implies true for $n+1$ by

symmetry"); (b) a proof of $\sum k^3 = n^4$ whose step "assumes the formula for $n+1$ and verifies it".

Exercise 3.10 (medium)**.** Prove: $n^3 - n$ is divisible by 6 for all $n \geq 0$, once by induction and once by the factorisation $n^3 - n = (n-1)n(n+1)$.

Exercise 3.11 (medium)**.** Prove by induction: $F_1 + F_2 + \cdots + F_n = F_{n+2} - 1$, where $F_1 = F_2 = 1$. (Fibonacci sums are AMC staples — see AMC 12 2024 B P18 [7].)

Exercise 3.12 (medium)**.** Prove: every $n \geq 1$ has a binary representation $n = \sum 2^{a_i}$ with distinct exponents, by strong induction. (Halve n and read the parity; the $n = 1$ base is 2^0.)

Exercise 3.13 (medium)**.** Prove by minimal counterexample: every rational number has a lowest-terms representative p/q with $\gcd(p, q) = 1$. (Take the smallest denominator.)

Exercise 3.14 (hard)**.** Prove that for every $n \geq 1$, a $2^n \times 2^n$ board missing *two* cells can never be tiled by L-trominos. (Area: $(4^n - 2)/3$ must be an integer — prove $4^n \equiv 1 \pmod 3$ by induction, so the count is never one.)

Exercise 3.15 (hard)**.** Extend `code/verify_tiling.py` so the hole can be *any* cell and the board any 2^k (try 16×16). Then prove the count invariant: the printed number of trominos is always $(4^n - 1)/3$, and prove by induction that $3 \mid 4^n - 1$ so the count is always an integer.

Exercise 3.16 (hard)**.** Strengthened induction. Prove directly that $\sum_{k=1}^{n} 1/k^2 < 2$ stalls, then prove the stronger $\sum_{k=1}^{n} 1/k^2 \leq 2 - 1/n$ by induction. Explain in one sentence why a stronger claim can be easier.

Exercise 3.17 (easy)**.** Prove by induction: $1 + 3 + 5 + \cdots + (2n-1) = n^2$. (Draw the staircase of Figure 3.2 first.)

Exercise 3.18 (medium)**.** Prove by induction that $6 \mid n^3 - n$ for every $n \geq 0$. Which property of $n^3 - n$ does the step use at the end?

Exercise 3.19 (medium)**.** Prove by *strong* induction: every integer $n \geq 12$ is a sum of 4's and 5's. State exactly which smaller cases the step reaches back to (Figure 3.4 is the right picture).

Exercise 3.20 (hard)**.** Find the flaw: "all horses are the same colour" is proved by induction — $n = 1$ trivial; given n same-coloured horses, in any set of $n+1$ the first n share a colour and the last n share a colour, so all $n+1$ do. Name the exact value of n where the step fails and why the overlap argument collapses there. (Compare Figure 3.6.)

Further reading

- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 5 — `https://courses.csail.mit.edu/6.042/spring18/mcs.pdf` — free PDF; ordinary, strong and structural induction plus the well-ordering equivalence this chapter quotes.
- R. Hammack, *Book of Proof*, 3rd ed., Ch. 10 — `https://richardhammack.github.io/BookOfProof/` — free PDF; the clearest gallery of induction proofs, including the inequality examples this chapter borrows.

- MIT OpenCourseWare, *6.042J Mathematics for Computer Science* (Fall 2010), Lectures 2–3 — https://ocw.mit.edu/6-042JF10 — the induction lectures; the "bogus proofs" tradition our autopsy section continues.
- Stanford CS103, *Mathematical Foundations of Computing* — https://web.stanford.edu/class/cs103/ — free reader; its induction guide is the best drill on base-window discipline.
- The Open Logic Project — https://openlogicproject.org/ — free texts; for readers who want induction as a formal deduction rule rather than a template.
- Cut-the-Knot, *Induction* pages — https://www.cut-the-knot.org/ — the classic fallacies collection; the fake-proof genre of §3.6 lives here.

WATCH INSTEAD

- **MIT OpenCourseWare — "MIT 6.042J Mathematics for Computer Science"** (playlist; induction at Lectures 2–3), https://www.youtube.com/playlist?list=PLB7540DEDD482705B — covers §3.1–3.4: the induction axiom, worked sums, and strong induction. (Verified 2026-10-11.)
- **SawFin Mathematics — "Discrete Math II – 5.2.1 Proof by Strong Induction"**, 16:21, https://www.youtube.com/watch?v=-vJhahLdmLI — covers §3.4: when the single-predecessor step is not enough. (Verified 2026-10-11.)

Notes

The domino template, the base-window discipline, and the strong-vs-ordinary reformulation follow MIT 6.042J's induction and well-ordering chapters, which also supplied the WOP $\Leftrightarrow$ induction equivalence and the "bogus proofs" tradition our autopsy section continues [4]. Velleman's induction chapter organises proofs by the same strategy-first pattern this book uses [2], and Hammack's chapter 10 supplied the inequality-by-induction examples [3]. The tromino theorem is the classic first structural induction (a Cut-the-Knot staple; the constructive proof here is the standard one [6]); Engel's induction chapter catalogues the choosing-the-variable and strengthen-the-claim moves of Section 3.7 [10]. Contest sequence problems named in this chapter (AMC 12 2024 A P21, AMC 12 2024 B P18, AIME 2021 I P10, AIME 2025 II P13, AMC 12 2023 A P20, AMC 12 2023 A P25) live on the AoPS archive [7] — they are inductions wearing arithmetic clothes.

CHAPTER 4

Sets, relations and functions

Every object in this book is built from three primitives: the *set* (a bag of things), the *relation* (a pairing rule), and the *function* (a relation that assigns each input exactly one output). After this chapter you will be able to prove statements about all three — in particular the two-direction discipline behind injective/surjective — and you will have seen the two proof machines that power all of cardinality: the *bijection argument* (count by matching) and *diagonalisation* (show a list can never be complete). These run constantly in the counting, probability and graph chapters.

BEFORE YOU START

Chapters 1–3. *Recap:* a *set* is a collection of distinct objects, written $\{1, 2, 3\}$; $x \in S$ means "x is in S." We build everything else on this one idea.

AFTER THIS CHAPTER YOU CAN

- compute with unions, intersections, complements, power sets and Cartesian products;
- decide whether a rule is a function, and whether it is injective, surjective or bijective, with a proof;
- check whether a relation is an equivalence relation and describe the partition it produces;
- explain why some infinite sets are bigger than others.

HOW TO THINK ABOUT THIS

Sets and functions are the nouns and verbs of the whole subject. Be patient with the definitions: they feel fussy, but each one is a precise answer to a natural question ("does every output get hit?" is surjective; "are any two inputs sent to the same output?" is the opposite of injective). When a definition confuses you, test it on a tiny set you can write out in full.

4.1 Sets, subsets and operations

Definition 4.1. A *set* is an unordered collection of distinct objects; $x \in A$ means "x is an element of A". $A \subseteq B$ means every element of A is an element of B; $A = B$ means $A \subseteq B$ and $B \subseteq A$ — that two-way containment is the standard proof template for set equalities.

The operations: union $A \cup B$, intersection $A \cap B$, difference $A \setminus B$, complement $\overline{A}$ (inside a fixed universe U). Figure 4.1 shows the geometry you should always draw first — with the warning that a Venn diagram is a *guide*, not a proof.

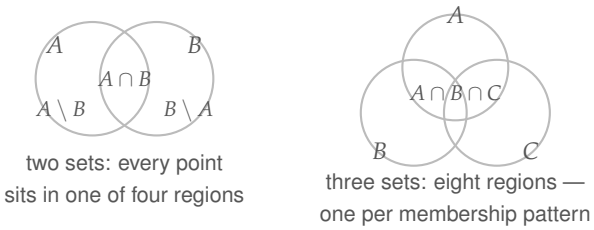


Figure 4.1: Draw before you prove. Notice three sets already give eight regions — which is why "the picture covers all cases" claims die at $n = 3$ sets. Use the diagram to locate the counterexample candidate, then write the element-chase proof.

Example 4.2. Prove the distributive law $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

Strategy. Equality of sets = two containments. Element-chase each direction: take an arbitrary x and unfold the definitions mechanically.

Solution. ($\subseteq$) Let $x \in A \cap (B \cup C)$. Then $x \in A$ and $x \in B \cup C$. If $x \in B$, then $x \in A \cap B \subseteq$ right side; if $x \in C$, then $x \in A \cap C \subseteq$ right side. ($\supseteq$) Let $x \in (A \cap B) \cup (A \cap C)$. Either way $x \in A$; and x lies in B or C, so $x \in B \cup C$. Hence $x \in A \cap (B \cup C)$. Both containments hold, so the sets are equal.

Exercise 4.1 (easy)**.** True or false, and fix the false ones: (a) $\{1\} \in \{1,2,3\}$; (b) $\{1\} \subseteq \{1,2,3\}$; (c) $1 \subseteq \{1,2,3\}$; (d) $\varnothing \in \{\varnothing\}$; (e) $\varnothing \subseteq A$ for every set A.

4.2 Power set and Cartesian product

Definition 4.3. The *power set* $\mathcal{P}(A)$ is the set of all subsets of A. If $|A| = n$ then $|\mathcal{P}(A)| = 2^n$ — each element independently votes in or out.

Definition 4.4. The *Cartesian product* $A \times B = \{(a,b) : a \in A, b \in B\}$ — ordered pairs. If $|A| = m$ and $|B| = n$ then $|A \times B| = mn$. Products iterate: $A^2 = A \times A$; $\mathbb{N} \times \mathbb{N}$ is the lattice of grid points of Figure 4.5.

Ordered means $(1,2) \neq (2,1)$; a product with itself carries a diagonal $\{(a,a)\}$ that shows up again in Section 4.6 for a very different reason.

Exercise 4.2 (easy)**.** List $\mathcal{P}(\{1,2,3\})$ and verify the count 2^3. How many subsets of $[n]$ have even size? (Prove your count by pairing each subset with its complement where possible, or hold it for Chapter 8.)

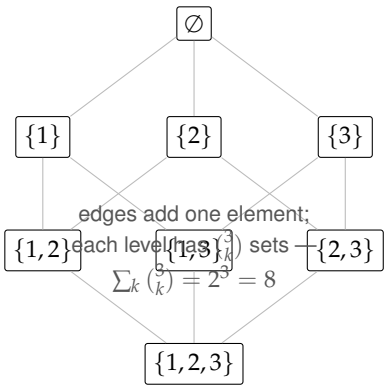


Figure 4.2: The power set of $\{1, 2, 3\}$ as a lattice. Notice: each upward edge adds exactly one element, and level k holds $\binom{3}{k}$ subsets — the 2^n count falls out of the structure, not just the formula.

4.3 Functions: definition, domain, image

Definition 4.5. A *function* $f : A \to B$ assigns to every $a \in A$ exactly one $b \in B$, written $f(a) = b$. A is the *domain*, B the *codomain*, and $f(A) = \{f(a) : a \in A\} \subseteq B$ the *image* (or *range*). Two functions are equal iff they have the same domain, same codomain, and agree on every input.

Discipline that prevents most beginner errors: codomain is a *claim*, image is a *fact*. "$f : \mathbb{Z} \to \mathbb{Z}$, $f(n) = n^2$" is a perfectly good function whose image $\{0, 1, 4, 9, \dots\}$ misses most of its codomain. When a problem asks you to prove something *onto*, it is the image that must be all of the codomain — Section 4.4 turns that into a template.

4.4 Injective, surjective, bijective — with templates

Definition 4.6. $f : A \to B$ is *injective* (one-to-one) if $f(a) = f(a') \Rightarrow a = a'$ — distinct inputs never collide. It is *surjective* (onto) if $f(A) = B$ — every $b \in B$ is hit. It is *bijective* if both hold.

Figure 4.3 shows what each does to the arrows. The proof templates are fixed — memorise the first line of each:

THE THREE TEMPLATES

- **Injective:** "Suppose $f(a) = f(a')$. Then … therefore $a = a'$." (Unfold the definition of f, cancel carefully.)
- **Surjective:** "Let $b \in B$. Take $a = \langle\text{candidate}\rangle$. Then $f(a) = \cdots = b$." (You must *produce* a preimage — guess it by solving $f(x) = b$.)
- **Bijective:** prove *both*, or exhibit an inverse g and verify $g \circ f = \mathrm{id}_A$ and $f \circ g = \mathrm{id}_B$.

Example 4.7. Prove: the composition of two injections is injective. That is, if $f : A \to B$ and $g : B \to C$ are injective, then $g \circ f$ is injective.

Strategy. Chase the definitions one layer at a time: write $(g \circ f)(a) = (g \circ f)(a')$, peel the outer layer with g's injectivity, then the inner with f's.

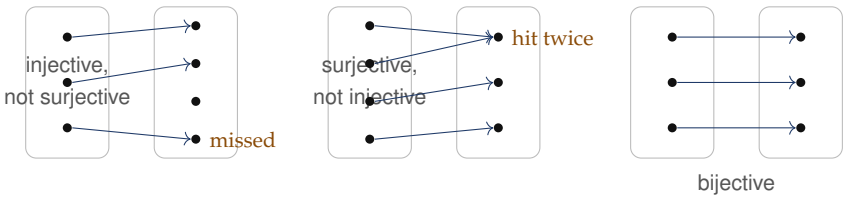


Figure 4.3: Read the arrows, not the words: injective forbids two arrows into one target, surjective forbids a missed target, bijective forbids both. The middle panel's "hit twice" mark and the left panel's "missed" mark are exactly the failures each template must rule out.

Solution. Suppose $(g \circ f)(a) = (g \circ f)(a')$, i.e. $g(f(a)) = g(f(a'))$. Since g is injective, $f(a) = f(a')$. Since f is injective, $a = a'$. Hence $g \circ f$ is injective.

The same one-layer proof gives: composition of surjections is surjective (exercise), and composition of bijections is bijective — which is why "built from bijections" is itself a counting technique (Section 4.6).

Exercise 4.3 (medium)**.** Prove injective or give a counterexample: (a) $f : \mathbb{Z} \to \mathbb{Z}$, $f(n) = 2n + 1$; (b) $f : \mathbb{Z} \to \mathbb{Z}$, $f(n) = n^2$; (c) $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^3 - x$.

4.5 Relations, equivalence relations, partitions

Definition 4.8. A *relation* on A is a subset $R \subseteq A \times A$; write $a \sim b$ (or aRb) for $(a, b) \in R$. R is *reflexive* if $a \sim a$ for all a; *symmetric* if $a \sim b \Rightarrow b \sim a$; *transitive* if $a \sim b \sim c \Rightarrow a \sim c$. All three together make an *equivalence relation*; its classes $[a] = \{b : b \sim a\}$ *partition* A — every element lands in exactly one class.

Example 4.9 (The parity classes)**.** On $\mathbb{Z}$, let $a \sim b$ iff $a - b$ is even. Show $\sim$ is an equivalence relation and describe its classes.

Strategy. Verify reflexive/symmetric/transitive straight on the definition — three tiny checks, no cleverness — then read the classes off the picture (Figure 6.1).

Solution. Reflexive: $a - a = 0$ is even. *Symmetric:* if $a - b$ is even then $b - a = -(a - b)$ is even. *Transitive:* if $a - b$ and $b - c$ are even then $a - c = (a - b) + (b - c)$ is a sum of evens, hence even. So $\sim$ is an equivalence relation. $a \sim 0$ iff a is even and $a \sim 1$ iff a is odd, so the classes are $[0] = \{\dots, -2, 0, 2, \dots\}$ and $[1] = \{\dots, -1, 1, 3, \dots\}$: the even and odd integers.

This is the trailer for Chapter 6: "same parity" is congruence mod 2, and mod m will give m classes with the same three-line verification.

Exercise 4.4 (medium)**.** Decide which of reflexive/symmetric/transitive each relation on $\mathbb{Z}$ has: (a) $a \sim b$ iff $a \leq b$; (b) $a \sim b$ iff $|a - b| \leq 1$; (c) $a \sim b$ iff $ab \geq 0$. Which are equivalence relations?

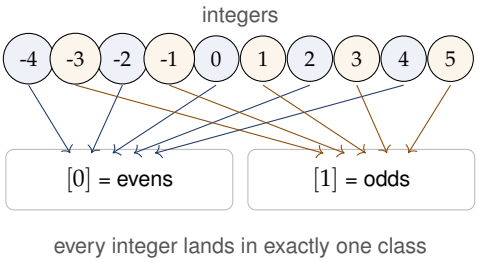


Figure 4.4: An equivalence relation sorts its set into bins. Notice nothing is left over and nothing is in two bins — "partition" is not a slogan, it is the theorem that classes never overlap and never miss anyone.

4.6 Cardinality: bijections count, diagonalisation escapes

Two finite sets have the same size iff a bijection exists between them — that is the *bijection principle*, and Chapter 8 spends a book on it. Applied to infinite sets it gives the first surprise:

Definition 4.10. A set A is *countably infinite* if a bijection $\mathbb{N} \to A$ exists; *countable* means finite or countably infinite; otherwise *uncountable*.

$\mathbb{Z}$ is countable (interleave $0, 1, -1, 2, -2, \ldots$). $\mathbb{Q}$ is countable — the snake of the next example is the standard proof sketch.

Example 4.11. Show $\mathbb{N} \times \mathbb{N}$ is countable.

Strategy. Draw a snake through the grid — a picture argument made precise. Visit anti-diagonals $x + y = k$ for $k = 2, 3, 4, \ldots$; each diagonal is finite, so every lattice point is reached at a finite step.

Solution. List pairs in order of increasing $x + y$, traversing each diagonal in the direction that continues the path — alternately decreasing-x and increasing-x, which is what keeps the trail continuous:

$$(1,1),\ (2,1),\ (1,2),\ (1,3),\ (2,2),\ (3,1),\ (4,1),\ \ldots$$

the path in Figure 4.5. Every (x, y) lies on the finite diagonal $x + y = x_0 + y_0$ and is enumerated at a finite position, so the list is a bijection $\mathbb{N} \to \mathbb{N} \times \mathbb{N}$. Hence $\mathbb{N} \times \mathbb{N}$ is countable.

And the second surprise — there are strictly more reals:

Example 4.12 (Cantor diagonalisation)**.** Prove: $[0, 1)$ is uncountable — no list of reals in $[0, 1)$ is complete.

Strategy. Given any candidate list, *manufacture the missing witness*: build a number whose n-th decimal digit differs from the n-th digit of the n-th row. It differs from every row, so it was never listed.

Solution. Suppose $r_1, r_2, r_3, \ldots$ is any list of numbers in $[0, 1)$, each written as decimals $0.d_{i1}d_{i2}d_{i3}\ldots$ (forbid the all-9s tail so every number has one expansion). Build $x = 0.e_1e_2e_3\ldots$

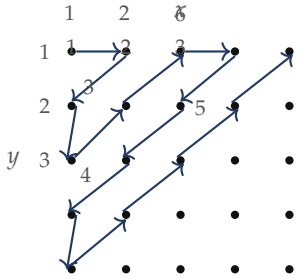


Figure 4.5: The snake enumeration of $\mathbb{N} \times \mathbb{N}$ (here $\mathbb{N} = \{1, 2, 3, \dots\}$). Notice why it works: every anti-diagonal $x + y = k$ is *finite*, so the path reaches any given point after finitely many steps — "eventually covers everything" is the whole content of countable.

by the rule $e_i = 4$ if $d_{ii} = 5$, else $e_i = 5$. For every i, x differs from r_i in position i, so $x \neq r_i$; and $x \in [0, 1)$ since its digits are 4s and 5s — never the ambiguous 9-tail. So x is a number in $[0, 1)$ absent from the list. No list is complete; $[0, 1)$ is uncountable.

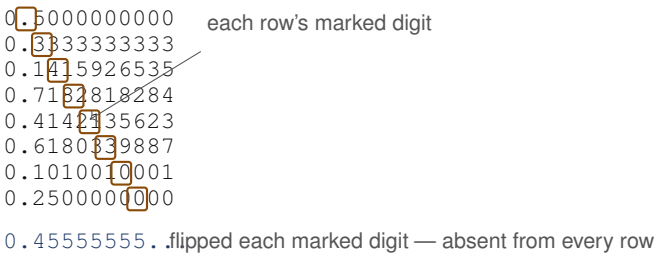


Figure 4.6: The flip rule at work. Notice two engineering details that make the argument watertight: the constructed digits are only 4s and 5s (no 9-tail, so x's expansion is unique), and each row is attacked at *its own* position — the new number escapes all rows simultaneously.

Checkpoint 4.1 *Predict before you run.* Listing 4.1 runs the construction on eight decimals. Before running: predict the constructed number's first two digits, and predict which row it could *still* equal even though it differs at the marked position. (Trick: none — differing in one decimal position is enough when expansions are unique.)

```python
def flip(digit): # never emits 9 -> unique expansion
    return "4" if digit == "5" else "5"

for i, row in enumerate(ROWS):
    ... # print row with position i+2 marked
    digits.append(flip(row[i+2]))
built = "0." + "".join(digits) + "..."
```

Listing 4.1: `diagonal_demo.py`: the manufacture-the-witness step, mechanised (full script in `code/diagonal_demo.py`).

Real output:

```
0.[5]000000000
0.3[3]33333333
0.14[1]5926535
```

```
0.718[2]818284
0.4142[1]35623
0.61803[3]9887
0.101001[0]001
0.2500000[0]00
constructed: 0.45555555...
differs from every row at its marked position
```

Eight rows, eight marked positions, one number that is provably on none of them. Infinite lists differ only in scale, not in kind.

Exercise 4.5 (easy)**.** Show each set is countable by describing the enumeration: (a) $\mathbb{Z}$; (b) the even integers; (c) $\mathbb{N} \times \{0, 1\}$.

4.7 The set $[n]$ and indexing conventions

Definition 4.13. $[n] = \{1, 2, \ldots, n\}$, the standard n-element set; $[0] = \varnothing$. Any set of size n is "a copy of $[n]$" via some bijection — that is the content of counting.

Conventions this book keeps: indices run $1..n$ unless declared otherwise; $a_1, \ldots, a_n$ is a *sequence* (ordered, may repeat) while $\{a_1, \ldots, a_n\}$ is a set (unordered, distinct); $x \in [n]$ means $1 \le x \le n$ integer. $\mathbb{N} = \{1, 2, 3, \ldots\}$ throughout — the olympiad convention of the notation guide — with $\mathbb{Z}_{\ge 0}$ written explicitly when 0 must be included.

4.8 Chapter summary

To prove…	Template	First line
$A = B$	two containments	"Let $x \in A$" / "Let $x \in B$"
f injective	unfold collision	"Suppose $f(a) = f(a')$"
f surjective	produce preimage	"Let $b \in B$; take $a = \ldots$"
f bijective	both, or inverse	verify $g \circ f$, $f \circ g$ identities
$\sim$ equivalence	R/S/T on definition	$a \sim a$; flip; add
A countable	enumerate	list that reaches every element
A uncountable	diagonalise	"Suppose $r_1, r_2, \ldots$ is a list"
count finite sets	bijection	match the objects, count $[n]$

Table 4.1: Every claim shape in this chapter has a named template and a required first line. Write the first line before you think — it is the part graders grade.

4.9 Common mistakes

- **Confusing $\in$ with $\subseteq$.** $1 \in \{1,2\}$ but $\{1\} \subseteq \{1,2\}$; the wrapped version is a different object. Check every statement's type before arguing about it.
- **Proving a bijection by one direction.** Injective-only and surjective-only examples are in Figure 4.3; one property says nothing about the other. Prove both, or exhibit the inverse.
- **"Uncountable" $\neq$ "infinite".** $\mathbb{Z}$ and $\mathbb{Q}$ are infinite *and* countable; the failure mode is assuming a list exists because the set is "just" infinite. Diagonalisation is precisely the argument that no list suffices for $[0,1)$.

4.10 Additional exercises

Exercise 4.6 (easy)**.** Prove by element-chase: $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$.

Exercise 4.7 (easy)**.** Prove De Morgan: $\overline{A \cup B} = \overline{A} \cap \overline{B}$. (Draw Figure 4.1's left panel first.)

Exercise 4.8 (easy)**.** Give an example of a function $f : \mathbb{Z} \to \mathbb{Z}$ that is injective but not surjective, and one that is surjective but not injective — and prove each property you claim.

Exercise 4.9 (easy)**.** On $\mathbb{Z}$, is $a \sim b$ iff $a \mid b$ an equivalence relation? Which of reflexive/symmetric/transitive hold?

Exercise 4.10 (medium)**.** Prove: the composition of two surjections is surjective. (Mirror Example 4.7; the harder half is producing the preimage.)

Exercise 4.11 (medium)**.** Show the positive rationals $\mathbb{Q}_{>0}$ are countable. (Snake the pairs (p,q), skip non-reduced fractions — explain why skipping is allowed.)

Exercise 4.12 (medium)**.** Let $a \sim b$ on $\mathbb{Z}$ iff $a - b$ is divisible by 3. Prove it is an equivalence relation and describe all its classes.

Exercise 4.13 (medium)**.** Prove: if $f : A \to B$ and $g : B \to C$ with $g \circ f$ injective, then f is injective. Does g have to be injective? Prove or counterexample.

Exercise 4.14 (hard)**.** Adapt diagonalisation to subsets: prove $\mathcal{P}(\mathbb{N})$ is uncountable by supposing a list $S_1, S_2, \ldots$ and building a subset that differs from every S_i on membership of i.

Exercise 4.15 (hard)**.** Prove Cantor's theorem: for *every* set A, no function $f : A \to \mathcal{P}(A)$ is surjective. (Consider the diagonal set $D = \{a \in A : a \notin f(a)\}$ — Russell's paradox in one line.)

Exercise 4.16 (hard)**.** Extend `code/diagonal_demo.py` so it works for base-3 expansions (digits 0–2), choosing a flip rule that still avoids ambiguity. Then prove from your construction that the numbers in $[0,1)$ with *only* digits $\{0,2\}$ in base 3 (the Cantor set) are uncountable.

Exercise 4.17 (easy)**.** Let $A = \{1,2,3,4\}$, $B = \{2,4,6\}$. Compute $A \cup B$, $A \cap B$, $A \setminus B$, $A \times B$'s cardinality, and $|\mathcal{P}(A \cap B)|$.

Exercise 4.18 (medium)**.** Prove or disprove: $f(n) = 2n + 1$ as a function $\mathbb{Z} \to \mathbb{Z}$ is (i) injective, (ii) surjective, (iii) bijective. If any fails, exhibit the counterexample. Then restrict the codomain to the odds and redo all three.

Exercise 4.19 (medium)**.** On $\mathbb{Z}$, define $a \sim b$ iff $a - b$ is divisible by 5. Prove this is an equivalence relation, list its classes, and say in one sentence why a partition and an equivalence relation are the same object.

Exercise 4.20 (hard)**.** Prove that the interval $(0, 1)$ and the open ray $(0, \infty)$ have the same cardinality by writing an explicit bijection (one of $x \mapsto x/(1-x)$ or its cousin) and checking it is injective and surjective — not by waving at "infinite means infinite".

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 1, 11–13 — https://richardhammack.github.io/BookOfProof/ — free PDF; sets, functions and cardinality with the two-containment and injective/surjective templates this chapter uses.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 4, 8 — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; the countability order and light Cantor treatment of §4.6.
- E. Chen, *Introduction to Functional Equations* — injective, surjective and bijective restated in solve-for-the-witness form; the bridge from this chapter to olympiad functional equations: https://web.evanchen.cc/handouts/FuncEq-Intro/FuncEq-Intro.pdf
- Stanford CS103, *Mathematical Foundations of Computing* — https://web.stanford.edu/class/cs103/ — free reader; its set-theory and functions units drill element chasing to fluency.
- The Open Logic Project — https://openlogicproject.org/ — free texts; for the axiomatic-set-theory view beneath the naive one taught here.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — https://ocw.mit.edu/6-042JF10 — the sets and cardinality lectures behind §4.6.

WATCH INSTEAD

- **TrevTutor — "INTRODUCTION to SET THEORY"**, 16:38, https://www.youtube.com/watch?v=tyDKR4FG3Yw — covers §4.1–4.2: sets, membership, subsets, power sets. (Verified 2026-10-11.)
- **TrevTutor — "INJECTIVE, SURJECTIVE, and BIJECTIVE FUNCTIONS"**, 17:14, https://www.youtube.com/watch?v=bZred_Ksz2k — covers §4.4: the three properties with arrow-diagram examples matching Figure 4.3. (Verified 2026-10-11.)

Notes

The element-chase template, the two-containment proof of set equality, and the treatment of relations/partitions follow Velleman's chapters 4–5 and 8 [2]; the injective/surjective templates and the composition exercises are Hammack's chapter 11–12 approach [3], with the definition conventions of Chen's *Introduction to Functional Equations* [11] (injections, surjections and bijections stated in solve-for-the-witness form). MIT 6.042J's chapters on sets and cardinality supplied the countability order of Section 4.6 and the light Cantor treatment [4]; diagonalisation as "manufacture the missing witness" is the standard presentation [3, 4]. The counting-flavoured contest problems this chapter feeds are AIME 2024 II P6, AIME 2023 II P11, AIME 2022 I P12, AIME 2010 I P7 and AMC 12 2023 A P24, all on the AoPS archive [7].

CHAPTER 5

Divisibility and primes

Number theory begins with one relation: $a \mid b$, "a divides b". This chapter builds the two machines it powers — the Euclidean algorithm (the fastest proof in mathematics: it *is* an algorithm and a proof of correctness at once) and unique factorisation — and ends with the first tool an olympiad problem reaches for: solving equations in integers. After this chapter you will be able to run Euclid by hand in seconds, extract Bézout coefficients from it, prove the digit criteria contestants quote without thinking, and solve any $ax + by = c$ completely.

BEFORE YOU START

Chapters 2–3 (proofs and induction). *Recap:* $a \mid b$ ("a divides b") means $b = ak$ for some integer k. A prime is an integer greater than 1 whose only positive divisors are 1 and itself. **Book 1, Chapter 14** turns this chapter into fast code.

AFTER THIS CHAPTER YOU CAN

- divide with remainder and use the Euclidean algorithm to compute a gcd by hand;
- find integers x, y with $ax + by = \gcd(a, b)$ (Bézout);
- factor into primes and use unique factorisation in a proof;
- solve a linear Diophantine equation or show it has no solution.

HOW TO THINK ABOUT THIS

Numbers hide structure: every integer is built from primes, and the gcd is the shared part of two numbers. The Euclidean algorithm works because the common divisors of a and b do not change when you replace a by its remainder; once you see *why* nothing is lost, the algorithm stops being a recipe and becomes obvious.

5.1 Divisibility and the division algorithm

Definition 5.1. For integers a, b with $a \neq 0$, *a divides b* (write $a \mid b$) means $b = ak$ for some integer k. Then a is a *divisor* (or factor) of b, and b is a *multiple* of a.

Note the type: $a \mid b$ is a *statement* (true or false), while a/b is a *number*. Writing "$3 \mid 6$" asserts; writing "$3/6$" computes. Mixing them is Common Mistake #1 and it misfires on every proof that unfolds the definition.

Basic rules, each a two-line element-chase: if $a \mid b$ and $a \mid c$ then $a \mid (b+c)$ and $a \mid (b-c)$; if $a \mid b$ then $a \mid bk$ for any integer k; if $a \mid b$ and $b \mid c$ then $a \mid c$. The engine behind all of them — and behind half of olympiad number theory — is the division algorithm.

Theorem 5.2 (Division algorithm)**.** *For integers a and b with $b > 0$, there exist unique integers q, r with $a = qb + r$ and $0 \le r < b$.*

Proof. Existence: let $q = \lfloor a/b \rfloor$ (the largest integer with $qb \le a$) and $r = a - qb$; then $0 \le r < b$ by the floor's definition. Uniqueness: if $a = q_1 b + r_1 = q_2 b + r_2$ with both remainders in $[0, b)$, then $(q_1 - q_2)b = r_2 - r_1$; the right side has absolute value $< b$ and is a multiple of b, so it is 0, giving $r_1 = r_2$ and $q_1 = q_2$. ∎

Every statement "$b \mid a$" is the special case $r = 0$. The remainder is the object; Chapter 6 builds arithmetic on it.

Exercise 5.1 (easy)**.** Prove from the definition: if $a \mid b$ and $a \mid c$, then $a \mid (3b - 5c)$.

5.2 Gcd and the Euclidean algorithm

Definition 5.3. The *greatest common divisor* $\gcd(a, b)$ of integers a, b (not both zero) is the largest integer dividing both. If $\gcd(a, b) = 1$, call a, b *coprime*.

Lemma 5.4. *For $a = qb + r$, $\gcd(a, b) = \gcd(b, r)$.*

Proof. Any d dividing a and b divides $r = a - qb$; any d dividing b and r divides $a = qb + r$. So the two pairs have *identical* divisor sets, hence identical greatest elements. ∎

Lemma 5.4 is the whole machine: replace (a, b) by $(b, a \bmod b)$ until the second entry is 0; the survivor is the gcd. Figure 5.1 shows why it is called an algorithm *and* a proof: tiling a 252×198 rectangle greedily with squares, the last square that fits is the gcd.

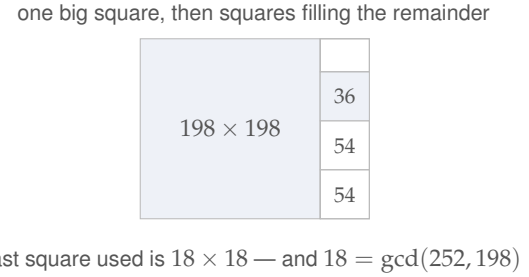


Figure 5.1: Euclid, geometrically. Notice what each square size is: the *side* of each fitted square is a quotient in the algorithm's trace — 198, 54, 36, 18 — and the smallest tile that tiles the whole rectangle is the answer.

Exercise 5.2 (easy)**.** Run the Euclidean algorithm on $\gcd(1071, 462)$ by hand, then check with `bezout.py` (change the two numbers).

5.3 Bézout's identity — and why the proof is an algorithm

Theorem 5.5 (Bézout). *For any integers a, b (not both zero), there exist integers x, y with $ax + by = \gcd(a, b)$.*

Two proofs, and the useful one is a procedure: run Euclid forward, then read the trace backwards — each remainder was itself a remainder, so the gcd unrolls into a combination. (The slick proof — smallest positive combination divides both — proves existence but hands you no x, y; contest work wants the coefficients.)

Example 5.6. Compute $\gcd(252, 198)$ and find integers x, y with $252x + 198y = \gcd(252, 198)$.

Strategy. Run the algorithm forward, then read it backwards: each line $a = qb + r$ rewrites as $r = a - qb$, and substituting upward keeps every term a combination of 252 and 198.

Solution. Forward:

$$252 = 1 \cdot 198 + 54, \quad 198 = 3 \cdot 54 + 36, \quad 54 = 1 \cdot 36 + 18, \quad 36 = 2 \cdot 18 + 0,$$

so gcd $= 18$. Backward: $18 = 54 - 1 \cdot 36 = 54 - (198 - 3 \cdot 54) = 4 \cdot 54 - 198 = 4 \cdot (252 - 198) - 198 = 4 \cdot 252 - 5 \cdot 198$. Check: $4 \cdot 252 - 5 \cdot 198 = 1008 - 990 = 18$. So $x = 4$, $y = -5$.

Checkpoint 5.1 *Predict before you run.* Listing 5.1 mechanises the back-substitution. Predict its Bézout line before scrolling to the output — and predict the sign pattern (why is at least one coefficient negative here? — a small exercise).

```
def euclid(a, b):
    steps = []
    while b:
        q, r = divmod(a, b); steps.append((a, q, b, r)); a, b = b, r
    return a, steps
...
A, B = 1, 0 # deepest: g = 1*a + 0*b
for a, q, b, r in reversed(steps):
    A, B = B, A - q*B # unwind r = a - q*b
```

Listing 5.1: `bezout.py`: Euclid forward, then read backwards (full script in `code/bezout.py`).

Real output:

```
Euclid(252,198):
  252 = 1*198 + 54
  198 = 3*54 + 36
  54 = 1*36 + 18
  36 = 2*18 + 0
gcd = 18
Bezout: 18 = 4*252 + -5*198 (check 18)

Solve 12x + 18y = 30:
  gcd(12,18) = 6, and 6 | 30 -> solutions exist
  particular: x0 = -5, y0 = 5
```

```
general: x = -5 + 3t, y = 5 - 2t
t=-1: x=-8, y=7, check 12x+18y = 30
t=0: x=-5, y=5, check 12x+18y = 30
t=2: x=1, y=1, check 12x+18y = 30
```

Bézout's corollary is the workhorse: if $a \mid bc$ and $\gcd(a,b) = 1$, then $a \mid c$. (Multiply $ax + by = 1$ by c.) One warning: the lemma *needs* coprimality — asserting $\gcd(a,b) = 1$ without checking it is Common Mistake #3.

Exercise 5.3 (medium)**.** Find integers x, y with $1071x + 462y = \gcd(1071, 462)$, by back-substitution (do Exercise 5.2 first).

5.4 Primes and the fundamental theorem

Definition 5.7. An integer $p \geq 2$ is *prime* if its only positive divisors are 1 and p; otherwise it is *composite*. (1 is neither — and the reason is the next theorem.)

Theorem 5.8 (Fundamental theorem of arithmetic)**.** *Every integer $n \geq 2$ has a unique factorisation $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ into primes (unique up to order).*

Existence is strong induction (Chapter 3): n is prime or splits $n = ab$ with $a, b < n$. Uniqueness uses Bézout's corollary: if a prime $p \mid ab$ then $p \mid a$ or $p \mid b$ (Euclid's lemma) — apply it to two factorisations and cancel one prime at a time. The poset of Figure 5.2 draws the divisor world of one number, $60 = 2^2 \cdot 3 \cdot 5$.

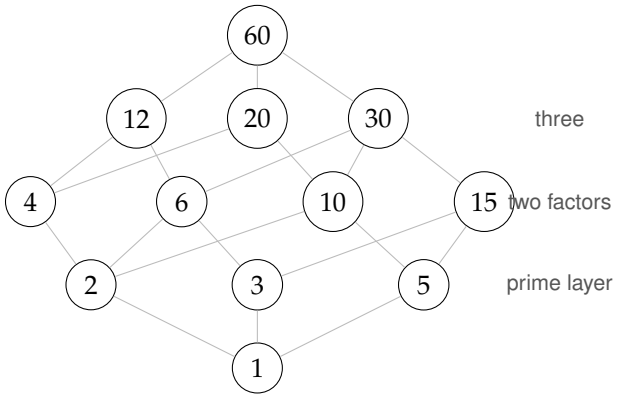


Figure 5.2: The divisors of 60, edges meaning "divides with nothing between". Notice the layering by factor count: every edge multiplies by a prime — the lattice *is* the factorisation $2^2 \cdot 3 \cdot 5$ drawn as a graph.

Exercise 5.4 (easy)**.** Draw the divisibility poset of 36 and of 30; why does 30's have the shape of a cube while 36's does not?

Example 5.9. Count the positive divisors of 360 without listing them, then count the positive divisors of any $n = p_1^{e_1} \cdots p_k^{e_k}$.

Strategy. A divisor of 360 is *constrained freedom*: it uses the same primes with smaller or equal exponents. Each exponent is an independent choice — so multiply the choices.

Solution. $360 = 2^3 \cdot 3^2 \cdot 5$. A divisor has the form $2^a 3^b 5^c$ with $0 \le a \le 3$, $0 \le b \le 2$, $0 \le c \le 1$: that is $4 \times 3 \times 2 = 24$ divisors (Figure 5.3). In general the number of positive divisors is $\tau(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$ — the "+1" per factor is the option *take none of that prime*.

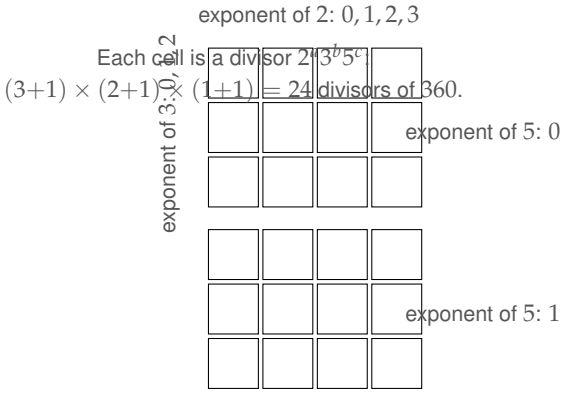


Figure 5.3: The 24 divisors of $360 = 2^3 \cdot 3^2 \cdot 5$ as a $4 \times 3 \times 2$ grid of exponent triples — the count $(e_1 + 1)(e_2 + 1)(e_3 + 1)$ is the grid's volume.

5.5 Euclid's infinitude of primes

Theorem 5.10. *There are infinitely many primes.*

Example 5.11. Prove it.

Strategy. Contradiction — from any finite list, build a number *no listed prime divides*: multiply them and add one.

Solution. Suppose $p_1, \ldots, p_k$ were all the primes. Consider $N = p_1 p_2 \cdots p_k + 1$. By Theorem 3.6 (Chapter 3) N has a prime factor p. But $N \bmod p_i = 1$ for every i, so no listed prime divides N: p is a new prime, contradiction.

Two warnings students always need: the proof does *not* claim $p_1 \cdots p_k + 1$ is prime ($2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30031 = 59 \cdot 509$ is composite) — only that *some* prime factor escapes the list. And this is a real proof by contradiction, not a construction: it gives no recipe for the next prime.

Exercise 5.5 (medium)**.** Prove there are infinitely many primes $p \equiv 3 \pmod 4$. (Copy Euclid: form $4p_1 \cdots p_k - 1$ and check its factors' residues.)

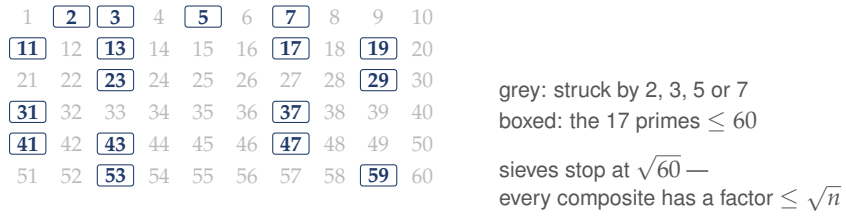


Figure 5.4: The sieve to 60: composites struck, 17 primes boxed. Notice the proof inside the picture — every composite ≤ 60 has a prime factor $\leq \sqrt{60}$, so crossing off multiples of $2, 3, 5, 7$ leaves only primes. That square-root bound is the same lemma that speeds up primality testing and powers the primality tests of the olympiad track [12].

5.6 Digit criteria and base representations

Theorem 5.12 (Base representation)**.** *For any base $b \geq 2$, every integer $n \geq 0$ has a unique expansion $n = d_k d_{k-1} \ldots d_1 d_0 = \sum d_i b^i$ with digits $0 \leq d_i < b$ and $d_k \neq 0$.*

Figure 5.5 shows the same number in two clothes. The digit criteria are polynomial evaluations: writing $n = \sum d_i 10^i = P(10)$, any criterion comes from what 10 does modulo the divisor.

Example 5.13. Prove the tests for 9 and 11: $9 \mid n$ iff $9 \mid$ digit-sum of n; $11 \mid n$ iff $11 \mid$ alternating digit-sum of n.

Strategy. Evaluate the polynomial at 10: since $10 \equiv 1 \pmod 9$, every power $10^i \equiv 1$, so $P(10) \equiv P(1) =$ the digit sum. Since $10 \equiv -1 \pmod{11}$, powers alternate $+1, -1, +1, \ldots$, giving the alternating sum. The criterion *is* the congruence.

Solution. Write $n = \sum_{i=0}^{k} d_i 10^i$. *Mod 9:* $10 \equiv 1 \pmod 9$, so $10^i \equiv 1$ and $n \equiv \sum d_i \pmod 9$; hence $9 \mid n$ iff $9 \mid \sum d_i$. *Mod 11:* $10 \equiv -1 \pmod{11}$, so $10^i \equiv (-1)^i$ and $n \equiv d_0 - d_1 + d_2 - \cdots \pmod{11}$; hence $11 \mid n$ iff 11 divides that alternating sum.

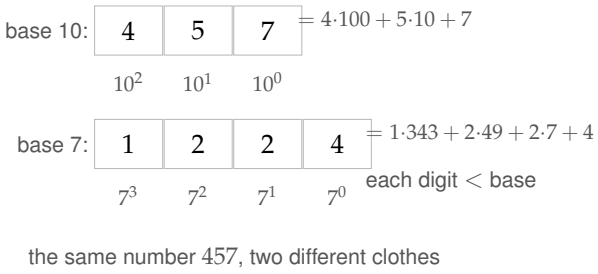


Figure 5.5: Place value in two bases. Notice what changes and what doesn't: the digits differ, the powers differ, the *number* is the same — base is a notation, not a property of n.

Same recipe, every divisor: 3 works like 9; $2, 5, 10$ read the last digit since $10 \equiv 0$; 4 and 25 read the last two since $100 \equiv 0$. Cut-the-Knot collects the whole family [6].

Exercise 5.6 (easy)**.** Derive the test for 7 in the style of Example 5.13: what is 10 mod 7, and why is the classical "double the last digit, subtract" rule equivalent?

5.7 Linear Diophantine equations

Theorem 5.14. *$ax + by = c$ has integer solutions iff $\gcd(a,b) \mid c$. If (x_0, y_0) is one solution, all solutions are $x = x_0 + \frac{b}{\gcd(a,b)}\,t,\ y = y_0 - \frac{a}{\gcd(a,b)}\,t$ for $t \in \mathbb{Z}$.*

Proof. Direction $\Rightarrow$: gcd divides $ax + by$. Direction $\Leftarrow$: Bézout gives $ax' + by' = \gcd$; multiply by $c/\gcd$. For the family: substituting shows every listed pair works; conversely, any two solutions differ by a solution of $ax + by = 0$, i.e. $\frac{a}{\gcd}x = -\frac{b}{\gcd}y$ — coprime factors force $x = \frac{b}{\gcd}t$. ∎

Example 5.15. Find all integer solutions of $12x + 18y = 30$.

Strategy. Two gates, in order: does $\gcd(12, 18) = 6$ divide 30? Yes — so solutions exist. Get one by Bézout (or by halving the equation to $2x + 3y = 5$ and spotting $x = 1, y = 1$); then the family is periodic with period $b/\gcd = 3$ in x.

Solution. Divide through by 6: $2x + 3y = 5$. Spot $x_0 = 1, y_0 = 1$ ($2 + 3 = 5$). By Theorem 5.14 all solutions are $x = 1 + 3t$, $y = 1 - 2t$ for $t \in \mathbb{Z}$. (The script's equivalent parametrisation $x = -5 + 3t$, $y = 5 - 2t$ is the same line re-indexed.) Check: $12(1 + 3t) + 18(1 - 2t) = 12 + 18 + 36t - 36t = 30$.

The structure is worth naming: solutions form a *particular solution plus a lattice* — a linear Diophantine equation is a line of integer points, and Bézout is how you land on it (Figure 5.6).

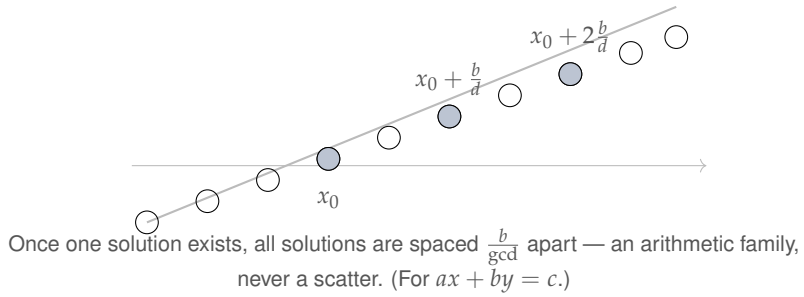


Figure 5.6: All solutions of $ax + by = c$ sit on one line, spaced $b/\gcd(a,b)$ apart in x — an arithmetic family from one seed x_0, never a scatter of independent hits.

Exercise 5.7 (medium)**.** Solve completely or prove impossible: (a) $15x + 35y = 100$; (b) $21x + 14y = 63$; (c) $34x + 55y = 1$ (consecutive Fibonacci numbers — predict gcd first).

5.8 Chapter summary

5.9 Common mistakes

- $a \mid b$ **read as a fraction.** $a \mid b$ is a proposition; a/b is a rational number. The error bites when unfolding: "$3 \mid 6 = 2$" is nonsense — the claim is $6 = 3 \cdot 2$.
- **"Prime" confused with "odd".** 9 and 15 are odd and composite; 2 is prime and even. The definition is about *divisors*, not parity — most students meet it as a slip in a proof that "assumes" a prime is odd.

Object	Machine	Watch
$a \mid b$	$b = ak$ unfolding	it is a statement, not a fraction
$\gcd(a, b)$	Euclid: $(a, b) \to (b, a \bmod b)$	log keeps every remainder
$ax + by = \gcd$	Bézout by back-substitution	one coefficient usually goes negative
prime factorisation	FTA: unique up to order	existence = strong induction
$ax + by = c$	gcd-gate then periodic family	check $\gcd \mid c$ FIRST
digit test for d	evaluate $P(10) \bmod d$	$10 \equiv 1, -1, 0$ cases
infinitude of primes	$p_1 \cdots p_k + 1$	shows a *new* prime, not *the* next

Table 5.1: The chapter in one page: every construction doubles as a proof. The right column is where hand-written solutions actually break.

- **Asserting $\gcd(a, b) = 1$ unproved.** Euclid's lemma and the family formula for Diophantine equations both *require* coprimality. "WLOG $\gcd(p, q) = 1$" is a licence only after you justify it (lowest terms) or divide it out explicitly.

5.10 Additional exercises

Exercise 5.8 (easy)**.** Prove: if $a \mid b$ and $b \mid a$ for positive integers, then $a = b$. (Where does "positive" enter?)

Exercise 5.9 (easy)**.** Compute $\gcd(2016, 784)$ and $\gcd(1001, 77)$ by Euclid; confirm with `code/bezout.py`.

Exercise 5.10 (easy)**.** Prove the divisibility tests for 4 and 25 from Example 5.13's method.

Exercise 5.11 (easy)**.** How many divisors does $360 = 2^3 \cdot 3^2 \cdot 5$ have? Prove the formula $d(n) = \prod(e_i + 1)$ from the FTA.

Exercise 5.12 (medium)**.** Prove $\gcd(a, b) \cdot \operatorname{lcm}(a, b) = ab$ for positive a, b. (Track one prime's exponent at a time.)

Exercise 5.13 (medium)**.** Prove: if $\gcd(a, b) = 1$ and $a \mid bc$, then $a \mid c$ — and find a counterexample when $\gcd(a, b) > 1$.

Exercise 5.14 (medium)**.** Show that among any three consecutive integers > 2, at least one is composite; and that $n! + 2, n! + 3, \ldots, n! + n$ gives $n - 1$ consecutive composites for any n.

Exercise 5.15 (medium)**.** Prove a square number never ends in digit $2, 3, 7$ or 8. (Reduce $(10a + b)^2 \bmod 10$.)

Exercise 5.16 (hard)**.** Prove Euclid's lemma in both directions: p prime iff $p \mid ab \Rightarrow p \mid a$ or $p \mid b$ — the reverse direction is a clean characterisation of primality.

Exercise 5.17 (hard)**.** Chickens cost 3 coins, goats 7: a farmer spends exactly 200 coins. Prove a solution exists and find all solutions ($3x + 7y = 200$, $x, y \geq 0$). Why is this Postage-with-a-budget?

Exercise 5.18 (hard). Extend `code/bezout.py` to print, for each Euclid step, the running coefficients (x_i, y_i) with $ax_i + by_i = r_i$ maintained as an invariant — the *iterative* extended algorithm. Prove its invariant by induction and verify on $(252, 198)$ and $(1071, 462)$.

Exercise 5.19 (easy). Compute $\tau(2025)$, $\tau(3600)$, and the number of *odd* divisors of 3600 — the odd-divisor count is the same product taken only over odd primes.

Exercise 5.20 (medium). Prove the converse of the gcd-gate for Diophantine equations is tight: if $d \mid c$ fails, no integer solution exists; then exhibit an *integers-only* equation with a solution whose real line has infinitely many integer points ($x - y = 0$) to see why "one solution" and "finitely many" are unrelated.

Exercise 5.21 (medium). A number n is *abundant* if $\sigma(n) > 2n$ where σ sums the divisors. Prove 945 is the smallest odd abundant number is plausible by computing $\sigma(945) = \sigma(3^3 \cdot 5 \cdot 7)$ using the multiplicative formula $\sigma(p^e) = (p^{e+1} - 1)/(p - 1)$, and verify $\sigma(945) = 1920$.

Exercise 5.22 (hard). Prove that for every $n \geq 2$ there is a prime strictly between n and $n!$: adapt Euclid's construction to $N = n! - 1$ — where does the argument break if you use $n! + 1$ instead?

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 4–5 — `https://richardhammack.github.io/BookOfProof/` — free PDF; divisibility, gcd and Euclid drilled with full proofs.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 9 — `https://courses.csail.mit.edu/6.042/spring18/mcs.pdf` — free PDF; the number-theory chapter with the gcd-gate and Euclid's lemma treated the way §5.3 and §5.7 use them.
- Y. Zhao, *Modular Arithmetic* — `https://yufeizhao.com/olympiad/mod2.pdf` — the handout that continues this chapter into orders, Fermat–Euler and Wilson.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — `https://ocw.mit.edu/6-042JF10` — number-theory lectures behind §5.2–§5.4.
- The Open Logic Project — `https://openlogicproject.org/` — free texts; set-theoretic groundwork for treating integers as a structure.
- Cut-the-Knot, divisibility articles — `https://www.cut-the-knot.org/` — the digit-criteria and sieve-as-proof perspective behind §5.6.

WATCH INSTEAD

- **MIT OCW 6.042J — "Lecture 4: Number Theory I"**, 1:20:24, `https://www.youtube.com/watch?v=NuY7szYSXSw` — Euclid's algorithm, gcd and divisibility done at the blackboard: §5.2–§5.4. (Verified 2026-10-11.)
- **Khan Academy — "Journey into cryptography" playlist** `https://www.youtube.com/playlist?list=PLSQl0a2vh4HA50QhFIirlEZRXG4yjcoGM` — its modular-arithmetic unit covers §5.6 and the congruence view. (Verified 2026-10-11.)

Notes

The presentation follows MIT 6.042J's number-theory chapter: division algorithm, Euclid-as-lemma-machine, Bézout by back-substitution, and the gcd-times-lcm identity [4]; Velleman's chapter on divisibility supplied the WOP-style proofs and the element-chase drills [2], and Hammack's proof chapters the gcd ·lcm and gcd-gate material [3]. The digit-criteria family and the sieve-as-proof framing follow Cut-the-Knot's divisibility articles [6]; Zhao's modular-arithmetic handout ladder (orders, Fermat–Euler, Wilson) is the recommended continuation into Chapter 6 [12]. The contest problems this chapter feeds — including AIME 2025 I P1, AIME 2025 I P5, AMC 12 2024 A P4, AMC 12 2024 A P9, AMC 12 2024 A P11, AIME 2024 II P7 and the classic remainder/digit items AIME 2018 I P2, AIME 2022 I P2, AIME 2020 I P3, AMC 12 2024 B P21 — are on the AoPS archive [7].

CHAPTER 6

Modular arithmetic

Chapter 5 ended on a remainder. This chapter makes remainders the whole number system: when "a and b leave the same remainder mod m" becomes an equality — congruence — arithmetic collapses to a finite clock where enormous questions (the last digits of 7^{2026}; the day of week of any date; which congruences have solutions at all) have finite answers. After this chapter you will be able to compute powers by cycle, solve linear congruences by the gcd-gate you already own, and read every AMC "last digit" item as a period-hunting problem.

BEFORE YOU START

Chapter 5: divisibility, gcd, Bézout. *Recap:* two numbers are *congruent mod m* when they leave the same remainder after dividing by m — the clock-face arithmetic where 13 o'clock is 1 o'clock.

AFTER THIS CHAPTER YOU CAN

- add, subtract and multiply congruences correctly and know why division needs care;
- find a multiplicative inverse mod m and know when it does not exist;
- solve $ax \equiv b \pmod{m}$ completely;
- find last digits and divisibility tests with congruences.

HOW TO THINK ABOUT THIS

Modular arithmetic means we only care about the remainder, like reading a clock. That is a huge simplification: infinitely many numbers collapse to m classes. The way of thinking is to ask, for any problem about divisibility or last digits, "can I replace each number by its remainder and work in the small world instead?"

6.1 Congruence as an equivalence

Definition 6.1. For a modulus $m \geq 1$, write $a \equiv b \pmod{m}$ ("a is congruent to b mod m") when $m \mid (a - b)$ — equivalently, when a and b leave the same remainder under the division algorithm.

Both clauses matter: the $m \mid (a - b)$ form is what proofs unfold; the same-remainder form is what intuition runs on. Congruence is an equivalence relation on $\mathbb{Z}$ (reflexive: $m \mid 0$; symmetric: $m \mid (a - b) \Rightarrow m \mid (b - a)$; transitive: m divides a sum of multiples) — and Figure 6.1 draws what that bought us for $m = 4$: a partition of all of $\mathbb{Z}$.

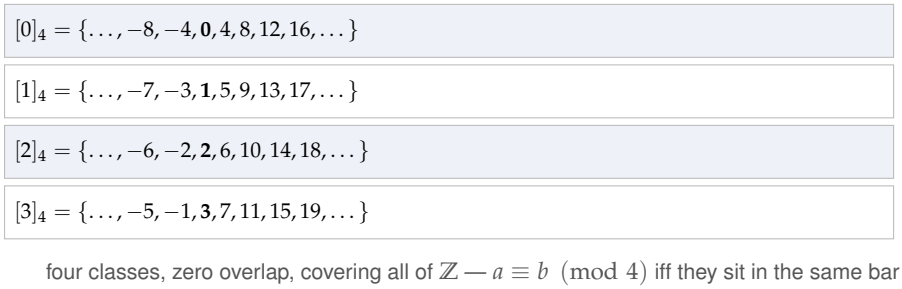


Figure 6.1: $\mathbb{Z}$ partitioned mod 4 into residue classes $[r]_4 = \{n : n \equiv r\}$. Notice the shape of equivalence itself: four classes, no overlap, nothing left out — Chapter 4's machinery doing real work.

Theorem 6.2 (Arithmetic respects congruence)**.** *If $a \equiv a' \pmod{m}$ and $b \equiv b' \pmod{m}$, then $a + b \equiv a' + b'$, $a - b \equiv a' - b'$, and $ab \equiv a'b' \pmod{m}$.*

The proof is one element-chase ($a - a' = mk$, expand) — but the content is the license: *inside any $+$, $-$, $\times$ computation you may swap any number for anything congruent to it.* Reduce first, compute second, always.

Remark. Division is conspicuously absent from Theorem 6.2. That is not an oversight — it is the subject of Section 6.3, and forgetting it is this chapter's most expensive mistake.

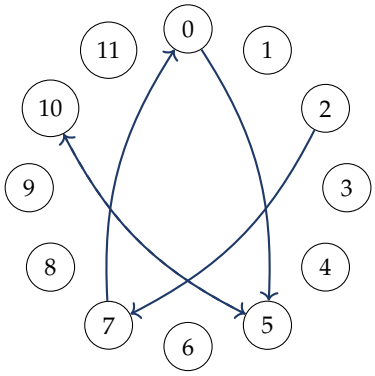
Figure 6.2 makes the arithmetic physical: mod 12 is the clock face you already know, and adding 5 hours is an arrow around it.

Exercise 6.1 (easy)**.** Prove transitivity from the definition: $a \equiv b$ and $b \equiv c \pmod{m}$ imply $a \equiv c \pmod{m}$.

6.2 The residue system $\mathbb{Z}/m\mathbb{Z}$

Definition 6.3. The *least residue* of a mod m is the unique $r \in \{0, 1, \ldots, m - 1\}$ with $a \equiv r \pmod{m}$ — the division-algorithm remainder. The *residue class* $[r]_m = \{n : n \equiv r \pmod{m}\}$, and $\mathbb{Z}/m\mathbb{Z}$ is the set of all m classes with the induced arithmetic.

Computing "mod m" then means: do integer arithmetic, drop to least residue at the end — or, better, reduce *as you go*, since Theorem 6.2 says the answer never changes. 7^{100}'s last digit never requires the 85-digit number.



$+5$ steps: $2 \to 7 \to 0 \to 5 \to 10$ (solid); then $+7$ (dashed): $10 \to 5$
arithmetic on the circle is arithmetic on residues

Figure 6.2: The residue clock mod 12. Notice the wrap: $7 + 5$ lands on 0, then continues to 5 — and a $+7$ step from 10 lands back on 5 again. Two different step sizes reaching the same class is the whole idea of "solutions mod m".

Exercise 6.2 (easy)**.** Compute the least residue of $2^{10} + 3^{10}$ mod 5 without evaluating either power fully.

6.3 Cancellation and multiplicative inverses

Here is where $\mathbb{Z}/m\mathbb{Z}$ stops being a field in disguise and starts being a ring worth studying. In $\mathbb{Q}$, $ab = ac$ implies $b = c$ for $a \neq 0$. Mod m, it does not: $2 \cdot 3 \equiv 2 \cdot 0 \pmod 6$, yet $3 \not\equiv 0$. Figure 6.3 shows why — the multiplication table is a permutation exactly when the row's factor is coprime to the modulus.

Theorem 6.4. *c has a multiplicative inverse mod m — some x with $cx \equiv 1 \pmod m$ — iff $\gcd(c, m) = 1$. Then $ca \equiv cb \pmod m$ implies $a \equiv b \pmod m$.*

Proof. Existence is Bézout, verbatim: $\gcd(c, m) = 1 \Rightarrow cx + my = 1$, and reading mod m gives $cx \equiv 1$. Failure direction: if $d = \gcd(c, m) > 1$, then $cx \equiv 1$ would force $d \mid 1$. For cancellation, $ca \equiv cb$ means $m \mid c(a - b)$; with $\gcd(m, c) = 1$, Euclid's lemma gives $m \mid (a - b)$. ∎

Inverses are computed, not guessed: run Euclid backwards (Example 5.6) and take the coefficient of c.

Exercise 6.3 (medium)**.** Find $7^{-1} \pmod{10}$ and $8^{-1} \pmod{13}$ by extended Euclid; verify each by multiplying back.

Example 6.5. Find $8^{-1} \pmod{15}$, then explain why 6 has no inverse mod 15.

Strategy. Inverse is Bézout in disguise: solve $8x + 15y = 1$ by extended Euclid and read the coefficient of 8 mod 15. For 6, the gcd-gate slams shut before you start.

Solution. $15 = 1 \cdot 8 + 7$; $8 = 1 \cdot 7 + 1$. Back-substitute: $1 = 8 - 7 = 8 - (15 - 8) = 2 \cdot 8 - 15$, so $8^{-1} \equiv 2 \pmod{15}$. Check: $8 \cdot 2 = 16 \equiv 1$. For 6: $\gcd(6, 15) = 3$, and $6x \equiv 1 \pmod{15}$

mod 5:

	1	**2**	**3**	**4**
1	1	2	3	4
2	2	4	1	3
3	3	1	4	2
4	4	3	2	1

every nonzero row is a permutation: each has a 1-entry ⇒ inverse

mod 6:

	1	**2**	**3**	**4**	**5**
1	1	2	3	4	5
2	2	4	0	2	4
3	3	0	3	0	3
4	4	2	0	4	2
5	5	4	3	2	1

shaded rows ($\gcd = 1$): still permutations ⇒ inverses.

unshaded rows collapse: no 1-entry, and 0 appears — $2 \cdot 3 \equiv 0$ kills cancellation.

Figure 6.3: Multiplication mod 5 vs mod 6. Notice what coprimality buys: mod 5 every nonzero row is a permutation (a 1 in every row: inverses for all). Mod 6, only the gcd $= 1$ rows permute; the row of 2 repeats $\{0, 2, 4\}$ with no 1 — and the 0-entries are zero divisors, the reason cancellation fails.

would mean $3 \mid 1$ — impossible. The row of 6 in a multiplication table mod 15 hits only multiples of 3, never 1.

6.4 Linear congruences

Theorem 6.6. *$ax \equiv b \pmod{m}$ has solutions iff $\gcd(a, m) \mid b$; then it has exactly $\gcd(a, m)$ solutions mod m: divide through by $d = \gcd(a, m)$ to reach $\frac{a}{d}x \equiv \frac{b}{d} \pmod{\frac{m}{d}}$ (coprime now), invert, and lift the answer back by $+k \cdot \frac{m}{d}$.*

This *is* Theorem 5.14 in disguise — $ax \equiv b \pmod{m}$ literally means $ax - my = b$.

Example 6.7. Solve $14x \equiv 6 \pmod{20}$.

Strategy. Gcd first — never cancel before checking. $\gcd(14, 20) = 2$ and $2 \mid 6$, so solutions exist and there are exactly two mod 20. Divide the *whole* congruence (modulus included) by 2, then invert the now-coprime coefficient.

Solution. Divide: $7x \equiv 3 \pmod{10}$. Since $7 \cdot 3 = 21 \equiv 1 \pmod{10}$, $7^{-1} \equiv 3$, so $x \equiv 3 \cdot 3 = 9 \pmod{10}$. Lifting back, the two solutions mod 20 are $x \equiv 9$ and $x \equiv 19$. Check: $14 \cdot 9 = 126 \equiv 6$ and $14 \cdot 19 = 266 \equiv 6 \pmod{20}$.

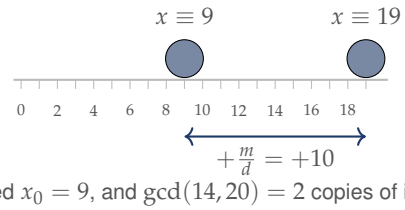


Figure 6.4: The $\gcd(a, m) = 2$ solutions of $14x \equiv 6 \pmod{20}$: one seed $x_0 = 9$ plus copies spaced $m/d = 10$ apart — a linear congruence answers with a finite arithmetic family, not one residue.

6.5 Modular patterns: last digits, power cycles, casting out nines

Powers of a mod m can produce at most m residues, so the sequence $a^1, a^2, a^3, \ldots \pmod{m}$ eventually repeats — and once it returns to 1 it loops with some *period*: exponents differing by the period give equal powers. "Last digit(s) of a^N" is never a big-number problem: it is "find the period, then reduce N mod the period."

Example 6.8. Find the last two digits of 7^{2026}.

Strategy. Hunt the period first: compute 7^k mod 100 until it returns to 1. Then the exponent only matters mod the period — 2026 reduces, 7^{2026} never has to exist.

Solution. $7^1 \equiv 07$, $7^2 \equiv 49$, $7^3 \equiv 343 \equiv 43$, $7^4 \equiv 301 \equiv 01 \pmod{100}$. Period $= 4$. Since $2026 = 4 \cdot 506 + 2$, $7^{2026} \equiv 7^2 \equiv 49 \pmod{100}$: the last two digits are **49**.

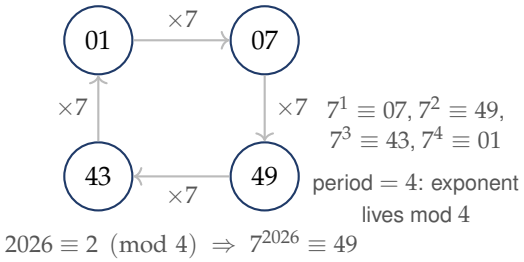


Figure 6.5: The power cycle of 7 mod 100: four residues on a loop. Notice where the exponent lives — not on the circle but *counting laps around it*: a^k depends only on k mod 4, which is why $2026 \equiv 2$ settles the question.

Checkpoint 6.1 *Predict before you run.* Listing 6.1 finds that same cycle mechanically. Predict the period for $(a, m) = (3, 7)$ before reading the output below — then check what Fermat's little theorem (the olympiad-track version is in [12]) predicts.

```
seen = {}; cur = 1 % m; k = 0
while cur not in seen:
    seen[cur] = k; cur = (cur * a) % m; k += 1
```

Listing 6.1: `powers_cycle.py`: multiply until a residue repeats (full script in `code/powers_cycle.py`).

Real output (defaults $a = 7$, $m = 100$; then 3 mod 7):

```
powers of 7 mod 100:
  7^0 == 1
  7^1 == 7
  7^2 == 49
  7^3 == 43
  7^4 == 1 (repeat)
cycle starts at k=0, period = 4
so 7^2026 mod 100 = 7^2 mod 100 = 49
---
powers of 3 mod 7:
  3^0 == 1
  3^1 == 3
  3^2 == 2
  3^3 == 6
  3^4 == 4
  3^5 == 5
  3^6 == 1 (repeat)
cycle starts at k=0, period = 6
so 3^2026 mod 7 = 3^4 mod 7 = 4
```

Example 6.9. Prove $n^3 - n$ is divisible by 6 for every integer n.

Strategy. A residue bash is a *finite* verification: there are only six classes mod 6, so check each once and the statement is settled for all n at once — no inductive step to invent, no factorisation to spot. The bash is the dumbest method in the chapter and also the most reliable.

Solution. Read $n^3 - n$ mod 6 for $n \equiv 0, 1, 2, 3, 4, 5$: $0 - 0 = 0$; $1 - 1 = 0$; $8 - 2 = 6 \equiv 0$; $27 - 3 = 24 \equiv 0$; $64 - 4 = 60 \equiv 0$; $125 - 5 = 120 \equiv 0$. Every class gives 0, so $6 \mid n^3 - n$ for all n. (The structural reason: $n^3 - n = (n-1)n(n+1)$, three consecutive integers — one divisible by 3, at least one even.)

Casting out nines is the same mechanism a level up: $10 \equiv 1 \pmod 9$ makes every integer congruent to its digit sum (Chapter 5 proved the criterion); the checksum survives addition, so "check the multiplication by digit-summing both sides" catches most slip errors. Note 9's blind spot: it cannot see a transposition of digits that changes the answer by a multiple of 9 — a checksum, not a proof.

Exercise 6.4 (medium)**.** Find the last two digits of 3^{2026} (hint: the period mod 100 is longer than 4 — hunt it with `powers_cycle.py`, then explain why $\varphi(100) = 40$ is an upper bound).

6.6 Two moduli at once: a CRT preview

What if a number must satisfy *two* congruences, $x \equiv 2 \pmod 3$ and $x \equiv 3 \pmod 5$? Hunting the finite list mod 15 finds $x \equiv 8$ — and the Chinese remainder theorem (Chapter 5's Bézout coefficients are its engine; the olympiad track develops it in full [12]) says the solution is unique mod $3 \cdot 5$ whenever the moduli are coprime. Chapter 5 solved one equation in integers; this is two conditions at once, and the same gcd-gate decides existence.

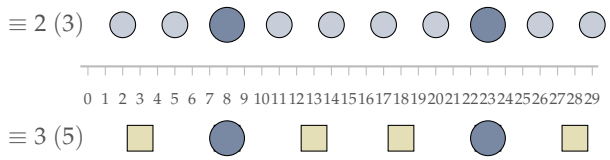


Figure 6.6: $x \equiv 2 \pmod 3$ (top) and $x \equiv 3 \pmod 5$ (bottom) tag the same numbers exactly once per 15-block: at $8, 23, \ldots$ — the single class $x \equiv 8 \pmod{15}$ that CRT guarantees when the moduli are coprime.

Example 6.10. What day of the week was October 10, 2026?

Strategy. Count in mod 7: January 1, 2026 was a Thursday (a verifiable anchor). Count the day-of-year of October 10 — a sum of month offsets, not a loop through the calendar — reduce mod 7, and step that many days from the anchor.

Solution. 2026 is not a leap year. Days before October: $31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30 = 273$, so October 10 is day 283 of the year, offset 282 days from January 1. $282 = 7 \cdot 40 + 2$, so $282 \equiv 2 \pmod 7$ and the day is Thursday $+2$ = **Saturday**.

6.7 Chapter summary

Object	Rule	Watch
$a \equiv b \pmod m$	$m \mid (a - b)$; equivalence on $\mathbb{Z}$	statement, not equation
$+, -, \times$	substitute freely, reduce early	exponent is NOT reduced mod m
$ca \equiv cb$	cancel iff $\gcd(c, m) = 1$	else divide modulus too
$c^{-1} \bmod m$	Bézout coefficient of c	exists iff coprime
$ax \equiv b$	gcd $\mid b$ gate; gcd solutions	divide modulus by d as well
$a^N \bmod m$	find period, reduce N mod period	period $\leq m$, usually φ
digit sum	$n \equiv$ digits mod 9	checksum, not proof

Table 6.1: The chapter in one page. Every rule is a Chapter-5 theorem wearing a clock face — the gcd still decides everything.

6.8 Common mistakes

- **Dividing by a non-unit.** $2x \equiv 4 \pmod 6$ "solves" to $x \equiv 2$ — but $x \equiv 5$ also works, and the real solution set is $\{2, 5\}$. Cancelling 2 without dividing the modulus deleted half the answer. Check $\gcd(c, m)$ *before* cancelling.
- **Writing $x = 3 \pmod 5$ inside algebra.** Congruence is a relation between *two* integers, not a decoration on one. The x that equals 3 is an integer; the x congruent to 3 mod 5 is a class. Mixing them produces "$3 = 8$" nonsense that quietly poisons a proof.
- **Reducing the exponent by the modulus.** $a^N \bmod m$ cycles with the *period* of a's powers — period 4 for 7 mod 100, not 100. 7^{2026} is not 7^{26}. Find the period first; only then does the exponent get reduced.

6.9 Additional exercises

Exercise 6.5 (easy)**.** Show $a \equiv b \pmod m$ iff a and b have the same least residue — and check whether $-7 \equiv 5 \pmod{12}$ (careful with the definition).

Exercise 6.6 (easy)**.** Compute $17 \cdot 23 \pmod 5$, $9^3 \pmod 7$, and $2026 \cdot 10^{10} \pmod 9$ by early reduction.

Exercise 6.7 (easy)**.** Which elements of $\mathbb{Z}/12\mathbb{Z}$ have inverses? List them with their inverses.

Exercise 6.8 (easy)**.** Use casting out nines to catch the error in $47 \times 83 = 3911$ — then produce a wrong answer the check *cannot* catch.

Exercise 6.9 (medium)**.** Solve $12x \equiv 8 \pmod{28}$ completely, and prove $15x \equiv 7 \pmod{21}$ has no solutions — state the gate that decides each.

Exercise 6.10 (medium)**.** Prove $n^3 - n$ is divisible by 6 for every integer n by checking the six residue classes — then prove it again by factoring. Which proof does the checker trust more, and why?

Exercise 6.11 (medium)**.** Find the last digit of 2026^{2026} and the last two digits of 9^{100} (period mod 4 vs mod 100).

Exercise 6.12 (medium)**.** Prove the "rule of 11" as a congruence ($10 \equiv -1$) and use it to test 121 121 and 9 876 543.

Exercise 6.13 (hard)**.** Prove that the powers of a mod m are *purely* periodic exactly when $\gcd(a, m) = 1$; otherwise a tail precedes the cycle (find the tail for 2 mod 12 and state the general rule).

Exercise 6.14 (hard)**.** Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ simultaneously. (Build it one congruence at a time — this is the CRT construction in embryo.)

Exercise 6.15 (hard)**.** Extend `powers_cycle.py` to print, for each base $a = 2, \ldots, 9$ and modulus m, the tail length and period — and conjecture the rule connecting them to $\gcd(a, m)$. Verify on $m = 12, 20, 100$.

Exercise 6.16 (easy)**.** Compute $2^{10} \bmod 11$, $5^6 \bmod 7$ and $10^{20} \bmod 3$ — find each in under a minute by reducing the base or spotting a short period first.

Exercise 6.17 (medium)**.** Solve $9x \equiv 6 \pmod{15}$ completely; then change the 6 to a 7 and prove there is no solution — same modulus, gcd-gate shut.

Exercise 6.18 (medium)**.** Prove that if $a \equiv b \pmod{m}$ and $a \equiv b \pmod{n}$ with $\gcd(m,n) = 1$, then $a \equiv b \pmod{mn}$ — and give a counterexample when $\gcd(m,n) > 1$ (the CRT preview's uniqueness half).

Exercise 6.19 (hard)**.** Find all solutions of $x^2 \equiv 1 \pmod{24}$. (Solve mod 3 and mod 8 separately, then combine — the answer is more than ± 1, which is the point.)

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 7 — https://richardhammack.github.io/BookOfProof/ — free PDF; congruence, inverses and the cancellation theorem.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 9 — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; modular arithmetic with the gcd-gate framing used here.
- Berkeley CS70 notes — https://www.eecs70.org/ — free lecture notes; the continuation into Fermat–Euler, CRT and RSA flagged in §7.1.
- Y. Zhao, *Modular Arithmetic* — https://yufeizhao.com/olympiad/mod2.pdf — orders and primitive roots for the olympiad track.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — https://ocw.mit.edu/6-042JF10 — the modular-arithmetic lectures this chapter follows.
- Stanford CS103, *Mathematical Foundations of Computing* — https://web.stanford.edu/class/cs103/ — free reader with extra congruence drills.

WATCH INSTEAD

- **MIT OCW — "Lecture 9: Modular Arithmetic"**, 1:19:45, https://www.youtube.com/watch?v=Nn8nBPyzzQ0 — congruence, clock arithmetic and exponent patterns: §6.1–§6.5. (Verified 2026-10-11.)
- **Zach Star — "This completely changed the way I see numbers: Modular Arithmetic Visually Explained"**, 20:33, https://www.youtube.com/watch?v=lJ3CD9M3nEQ — the visual, why-it-matters motivation for §6.1 and §6.5. (Verified 2026-10-11.)

Notes

The equivalence-first presentation and the cancellation theorem follow MIT 6.042J's modular-arithmetic sections [4]; Velleman's congruence chapter supplied the residue-partition viewpoint [2], and Hammack's the clock-face intuition [3]. Berkeley CS70's modular note is the recommended next read (it pushes straight into Fermat–Euler, CRT and RSA [13]); Zhao's olympiad handout covers the orders/primitive-roots continuation [12]. Contest items powered by this chapter's machinery are on the AoPS archive [7]: power cycles

(AMC 12 2024 B P14, AIME 2024 I P13), last-digit and remainder problems (AIME 2020 I P10–12, AIME 2010 I P2, AIME 2018 I P11), linear congruences (AIME 2023 I P7, AIME 2025 II P2) and CRT-flavoured counting (AIME 2023 II P15).

CHAPTER 7

The Chinese remainder theorem and Fermat–Euler

Chapter 6 solved one congruence at a time. This chapter joins them — the Chinese remainder theorem says a system of congruences with coprime moduli always has exactly one solution class mod the product — and then proves the two big structure theorems of prime moduli: Fermat's little theorem and Euler's totient theorem. These are the workhorses behind almost every "compute a huge power mod m" problem in the archive. After this chapter you will be able to solve CRT systems by construction, reduce exponents by the right period (φ, not m), and recognise when a congruence problem is really asking for an order.

BEFORE YOU START

Chapters 5–6. *Recap:* modulo m, every integer is equivalent to one of the m residues $0, 1, \ldots, m-1$, and a has an inverse exactly when $\gcd(a, m) = 1$.

AFTER THIS CHAPTER YOU CAN

- solve a system of congruences with coprime moduli by the Chinese remainder theorem;
- reduce huge powers using Fermat's little theorem and Euler's theorem;
- compute $\varphi(n)$ and the order of an element;
- build a check-digit scheme and prove what errors it catches.

HOW TO THINK ABOUT THIS

The Chinese remainder theorem says that knowing a number's remainders modulo several coprime numbers is the same as knowing it modulo their product. Think of it as describing a place by two separate coordinates. Fermat and Euler say that powers eventually repeat in a regular way, which is why enormous exponents can be shrunk before you compute anything.

7.1 The Chinese remainder theorem

Theorem 7.1 (CRT). *If $m_1, \dots, m_k$ are pairwise coprime, the system $x \equiv r_i \pmod{m_i}$ has exactly one solution modulo $M = m_1 m_2 \cdots m_k$.*

Existence *is* the construction, and the construction is the two- modulus case repeated: to merge $x \equiv r_1 \pmod{m_1}$ with $x \equiv r_2 \pmod{m_2}$, write $x = r_1 + m_1 t$ and demand $m_1 t \equiv r_2 - r_1 \pmod{m_2}$ — solvable because $\gcd(m_1, m_2) = 1$ gives m_1 an inverse (Theorem 6.4). Uniqueness: two solutions differ by a multiple of every m_i, and pairwise coprimality makes that a multiple of M.

Example 7.2. Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$.

Strategy. Satisfy one modulus, then *add multiples* to satisfy the next: parametrise the current solution as $x = r + mt$ and solve the linear congruence for t — Chapter 6's entire toolbox inside Chapter 7's loop.

Solution. $x \equiv 2 \pmod 3$: write $x = 2 + 3t$. Mod 5: $2 + 3t \equiv 3$, $3t \equiv 1$, $t \equiv 2$ — so $x \equiv 8 \pmod{15}$. Fold again: $x = 8 + 15s$; mod 7: $8 + 15s \equiv 2$, i.e. $15s \equiv -6 \equiv 1 \pmod 7$ — and $15 \equiv 1 \pmod 7$, so $s \equiv 1$. Hence $x \equiv 23 \pmod{105}$ — the unique class, verified in the listing below ($23 \bmod 3 = 2$, $\bmod 5 = 3$, $\bmod 7 = 2$).

Figure 7.1 draws the two-modulus view: residues on two different rulers, and the solution is the tick where both land.

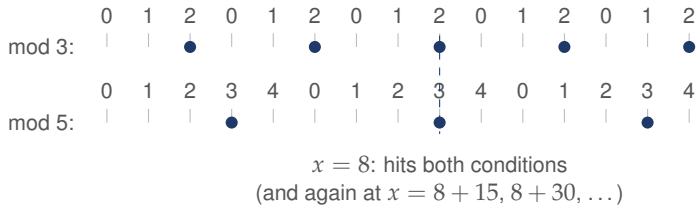


Figure 7.1: CRT as sliding rulers: $x \equiv 2 \pmod 3$ (top dots) and $x \equiv 3 \pmod 5$ (bottom dots) meet at $x = 8$, and again every 15 — the period is the *product*, which is why coprimality matters.

Checkpoint 7.1 *Predict before you run.* Listing 7.1 solves the system two ways — brute scan and the folding construction. Predict the intermediate fold $x \equiv ? \pmod{15}$ before reading the output.

```
def crt_pair(r1, m1, r2, m2):
    g, s, _ = egcd(m1, m2)
    t = (s * (r2 - r1) // g) % (m2 // g)
    return (r1 + m1 * t) % (m1 * m2 // g), m1 * m2 // g
```

Listing 7.1: `crt_solver.py`: fold two congruences at a time via Bézout (full script in `code/crt_solver.py`).

Real output:

```
system: x == 2 (mod 3), x == 3 (mod 5), x == 2 (mod 7)
brute force scan: [23]
  folded: x == 8 (mod 15)
  folded: x == 23 (mod 105)
constructive: x == 23 (mod 105)
unique mod 105: 23; check [2, 3, 2]
```

Exercise 7.1 (easy). Solve $x \equiv 1 \pmod 4$, $x \equiv 4 \pmod 9$ by the folding method, and verify against `crt_solver.py`.

7.2 Three ways to hold the theorem

Chen's handout presents CRT as three lenses on the same fact, and each lens wins a different fight [16]:

- **Construction** (what we just did): given target residues, *build* an x with them. Use when the problem asks "find an integer such that ..." — the theorem licenses your assembly.
- **Lifting**: if $x \equiv r$ holds mod every part, it holds mod the product. $x \equiv 1 \bmod 2, 3, 5$ means $x \equiv 1 \pmod{30}$ — surprising congruences often decompose this way.
- **Destruction**: a congruence mod M can be *split* into its prime-power parts, which are easier to reason about one at a time. "$a^2 \equiv 1 \pmod{24}$" is really "mod 8 and mod 3 simultaneously" — and each half is nearly free.

The skill is recognising which direction the problem is already facing: systems want construction, big moduli want destruction, and "show it holds for all n" often wants lifting.

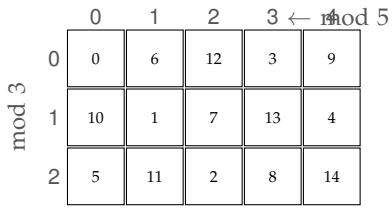


Figure 7.2: CRT as a bijection: the 15 residues fill the $(\bmod 3) \times (\bmod 5)$ grid exactly once each. Construction says every cell is reachable; lifting says the coordinates determine the cell's content.

7.3 Systems in practice

The theorem's hypothesis is load-bearing. With $\gcd(m_1, m_2) = d > 1$, a solution exists *iff* $r_1 \equiv r_2 \pmod{d}$ — the consistency check — and then the class is unique only mod $\operatorname{lcm}(m_1, m_2)$. $x \equiv 1 \pmod 4$, $x \equiv 3 \pmod 6$: consistent ($1 \equiv 3 \pmod 2$), solutions $x \equiv 9 \pmod{12}$ ($9 \bmod 4 = 1$, $9 \bmod 6 = 3$). But $x \equiv 1 \pmod 4$, $x \equiv 4 \pmod 6$ is inconsistent: no solution, and no amount of Bézout saves it. The gate is cheap — always run it before constructing.

Exercise 7.2 (easy)**.** Decide, without solving: is $x \equiv 2 \pmod 6$, $x \equiv 5 \pmod 9$ consistent? If yes, solve it; if no, say what the gate detected.

7.4 Fermat's little theorem, two ways

Theorem 7.3 (Fermat's little theorem)**.** *For prime p and $p \nmid a$: $a^{p-1} \equiv 1 \pmod p$. Equivalently $a^p \equiv a \pmod p$ for all a.*

Example 7.4. Prove it by the multiplication-permutation argument.

Strategy. Multiply the nonzero residues $1, 2, \ldots, p-1$ by a: because a is coprime to p, that map is a *bijection* — the same set reshuffled (Figure 7.3). Two lists with the same members have the same product; the rest is cancellation.

Solution. $\{a \cdot 1, a \cdot 2, \ldots, a \cdot (p-1)\} \pmod p$ are the nonzero residues in some order (no two collide: $ai \equiv aj$ would cancel a — coprimality again). Hence $a \cdot 2a \cdots (p-1)a \equiv 1 \cdot 2 \cdots (p-1) \pmod p$, i.e. $a^{p-1}(p-1)! \equiv (p-1)!$. Since $\gcd((p-1)!, p) = 1$, cancel: $a^{p-1} \equiv 1 \pmod p$.

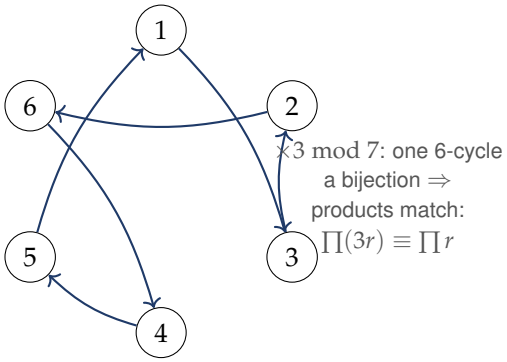


Figure 7.3: $\times 3 \bmod 7$ is a bijection — here a single 6-cycle. Notice the proof hiding in the picture: the products of all arrows' endpoints and origins are the same set, so $3^6 \cdot 6! \equiv 6!$.

Example 7.5 (FLT by counting)**.** Prove $a^p \equiv a \pmod p$ by a combinatorial argument.

Strategy. Count a^p strings over a colours, remove the a monochrome ones, and partition the rest into rotation classes — primality is what forces every class to have full size p.

Solution. Of the a^p strings on an alphabet of size a, exactly a are monochrome. Every other string has all p of its rotations distinct (a smaller orbit would give a nontrivial divisor of p — see Figure 7.4), so the remaining $a^p - a$ strings split into classes of size p. Hence $p \mid a^p - a$.

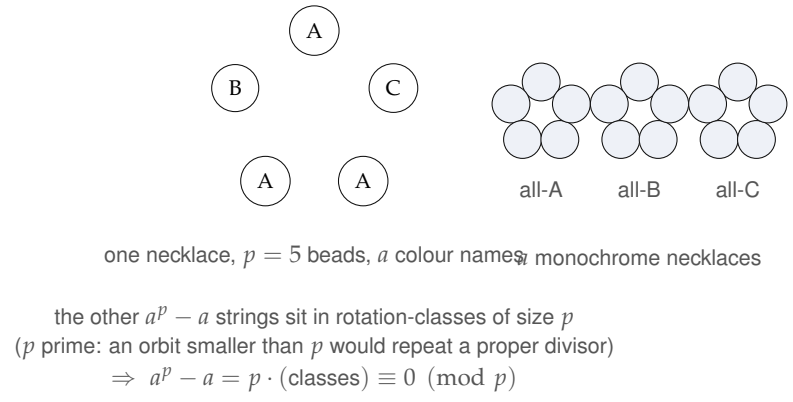


Figure 7.4: The necklace proof of $a^p \equiv a \pmod{p}$. Notice why p must be prime: only then does every non-monochrome string rotate into *exactly* p distinct strings — a composite modulus would allow smaller orbits and break the count.

Exercise 7.3 (medium)**.** Compute 2^{100} mod 13 and 7^{222} mod 11 by Fermat reduction; then verify with `powers_cycle.py`.

7.5 Euler's totient and Euler's theorem

Definition 7.6. $\varphi(n)$ counts the integers in $\{1, \dots, n\}$ coprime to n. For $n = \prod p_i^{e_i}$: $\varphi(n) = n\prod(1 - 1/p_i) = \prod p_i^{e_i - 1}(p_i - 1)$.

Theorem 7.7 (Euler)**.** *If* $\gcd(a, n) = 1$*:* $a^{\varphi(n)} \equiv 1 \pmod{n}$.

The proof is Fermat's verbatim, with "nonzero residues" replaced by "units" — the $\varphi(n)$ residues coprime to n form a group under multiplication, and multiplying by a unit permutes them exactly as in Figure 7.3. Fermat is the special case $\varphi(p) = p - 1$. Figure 7.5 charts φ to 36: the primes are the spikes where $n - 1$ is achieved.

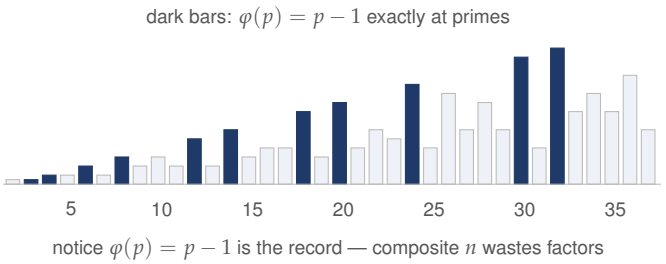


Figure 7.5: $\varphi(n)$ for $n \le 36$. Notice the records: $\varphi(p) = p - 1$ at primes (dark bars), and values like $\varphi(30) = 8$ show the product discount $30(1 - \frac{1}{2})(1 - \frac{1}{3})(1 - \frac{1}{5})$ at work.

Example 7.8. Find the last three digits of 3^{400}.

Strategy. Last three digits is mod 1000; Euler reduces the exponent mod $\varphi(1000)$. Compute $\varphi(1000) = 1000(1-\frac{1}{2})(1-\frac{1}{5}) = 400$ — the exponent *is* the order's multiple.

Solution. $\gcd(3, 1000) = 1$, so $3^{400} = 3^{\varphi(1000)} \equiv 1 \pmod{1000}$: the last three digits are **001**.

Exercise 7.4 (medium)**.** Find the last three digits of 7^{1000} and of 3^{401} (reduce each exponent mod the period from WE 7.8 first; mind the remainder this time).

7.6 Orders modulo p

Euler gives *an* exponent that returns 1; the *order* of a mod m is the *smallest* positive one, written $\mathrm{ord}_m(a)$. Two facts carry the theory: ord always divides $\varphi(m)$ (long division of the exponent), and $a^k \equiv 1$ iff the order divides k. Chapter 6's power cycles were orders in plain clothes — 7 has order 4 mod 100, and $4 \mid \varphi(100) = 40$. A unit whose order is $\varphi(m)$ is a *primitive root*; mod primes one always exists (proved in the olympiad track [12]).

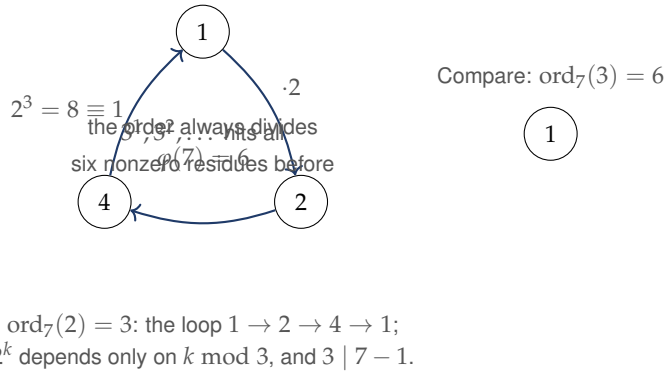


Figure 7.6: Order on a wheel: 2^k mod 7 loops $1 \to 2 \to 4 \to 1$, so $\mathrm{ord}_7(2) = 3 \mid 6$. The order is the loop's length — and it always divides $\varphi(m)$.

Exercise 7.5 (medium)**.** Find $\mathrm{ord}_{11}(2)$ and $\mathrm{ord}_{11}(10)$; both divide 10 — which hits the bound, and what does that mean?

7.7 Wilson's theorem (enrichment)

Theorem 7.9. $(p-1)! \equiv -1 \pmod{p}$ *for prime* p.

The proof pairs each residue with its inverse: mod p, the only self-inverse units are ± 1 ($x^2 \equiv 1$ factors as $p \mid (x-1)(x+1)$, and primality gives no zero divisors). Every other unit cancels in pairs, leaving $1 \cdot (-1)$. The converse is also true — the congruence *characterises* primes — but useless as a primality test ($p!$ is huge); its real job is inside other proofs.

7.8 Applications: last digits again, check digits

Chapter 6 computed last digits by hunting periods; Euler supplies the upper bound on the hunt. And the same machinery hides in every checkout line: the ISBN-10 check digit $d_{10} = \sum_{i=1}^{9} i \cdot d_i \bmod 11$ catches *any* single-digit error and any transposition of adjacent digits — the second claim is exactly that the weights i are distinct mod 11.

Example 7.10. Show that $t^2 + t + 1$ has arbitrarily many distinct prime divisors as t ranges over the positive integers.

Strategy. Euclid's infinitude argument, lifted onto the value-set: given any finite list of primes hitting the values, evaluate f at their product — every listed prime divides the argument, so the value is $\equiv f(0) = 1$ at each one, and every prime factor of the value is new.

Solution. Suppose only $p_1, \ldots, p_k$ ever divide values of $f(t) = t^2 + t + 1$. Set $Q = p_1 p_2 \cdots p_k$ and evaluate $f(Q) = Q^2 + Q + 1$. For each listed p_i: $Q \equiv 0 \pmod{p_i}$, so $f(Q) \equiv f(0) = 1 \pmod{p_i}$ — no listed prime divides $f(Q)$. But $f(Q) > 1$ has some prime factor q, which is therefore a new prime dividing a value of f: contradiction. The same trick works for any nonconstant polynomial with $f(0) = \pm 1$ (USAMO 2008 P1 builds the full construction on this idea [16]).

7.9 Chapter summary

Tool	Condition	Delivers
CRT (coprime)	m_i pairwise coprime	one class mod $\prod m_i$
CRT (general)	consistent iff $r_i \equiv r_j \pmod{\gcd}$	class mod lcm
Fermat	p prime, $p \nmid a$	$a^{p-1} \equiv 1 \pmod{p}$
Euler	$\gcd(a, n) = 1$	$a^{\varphi(n)} \equiv 1 \pmod{n}$
totient	$n = \prod p^e$	$n \prod (1 - 1/p)$; never trial-count
order	$\text{ord} \mid \varphi(m)$	exact period, divides φ
last digits of a^N	reduce N mod ord, not m	finite loop

Table 7.1: The chapter in one page: every tool is a gate plus a formula. The second column is where the proofs actually check you.

7.10 Common mistakes

- **CRT on non-coprime moduli.** The construction proceeds whether or not the system is consistent — and returns a confident wrong answer. Gate first: $r_i \equiv r_j \pmod{\gcd(m_i, m_j)}$ for every pair, else the system is just unsatisfiable.
- **Fermat off-by-one.** $a^{p-1} \equiv 1$ needs $p \nmid a$ *and* exponent $p - 1$ — writing $a^p \equiv 1$ is the classic slip (the correct all-a form is $a^p \equiv a$). Reduce exponents mod $p - 1$, not mod p.

- **Totient by trial.** Counting coprimes to n by hand is $O(n)$ and error-prone; the product $n\prod(1-1/p)$ needs only the factorisation. $\varphi(1000) = 400$ in one line, not 400 checks.

7.11 Additional exercises

Exercise 7.6 (easy)**.** Solve $x \equiv 1 \pmod 2$, $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$ by folding; verify with `crt_solver.py`.

Exercise 7.7 (easy)**.** Show $n^5 \equiv n \pmod{30}$ for every n — split into the three moduli (destruction lens) and use Fermat on each.

Exercise 7.8 (easy)**.** Compute $\varphi(360)$ and $\varphi(97)$; list the units of $\mathbb{Z}/8\mathbb{Z}$ and verify each squares to 1.

Exercise 7.9 (easy)**.** Find the last two digits of 3^{100} via Euler ($\varphi(100)$) and via the cycle picture of Chapter 6 — confirm they agree.

Exercise 7.10 (medium)**.** Prove the totient product formula for prime powers first: $\varphi(p^e) = p^e - p^{e-1}$ by direct count; then argue multiplicativity from CRT (a unit mod mn is a pair of units).

Exercise 7.11 (medium)**.** Solve $x \equiv 4 \pmod 6$, $x \equiv 2 \pmod 8$ after checking consistency; state the modulus of the answer class.

Exercise 7.12 (medium)**.** Prove $n^{13} \equiv n \pmod{2730}$ for all n — the lcm of $\{p-1 : p \mid 2730\}$ is the engine.

Exercise 7.13 (medium)**.** Show that if $\operatorname{ord}_m(a) = k$, then $a^i \equiv a^j$ iff $i \equiv j \pmod k$ — the power cycle is exactly a cycle.

Exercise 7.14 (hard)**.** Find all $n \le 60$ with $\varphi(n) = 12$, and prove your list is complete (what does $p-1 \mid 12$ force?).

Exercise 7.15 (hard)**.** Prove that $x^2 \equiv 1 \pmod p$ has *only* the solutions $x \equiv \pm 1$ — and that this fails mod 8: find all four.

Exercise 7.16 (hard)**.** Extend `crt_solver.py` to non-coprime systems: run the consistency gate, emit the class mod lcm, and report inconsistent systems explicitly. Verify on $x \equiv 4 \pmod 6$, $x \equiv 2 \pmod 8$ and a deliberately broken system.

Exercise 7.17 (easy)**.** Solve $x \equiv 1 \pmod 4$, $x \equiv 0 \pmod 9$ by hand — multiples of 9 ending in the right class — and check against `crt_solver.py`.

Exercise 7.18 (medium)**.** Find the last two digits of 3^{400} two ways: Euler's theorem mod 100 (what is $\varphi(100)$ and does it apply?) and the power-cycle method of Chapter 6. Reconcile the answers.

Exercise 7.19 (medium)**.** Prove $\operatorname{ord}_m(a)$ divides $\varphi(m)$ whenever $\gcd(a,m) = 1$ by writing $\varphi(m) = q \cdot \operatorname{ord} + r$ with $0 \le r < \operatorname{ord}$ and showing $a^r \equiv 1$ forces $r = 0$.

Exercise 7.20 (hard)**.** A number leaves remainder $2, 3, 4$ on division by $5, 6, 7$ respectively. Show it is $x \equiv -1 \pmod{5 \cdot 6 \cdot 7/\gcd\text{-corrected}}$, i.e. solve with the non-coprime gate and state the class mod $\operatorname{lcm}(5,6,7) = 210$.

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 7 — https://richardhammack.github.io/BookOfProof/ — free PDF; CRT, Fermat and Euler drilled gently.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 9 — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; CRT and Euler's theorem with the ring viewpoint.
- E. Chen, *The Chinese Remainder Theorem* — https://web.evanchen.cc/handouts/CRT/CRT.pdf — the three-lenses presentation behind §7.2.
- Y. Zhao, *Modular Arithmetic* — https://yufeizhao.com/olympiad/mod2.pdf — orders and primitive roots beyond §7.6.
- Berkeley CS70 notes — https://www.eecs70.org/ — free lecture notes; CRT + RSA carried through to working code.
- Cut-the-Knot, number theory — https://www.cut-the-knot.org/ — alternate proofs of Fermat, Euler and Wilson for comparison.

WATCH INSTEAD

- **Numberphile — "Chinese Remainder Theorem and Cards"**, 11:13, https://www.youtube.com/watch?v=l9dXo5f3zDc — Tokieda's card-trick CRT, including the non-coprime case of §7.3. (Verified 2026-10-11.)
- **Socratica — "Fermat's Little Theorem"**, 10:35, https://www.youtube.com/watch?v=w0ZQvZLx2KA — the permutation proof of §7.4 done carefully. (Verified 2026-10-11.)

Notes

The three-lenses framing (construction, lifting, destruction) and the USAMO 2008 P1 connection follow Chen's CRT handout [16]. The folding construction and non-coprime consistency gate follow 6.042J's CRT sections [4]; Velleman supplied the residue-system proofs [2]; Berkeley CS70's notes push on into RSA from exactly this point [13], and Zhao's handouts carry orders and primitive roots to olympiad depth [12]. The necklace proof of Fermat is the standard counting argument (cf. OTIS [9]). Contest items this chapter powers — CRT systems (AIME 2023 I P7, AIME 2025 II P2), Euler reductions (AIME 2024 I P13), remainder cycles (AIME 2020 I P12), orders and φ (AIME 2010 I P14, AIME 2021 I P7), plus USAJMO 2024 P3 and USAMO 2008 P1 — are on the AoPS archive [7].

CHAPTER 8

Counting principles

Combinatorics is arithmetic with a story attached. Every formula in this chapter — $\binom{n}{k}$, $n!$, the stars-and-bars count — is a count of *something concrete*, and the fastest proofs are the ones that name the object and count it two ways. After this chapter you will be able to read a counting problem, name the underlying set, pick the right model (ordered vs unordered, distinct vs repeated), and verify small cases against brute force before trusting a closed form.

BEFORE YOU START

Chapters 2–4. No number theory is needed here. *Recap:* counting means finding the size of a finite set without listing it; $n! = 1 \cdot 2 \cdots n$ is the number of ways to order n different things.

AFTER THIS CHAPTER YOU CAN

- choose between the sum and product rules and know when each applies;
- count arrangements, selections and distributions with factorials, binomials and stars-and-bars;
- prove a counting identity by a bijection or a committee argument;
- use the pigeonhole principle to prove that something must exist.

HOW TO THINK ABOUT THIS

Counting is about structure, not formulas. Before reaching for $\binom{n}{k}$, ask: "what are the choices, in what order do I make them, and do different choice-paths ever produce the same object?" If you can describe how to *build* each object exactly once, you have the count.

8.1 Sum and product rules

Two rules generate every formula in the chapter:

- **Sum rule.** If the objects split into disjoint cases of sizes a and b, there are $a + b$ objects. "Disjoint" is the licence — shared cases get double-counted.
- **Product rule.** If an object is built by a sequence of independent choices of sizes $a_1, a_2, \ldots, a_k$, there are $a_1 a_2 \cdots a_k$ objects — one per root-to-leaf path in Figure 8.1's decision tree.

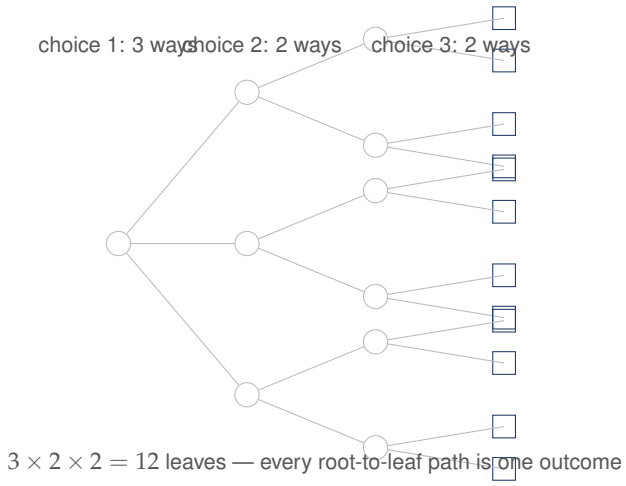


Figure 8.1: The product rule as a decision tree. Notice the multiplication is in the *shape*: each level doubles the fan-out, and leaves are outcomes — not choices. Counting *paths*, not *nodes*, is the discipline.

The mistakes are symmetric: multiplying when cases should be added ("3 shirts or 4 trousers" is $3+4$, not $3\cdot 4$), and adding where a build-process should multiply ("3 shirts then 4 trousers" is an outfit count, $3\cdot 4$). The diagnostic is always: is the second choice *conditional on* the first?

8.2 Permutations and factorials

Orderings of n distinct objects: $n!$ — the product rule applied n times (n choices for first, $n-1$ for second, …). A k-permutation (ordered selection of k from n) is $n!/(n-k)!$: the last $n-k$ slots "don't matter", and dividing by their arrangements is honest cancellation, not a trick.

Example 8.1. How many distinct arrangements does MISSISSIPPI have?

Strategy. Count as if all 11 letters were distinct, then undo the overcount: identical letters can't be told apart, so each arrangement was counted once per internal shuffle of every repeated letter.

Solution. 11 letters: one M, four I's, four S's, two P's. Naive count 11! treats the I's as distinguishable $I_1,\dots,I_4$ — each real arrangement appears 4! times (I-shuffles) $\times$ 4! (S-shuffles) $\times$ 2! (P-shuffles). So the count is $\dfrac{11!}{4!\,4!\,2!} = 34\,650$. The same division-by-shuffles gives the multinomial formula $\binom{n}{n_1,\dots,n_k}$ in general.

Exercise 8.1 (easy)**.** How many ways to award gold, silver, bronze in a 20-runner race? What if ties were allowed — does the formula still apply?

8.3 Binomial coefficients and Pascal's triangle

Definition 8.2. $\binom{n}{k}$ ("n choose k") is the number of k-element subsets of an n-element set: $\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$.

Pascal's triangle builds the table by the rule $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ — itself a sum-rule proof (the subsets either contain element n or don't). Figure 8.2 highlights the hockey-stick identity, a workhorse for summing columns.

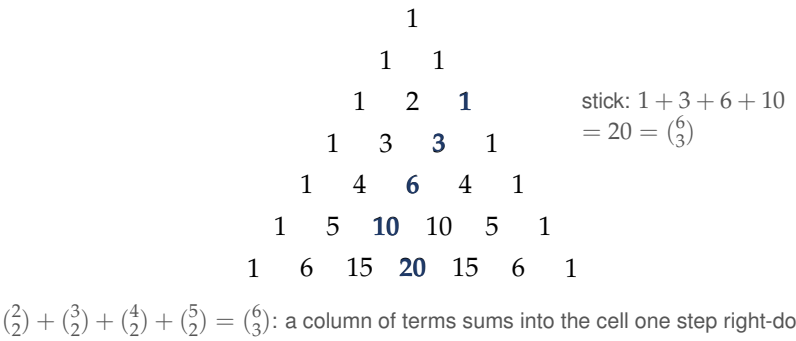


Figure 8.2: Hockey-stick in Pascal's triangle. Notice the highlighted stick $1 + 3 + 6 + 10$ summing to the cell one step right-down, 20: the identity $\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$ is the triangle's own shape.

8.4 The binomial theorem, proved two ways

Theorem 8.3 (Binomial theorem)**.** $(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}$.

Proof 1 (committee). Expanding $(x+y)^n$ is a product-rule walk: from each of the n factors pick x or y. The term $x^k y^{n-k}$ appears once for every way to choose which k factors gave x — that is the definition of $\binom{n}{k}$. *Proof 2 (induction).* Multiply $(x+y)^{n-1}$ by $(x+y)$ and collect $x^k y^{n-k}$: it arrives from $x \cdot x^{k-1}y^{n-k}$ (coefficient $\binom{n-1}{k-1}$) plus $y \cdot x^k y^{n-1-k}$ ($\binom{n-1}{k}$), and Pascal's rule completes it.

Setting $x = y = 1$ gives $\sum_k \binom{n}{k} = 2^n$ — the next example's committee argument proves the same identity directly.

Example 8.4. Prove $\sum_{k=0}^{n} \binom{n}{k} = 2^n$ by counting in two ways.

Strategy. Count the same object by two different routes and set the counts equal: a subset of $[n]$ can be counted *by size* ($\binom{n}{k}$ per size, summed) or *by membership* (each element independently in or out: 2^n). Same set, two formulas.

Solution. The left side counts subsets of $[n]$ partitioned by cardinality: $\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{n}$. The right side counts the same subsets by the product rule: n independent in/out decisions, 2^n outcomes. Equal by definition of "counting the same thing" — that *is* the proof.

Exercise 8.2 (medium)**.** Prove $\sum_k k\binom{n}{k} = n\,2^{n-1}$ the same way (count "subsets with a marked element" two ways).

8.5 Bijection is counting

Chapter 4's bijections pay rent here: a bijection between two finite sets makes their sizes equal, so *to count A, count an easier B*. That is why "counting in two ways" proves identities and why renaming the object is the first move on a hard problem.

Example 8.5. Count lattice paths on an $m \times n$ grid: paths from $(0,0)$ to (m,n) using only right-steps R and up-steps U.

Strategy. Name the bijection: a path is a word over $\{R, U\}$ with exactly m R's and n U's — length $m+n$, and the path is determined by which positions are R.

Solution. Paths $\leftrightarrow$ words of length $m+n$ with m R's is a bijection (Figure 8.3). Choosing which m of the $m+n$ positions are R determines the word, so the count is $\binom{m+n}{m}$.

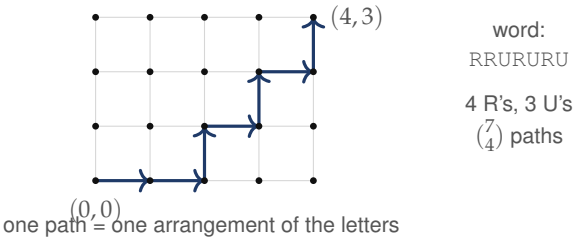


Figure 8.3: The lattice-path bijection. Notice the word RRURURU drawn as the path it encodes — the picture *is* the bijection, not an illustration of it.

8.6 Stars and bars

Theorem 8.6. *The equation $x_1 + x_2 + \cdots + x_k = n$ has $\binom{n+k-1}{k-1}$ solutions in nonnegative integers ($\binom{n-1}{k-1}$ in positive).*

Example 8.7. Solve $x_1 + x_2 + x_3 + x_4 = 12$ in nonnegative integers.

Strategy. Draw it before counting: write n stars in a row, place $k-1$ bars among the $n+k-1$ symbol positions — the bars cut the stars into k groups, and group sizes *are* the solution.

Solution. Figure 8.4 shows one arrangement: $\star\star\star|\,\star\star|\,\star\star\star\star|\,\star\star\star$ for $(3,2,4,3)$. Placing 3 bars among $12+3=15$ positions gives $\binom{15}{3} = 455$ solutions.

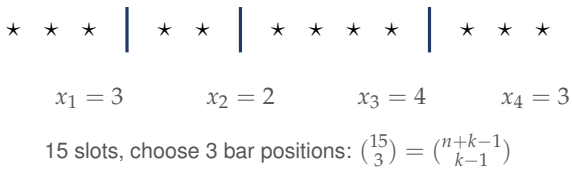


Figure 8.4: Stars and bars for $x_1 + x_2 + x_3 + x_4 = 12$. Notice what a "solution" looks like physically: $k-1$ bars carving n stars into ordered piles — order is free because the piles have names.

Positive solutions are the same picture with one star pre-committed per group: put 1 in each x_i first, distribute $n-k$ stars freely.

Exercise 8.3 (medium)**.** Count solutions of $x_1 + x_2 + x_3 = 10$ with all $x_i \geq 2$. (Shift variables first — then stars and bars.)

8.7 The pigeonhole principle

Theorem 8.8 (Pigeonhole)**.** *If $n + 1$ objects go into n boxes, some box gets at least two. More generally, N objects in k boxes put at least $\lceil N/k \rceil$ in one box.*

Trivially true — and load-bearing in olympiad proofs because the *boxes are a design choice*: partition the objects cleverly and "two in one box" becomes "two objects sharing a property".

Example 8.9. Show that among any 51 numbers from $\{1, \ldots, 100\}$, two differ by a factor of 2.

Strategy. The boxes must make "differing by a power of two" mean "same box". Every integer is (odd part) $\times$ (a power of two); numbers with the *same odd part* differ only by powers of two.

Solution. Write each number as $2^e \cdot o$ with o odd; there are exactly 50 odd parts in $\{1, \ldots, 100\}$, hence 50 boxes (Figure 8.5). 51 numbers force two into one class: $a = 2^e o$ and $b = 2^f o$ with $e \neq f$, so one divides the other by a factor of $2^{|e-f|} \geq 2$. Sharp bound: the 50 odd numbers are a 50-element set with no such pair.

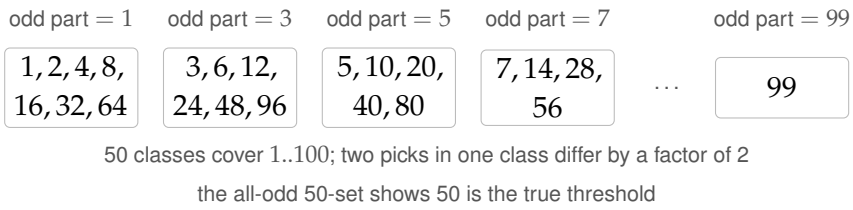


Figure 8.5: The odd-part partition of 1..100 into 50 classes. Notice the proof is in the box design: inside one class, two elements differ by a pure power of 2 — the "same box" condition was engineered to be exactly the conclusion.

Checkpoint 8.1 *Predict before you run.* Listing 8.1 checks the chapter's three closed forms against brute force. Predict which count is *not* a power of two before reading the output.

```
enum = sum(1 for w in combinations(range(m + k), m))
```

Listing 8.1: `count_check.py`: enumeration vs closed forms (full script in `code/count_check.py`).

Real output:

```
subsets: n=4
  enum=16 closed=2^4=16
lattice paths: 3x2 grid
  enum=10 closed=C(5,3)=10
stars and bars: x1+x2+x3+x4 = 6, x >= 0
```

```
  enum=84 closed=C(9,3)=84
pigeonhole demo: 51 numbers from 1..100, two differ by 2x
  all-odd set [1, 3, 5, 7, 9]...[95, 97, 99]: no pair doubles -> 50
     evades
  adding any even e makes e/2 or 2e odd-divisor-equal
```

8.8 Committee proofs

The committee argument generalises WE 8.4: an identity between two formulas is proved by exhibiting *one* set both sides count. The discipline is identical to Chapter 2's proof hygiene — name the object, then let each side count it under a different strategy. $\binom{n}{k} = \binom{n}{n-k}$: choosing a committee chooses its non-members. Vandermonde $\sum_i \binom{m}{i}\binom{n}{k-i} = \binom{m+n}{k}$: a k-committee from m women + n men splits by the number of women chosen. Every algebraic identity in this chapter has a committee behind it — finding it is the exercise.

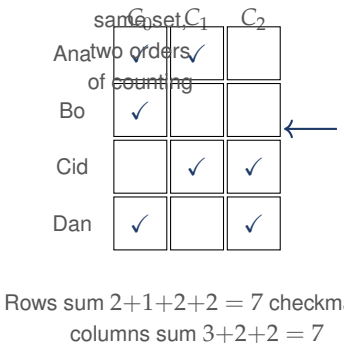


Figure 8.6: Double counting in its rawest form: memberships as a $\{\text{person}\} \times \{\text{committee}\}$ table. Row-sums and column-sums both total 7 — and that tautology, $\sum_i |C_i| = \sum_j |P_j|$, is the engine of every committee proof.

Exercise 8.4 (medium). Prove Vandermonde's identity by naming the set both sides count.

8.9 Chapter summary

8.10 Common mistakes

- **Multiply where cases add.** "Pick 1 of 3 or 1 of 4" is 7 outcomes; $3 \cdot 4$ counts *pairs*, a different object. Ask: is the second choice conditional on the first?
- **Ordered where unordered was meant.** Dealing a 5-card hand by $52 \cdot 51 \cdots 48$ counts ordered hands; each real hand appears 5! times. The fix "divide by k!" is honest here — but only when every target object was overcounted *equally*.
- **Ignoring "indistinguishable".** Stars and bars counts named piles (the x_i have labels). If the problem's piles are unnamed, the same diagram overcounts — partitions of n have no closed form, and pretending they do is the mistake.

Object	Count	Watch
ordered sequence, no repeats	$n!$, or $n!/(n-k)!$ for length k	ties need their own rule
k-subset of $[n]$	$\binom{n}{k}$	unordered: no $k!$ factor
paths with m R, n U	$\binom{m+n}{m}$	the word is the object
$x_1 + \cdots + x_k = n$, $x_i \geq 0$	$\binom{n+k-1}{k-1}$	shift for lower bounds
same set, two routes	identity for free	name the set first
N objects in k boxes	one gets $\geq \lceil N/k \rceil$	design the boxes

Table 8.1: Counting in one page: pick the model (ordered? distinct? bounded?), then let a bijection or a committee do the proving.

8.11 Additional exercises

Exercise 8.5 (easy)**.** Count 4-letter "words" from $\{a, b, c, d, e\}$: (a) any letters; (b) distinct letters; (c) distinct with no two vowels adjacent.

Exercise 8.6 (easy)**.** Evaluate $\binom{10}{3}$ and $\binom{10}{7}$ two ways each; which computation is shorter and why does symmetry predict it?

Exercise 8.7 (easy)**.** Count committees of 5 from 12 people if two specific people refuse to serve together. (Two clean cases — don't subtract blindly.)

Exercise 8.8 (easy)**.** From a standard deck: how many 5-card hands are all hearts? All one suit? Exactly 3 aces?

Exercise 8.9 (medium)**.** Prove $\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}$ by counting n-subsets of $[2n]$ a second way.

Exercise 8.10 (medium)**.** Count lattice paths $(0,0) \to (6,4)$ that pass through $(3,2)$ — then count those that avoid it. Which was easier to count, and why?

Exercise 8.11 (medium)**.** Count nonnegative solutions of $x_1 + x_2 + x_3 + x_4 = 20$ with $x_1 \leq 6$. (Inclusion-exclusion on the bad side.)

Exercise 8.12 (medium)**.** Prove: among any 6 integers, two have difference divisible by 5. Generalise: $n+1$ integers give a pair with difference divisible by n. What are the pigeons, what are the boxes?

Exercise 8.13 (medium)**.** Show that any 10 distinct two-digit numbers contain two disjoint subsets with equal sum. (Count subsets vs possible sums.)

Exercise 8.14 (hard)**.** Prove $\binom{n}{0} + \binom{n}{2} + \binom{n}{4} + \cdots = 2^{n-1}$ by a committee argument *and* by the binomial theorem (evaluate at $(1,1)$ and $(1,-1)$).

Exercise 8.15 (hard)**.** Derive the positive-solutions formula $\binom{n-1}{k-1}$ for $x_1 + \cdots + x_k = n$ directly from the diagram (bars between stars, at most one per gap), then reconcile it with the $x_i \geq 1$ shift.

Exercise 8.16 (hard). Extend `count_check.py` to enumerate *restricted* stars-and-bars solutions ($x_i \le c_i$) for $k = 3$ and verify the inclusion–exclusion count (the tool of Chapter 9) on 200 random cases.

Exercise 8.17 (easy). Count the arrangements of BANANA and of BOOKKEEPER — three double letters in a row in the second one is not a special case, just the same division-by-shuffles.

Exercise 8.18 (medium). A class of 12 students splits into three groups of 4 for projects. Count the divisions — and then recount if the groups are labelled *Red, Green, Blue*. (What does labelling multiply by?)

Exercise 8.19 (hard). Prove $\sum_{k=0}^{n} k\binom{n}{k} = n\,2^{n-1}$ two ways: once by counting (a committee plus a named chair) and once by differentiating $(1+x)^n$. Say which proof generalises to $\sum_k k^2\binom{n}{k}$ and why.

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 3 — `https://richardhammack.github.io/BookOfProof/` — free PDF; counting, factorials and the binomial theorem.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 14–17 — `https://courses.csail.mit.edu/6.042/spring18/mcs.pdf` — free PDF; the sum/product rules, pigeonhole and counting-in-two-ways framing used throughout this chapter.
- Y. Zhao, *Bijections* — `https://yufeizhao.com/olympiad/bijections.pdf` — the bijective proof drill behind §8.5–§8.8.
- Berkeley CS70 notes — `https://www.eecs70.org/` — counting and discrete probability with worked problems.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — `https://ocw.mit.edu/6-042JF10` — the counting lectures behind §8.1–§8.7.
- Cut-the-Knot, combinatorics — `https://www.cut-the-knot.org/` — committee proofs of the binomial identities in §8.3–§8.8.

WATCH INSTEAD

- **Spanning Tree — "What Is the Pigeonhole Principle?"**, 8:23, `https://www.youtube.com/watch?v=B2A2pGrDG8I` — the principle plus three real applications (chessboard, planets, compression): §8.7. (Verified 2026-10-11.)
- **TrevTutor — "Discrete Math" playlist** `https://www.youtube.com/playlist?list=PLDDGPdw7e6Ag1EIznZ-m-qXu4XX3A0cIz` — its counting unit covers §8.1–§8.6 (permutations, combinations, stars and bars). (Verified 2026-10-11.)

Notes

The two-rules-first presentation and committee-proof discipline follow Hammack's counting chapter [3]; 6.042J's counting-via- bijection sections supplied the lattice-path and "counting in two ways" framing [4]; CS70's counting notes carry the same material with more identities [13]; Zhao's bijection and counting-in-two-ways handouts are the olympiad continuation [12], and Engel's counting chapter the hard-end drill [10]. Contest items powered by this chapter — paths and stars-and-bars (AIME 2025 I P3, AIME 2024 I P6, AIME 2024 II P3), committee identities (AIME 2023 II P10, AIME 2020 I P7), pigeonhole constructions (AIME 2025 II P10–11, AIME 2022 I P6, AIME 2018 I P9–10, AIME 2010 I P10) and the distribution-flavoured items AIME 2025 I P10, AIME 2023 I P11, AIME 2021 I P4, AIME 2020 I P5, AMC 12 2024 B P16, AMC 12 2023 A P13 — are on the AoPS archive [7].

CHAPTER 9

Inclusion–exclusion and generating functions

Chapter 8 counted objects by naming them. This chapter handles the two situations where naming fails: when the condition is a *union* of overlapping bad events (count the complement by inclusion–exclusion), and when the constraint is a *sum* (track it as an exponent in a generating function, where multiplication does the bookkeeping). After this chapter you can derive derangements and surjection counts from one formula, extract φ as a PIE special case, and turn "how many ways to make n" into a coefficient extraction.

BEFORE YOU START

Chapter 8 (counting rules, binomials). *Recap:* to count "A or B" you add $|A| + |B|$ and subtract what you counted twice, $|A \cap B|$. Inclusion–exclusion is that idea for many sets at once.

AFTER THIS CHAPTER YOU CAN

- count objects with "at least one of these bad properties" using inclusion–exclusion;
- count derangements, surjections and numbers coprime to n;
- write a generating function for a counting problem and multiply two of them to combine choices;
- read a coefficient of a generating function as a count.

HOW TO THINK ABOUT THIS

When you count objects with "at least one" of several properties, directly listing them double-counts. Inclusion–exclusion corrects this like a careful accountant: add everything, subtract overlaps, add back what was subtracted too often. A generating function is a different idea: a clothesline that carries the whole sequence of answers at once, so you can combine counting problems by multiplying.

9.1 Two sets, three sets, then the formula

$|A \cup B| = |A| + |B| - |A \cap B|$: the overlap was counted twice, so subtract it once. For three sets, pairwise subtractions over-remove the triple overlap, which is added back (Figure 9.1).

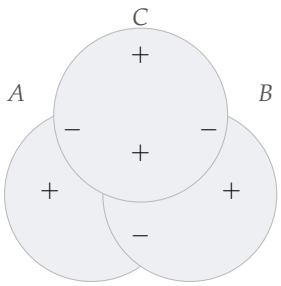
Theorem 9.1 (Inclusion–exclusion). *For finite sets $A_1, \dots, A_n$,*

$$\Big|\bigcup_{i=1}^{n} A_i\Big| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n+1} |A_1 \cap \cdots \cap A_n|.$$

Example 9.2. Prove the general PIE formula.

Strategy. Show every element's total contribution is exactly $+1$: an element in t of the sets contributes $\binom{t}{1} - \binom{t}{2} + \binom{t}{3} - \cdots = 1 - (1-1)^t = 1$ — algebra as bookkeeping. (The indicator-function version: expand $1 - \prod_i (1 - \mathbb{1}_{A_i})$.)

Solution. Fix an element x lying in exactly $t \geq 1$ of the sets. In the j-th alternating sum it is counted $\binom{t}{j}$ times, so its total weight is $\sum_{j \geq 1} (-1)^{j+1} \binom{t}{j} = 1 - \sum_{j=0}^{t} (-1)^j \binom{t}{j} = 1 - (1-1)^t = 1$. Every element contributes exactly once, hence the formula.



each element is counted $+1$ exactly once: singles $+$, pairs $-$, triple $+$

Figure 9.1: PIE's alternating correction on three sets. Notice the sign of each region equals the parity of how many sets cover it — the formula is not subtracting "mistakes" but assigning each element weight $+1$.

9.2 Derangements

Example 9.3. Count derangements of 4: permutations with no fixed point.

Strategy. Count the complement: let A_i = permutations fixing i. Then $|A_{i_1} \cap \cdots \cap A_{i_k}| = (n-k)!$ for every k-fold intersection — the bad events are easy to intersect, which is the whole reason PIE works here.

Solution. Figure 9.2 shows the bar stack: $D_4 = 4! - \binom{4}{1}3! + \binom{4}{2}2! - \binom{4}{3}1! + \binom{4}{4}0! = 24 - 24 + 12 - 4 + 1 = 9$. Generally $D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!} \approx n!/e$ — about a third of all permutations are derangements.

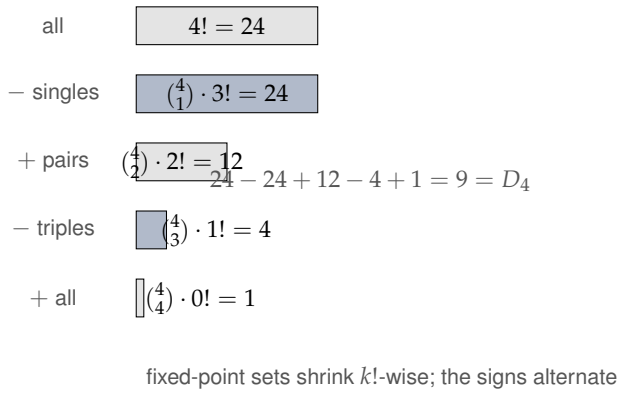


Figure 9.2: PIE for D_4 as a bar stack. Notice the k-fold intersections shrink factorially while their counts $\binom{4}{k}$ grow — the two effects nearly cancel at the top, which is why derangements stay a constant fraction of $n!$.

9.3 Surjections

Same machine, different bad events: onto maps $[n] \to [k]$ are counted by excluding the maps that miss a chosen point. A_i = maps missing target i; then $|A_{i_1} \cap \cdots \cap A_{i_j}| = (k-j)^n$, giving

$$\#\{\text{surjections}\} = \sum_{j=0}^{k} (-1)^j \binom{k}{j} (k-j)^n \;=\; \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n.$$

(The second form reindexes $j \mapsto k-j$; the sign it carries is $(-1)^{k-j}$, not $(-1)^j$.)

Example 9.4. Count the surjections $[5] \to [3]$.

Strategy. PIE with bad events "miss target i" — all 3^5 functions, subtract those missing one box, add back those missing two.

Solution. $3^5 - \binom{3}{1}2^5 + \binom{3}{2}1^5 = 243 - 96 + 3 = 150$. (Figure 9.3 draws the same count at $n = 4$.) Sanity check: $150/3! = 25 = S(5,3)$ — the Stirling number, since a surjection partitions the domain into 3 unlabelled fibres then names them.

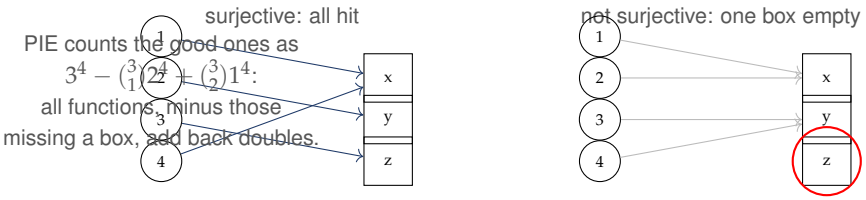


Figure 9.3: Surjections by PIE: all k^n maps minus those missing a chosen box. The empty-box case is exactly what the $j = 1$ term removes — and the $j = 2$ term restores the double-counted pair-misses.

9.4 Euler's totient by PIE

Example 9.5. Compute $\varphi(360)$ by PIE.

Strategy. Coprimality is "no prime factor shared" — the bad events are the primes $2, 3, 5$ of $360 = 2^3 \cdot 3^2 \cdot 5$: exclude multiples of each, add back pairwise multiples.

Solution. $\varphi(360) = 360 - \frac{360}{2} - \frac{360}{3} - \frac{360}{5} + \frac{360}{6} + \frac{360}{10} + \frac{360}{15} - \frac{360}{30} = 360 - 180 - 120 - 72 + 60 + 36 + 24 - 12 = 96$. This matches Chapter 7's product formula $360 \cdot \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{4}{5} = 96$ — PIE *is* the proof of that formula: the alternating expansion factors as $n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$.

9.5 Ordinary generating functions

Definition 9.6. The ordinary generating function of a sequence (a_n) is the formal series $A(x) = \sum_{n \geq 0} a_n x^n$; $[x^n]A(x)$ denotes the coefficient a_n.

"Formal" is the licence: x is never evaluated, so convergence is irrelevant — the series is a coat-hanger for the coefficients, and $[x^n]$ is extraction, not evaluation.

9.6 The catalog and the convolution rule

Workhorses: $\frac{1}{1-x} = \sum x^n$; $\frac{1}{1-ax} = \sum a^n x^n$; $\frac{1}{(1-x)^2} = \sum (n+1)x^n$; $\frac{1}{1-x^k}$ counts multiples of k; $(1+x)^n = \sum \binom{n}{k} x^k$ (the binomial theorem is a GF). The key move: **multiplication convolves** — if $C = A \cdot B$ then $[x^n]C = \sum_k a_k b_{n-k}$, which is exactly the sum rule applied to splitting a total between two independent parts.

9.7 GFs for counting

Example 9.7. Count ways to make 30 with coins of $5, 10, 25$ (order irrelevant, unlimited supply).

Strategy. Model each coin type as the geometric factor $1 + x^c + x^{2c} + \cdots = \frac{1}{1-x^c}$ — choosing how many of that coin. The product's coefficient $[x^{30}]$ counts triples (a, b, c) with $5a + 10b + 25c = 30$: the convolution does the bookkeeping.

Solution. $\frac{1}{(1-x^5)(1-x^{10})(1-x^{25})} = 1 + x^5 + 2x^{10} + 2x^{15} + 3x^{20} + 4x^{25} + 5x^{30} + \cdots$ (Figures 9.4, 9.5), so $[x^{30}] = 5$. Listing 9.1 computes this by truncated-series multiplication and brute-force verifies it.

Checkpoint 9.1 *Predict before you run.* Before looking at Listing 9.1's output: predict whether $[x^{30}]$ for coins $\{5, 10, 25\}$ is odd or even, and name one of the five solutions.

```
for c in coins:
    ng = g[:] # convolve with the coin's series
```

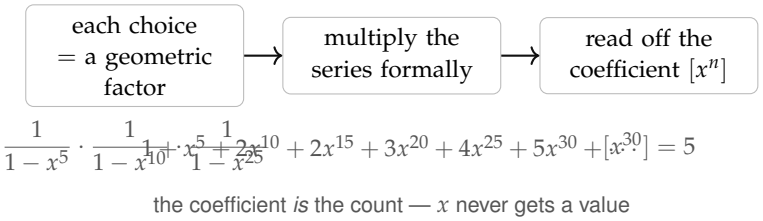


Figure 9.4: Coefficient extraction as a pipeline. Notice x is a formal placeholder — the coefficient *is* the count, and no value is ever substituted.

factor	$[x^0]$	$[x^5]$	$[x^{10}]$	$[x^{15}]$	$[x^{20}]$	$[x^{25}]$	$[x^{30}]$
$1/(1-x^5)$	1	1	1	1	1	1	1
$\times\, 1/(1-x^{10})$	1	1	2	2	3	3	4
$\times\, 1/(1-x^{25})$	1	1	2	2	3	4	**5**

Figure 9.5: Partial products for the coin GF through x^{30}. Notice each new factor adds the previous row shifted by the coin's step — the table *is* the convolution, run by hand.

```
    for k in range(c, n + 1, c):
        for j in range(n - k + 1):
            ng[j + k] += g[j]
```

Listing 9.1: `gf_coin.py`: GF coefficients by truncated-series multiplication + brute-force check (script in `code/gf_coin.py`).

Real output:

```
GF product for coins {5,10,25}, terms through x^30:
  coefficients [x^0..x^30] nonzero: {0: 1, 5: 1, 10: 2, 15: 2, 20: 3,
     25: 4, 30: 5}
[x^30] by GF: 5
[x^30] by brute force: 5
```

Constrained integer solutions are the same shape: $x_1 + x_2 + x_3 = n$ with $x_i \le c_i$ is $[x^n]$ of $\prod_i (1 + x + \cdots + x^{c_i}) = \prod_i \frac{1-x^{c_i+1}}{1-x}$ — the stars-and-bars count corrected by PIE, with the correction factors already built into the numerator.

9.8 GFs meet recurrences

A recurrence is a GF equation: Fibonacci $f_n = f_{n-1} + f_{n-2}$ translates to $F(x) = x + xF(x) + x^2F(x)$, hence $F(x) = \frac{x}{1-x-x^2}$ — coefficients recoverable by partial fractions. That is Chapter 10's machinery; the point here is the conversion habit: recurrence in, rational function out, coefficients back.

Exercise 9.1 (easy)**.** How many integers in 1..1000 are divisible by 3 or 5? By neither?

Exercise 9.2 (medium)**.** Compute D_5 from the general formula; check that $D_5/5!$ is closer to $1/e$ than $D_4/4!$ was.

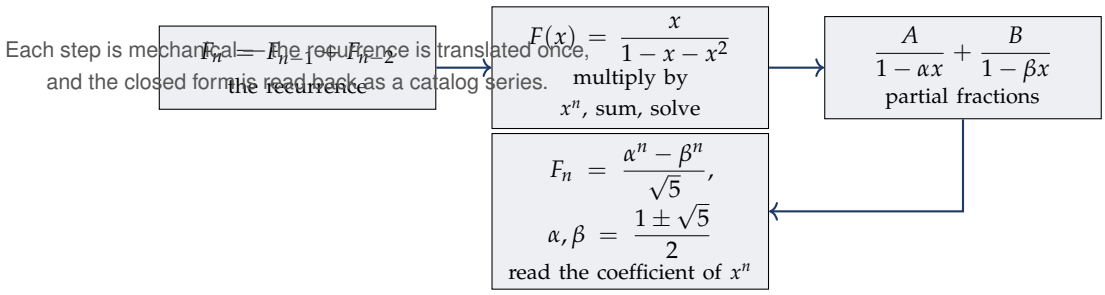


Figure 9.6: The three-station pipeline: a recurrence becomes a rational function by algebra, partial fractions split it into catalog pieces, and the coefficient of x^n is read off as a closed form. Every step is mechanical — the creativity is choosing to convert at all.

Exercise 9.3 (medium)**.** What is $[x^{12}]$ of $\frac{1}{(1-x^2)(1-x^3)}$? Interpret it as a coin count.

Exercise 9.4 (medium)**.** Count surjections $[4] \to [3]$ and verify against 3^4 minus the miss-a-point cases by hand.

9.9 Chapter summary

Want	Tool	Shape
union of bad events	PIE	alternating intersections
no fixed point	derangement	$n!\sum(-1)^k/k!$
onto maps $[n] \to [k]$	surjection count	$\sum(-1)^j\binom{k}{j}j^n$
coprimes in $[n]$	φ via PIE	$n\prod_{p\mid n}(1-1/p)$
constrained sums	generating function	$[x^n]\prod(\cdot)$
recurrences	GF algebra	solve for F, extract $[x^n]$

Table 9.1: One page: bad events → PIE; sum constraints → coefficient extraction; recurrences → GF algebra.

9.10 Common mistakes

- **Stopping PIE at single sets.** $|A \cup B \cup C| \neq |A| + |B| + |C|$ even when pairs are subtracted — the triple overlap must come back. Sign discipline: subtract intersections of *even* order, add *odd* order, and check a single element's weight is $+1$.
- **Treating $[x^n]$ as evaluation.** $[x^{30}]$ is the coefficient of the series, not the value at $x = 30$. GFs are formal: x is a bookkeeping placeholder, never a number.
- **Reading the wrong coefficient.** After multiplying factors, the answer lives at the *combined* exponent — $[x^n]$ of the product, where n is the total being made, not at any factor's own exponent.

9.11 Additional exercises

Exercise 9.5 (easy)**.** Of 200 students, 120 take maths, 90 take physics, 50 take both. How many take neither?

Exercise 9.6 (easy)**.** Compute $\varphi(1000)$ by PIE (prime factors $2, 5$) and check against Chapter 7's product formula.

Exercise 9.7 (easy)**.** Extract $[x^6]$ from $\frac{1}{1-x} \cdot \frac{1}{1-x^3}$ and name the counting problem it answers.

Exercise 9.8 (easy)**.** List the 9 derangements of 4 elements explicitly (or write the generator) and verify a sample of three fix nothing.

Exercise 9.9 (medium)**.** Count ways to make 40 with coins $\{5, 10, 25\}$ using the partial-product method; verify against `gf_coin.py`.

Exercise 9.10 (medium)**.** Prove the derangement recurrence $D_n = (n-1)(D_{n-1} + D_{n-2})$ directly (where does element 1 go?), then reconcile with the PIE formula.

Exercise 9.11 (medium)**.** Show $\varphi(n) = n \prod_{p \mid n}(1 - 1/p)$ by PIE over the distinct prime factors — why do only the *distinct* primes enter?

Exercise 9.12 (medium)**.** Count solutions of $x_1 + x_2 + x_3 = 12$ with each $x_i \le 5$, once by PIE-on-stars-and-bars and once as $[x^{12}](1 + x + \cdots + x^5)^3$.

Exercise 9.13 (hard)**.** Prove the surjection formula $\sum_j (-1)^j \binom{k}{j} j^n$ gives 0 when $n < k$ — explain combinatorially why it *must*.

Exercise 9.14 (hard)**.** Derive the GF for partitions into distinct parts and into odd parts, and prove the two counts are equal by manipulating the products.

Exercise 9.15 (hard)**.** Extend `gf_coin.py` to print the coefficient table for any coin list and any n, then use it to find the largest amount that cannot be made with $\{6, 10, 15\}$ (answer: 29; prove it is largest by induction on residues mod 5 — and first check why the question only makes sense when gcd $= 1$).

Exercise 9.16 (easy)**.** How many integers in 1..360 are coprime to 360? Count by PIE and check against $\varphi(360) = 96$ from Example 9.5.

Exercise 9.17 (medium)**.** A club has 10 members; in how many ways can three disjoint committees of $2, 3, 4$ be formed (the leftovers form no committee)? Then recount if the committees are labelled A, B, C.

Exercise 9.18 (medium)**.** Compute the GF of the sequence $a_n =$ number of nonnegative solutions of $x_1 + 2x_2 + 5x_3 = n$ as a product, and extract $[x^{10}]$ by systematic coefficient bookkeeping.

Exercise 9.19 (hard)**.** Prove $D_n/n! = \sum_{k=0}^{n} (-1)^k/k!$ converges to $1/e$ at the rate $1/(n+1)!$: bound the tail of the series, and say how large n must be before rounding $n!/e$ gives D_n exactly.

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 3 — https://richardhammack.github.io/BookOfProof/ — free PDF; inclusion–exclusion as a counting complement.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 15 — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; PIE, derangements and generating functions.
- E. Chen, *Summation* — the GF and coefficient-extraction viewpoint behind §9.5–§9.8: https://web.evanchen.cc/handouts/Summation/Summation.pdf
- Y. Zhao, *Double Counting* — olympiad-grade PIE and two-ways counting: https://yufeizhao.com/olympiad/doublecounting_mop.pdf
- Berkeley CS70 notes — https://www.eecs70.org/ — counting with worked problems.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — https://ocw.mit.edu/6-042JF10 — counting and generating functions lectures.

WATCH INSTEAD

- **TrevTutor — "GENERATING FUNCTIONS"**, 18:41, https://www.youtube.com/watch?v=-drdeNMoe8w — formal power series and coefficient extraction: §9.5–§9.7. (Verified 2026-10-11.)
- **Mathispower4u — "Introduction to Generating Functions for Sequences"**, 5:37, https://www.youtube.com/watch?v=h39O4rUJPQo — the sequence-to- function idea in five minutes: §9.5. (Verified 2026-10-11.)

Notes

PIE's indicator-function proof and the derangement treatment follow Hammack's counting chapter [3] and 6.042J's counting sections [4]; the GF-as-bookkeeping viewpoint is CS70's combinatorial-proofs note [13] and Chen's summation handout's GF section [15]; Engel supplies the hard-end drill [10]. Contest items powered by this chapter — PIE counts (AIME 2024 II P1, AIME 2010 I P7), derangements and surjections (AIME 2022 I P12–13, AIME 2015 I P12, AIME 2018 I P12), GF extractions (AIME 2023 II P7, AIME 2023 II P11, AMC 12 2023 A P24, AIME 2025 II P8) — are on the AoPS archive [7].

CHAPTER 10

Recurrences

A recurrence describes a sequence by its own past — the natural output of the case splits from Chapter 8 and Chapter 3. This chapter is about turning that description into either a closed form (characteristic equations, generating functions) or at least a growth order. After this chapter you can set up a recurrence from a problem's own structure, solve the linear homogeneous ones, and predict how fast the answer grows.

BEFORE YOU START

Chapters 3 (induction) and 8–9 (counting, generating functions). *Recap:* a recurrence defines each term from earlier ones, like $F_n = F_{n-1} + F_{n-2}$; a closed form gives F_n directly.

AFTER THIS CHAPTER YOU CAN

- guess a closed form from the first few terms and prove it by induction;
- solve linear recurrences with the characteristic equation;
- set up a recurrence for a tiling or counting problem;
- estimate how fast a sequence grows.

HOW TO THINK ABOUT THIS

A recurrence tells you how to get the next term from earlier ones, which is exactly how you would explain the pattern to a friend. The goal is to turn that step-by-step description into a direct formula. Guess from small cases, then *prove* the guess by induction; do not trust a pattern until it is proved.

10.1 Recurrence vs closed form

A *recurrence relation* gives a_n in terms of earlier terms plus base cases; a *closed form* gives a_n as a function of n directly. The recurrence is usually what the problem gives you for free; the closed form is what you earn by solving it. Both are honest answers — but a closed form that was never verified against the recurrence is just a conjecture.

10.2 Guessing, then proving

The standard pipeline: unroll or compute a few values, conjecture a closed form, then prove it by induction — the guess supplies the formula, induction supplies the licence.

Example 10.1. Solve the Towers of Hanoi recurrence $H_n = 2H_{n-1} + 1$, $H_1 = 1$.

Strategy. Compute four values first — never conjecture from one data point. $1, 3, 7, 15$ is $2^n - 1$; the recursion tree (Figure 10.1) shows *why*: each level doubles, and the $+1$ is the single real move of the biggest disk.

Solution. Claim $H_n = 2^n - 1$. Base: $H_1 = 1 = 2^1 - 1$. Step: $H_n = 2H_{n-1} + 1 = 2(2^{n-1} - 1) + 1 = 2^n - 1$ by the induction hypothesis. Unwrapping gives the same sum directly: $H_n = 1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1$.

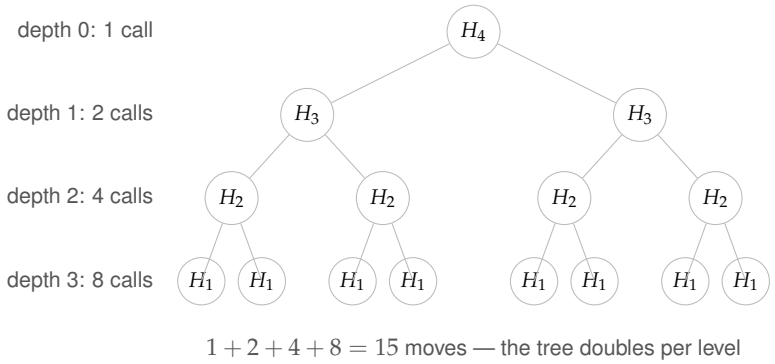


Figure 10.1: The Hanoi recursion tree, four levels deep ($1 + 2 + 4 + 8$ calls). Notice the shape is the proof: $1 + 2 + 4 + 8$ calls, one real move per call — the doubling you see *is* the $2^n - 1$.

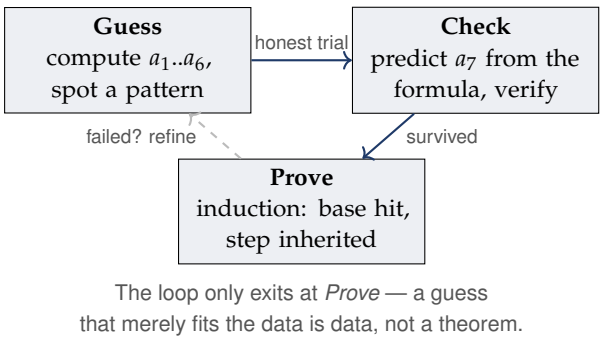


Figure 10.2: The honest pipeline: a computed guess is *evidence*, the check is a filter, and induction is what converts pattern into theorem. Skipping the loop is how false patterns get published.

10.3 Characteristic equations

For $a_n = p\,a_{n-1} + q\,a_{n-2}$, guess $a_n = r^n$ (geometric ansatz — geometric sequences are the only ones where each term is a constant ratio of the previous two, and the recurrence is asking exactly that question). Substituting gives $r^n = p\,r^{n-1} + q\,r^{n-2}$, i.e. $r^2 = pr + q$ — the *characteristic equation*.

Example 10.2. Solve $a_n = 5a_{n-1} - 6a_{n-2}$ with $a_0 = 2$, $a_1 = 5$.

Strategy. The ansatz r^n reduces the recurrence to algebra; two roots mean the general solution is a combination, and the base cases pin down the weights.

Solution. $r^2 - 5r + 6 = 0$ gives $r = 2, 3$ (Figure 10.3). $a_n = A \cdot 2^n + B \cdot 3^n$; $a_0 = A + B = 2$, $a_1 = 2A + 3B = 5$ gives $A = B = 1$, so $a_n = 2^n + 3^n$. Check: $a_2 = 5 \cdot 5 - 6 \cdot 2 = 13 = 4 + 9$.

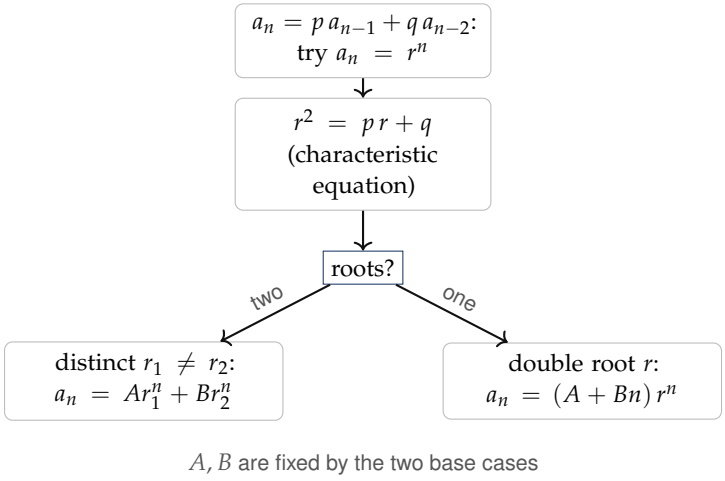


Figure 10.3: The characteristic-equation flowchart. Notice a double root doesn't halve the answer — it buys an extra n factor: $(A + Bn)r^n$, because nr^n is a second independent solution.

10.4 Fibonacci, tilings, and derangements

Example 10.3. Count domino tilings of a $2 \times n$ strip.

Strategy. Split on the *last* tile's shape, not the first — the suffix of a tiling is a shorter tiling, which is exactly the form a recurrence needs.

Solution. Figure 10.4: tilings of $2 \times n$ end either in a vertical domino (the rest is a $2 \times (n-1)$ tiling, a_{n-1} ways) or in a horizontal pair (a_{n-2} ways). So $a_n = a_{n-1} + a_{n-2}$ with $a_1 = 1$, $a_2 = 2$ — the Fibonacci numbers shifted, and Figure 10.5 shows the ratio converging to $\varphi = \frac{1+\sqrt{5}}{2}$. Binet's formula $F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$ (with $\psi = \frac{1-\sqrt{5}}{2}$) is exactly the two-root case of §10.3.

Example 10.4. Prove the derangement recurrence $D_n = (n-1)(D_{n-1} + D_{n-2})$.

Strategy. Classify derangements by where element 1 lands: it goes to one of $n-1$ positions k; then ask where element k goes.

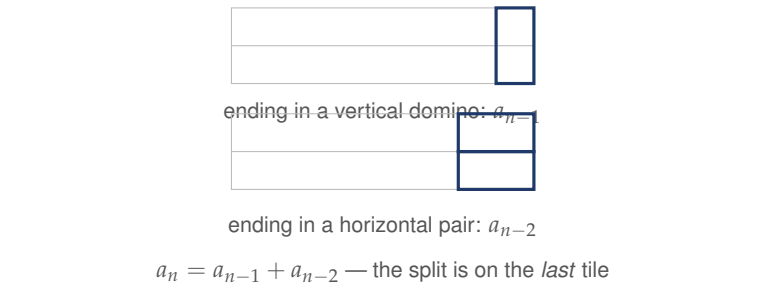


Figure 10.4: The tiling split on the last domino. Notice only two endings are possible and they account for disjoint, exhaustive cases — that is what licences $a_n = a_{n-1} + a_{n-2}$.

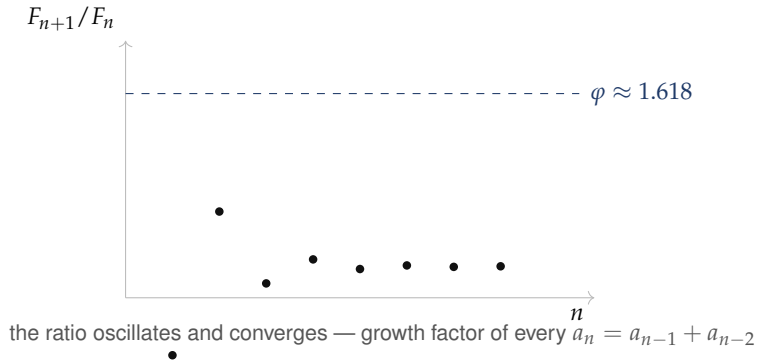


Figure 10.5: $F_{n+1}/F_n \to \varphi$. Notice the oscillation about the limit — the shrinking second root $\psi \approx -0.618$ flips sign each step, which is the oscillation made visible.

Solution. Fix where 1 lands: $n-1$ choices of position k. Now element k either lands in position 1 (the rest is a derangement of the remaining $n-2$ elements, D_{n-2} ways) or anywhere else (element k still can't go home — treat position 1 as its forbidden spot; the rest is a derangement of $n-1$ elements, D_{n-1} ways). Total $(n-1)(D_{n-1} + D_{n-2})$.

Checkpoint 10.1 *Predict before you run.* Listing 10.1 runs the recurrence against Binet's formula in floating point. Predict: does the closed form stay exactly right in floats, and if not, where does it first break?

```
def fib_binet(n):
    return (PHI ** n - PSI ** n) / sqrt(5)
```

Listing 10.1: `fib_closed.py`: recurrence vs Binet — watch the floats (script in `code/fib_closed.py`).

Real output:

```
n | recurrence | Binet (float) | round | ok?
 10 | 55 | 5.500000e+01 | 55 | True
 20 | 6765 | 6.765000e+03 | 6765 | True
 40 | 102334155 | 1.023342e+08 | 102334155 | True
 50 | 12586269025 | 1.258627e+10 | 12586269025 | True
 60 | 1548008755920 | 1.548009e+12 | 1548008755920 | True
 70 | 190392490709135 | 1.903925e+14 | 190392490709135 | True
 71 | 308061521170129 | 3.080615e+14 | 308061521170130 | False
```

```
 72 | 498454011879264 | 4.984540e+14 | 498454011879265 | False
first float failure at n = 71 (double precision; exact answer needs
   ints)
```

Exact in real arithmetic, not in floats: Binet drifts by $n = 71$ in double precision. The closed form is a theorem about reals; the recurrence on integers never lies.

10.5 Solving by generating functions

Chapter 9's conversion, applied: for Fibonacci let $F(x) = \sum F_n x^n$. The recurrence $F_n = F_{n-1} + F_{n-2}$ multiplied out gives $F(x) = x + xF(x) + x^2F(x)$, so $F(x) = \frac{x}{1-x-x^2}$; partial fractions on the denominator's roots φ, ψ recover Binet. The GF turns "solve a recurrence" into "solve one equation, then do algebra".

Example 10.5. Solve $a_n = 3a_{n-1} + 2$, $a_0 = 1$, by generating functions.

Strategy. One conversion: write $A(x) = \sum a_n x^n$, multiply the recurrence by x^n and sum $n \geq 1$, then solve the linear equation for A and extract coefficients from a partial-fraction split.

Solution. $\sum_{n\geq 1} a_n x^n = 3\sum_{n\geq 1} a_{n-1}x^n + 2\sum_{n\geq 1} x^n$ gives $A - 1 = 3xA + \frac{2x}{1-x}$, so $A(x) = \frac{1+x}{(1-x)(1-3x)} = \frac{2}{1-3x} - \frac{1}{1-x}$. Reading coefficients: $a_n = 2 \cdot 3^n - 1$. Check: $a_1 = 5 = 2 \cdot 3 - 1$ and $a_2 = 17 = 2 \cdot 9 - 1$; the recurrence holds since $3(2 \cdot 3^{n-1} - 1) + 2 = 2 \cdot 3^n - 1$.

10.6 A feel for growth orders

You can often read the growth order off the recursion tree without solving: Hanoi splits one call into two calls each one size smaller — 2^n. A recurrence like $T(n) = 2T(n/2) + n$ (merge sort) splits into two half-size problems; the tree has $\log_2 n$ levels of n work each, so $T(n) \approx n \log_2 n$. Chapter-17's algorithms live in this vocabulary; the discipline is to see the *tree* before reaching for the *formula*.

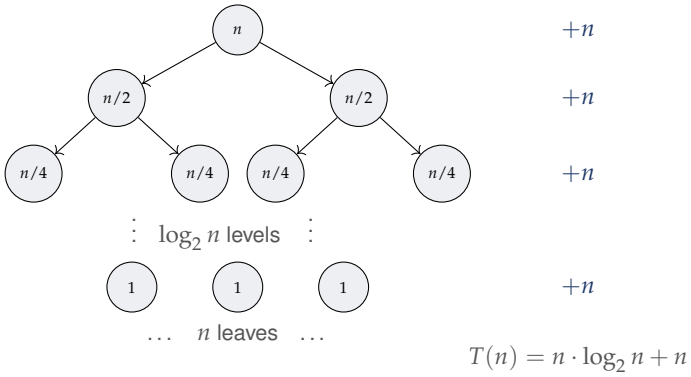


Figure 10.6: The recursion tree of $T(n) = 2T(n/2) + n$: $\log_2 n$ levels, n units of work per level. The closed form is read off the shape — $n \log_2 n$, before any formula is invoked.

Exercise 10.1 (easy)**.** Unroll $b_n = 3b_{n-1}$, $b_0 = 2$; closed form? Verify b_4.

Exercise 10.2 (medium). Solve $a_n = 4a_{n-1} - 4a_{n-2}$, $a_0 = 1$, $a_1 = 4$ (double root — use Figure 10.3's second branch).

Exercise 10.3 (medium). Set up the recurrence for climbing an n-stair taking 1 or 2 steps, solve it, and say which sequence this is.

Exercise 10.4 (medium). From `fib_closed.py`'s table: why does the float failure show up only near $n \approx 70$, not earlier? (How many significant digits does a double carry?)

10.7 Chapter summary

Shape	Method	Growth
$a_n = c\,a_{n-1} + d$	unroll / geometric	c^n
$a_n = pa_{n-1} + qa_{n-2}$	characteristic eq.	$r_{\max}^n$
first-order, side term, $c = 1$	unroll + sum	like the side term
split on last object	set up + solve	per the roots
linear, any order	generating function	partial fractions
halving splits	recursion tree	$n \log n$-ish

Table 10.1: One page: name the shape, pick the method, predict the growth.

10.8 Common mistakes

- **Off-by-one base cases.** $a_1 = 1, a_2 = 2$ for tilings is not $F_1 = 1, F_2 = 1$: the answer is Fibonacci *shifted*. Solve with the problem's own bases, then reconcile the indices.
- **Circular proof.** "Assume the closed form, plug it into itself" proves nothing — the induction must verify the form *against the recurrence*, not against itself.
- **Formula where the count was the answer.** A closed form that doesn't evaluate to integers (Binet in floats past $n \approx 70$) is a liability; when the question wants a count, keep the integers and treat the formula as a description of growth.

10.9 Additional exercises

Exercise 10.5 (easy). Compute a_5 for $a_n = 2a_{n-1} + 3$, $a_0 = 1$; then find the closed form (hint: $a_n + c$ geometric for the right c).

Exercise 10.6 (easy). Write the recurrence for "binary strings with no two adjacent 1s" of length n and identify the sequence.

Exercise 10.7 (easy). Solve $a_n = 7a_{n-1}$, $a_0 = 5$; what is the growth factor?

Exercise 10.8 (easy). Solve $a_n = a_{n-1} + n$, $a_0 = 0$ by unwrapping; name the closed form.

Exercise 10.9 (medium). Solve $a_n = 6a_{n-1} - 9a_{n-2}$, $a_0 = 0$, $a_1 = 3$.

Exercise 10.10 (medium)**.** Show the $2 \times n$ tiling count with dominoes *and* 2×2 squares allowed satisfies $b_n = b_{n-1} + 2b_{n-2}$, and solve it.

Exercise 10.11 (medium)**.** Use the derangement recurrence to compute D_5, D_6 from $D_3 = 2, D_4 = 9$, and check $D_6/6!$ against $1/e$.

Exercise 10.12 (medium)**.** Find the GF for $a_n = 2a_{n-1} + 3^n$, $a_0 = 1$: set $A(x)$, solve for it, and extract the closed form by partial fractions.

Exercise 10.13 (hard)**.** Prove Binet's formula by induction *and* derive it from $F(x) = x/(1 - x - x^2)$ — show the two answers coincide term by term.

Exercise 10.14 (hard)**.** Extend `fib_closed.py` to use `Decimal` at 50 digits: does Binet's failure move to a larger n, and does *exact* arithmetic (algebraic form) ever fail? Explain.

Exercise 10.15 (easy)**.** Solve $a_n = 2a_{n-1} + 1$, $a_0 = 0$, three ways: unrolling, the guess-check-prove loop, and the GF of Example 10.5.

Exercise 10.16 (medium)**.** Solve $a_n = a_{n-1} + n$, $a_0 = 1$ (arithmetic forcing term — which catalog GF produces n?).

Exercise 10.17 (medium)**.** A path on a staircase of n stones uses steps of 1 or 3. Set up and solve the recurrence for $n \leq 9$; where does the characteristic equation's missing root enter?

Exercise 10.18 (medium)**.** Draw the recursion tree of $T(n) = 3T(n/2) + n$ and read off the growth order without a formula. Compare against $2T(n/2) + n$ — what does the branching factor do to the level sums?

Exercise 10.19 (hard)**.** Solve the nonhomogeneous $a_n = 4a_{n-1} - 4a_{n-2} + 2^n$, $a_0 = a_1 = 1$ (resonance: the forcing term is itself a solution of the homogeneous part — what replaces the usual ansatz?).

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 21 — `https://richardhammack.github.io/BookOfProof/` — free PDF; induction on sequences and recurrences.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 22 — `https://courses.csail.mit.edu/6.042/spring18/mcs.pdf` — free PDF; recurrences, characteristic equations and the recursion-tree method.
- E. Chen, *Summation* — generating-function solutions of linear recurrences: `https://web.evanchen.cc/handouts/Summation/Summation.pdf`
- Y. Zhao, *Summations and Combinatorial Identities* — olympiad recurrences and the identities they generate: `https://yufeizhao.com/olympiad/comb1.pdf`
- MIT OpenCourseWare, *6.042J* (Fall 2010) — `https://ocw.mit.edu/6-042JF10` — the recurrences lectures this chapter follows.
- Cut-the-Knot, Fibonacci — `https://www.cut-the-knot.org/` — alternate derivations of Binet's formula for comparison with §10.5.

WATCH INSTEAD

- **TrevTutor — "RECURRENCE RELATIONS"**, 15:24, https://www.youtube.com/watch?v=eAaP4XaB8hM — Fibonacci and geometric recurrences, setting up the §10.2–§10.3 pipeline. (Verified 2026-10-11.)
- **TrevTutor — "HOMOGENEOUS RECURRENCE RELATIONS"**, 25:15, https://www.youtube.com/watch?v=7mhvA5L7KqY — characteristic equations worked end to end: §10.3–§10.4. (Verified 2026-10-11.)

Notes

The recursion-tree viewpoint and the characteristic-equation pipeline follow 6.042J's recurrences chapter [4]; the GF-solution section uses Chen's Summation handout [15] as the worked source; Velleman's recursion section supplied the guess-then-induct pipeline [2], and Engel the drill level [10]. Contest items powered by this chapter — linear recurrences (AMC 12 2024 A P21, AMC 12 2024 B P18, AIME 2021 I P10), tiling recurrences (AIME 2025 II P13, AMC 12 2023 A P20, AMC 12 2023 A P25) and combined counting-recurrence items (AIME 2015 I P9, AIME 2023 I P10) — are on the AoPS archive [7].

CHAPTER 11

Graphs, trees and planarity

A graph is Chapter 4's relation made drawable: vertices, edges, and nothing else — yet from that skeleton come the oldest theorem of network reasoning (handshake), the simplest connectivity story (trees), Euler's walk through Königsberg, and a counting formula that forbids whole graphs from living in the plane. After this chapter you can model a problem as a graph, run degree-parity arguments, and use Euler's formula as a non-planarity weapon.

BEFORE YOU START

Chapters 2–4 (proofs and sets). *Recap:* a *graph* is a set of vertices and a set of edges, each edge a pair of vertices. **Book 1, Chapter 7** implements the same objects in code.

AFTER THIS CHAPTER YOU CAN

- prove the handshake lemma and use degree counting in a proof;
- characterise bipartite graphs and trees and prove the equivalent definitions;
- decide whether a graph has an Eulerian trail;
- use Euler's formula to prove that a graph cannot be drawn without crossings.

HOW TO THINK ABOUT THIS

A graph strips a situation down to dots and lines, and the theorems here are about what that skeleton forces. Handshake counting shows how much a single trick (count the same thing two ways) can do. When you meet a graph proof, ask what is being counted, and whether you can count it once from each side.

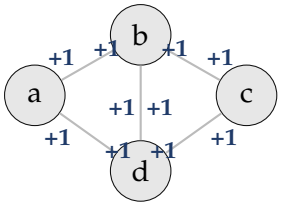
11.1 Graphs and the handshake lemma

Definition 11.1. A graph $G = (V, E)$ is a set of vertices with edges as unordered pairs $\{u, v\}$; $\deg(v)$ counts the edges at v.

Example 11.2. Prove that at any party, an even number of people shook an odd number of hands.

Strategy. Count edge-ends two ways — the double-counting habit from Chapter 8: each edge contributes exactly 2 to the degree sum (Figure 11.1).

Solution. $\sum_v \deg(v) = 2|E|$ (each edge has two ends). An odd number of odd terms can't sum to the even right side, so the number of odd-degree vertices is even.



$\sum_v \deg(v) = 2|E|$: five edges, ten ends counted

Figure 11.1: The handshake lemma is a double count. Notice every edge carries two $+1$ markers — the sum of degrees is $2|E|$ because the count is per endpoint, not per edge.

11.2 Walks, paths, cycles, connectivity

A *walk* allows repeats; a *path* forbids repeated vertices; a *cycle* returns to its start (length ≥ 3). A graph is *connected* when every pair is joined by a path; the maximal connected pieces are its *components*. The terms differ in exactly what repetition is allowed — a frequent exam-time collision.

11.3 Bipartite graphs

A graph is bipartite iff V splits into two independent classes — equivalently, iff it contains *no odd cycle* (an odd cycle needs a third colour; a 2-colouring propagates consistently when no odd cycle exists to break it).

Example 11.3. Decide whether the cube graph Q_3 (vertices = binary strings of length 3, edges = single-bit flips) is bipartite.

Strategy. Either produce a 2-colouring or exhibit an odd cycle — the theorem says exactly one will exist. Try the obvious candidate first: parity of the string's weight.

Solution. Colour a string by the parity of its number of 1s: $\{000, 011, 101, 110\}$ vs $\{001, 010, 100, 111\}$. Every edge flips exactly one bit, hence flips parity — so every edge crosses the classes, and Q_3 is bipartite. (Generalised: every hypercube Q_d is bipartite by weight parity; and the same weight-parity trick settles the grid graphs $P_m \times P_n$.)

11.4 Trees

A tree is connected and acyclic. Every tree on $n \geq 2$ vertices has a leaf (a degree-1 vertex): start at any vertex, walk without repeating. The tree is acyclic, so the walk cannot revisit and must stop; the endpoint has degree 1 — an unvisited neighbor would extend the walk, a visited one would close a cycle.

Example 11.4. Prove a tree on n vertices has exactly $n - 1$ edges.

Strategy. Remove the extremal object: a leaf exists, deleting it keeps the graph a tree, and the count drops by exactly one — induction with the induction doing real work (Figure 11.2).

Solution. Induct on n; $n = 1$ gives 0 edges. For $n \geq 2$ pick a leaf ℓ: $T - \ell$ is still connected (only ℓ's single neighbour touched it) and acyclic, hence a tree on $n - 1$ vertices, so by induction it has $n - 2$ edges. Adding ℓ's edge back gives $n - 1$.

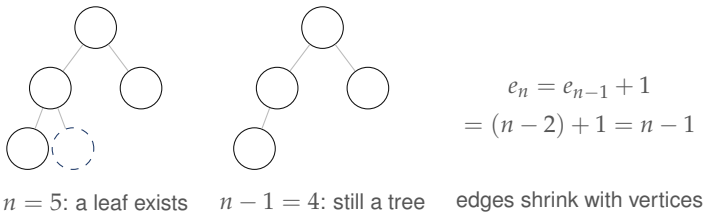


Figure 11.2: Leaf removal as the induction step. Notice deleting a leaf preserves treeness — connectedness survives because a leaf's edge was its only link.

A spanning tree of a connected graph is a tree using all its vertices: repeat edge-deletion on cycles until none remain, $n - 1$ edges minimum.

11.5 Eulerian trails

Example 11.5. Decide whether Königsberg's seven bridges admit a walk using each exactly once.

Strategy. Every *intermediate* visit to a land mass consumes two edges — one in, one out — so interior vertices need even degree; only start and end may be odd.

Solution. The multigraph of Figure 11.3 has all four vertices odd. An Eulerian trail allows exactly 0 or 2 odd vertices: no such walk exists. The same parity test decides any graph (Listing 11.1).

Checkpoint 11.1 *Predict before you run.* Listing 11.1 checks connectivity and degree parity on four graphs, printing one of three statuses. Predict which of {Königsberg multigraph, 5-cycle, 4-path} gets an Eulerian circuit versus a mere trail.

```
odd = sum(len(adj[v]) % 2 for v in range(n))
```

Listing 11.1: `euler_check.py`: DFS connectivity + odd-degree count (full script in `code/euler_check.py`).

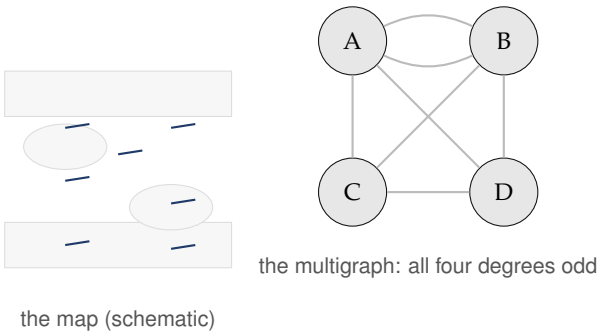


Figure 11.3: Königsberg as a multigraph. Notice the map's geometry is irrelevant — only "which land masses share a bridge" survives, which is exactly the move that turns a puzzle into graph theory.

Real output:

```
Konigsberg multigraph: no Eulerian trail
5-cycle: Eulerian circuit
path of 4: Eulerian trail (no circuit)
random 6v/8e: Eulerian trail (no circuit)
```

11.6 Planar graphs and Euler's formula

Theorem 11.6 (Euler's formula)**.** *A connected planar embedding obeys* $V - E + F = 2$.

(Figure 11.4 works it twice.) Corollaries by double-counting edge-face incidence: $E \leq 3V - 6$ for simple planar graphs ($V \geq 3$), and $E \leq 2V - 4$ when the graph is triangle-free (every face needs ≥ 4 sides).

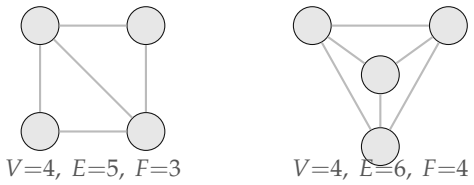


Figure 11.4: Euler's formula checked on two embeddings. Notice F counts the outside face too — the plane's "outside" is a face in every sense the formula needs.

Example 11.7. Prove $K_{3,3}$ is nonplanar.

Strategy. $K_{3,3}$ is bipartite, hence triangle-free: the sharper bound $E \leq 2V - 4$ applies. Count and contradict.

Solution. $V = 6$, $E = 9$; a bipartite planar graph needs $E \leq 2 \cdot 6 - 4 = 8 < 9$ — impossible. (For K_5: $E = 10 > 3 \cdot 5 - 6 = 9$; both failures are in Figure 11.5.)

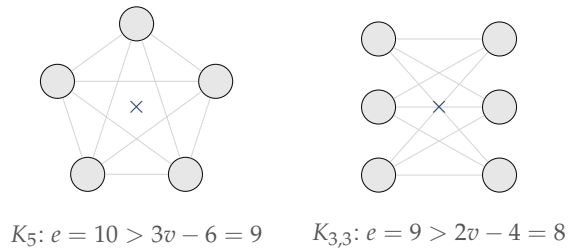


Figure 11.5: K_5 and $K_{3,3}$ with their inevitable crossings marked. Notice the proofs are edge-counts, not the drawing — the picture is intuition, Euler's formula is the licence.

11.7 Graph colouring basics

A proper colouring assigns colours so no edge is monochromatic; $\chi(G)$ is the fewest colours needed. Bipartite $\Rightarrow \chi = 2$ (the definition repackaged); odd cycles need 3; and a vertex of degree d bounds $\chi \leq d_{\max} + 1$ greedily.

Exercise 11.1 (easy)**.** A graph has degrees $3, 3, 2, 2, 1, 1$. Is it possible? Check the handshake lemma first.

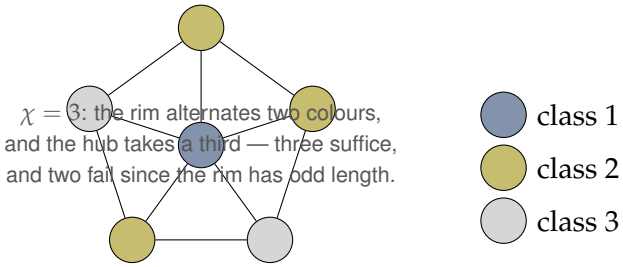


Figure 11.6: A proper 3-colouring of the 5-wheel: the rim alternates two colours and the hub takes a third. Notice the lower bound is forced — an odd rim can never be 2-coloured, so $\chi = 3$ is proved, not guessed.

Exercise 11.2 (medium)**.** Prove a tree is bipartite.

Exercise 11.3 (medium)**.** Does the graph K_4 minus one edge have an Eulerian trail? A circuit? Decide by parity, don't draw the trail.

Exercise 11.4 (medium)**.** Use Euler's formula to show every planar simple graph has a vertex of degree ≤ 5.

11.8 Chapter summary

Fact	Statement	Proof tool
handshake	$\sum \deg = 2\lvert E\rvert$	double counting
tree	n vertices $\Rightarrow n-1$ edges	leaf induction
bipartite	$\Leftrightarrow$ no odd cycle	2-colour propagation
Euler trail	conn. + ≤ 2 odd degrees	parity argument
Euler formula	$V - E + F = 2$ planar	induction on edges
planar bound	$E \leq 3V - 6$; $2V - 4$ if no $\triangle$	face-edge count

Table 11.1: One page: degrees and double counts, then Euler twice — trails by parity, planarity by formula.

11.9 Common mistakes

- **Walk $\neq$ path $\neq$ cycle.** A walk may repeat; a path may not; a cycle returns home. Many "proofs" break by silently upgrading a walk to a path.
- **Using $E \leq 3V - 6$ backwards.** Satisfying the bound does *not* prove planarity — it is a necessary condition only; nonplanarity is proved by violating it.
- **"Looks bipartite."** The property is odd-cycle-free, not visually two-sided; a twisted drawing can hide an odd cycle.

11.10 Additional exercises

Exercise 11.5 (easy)**.** Can a graph have degree sequence $5, 4, 3, 2, 1, 0$? What invariant decides before you try to draw it?

Exercise 11.6 (easy)**.** How many edges does a forest of k components on n vertices have? Generalise the tree count.

Exercise 11.7 (easy)**.** Is the cube graph (8 vertices, edges of a cube) bipartite? Eulerian?

Exercise 11.8 (easy)**.** How many spanning trees does a cycle C_n have?

Exercise 11.9 (medium)**.** Prove every connected graph with all even degrees has an Eulerian circuit (Hierholzer's idea: stitch cycles).

Exercise 11.10 (medium)**.** Show a tree with a vertex of degree d has at least d leaves.

Exercise 11.11 (medium)**.** Prove a connected graph where every vertex has degree 2 is a cycle.

Exercise 11.12 (medium)**.** The Petersen graph has $V = 10$, $E = 15$. Is it planar? (Its girth is 5 — use the right bound.)

Exercise 11.13 (hard)**.** Prove K_5 nonplanar without citing the $3V - 6$ bound — derive the bound first from $V - E + F = 2$.

Exercise 11.14 (hard)**.** Show that a planar graph and its dual have complementary edge counts, and that the dodecahedron's dual is the icosahedron (20 faces, 12 vertices).

Exercise 11.15 (hard)**.** Extend `euler_check.py` to output one actual Eulerian trail (Hierholzer's algorithm) and verify it uses each edge exactly once on 200 random even-degree graphs.

Exercise 11.16 (easy)**.** A party of 9 people: show someone shook hands with an even number of others — or show the statement is false as stated, and repair it.

Exercise 11.17 (medium)**.** Show every tree is bipartite two ways: by the odd-cycle criterion and by an explicit 2-colouring (pick a root, colour by depth parity).

Exercise 11.18 (medium)**.** A connected planar graph has $V = 8$, $E = 12$. Find F, then show no convex polyhedron can have 7 edges (use the handshake lemma on the dual — each face has ≥ 3 sides).

Exercise 11.19 (hard)**.** Prove every planar graph has a vertex of degree ≤ 5, and hence is 6-colourable by greedy induction (the warmup for the 5- and 4-colour theorems).

Further reading

- R. Hammack, *Book of Proof*, 3rd ed., Ch. 14 — https://richardhammack.github.io/BookOfProof/ — free PDF; graphs, Eulerian trails and colouring gently done.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, Ch. 10–12 — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; the handshake lemma, trees and planarity as taught here.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — https://ocw.mit.edu/6-042JF10 — the graph-theory lectures.
- Y. Zhao, *Graph Theory* — olympiad-grade problems on trees, cycles and colourings: https://yufeizhao.com/olympiad/comb3.pdf
- Berkeley CS70 notes — https://www.eecs70.org/ — free lecture notes; graphs with the algorithms continuation.
- The Open Logic Project — https://openlogicproject.org/ — for the formal graph-as- structure view beneath the pictorial one.

WATCH INSTEAD

- **Reducible — "Introduction to Graph Theory: A Computer Science Perspective"**, 16:25, https://www.youtube.com/watch?v=LFKZLXVO-Dg — definitions, terminology and why graphs matter: §11.1–§11.4. (Verified 2026-10-11.)
- **Numberphile — "A Breakthrough in Graph Theory"**, 24:56, https://www.youtube.com/watch?v=Tnu_Ws7Llo4 — graph colouring and the Hedetniemi conjecture's fall: §11.7. (Verified 2026-10-11.)

Notes

The walk/path/cycle hygiene and the planar sections follow 6.042J's graph chapters [4]; CS70's graph notes carry the same material with more algorithms [13]; Engel's graph chapter supplies the olympiad drill [10], and Cut-the-Knot the Königsberg framing [6]. Contest items powered by this chapter — graph-distance and path-state framings (AMC 12 2023 A P21, AIME 2024 I P3), cycle and polygon walks (AIME 2018 I P10, AIME 2018 I P14, AIME 2021 I P12), plus constraint graphs (AMC 12 2024 A P22, AIME 2023 I P3) — are on the AoPS archive [7].

CHAPTER 12

Probability on finite spaces

Probability on finite spaces is Chapter 8's counting with a denominator: #good/#all. The new content is what happens when events interact — conditioning, independence, and expectation — and the flagship result is that expected values add even when the events do not. After this chapter you can model a random experiment as a sample space, run conditional probabilities through a tree, and use linearity of expectation where a direct count would be hopeless.

BEFORE YOU START

Chapters 8–9 (counting, inclusion–exclusion). *Recap:* with equally likely outcomes, probability of an event is (outcomes in the event) divided by (all outcomes). So finite probability is counting wearing a different hat.

AFTER THIS CHAPTER YOU CAN

- set up a sample space and compute probabilities by counting;
- apply conditional probability and independence correctly, including when independence fails;
- compute an expected value by linearity of expectation;
- explain Simpson's paradox with a concrete table.

HOW TO THINK ABOUT THIS

Probability on finite spaces is counting in disguise: list the equally likely outcomes, count those that match. The most important habit is to be explicit about the sample space before computing anything, because most famous paradoxes come from quietly changing what the outcomes are.

12.1 Sample spaces and uniform models

A sample space Ω is the set of all outcomes; an event is a subset. Uniform model: every outcome equally likely, so $P(A) = |A|/|\Omega|$ — probability becomes combinatorics. The uniformity must be *verified* ("equally likely" is a claim, not a decoration): dealing cards uniformly is fine; summing two dice is not uniform over the sums.

12.2 Events, complements, "at least one"

$P(\overline{A}) = 1 - P(A)$. The complement move converts "at least one" into "none", and "none" factorises (Figure 12.1): $P(\text{at least one hit}) = 1 - P(\text{all misses})$.

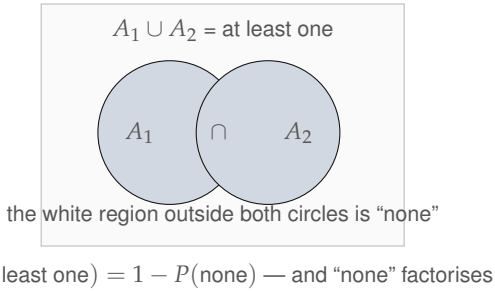


Figure 12.1: "At least one" as a complement. Notice the white region is the only part counted — and it factorises, which is why the complement is the computable side.

Example 12.1. How many people until a shared birthday is more likely than not?

Strategy. Compute the complement "all distinct" — it is a falling product, easy to evaluate; the event itself is an awkward union.

Solution. $P(\text{all distinct}) = \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365-k+1}{365}$; at $k = 23$ this first dips below $1/2$ (≈ 0.493), so the threshold is 23 people. The paradox is that pair-counting, not person-counting, drives it: $\binom{23}{2} = 253$ pairs.

12.3 Conditional probability

Definition 12.2. $P(A \mid B) = \dfrac{P(A \cap B)}{P(B)}$ for $P(B) > 0$ — the sample space shrinks to B; the four-step method is: draw the tree, compute the branch weights, take the intersection, divide.

Example 12.3. A prize draw first picks a tier — gold with probability $1/4$, plain $3/4$ — then pays out with probability $1/2$ on gold, $1/6$ on plain. Given that a prize was won, what is the chance it came from the gold tier?

Strategy. Define the event algebra before touching numbers: want $P(\text{gold} \mid \text{win})$; build the tree, weight each leaf, then divide the gold-leaf weight by the win leaves' total (Figure 12.2).

Solution. Leaves: gold-win $= \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8}$; plain-win $= \frac{3}{4} \cdot \frac{1}{6} = \frac{1}{8}$; the other two leaves (gold-lose $\frac{1}{8}$, plain-lose $\frac{5}{8}$) condition out — they are not wins. So $P(\text{gold} \mid \text{win}) = \frac{1/8}{1/8+1/8} = \frac{1}{2}$:

knowing a prize was won doubles the chance it was gold, since plain pays out at only a third of gold's rate.

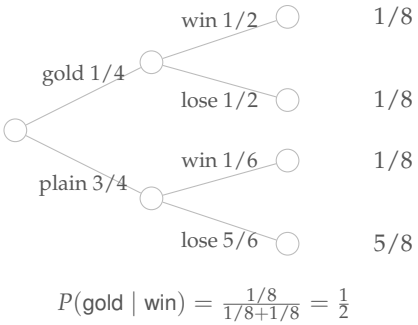


Figure 12.2: The four-step method on the prize draw. Notice the conditioning just deletes leaves: the answer lives entirely inside the win leaves' mass, and every leaf's weight is the product along its path.

12.4 Independence, and when it fails

A and B are independent iff $P(A \cap B) = P(A)P(B)$ — equivalently $P(A \mid B) = P(A)$. It is a property to *prove*, not an assumption to reach for: two people sharing a birthday is roughly independent of *another* pair sharing, but within one group the events collide. The test is always the product rule: does knowing B change A's probability?

	heads	tails	
heads	1/4	1/4	
tails	1/4	1/4	

every cell $= \frac{1}{2} \cdot \frac{1}{2}$: coin 2's outcome changes nothing about coin 1's probability

Figure 12.3: Two fair coins: the joint table factors cell-by-cell. Notice independence is a *factorisation* claim — every entry is a product of marginals, which is exactly what $P(A \cap B) = P(A)P(B)$ says.

Example 12.4. Toss two fair coins; let A = "first heads", B = "second heads", C = "outcomes agree". Show A, B, C are *pairwise* independent yet not jointly independent.

Strategy. Check the definition literally: every pair must factor, and the triple intersection must fail to factor.

Solution. $P(A) = P(B) = P(C) = 1/2$. Each pair's intersection is a single outcome: $A \cap B = \{HH\}$, $A \cap C = \{HH\}$, $B \cap C = \{HH\}$, each of probability $1/4 = (1/2)(1/2)$ — pairwise independence holds. But $A \cap B \cap C = \{HH\}$ has probability $1/4 \neq 1/8 = P(A)P(B)P(C)$. Equivalently: knowing A *and* B determines C exactly. Pairwise independence does not imply joint independence — the conjunction can carry information no single pair reveals.

12.5 Random variables and expectation

A random variable is a function $X : \Omega \to \mathbb{R}$; $E[X] = \sum_\omega X(\omega)\,P(\omega)$. The definition is deceptively plain — its power is in the next section's linearity.

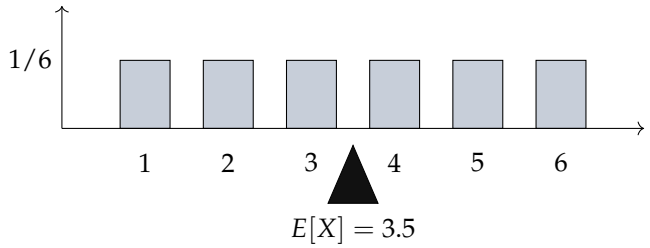


Figure 12.4: Expectation as a balance point: the pmf of a fair die balances at 3.5. Notice $E[X]$ need not be a value X ever takes — it is a centre of mass, not a prediction.

12.6 Linearity of expectation

Theorem 12.5. $E[X_1 + X_2 + \cdots + X_n] = E[X_1] + \cdots + E[X_n]$ — *always, with no independence hypothesis.*

Example 12.6. Find the expected number of fixed points of a random permutation of n elements.

Strategy. Do not count the distribution — it depends horribly on n. Write $X = \sum_i \mathbb{1}\{\sigma(i) = i\}$; each indicator has $E = 1/n$ (position i is fixed with probability $1/n$), and linearity needs nothing else (Figure 12.5).

Solution. $E[X] = \sum_{i=1}^{n} \frac{1}{n} = 1$, for every n — exact, and Listing 12.1 confirms it empirically.

σ	$\mathbb{1}_1$	$\mathbb{1}_2$	$\mathbb{1}_3$	$\mathbb{1}_4$	X
1234	1	1	1	1	4
1243	1	1	0	0	2
1324	1	0	1	0	2
2143	0	0	0	0	0
4321	0	0	0	0	0
⋮					

24 rows

column sums: each $\mathbb{1}_i$ totals $3! = 6$; $\sum X = 4 \cdot 6 = 24$; $E[X] = 24/24 = 1$

Figure 12.5: The indicator table idea ($n = 4$ shown compactly). Notice the proof is a column count: each $\mathbb{1}_i$ column sums to 3! over the 4! rows — the indicators are nowhere near independent, and linearity does not care.

Checkpoint 12.1 *Predict before you run.* Listing 12.1 simulates permutations of 6 for 10,000 trials. Predict the estimate to two decimals before reading the output — and the size of the residual.

```
total += sum(p[i] == i for i in range(n))
```

Listing 12.1: `fixed_points.py`: Monte Carlo vs exact theory (script in `code/fixed_points.py`).

Real output:

```
permutations of n = 6, 10000 Monte Carlo trials
estimated E[fixed points] = 1.0048
exact value = 1 (n * 1/n, linearity of expectation)
relative error = 0.48%
```

12.7 Simpson's paradox

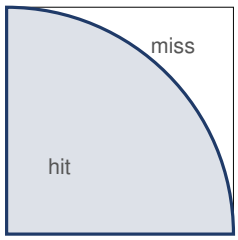
Example 12.7. Drug A beats drug B separately among mild and severe cases, yet loses overall — construct the table.

Strategy. Weighted sums on a $2 \times 2 \times 2$ table: rates can win per stratum and still lose when the groups' sizes skew the mixture. Arithmetic, not opinion.

Solution. A wins both strata — mild: $80/100 = .80$ vs B's $79/100 = .79$; severe: $20/100 = .20$ vs B's $5/50 = .10$ — yet loses the pool: A $= 100/200 = .50$ against B $= 84/150 = .56$. The trick is the mix: B's patients were mostly mild, where everyone's rate is high, so B's pooled rate rides the mild stratum's weight. Weighted means move with the weights — that is the whole paradox.

12.8 Geometric probability preview

When the space is continuous, counts become areas or volumes: a uniform point in the unit square hits the quarter-circle with probability $\pi/4$ (Figure 12.6) — the same "favourable/total" shape, measured instead of counted. The discrete habits (conditioning, complements, indicators) all carry over.



unit square, quarter-circle of radius 1

$$P(\text{hit}) = \frac{\pi/4}{1} = \frac{\pi}{4}$$ — areas replace counts

Figure 12.6: Geometric probability: the region's *area* replaces the count. Notice nothing else changes — favourable over total, just measured.

Exercise 12.1 (easy)**.** Two dice: $P(\text{sum} = 7)$? Explain why $1/11$ is wrong and what the sample space actually is.

Exercise 12.2 (medium)**.** Three fair coin flips: find $P(\text{at least one head})$ by complement, then the probability of exactly one head.

Exercise 12.3 (medium)**.** Expected number of aces in a 5-card hand — by indicators, not by the hypergeometric formula.

Exercise 12.4 (medium)**.** A test is 99% accurate on a disease with prevalence $1/1000$. Compute $P(\text{disease} \mid \text{positive})$ — the prosecutor's fallacy gap.

12.9 Chapter summary

Tool	Formula	When
uniform count	$\lvert A\rvert/\lvert\Omega\rvert$	verify uniformity first
complement	$1 - P(\overline{A})$	"at least one"
conditioning	$P(A \cap B)/P(B)$	tree + four steps
independence	$P(A)P(B)$	prove it, don't assume
linearity	$\sum E[\mathbb{1}_i]$	indicators, always
geometric	area ratio	continuous spaces

Table 12.1: One page: counting with a denominator, plus the three rules that survive when events interact.

12.10 Common mistakes

- **Assuming independence.** "Looks unrelated" is not a proof; the product rule is a theorem-level claim that must be verified. The birthday paradox is exactly the price of assuming it.
- **Adding where complements apply.** $P(A \cup B)$ is not $P(A) + P(B)$ in general — that is Chapter 9's PIE correction again; "at least one" wants $1 - P(\text{none})$.
- **Flipping the condition.** $P(A \mid B) \neq P(B \mid A)$: $P(\text{positive} \mid \text{disease}) = .99$ says nothing about $P(\text{disease} \mid \text{positive}) \approx .09$ at $1/1000$ prevalence — the prosecutor's fallacy, and it is arithmetic, not intuition.

12.11 Additional exercises

Exercise 12.5 (easy)**.** $P(\text{both heads})$ in two fair flips; then $P(\text{both heads} \mid \text{at least one head})$. Same event — different space.

Exercise 12.6 (easy)**.** Expected value of a fair die; then $E[X^2]$ and check that $E[X^2] \neq (E[X])^2$ — what does the gap measure?

Exercise 12.7 (easy)**.** Draw four without replacement from $\{1, \ldots, 10\}$; find the probability all are even.

Exercise 12.8 (easy)**.** Rolls of two dice: is "sum 7" independent of "first die 3"? Check by the product rule, not by feel.

Exercise 12.9 (medium)**.** Expected number of runs (maximal same-face blocks) in n fair flips, via indicators on the boundaries.

Exercise 12.10 (medium)**.** Coupon collector for $n = 3$: expected draws to see all three types — split into geometric phases and add.

Exercise 12.11 (medium)**.** Three prisoners and the jailer's reveal: compute $P(\text{spared} \mid \text{jailer names a prisoner})$ — where does the naive $1/2$ go wrong?

Exercise 12.12 (medium)**.** Derive the birthday threshold estimate $k \approx \sqrt{2 \cdot 365 \log 2}$ from the product formula (take logs, approximate $\log(1 - x) \approx -x$).

Exercise 12.13 (medium)**.** Expected number of inversions in a random permutation of n — indicators on pairs.

Exercise 12.14 (hard)**.** Prove $\mathrm{Var}(X) = E[X^2] - (E[X])^2$ and compute the variance of the fixed-point count (hint: indicators plus pairwise products).

Exercise 12.15 (hard)**.** Simpson's paradox: give a $2 \times 2 \times 2$ table where A wins both strata by ≥ 10 points but loses overall by ≥ 5, and prove the sizes must be skewed for this to happen.

Exercise 12.16 (hard)**.** Extend `fixed_points.py` to track the empirical *maximum* fixed-point count per trial over growing n and describe the maximum's behavior (conjecture: bounded in distribution with a slowly growing tail — collect data and state what you actually see).

Exercise 12.17 (easy)**.** Two dice: find $P(\text{sum} = 7 \mid \text{first die} \geq 4)$.

Exercise 12.18 (medium)**.** Show that if A and B are independent, so are A and B^c — independence survives complements (one line of algebra, not intuition).

Exercise 12.19 (hard)**.** 100 fair coins are flipped; find the expected number of runs of $k = 3$ consecutive heads by linearity, then verify against `fixed_points.py`-style simulation.

Further reading

- J. Blitzstein and J. Hwang, *Introduction to Probability* (free online edition) — https://probabilitybook.net — the Stat 110 text; story-based probability at exactly this chapter's level.
- MIT OpenCourseWare, *6.042J* (Fall 2010) — https://ocw.mit.edu/6-042JF10 — probability for CS; discrete, combinatorial, no measure theory.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, probability chapters — https://courses.csail.mit.edu/6.042/spring18/mcs.pdf — free PDF; linearity of expectation with combinatorial applications.
- Berkeley CS70 notes — https://www.eecs70.org/ — free lecture notes; discrete probability, conditioning and the birthday paradox.

- E. Chen, *Expected Uses of Probability* — https://web.evanchen.cc/handouts/ProbabilisticMethod/ProbabilisticMethod.pdf — olympiad linearity tricks and the probabilistic method.
- R. Hammack, *Book of Proof*, 3rd ed. — https://richardhammack.github.io/BookOfProof/ — for tightening the probabilistic-argument writing style.

WATCH INSTEAD

- **Harvard Stat 110 — "Lecture 1: Probability and Counting" (Joe Blitzstein)**, 46:29, https://www.youtube.com/watch?v=KbB0FjPg0mw — sample spaces and the naive definition, exactly the tone of §12.1–§12.2. (Verified 2026-10-11.)
- **Harvard Stat 110 full playlist (Blitzstein)**, 34 lectures, https://www.youtube.com/playlist?list=PLLVplP8OIVc8EktkrD3Q8td0GmId7DjW0 — the entire free course this chapter previews. (Verified 2026-10-11.)

Notes

The four-step method, tree diagrams, linearity of expectation and the Simpson's-paradox setup follow 6.042J's probability chapters [4]; Berkeley CS70's probability note sequence carries the same material with more rigor [13]; Chen's *Expected Uses of Probability* supplies the indicator-table worked pattern [14], and Zhao's counting-adjacent handouts the olympiad drills [12]. Contest items powered by this chapter — conditional probability and trees (AIME 2024 I P4, AIME 2023 I P6, AIME 2021 I P1, AIME 2015 I P2), expectation by linearity (AIME 2024 I P11, AIME 2021 I P12, AIME 2018 I P12, AIME 2023 II P6), complements and birthdays (AIME 2023 I P1, AIME 2020 I P9, AIME 2018 I P3, AIME 2015 I P5), plus AMC items (AMC 12 2023 A P5, AMC 12 2023 A P17, AMC 12 2024 A P16, AMC 12 2024 A P20, AMC 12 2024 B P9, AMC 12 2024 B P17, AIME 2010 I P1, AIME 2010 I P4, AIME 2025 II P7) — are on the AoPS archive [7].

CHAPTER 13

Inequalities

Half of olympiad algebra is inequalities — and almost all of it grows from one seed: a real square is nonnegative. This chapter builds the classical toolkit ($x^2 \geq 0$, AM-GM, Cauchy–Schwarz, Schur, Muirhead), and just as importantly, the *discipline*: an inequality claim is incomplete until you name the equality case. After this chapter you can pick a tool for a target, prove the bound, and exhibit the case where it is sharp.

BEFORE YOU START

Chapters 2–3 (proofs, induction). *Recap:* an inequality proof is a chain of steps that each preserve the direction. Multiplying by a negative number reverses it; squaring only works on nonnegative numbers.

AFTER THIS CHAPTER YOU CAN

- prove an inequality from $x^2 \geq 0$ and by rearranging;
- apply AM–GM, power-mean and Cauchy–Schwarz, and state exactly when equality holds;
- choose a strategy for an olympiad inequality;
- sanity-check a proposed inequality numerically before trying to prove it.

HOW TO THINK ABOUT THIS

Almost every inequality begins with "a square cannot be negative." Look for places where you can create a square, or where two quantities are being compared by a hidden balance. When an inequality is tight (equal) for some values, those values tell you which technique to try.

13.1 What's allowed: the algebra of $<$

From the axioms of an ordered field: adding the same quantity preserves $<$; multiplying by a *positive* quantity preserves it; multiplying by a *negative* quantity *flips* it; taking reciprocals of positive quantities flips it; $<$ is transitive. Every step of an inequality proof must be one of these — the most common beginner error is multiplying both sides by an expression whose sign was never fixed.

13.2 Squares are nonnegative

$(a-b)^2 \geq 0$ with equality iff $a = b$. Expanding gives $a^2 + b^2 \geq 2ab$: the entire classical apparatus is this fact wearing cleverer clothes.

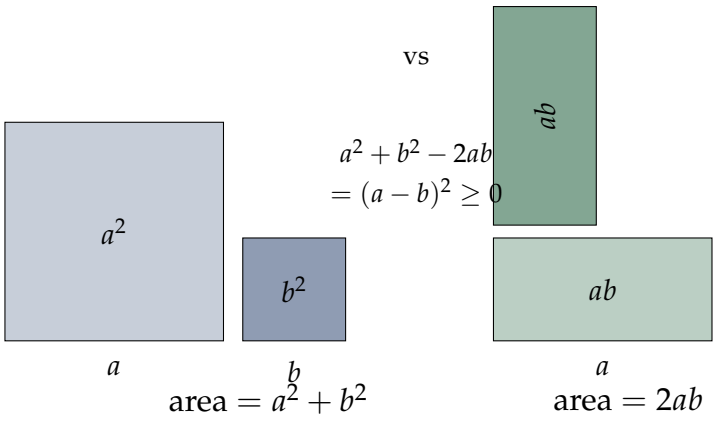


Figure 13.1: $a^2 + b^2 \geq 2ab$ as area bookkeeping: the two squares outgrow the two rectangles by exactly $(a-b)^2$. Notice the equality case is visible — the leftover square vanishes exactly when $a = b$.

Example 13.1 (AM–GM for $n = 4$, from scratch)**. Strategy.** We may not assume AM–GM — we are proving it. The only "free" inequality is the square. Build up: $n = 2$, then bootstrap four at once, and look at the pattern that generalizes.

Solution. $n = 2$: $(x-y)^2 \geq 0$ gives $\frac{x+y}{2} \geq \sqrt{xy}$, equality iff $x = y$. For $n = 4$:

$$\frac{a+b+c+d}{4} = \frac{\frac{a+b}{2} + \frac{c+d}{2}}{2} \geq \frac{\sqrt{ab} + \sqrt{cd}}{2} \geq \sqrt{\sqrt{ab}\sqrt{cd}} = (abcd)^{1/4},$$

each step an instance of $n = 2$. Equality forces $a = b$, $c = d$, and $\sqrt{ab} = \sqrt{cd}$, i.e. $a = b = c = d$. The shape ("pair up, apply $n = 2$ twice") is exactly Cauchy's argument: prove $n = 2$, prove $n \Rightarrow 2n$, and *backwards*-fill any n by augmenting with copies of the mean. This is a second specimen of forward–backward induction (compare §3).

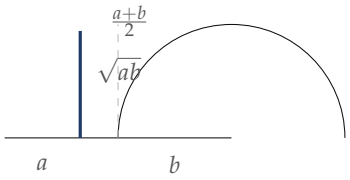
13.3 AM–GM

Theorem 13.2 (AM–GM)**.** *For $a_1, \ldots, a_n \geq 0$, $\frac{a_1 + \cdots + a_n}{n} \geq (a_1 \cdots a_n)^{1/n}$, with equality iff all a_i are equal.*

Figure 13.2 shows the classical picture for $n = 2$: a chord of length a then b subtends a semicircle; the radius $(a+b)/2$ always clears the chord-height $\sqrt{ab}$.

Example 13.3 (Minimize $x + \frac{1}{x}$ on $x > 0$)**. Strategy.** Equality cases guide proofs. Guess the symmetric candidate: $x = 1/x$, i.e. $x = 1$, value 2. Now *prove* it — which means exhibiting an inequality sharp exactly at $x = 1$.

Solution. $x + \frac{1}{x} - 2 = \frac{(x-1)^2}{x} \geq 0$ since $x > 0$. Equality iff $x = 1$. Two halves: (a) bound $x + 1/x \geq 2$ for all $x > 0$; (b) attainability, at $x = 1$. Drop either half and the claim is incomplete — "the minimum is 2" requires both a proof *and* a witness.

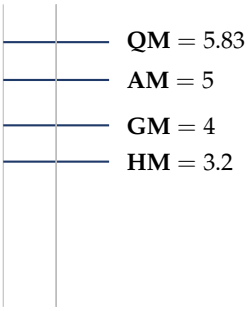


the dashed radius always clears the chord's height

Figure 13.2: AM–GM as geometry. Notice the dashed radius $(a+b)/2$ is never shorter than the perpendicular $\sqrt{ab}$ — and they coincide exactly when $a = b$.

13.4 The power-mean chain

For $a, b > 0$: $\underbrace{\sqrt{\frac{a^2+b^2}{2}}}_{\text{QM}} \geq \underbrace{\frac{a+b}{2}}_{\text{AM}} \geq \underbrace{\sqrt{ab}}_{\text{GM}} \geq \underbrace{\frac{2}{1/a+1/b}}_{\text{HM}}$, all equal iff $a = b$. Figure 13.3 evaluates the whole ladder at $(2, 8)$.



for $(2, 8)$

four rungs, one theorem — equality at all four iff $a = b$

Figure 13.3: The power-mean ladder on $(2, 8)$: QM $5.83 \geq$ AM $5 \geq$ GM $4 \geq$ HM 3.2. Notice the four rungs collapse to one exactly when $a = b$.

13.5 Cauchy–Schwarz

Theorem 13.4 (Cauchy–Schwarz)**.** $(\sum a_i^2)(\sum b_i^2) \geq (\sum a_i b_i)^2$, *with equality iff the vectors are proportional.*

Proof: $\sum (a_i t + b_i)^2 \geq 0$ is a quadratic in t with no sign change, so its discriminant is nonpositive — that discriminant is exactly the claim. The *Engel form*: for $b_i > 0$, $\sum \frac{a_i^2}{b_i} \geq \frac{(\sum a_i)^2}{\sum b_i}$ — "Titu's lemma" — obtained by Cauchy on $(a_i/\sqrt{b_i})^2$ and $(\sqrt{b_i})^2$.

Example 13.5 (Nesbitt's inequality)**.** For $a, b, c > 0$: $\frac{a}{b+c} + \frac{b}{c+a} + \frac{c}{a+b} \geq \frac{3}{2}$.

Strategy. The Engel form eats exactly this shape: write each term as $a^2/(a(b+c))$ so denominators combine.

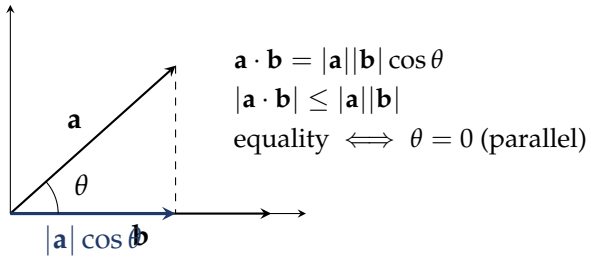


Figure 13.4: Cauchy–Schwarz is a disguised trigonometric fact: $\mathbf{a}\cdot\mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$ cannot exceed $|\mathbf{a}||\mathbf{b}|$, with equality only when the vectors are parallel. Notice the discriminant proof manufactures exactly this $\cos\theta \le 1$ bound.

Solution.

$$\sum_{cyc}\frac{a^2}{a(b+c)} \ge \frac{(a+b+c)^2}{2(ab+bc+ca)} \ge \frac{3}{2} \iff (a+b+c)^2 \ge 3(ab+bc+ca),$$

which is $\frac{1}{2}\sum(a-b)^2 \ge 0$. Equality iff $a=b=c$.

13.6 Equality conditions

Every optimization problem is two problems: prove the bound, then find where it binds. The second half is skipped constantly — and it is the half that tells you *which* inequality to use. Apply AM–GM only where its equality case (a_i all equal) is attainable in the problem's constraints; otherwise the "minimum" you found is unattainable and the answer is wrong.

Example 13.6 (Reading the equality case off the proof)**.** Show $a^2+b^2+c^2 \ge ab+bc+ca$ for all reals, and find exactly when equality holds.

Strategy. Rearrange into a sum of squares; the equality case then reads itself off — every square must vanish simultaneously.

Solution. $2(a^2+b^2+c^2)-2(ab+bc+ca) = (a-b)^2+(b-c)^2+(c-a)^2 \ge 0$, dividing by 2 gives the claim. Equality needs all three squares zero, i.e. $a=b=c$. The same two-step structure solves every problem of this shape: first the bound, then the configuration attaining it — and the sum-of-squares form makes the second step mechanical.

13.7 Muirhead and Schur

For symmetric sums use the bracket notation: $[3,0,0] = \sum_{sym} x^3/2$ conventions vary; we use $\sum_{cyc} x^3$ style — spelled out when ambiguity looms. Muirhead: if exponent vector α *majorizes* β (partial sums of sorted components), then $\sum_{sym} x^\alpha \ge \sum_{sym} x^\beta$ for $x_i > 0$. Figure 13.5 pictures $[3,0,0]$ majorizing $[2,1,0]$.

Example 13.7 (Schur, derived not stated)**. Strategy.** Don't invoke Schur as a black box — manufacture it. $\sum(a-b)(a-c)(a+b-c)$-type thinking aside, the honest derivation: expand the nonnegativity of $\sum a(a-b)(a-c)$.

Solution. For $a, b, c \geq 0$, claim $\sum_{cyc} a(a-b)(a-c) \geq 0$: WLOG $a \geq b \geq c$; group the first two terms: $a(a-b)(a-c) + b(b-c)(b-a) = (a-b)^2(a+b-c) \geq 0$ since $a \geq b$ and $a+b \geq c$; the last term $c(c-a)(c-b) \geq 0$ since both factors are nonpositive. The full check is exr:i2. Expanding:

$$\sum_{cyc} a^3 + 3abc \;\geq\; \sum_{sym} a^2 b \quad \text{(Schur of degree 3)},$$

equality iff $a = b = c$ or two are equal and the third is 0. This is the standard repair for "AM–GM won't reach" symmetric cubics.

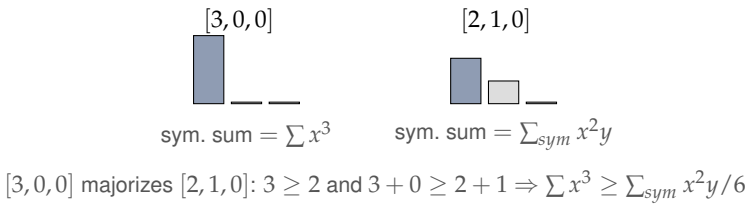


Figure 13.5: Majorization: $[3,0,0]$'s bars dominate $[2,1,0]$'s cumulatively. Notice this is a statement about exponent *distribution*, not values.

13.8 Strategy selection

- *Homogenize*: scale until a free parameter (a sum, a product) is a fixed value — legitimate when the inequality is homogeneous.
- *Normalize*: assume $a+b+c = 1$ (or $abc = 1$) to eliminate a constraint before the real work.
- *Tangent line*: guess where equality hits, fit the tangent of each summand there, prove summand $\geq$ tangent pointwise (Figure 13.6). Sharpness is built in at the touching point.

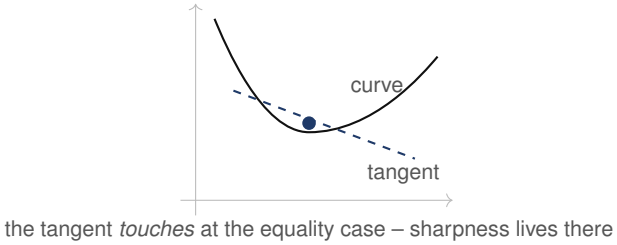


Figure 13.6: Tangent-line method: the curve sits above its tangent, touching at the equality point. Notice sharpness is *designed in* by choosing the tangent at the equality case.

13.9 Summary

Tool	Statement	Equality
$x^2 \geq 0$	$a^2 + b^2 \geq 2ab$	$a = b$
AM–GM	$\bar{a} \geq \sqrt[n]{\prod a_i}$	all equal
Power-mean	QM≥AM≥GM≥HM	all equal
Cauchy–Schwarz	$\sum a_i^2 \sum b_i^2 \geq (\sum a_i b_i)^2$	proportional
Engel	$\sum a_i^2/b_i \geq (\sum a_i)^2/\sum b_i$	$a_i \propto b_i$
Schur	$\sum a^3 + 3abc \geq \sum_{sym} a^2 b$	equal, or $a = b, c = 0$
Muirhead	majorized exponents ⇒ bigger sym sum	all equal

13.10 Code: numerical sanity checks

An inequality you "believe" should survive a Monte-Carlo scan — `ineq_scan.py` checks Nesbitt and AM-GM on 10 000 random positive triples and grid-searches the $x + 1/x$ minimum.

Checkpoint 13.1 Predict: what value will the Nesbitt minimum hover just above, and at which kind of triple? Then run `python3 ineq_scan.py`.

```
Nesbitt over 10000 random triples: min = 1.5003 >= 1.5
  at (a,b,c) = (3.81, 3.848, 3.951)
AM-GM held on all 10000 triples: True
x + 1/x minimizer on grid: x = 1.0, value = 2.0000
theory: equality at x = 1, minimum value = 2
```

The empirical minimum sits just above 1.5 at a nearly-equal triple — exactly the theorem's equality case, found by the experiment rather than assumed. Sampling never proves, but a violated sample would disprove; here 10 000 samples land where theory says they must. `test_13_ineq.sh` re-runs and greps the summary lines.

13.11 Common mistakes

- *Flipping on an unsigned multiplier.* Multiplying by "$a - b$" without fixing its sign silently reverses $<$. Fix signs first — or add, which never flips.
- *Bound without witness.* "Minimum is 2" needs both $x + 1/x \geq 2$ *and* an x attaining it. Most lost marks are the missing second half.
- *AM–GM where equality is impossible.* If the constraint forces $a \neq b$, AM–GM's equality case is unattainable and the bound doesn't bind — pick a tool whose equality case *is* feasible.

Exercises

Inline

Exercise 13.1 (easy)**.** Prove $\frac{a+b}{2} \le \sqrt{\frac{a^2+b^2}{2}}$ for $a, b > 0$, and state the equality case, using only $(a-b)^2 \ge 0$.

Solution. Squaring: equivalent to $(a+b)^2 \le 2(a^2+b^2)$, i.e. $0 \le (a-b)^2$. Equality iff $a = b$.

Exercise 13.2 (easy)**.** Complete the WLOG argument in Schur's inequality: for $a \ge b \ge c \ge 0$, show $\sum_{cyc} a(a-b)(a-c) \ge 0$.

Solution. Group the first two terms: $a(a-b)(a-c) + b(b-c)(b-a) = (a-b)\big[a(a-c) - b(b-c)\big] = (a-b)\big[a^2 - ac - b^2 + bc\big] = (a-b)^2(a+b-c) \ge 0$ since $a \ge b$ and $a+b \ge c$. The last term $c(c-a)(c-b) \ge 0$ because $c-a \le 0$ and $c-b \le 0$. Total ≥ 0.

Exercise 13.3 (medium)**.** For $x, y > 0$ with $x + y = 1$, prove $(1 + \frac{1}{x})(1 + \frac{1}{y}) \ge 9$.

Solution. Expand: $1 + \frac{x+y}{xy} + \frac{1}{xy} = 1 + \frac{2}{xy}$, and $xy \le \left(\frac{x+y}{2}\right)^2 = \frac{1}{4}$ by AM–GM, so $1 + \frac{2}{xy} \ge 1 + 8 = 9$; equality at $x = y = \frac{1}{2}$.

Exercise 13.4 (medium)**.** By Engel's form, prove for $a, b, c > 0$: $\frac{a^2}{b} + \frac{b^2}{c} + \frac{c^2}{a} \ge a + b + c$.

Solution. $\sum a^2/b \ge (a+b+c)^2/(a+b+c) = a+b+c$; equality at $a = b = c$.

Additional

Exercise 13.5 (easy)**.** Show $x^2 + y^2 + z^2 \ge xy + yz + zx$; find the equality case.

Solution. Multiply by 2: $(x-y)^2 + (y-z)^2 + (z-x)^2 \ge 0$; equality iff $x = y = z$.

Exercise 13.6 (easy)**.** Prove $\frac{2ab}{a+b} \le \sqrt{ab}$ for $a, b > 0$ (HM≤GM on two variables).

Solution. Equivalent to $2\sqrt{ab} \le a + b$, i.e. $(\sqrt{a} - \sqrt{b})^2 \ge 0$.

Exercise 13.7 (easy)**.** Prove or disprove: if $a \ge b$ then $a^2 \ge b^2$ — and repair the false version.

Solution. False without signs ($-1 \ge -2$). True for $a \ge b \ge 0$, or with $|a| \ge |b|$.

Exercise 13.8 (easy)**.** Minimize $\frac{x^2+2}{\sqrt{x^2+1}}$ on real x.

Solution. Let $u = \sqrt{x^2+1} \ge 1$: $u + 1/u \ge 2$ but equality $u = 1$ forces $x = 0$ — it *is* attainable; minimum 2 at $x = 0$.

Exercise 13.9 (medium)**.** Prove for $a, b, c > 0$ with $abc = 1$: $a + b + c \ge 3$, and extend: minimize $a + b + c + d$ given $abcd = 1$ — what does the equality case look like?

Solution. AM–GM directly: $a+b+c \geq 3\sqrt[3]{abc} = 3$, equality at $a=b=c=1$; likewise $a+b+c+d \geq 4$ with all $= 1$.

Exercise 13.10 (medium)**.** Prove Schur of degree 3 implies: for $a,b,c>0$, $(a+b-c)(b+c-a)(c+a-b) \leq abc$.

Solution. Expand the left minus right; it rearranges exactly to the negation of Schur's form. (This is the substitution trick behind Ravi.)

Exercise 13.11 (medium)**.** Minimize $\frac{1}{x}+\frac{1}{y}$ subject to $x+y=2$, $x,y>0$, and state the minimizer.

Solution. $\frac{x+y}{xy} = \frac{2}{xy} \geq \frac{2}{1} = 2$ since $xy \leq 1$; minimizer $x=y=1$, value 2.

Exercise 13.12 (medium)**.** Verify Muirhead's smallest case on two variables: $[2,0]$ majorizes $[1,1]$, and show $\sum_{sym} x^2 \geq \sum_{sym} xy$ recovers a familiar inequality.

Solution. On (x,y): $\sum_{sym} x^2 = 2x^2+2y^2$, $\sum_{sym} xy = 2xy$, so the claim is $x^2+y^2 \geq 2xy$ — the square. Check the majorization: $2 \geq 1$ and $2+0 = 1+1$.

Exercise 13.13 (hard)**.** Prove the tangent-line method on one summand: show $\frac{x^3}{1+x^2} \geq x - \frac{1}{2}$ for $x>0$, equality at $x=1$, and assemble a proof that for $a+b+c=3$, $a,b,c>0$, $\sum \frac{a^3}{1+a^2} \geq \frac{3}{2}$.

Solution. $\frac{x^3}{1+x^2} - \left(x-\frac{1}{2}\right) = \frac{x^3-(x-\frac{1}{2})(1+x^2)}{1+x^2} = \frac{\frac{1}{2}(x-1)^2}{1+x^2} \geq 0$ on expanding the numerator: $x^3 - x - x^3 + \frac{1}{2} + \frac{1}{2}x^2 = \frac{1}{2}(x^2-2x+1)$. Equality iff $x=1$. Sum over a,b,c with $a+b+c=3$: $\sum \frac{a^3}{1+a^2} \geq \sum\left(a-\frac{1}{2}\right) = 3 - \frac{3}{2} = \frac{3}{2}$, equality at $a=b=c=1$ — the tangent was fitted exactly there.

Exercise 13.14 (hard)**.** Implement-or-derive: extend `ineq_scan.py` to scan the Schur inequality $\sum a^3 + 3abc \geq \sum_{sym} a^2 b$ on 10^4 random triples, and print the minimum of LHS$-$RHS.

Solution. One loop; the scan's minimum should approach 0 at near-equal triples, matching the equality case — a falsification would show up as a strictly negative minimum.

Exercise 13.15 (hard)**.** Prove Cauchy's backward step: if AM–GM holds for $2n$ it holds for n, by taking the n numbers plus n copies of their arithmetic mean.

Solution. Let $m = \bar{a}$. Apply the $2n$ case to $(a_1,\ldots,a_n,m,\ldots,m)$: mean is still m, geometric mean $\sqrt[2n]{\prod a_i \cdot m^n} \leq m$; squaring out m^n gives $\prod a_i \leq m^n$.

Exercise 13.16 (easy)**.** For $x,y>0$ with $xy=1$, find the minimum of x^2+y^2 and state the equality case.

Solution. $x^2+y^2 \geq 2xy = 2$ by $a^2+b^2 \geq 2ab$; equality at $x=y=1$.

Exercise 13.17 (medium)**.** Show for positive a, b, c: $(a+b+c)\left(\frac{1}{a}+\frac{1}{b}+\frac{1}{c}\right) \geq 9$ — once by Cauchy–Schwarz in Engel form and once by AM–GM twice.

Solution. Engel: $\sum 1/a = \sum 1^2/a \geq (1+1+1)^2/(a+b+c)$. Or AM–GM on $a+b+c \geq 3(abc)^{1/3}$ and on $1/a+1/b+1/c \geq 3/(abc)^{1/3}$, then multiply. Equality at $a=b=c$ either way.

Exercise 13.18 (medium)**.** Minimize $\frac{x^2+2}{\sqrt{x^2+1}}$ over real x — and find the trap in the naive AM–GM answer.

Solution. AM–GM on $u+1/u$ for $u=\sqrt{x^2+1} \geq 1$ suggests min $= 2$ at $u=1$, i.e. $x=0$ — attainable, so the answer really is 2 at $x=0$; the point is that the equality case must be checked for reachability (here it is: $u=1$ occurs), unlike the classic $u \geq 2$ variants where $u=1$ is forbidden.

Exercise 13.19 (hard)**.** Prove Nesbitt a second way: write $\frac{a}{b+c}+1 = \frac{a+b+c}{b+c}$ and conclude by AM–HM on the three denominators.

Solution. $\sum \frac{a}{b+c} = \sum \frac{a+b+c}{b+c} - 3$. By AM–HM (or the $\sum 1/x \geq 9/\sum x$ exercise above) on $b+c, c+a, a+b$: $\sum \frac{1}{b+c} \geq \frac{9}{2(a+b+c)}$, so $\sum \frac{a+b+c}{b+c} \geq \frac{9}{2}$ and the claim follows with equality at $a=b=c$.

Notes

The AM–GM forward–backward induction is Cauchy's original argument [9]; the power-mean ladder and equality discipline follow the Excerpts' chapter on sharpness-vs-tractability [9]. The discriminant proof of Cauchy–Schwarz and the Engel form are standard [10]; Schur and Muirhead statements follow [9]. Tangent-line and normalization strategies are catalogued in [17]. For the "prove the bound + exhibit the case" two-half structure see [2]. Contest uses: 2024 AIME II P4; 2022 AIME I P15; 2022 AIME I P7; 2024 AMC 12B P13; 2010 AIME I P9; 2025 AIME II P15; 2024 AMC 12B P3; 2025 AIME I P14; 2023 AIME I P2.

Further reading

- *Basics of Olympiad Inequalities* (S. Riasat) — `https://web.evanchen.cc/handouts/Ineq/en.pdf` — free PDF; the standard olympiad inequalities reference at this level.
- E. Chen's inequality handouts — tangent-line tricks and harder examples: `https://web.evanchen.cc/handouts/`
- The AoPS wiki's curated inequalities articles (search "Olympiad inequalities" on): `https://artofproblemsolving.com`
- A. Engel, *Problem-Solving Strategies*, inequalities chapters — the drill source used throughout the notes.
- Berkeley CS70 notes — `https://www.eecs70.org/` — for the probabilistic and CS-side uses of the same bounds.

- R. Hammack, *Book of Proof* — for writing the equality-case half of an inequality proof cleanly: https://richardhammack.github.io/BookOfProof/

WATCH INSTEAD

- **mathophilia — "Proving the AM-GM Inequality with ZERO Calculus"**, 9:22, https://www.youtube.com/watch?v=bA01HNj2mDw — AM–GM for $n = 2$, then induction and Cauchy's forward–backward induction: the AM–GM sections. (Verified 2026-10-11.)
- **Khan Academy — Linear Algebra playlist** — https://www.youtube.com/playlist?list=PLFD0EB975BA0CC1E0 — contains Sal's Cauchy–Schwarz lectures: the Cauchy–Schwarz section. (Verified 2026-10-11.)

Notes

CHAPTER 14

Functional equations

A functional equation asks for every function satisfying a relation like $f(x+y) = f(x) + f(y)$ — not a value, a *family*. Solving one is a two-directions discipline: find candidates, then prove nothing else works. This chapter is the toolkit: candidate guessing, substitution tactics, injectivity/surjectivity milestones, Cauchy's equation done honestly, and the feedback step most solutions skip — verifying the answer. After it you can carry a functional equation from first plug-in to a verified closed form.

BEFORE YOU START

Chapters 2–4 (proofs, functions). *Recap:* a functional equation asks for *all functions* satisfying a rule, like $f(x+y) = f(x) + f(y)$. Your job is to find them all and prove there are no others.

AFTER THIS CHAPTER YOU CAN

- substitute special values to extract information from a functional equation;
- prove injectivity or surjectivity of an unknown function;
- verify that a candidate solution actually works;
- find a family of solutions by brute-force search and then prove it.

HOW TO THINK ABOUT THIS

A functional equation is a puzzle in which the unknown is an entire function. Start by playing: plug in 0, 1, equal values, and see what the equation *forces*. Then decide what kind of function it must be and finally check that your candidate truly works; many attempts lose points by skipping that last check.

14.1 What "solve over K" means

"Solve $f(x+y) = f(x) + f(y)$ over $\mathbb{Q}$" means: determine *all* $f : \mathbb{Q} \to \mathbb{Q}$ satisfying it, and verify each claimed solution satisfies it. The domain matters twice — it decides which candidates exist (over $\mathbb{R}$, Cauchy's equation has infinitely many wild solutions; over $\mathbb{Q}$, only kx) and it licenses which inductions you may run ($\mathbb{N} \to \mathbb{Z} \to \mathbb{Q}$).

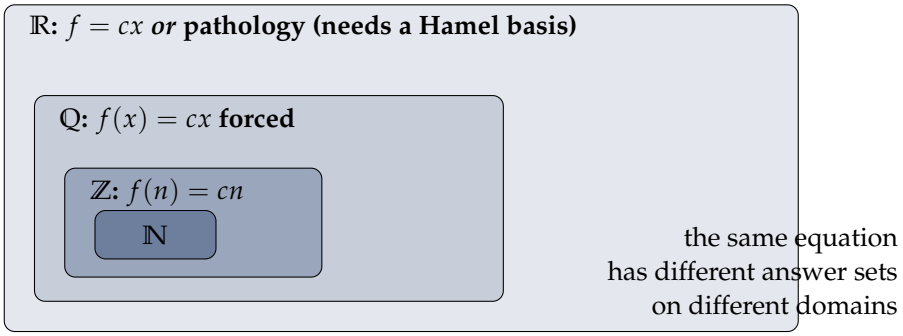


Figure 14.1: The domain ladder for $f(x+y) = f(x) + f(y)$. Notice the answer set only stabilizes into "$f = cx$" where the inductions reach — on $\mathbb{R}$ the tame answer is joined by unclassifiable pathological solutions.

14.2 Guessing candidates

Before proving uniqueness, guess: try $f = 0$, $f = c$, $f = kx$, $f = x + c$, $f = kx + c$, monomials, low-degree polynomials. Each guess that survives is a lead on the general shape — and a sanity check for later (if your "proof" kills a function you already verified, the proof is wrong).

14.3 Substitution tactics

The engine of every solution: choose substitutions that make terms *collide*.

- *Zeros* ($x = 0$ or $y = 0$) collapse sums and expose constants.
- *Swaps* ($x \leftrightarrow y$) exploit symmetric left sides to equate right sides.
- *Plugs* ($x = u$ for a specific u, or $x = f(u)$) force a value to appear where you need it.

Figure 14.3 is the loop: every substitution should earn a fact; a substitution that produces $0 = 0$ taught you nothing — try a different one. Figure 14.2 lists the five workhorse substitutions in the order to try them.

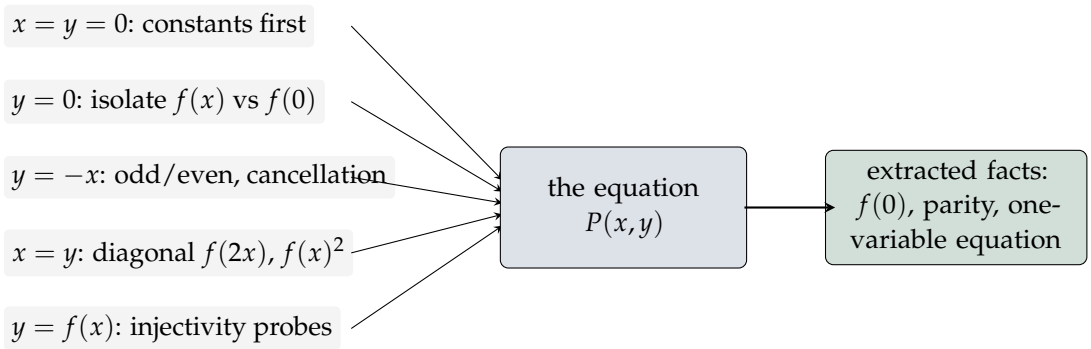


Figure 14.2: The substitution toolbox: each plug is chosen to make two terms collide and cancel. Notice they are ordered by cheapness — zeros first, injectivity probes last.

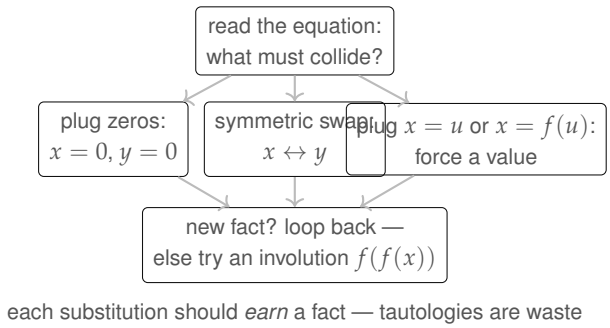


Figure 14.3: Substitution tactics as a decision loop. Notice the goal of each move: make two previously-distinct terms coincide.

14.4 Milestones: injectivity, surjectivity, involutions

Structural facts convert equation-juggling into set-juggling: f injective lets you cancel $f(\cdot) = f(\cdot)$; surjective lets you rename any value as $f(\text{something})$; and an *involution* — $f(f(x)) = x$ — hands you both for free (Figure 14.4).

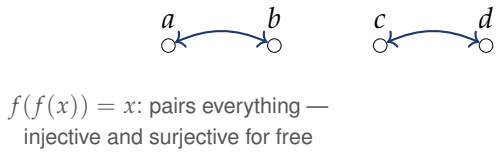


Figure 14.4: An involution pairs the domain. Notice why it is bijective: $f(a) = f(b) \Rightarrow f(f(a)) = f(f(b)) \Rightarrow a = b$.

Example 14.1 (An involution equation, Kyrgyzstan-2012 style)**.** Solve over $\mathbb{R}$: $f\big(x\, f(y)\big) = y\, x$.

Strategy. The left side buries $f(y)$ inside f — expose it. Plugging $x = 1$ should peel out $f(f(y))$; if that comes out as an involution, bijectivity is free and $y = f(t)$ substitutions become available.

Solution. $x = 1$: $f(f(y)) = y$ — an involution, so f is a bijection (Figure 14.4). Now substitute $y = f(t)$: using $f\big(x\, f(f(t))\big) = f(t)\, x$, i.e. $f(xt) = x\, f(t)$. Setting $t = 1$: $f(x) = f(1)\, x$, so f is linear, $f(x) = cx$. Feed back: $c^2\, xy = xy$ for all x, y, so $c = \pm 1$. Verification: $f(x) = x$ gives $f(xf(y)) = xy$; $f(x) = -x$ gives $f(-xy) = xy$. *Both* are solutions — the answer is the two-element set $\{x \mapsto x, x \mapsto -x\}$.

14.5 Cauchy's equation over $\mathbb{Q}$

Example 14.2 (Solve $f(x+y) = f(x) + f(y)$ over $\mathbb{Q}$)**. Strategy.** The candidate is $f = kx$. Rather than prove $f = kx$ directly, expose the free parameter $k = f(1)$ and expand the domain in steps: $\mathbb{N}$ by induction, $\mathbb{Z}$ by a sign trick, $\mathbb{Q}$ by clearing denominators (Figure 14.5).

Solution. $x = y = 0$: $f(0) = 0$. Induction gives $f(n) = nf(1)$ on $\mathbb{N}$. Then $0 = f(n + (-n)) = f(n) + f(-n)$ gives $f(-n) = -nf(1)$, so $\mathbb{Z}$ is done. For $p/q \in \mathbb{Q}$: $f(p) = f(q \cdot \frac{p}{q}) = q\, f(\frac{p}{q})$, so $f(\frac{p}{q}) = \frac{p}{q} f(1)$. Hence $f(x) = f(1)x$ for all $x \in \mathbb{Q}$ — and kx does satisfy the equation, so this is the complete list. Over $\mathbb{R}$ the pipeline stops: the rationals are dense but algebra alone doesn't bridge (wild solutions exist without regularity).

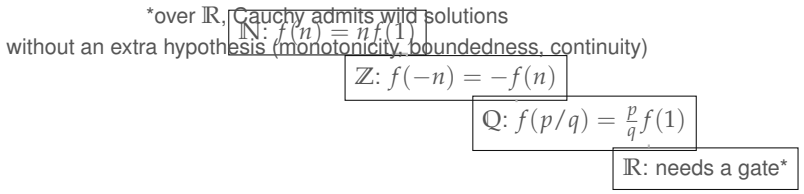


Figure 14.5: The domain-expansion staircase for Cauchy's equation. Notice each step uses the previous one — and the last step to $\mathbb{R}$ needs a gate (monotone, bounded, continuous), not more algebra.

Example 14.3 (Jensen's equation by shifting)**.** Solve over $\mathbb{Q}$: $f(\frac{x+y}{2}) = \frac{f(x)+f(y)}{2}$.

Strategy. Constants satisfy it — kill that family first by shifting: $g(x) = f(x) - f(0)$ satisfies the same equation and has $g(0) = 0$.

Solution. With $g(0) = 0$, set $y = 0$: $g(x/2) = g(x)/2$. Then $g(\frac{x+y}{2}) = \frac{g(x)+g(y)}{2}$ becomes $g(x+y) = g(x) + g(y)$ after replacing $x + y \mapsto$ double — Cauchy again, so $g(x) = kx$ and $f(x) = kx + f(0)$. Verify: affine functions satisfy Jensen. The shift isolated the constant family so the equation became one already solved.

14.6 The pointwise trap

Suppose you prove "for each t, $f(t) = \pm t$". You have *not* proved $f(t) = t$ or $f(t) = -t$: the sign may depend on t (Figure 14.6). Kill the gap by mixing: plug $f(a) = a$, $f(b) = -b$ into the equation and derive a contradiction.

Example 14.4 (Trap autopsy)**. Strategy.** In $f(f(x)) = x$ + Cauchy-shaped problems the sign pattern survives mixing only under extra structure. Exhibit the gap: $f(t) = t$ on $[0, \infty)$, $f(t) = -t$ on $(-\infty, 0)$ satisfies $|f| = |x|$ pointwise — and fails most equations.

Solution. Take $f(xf(y)) = yx$ again: after proving $f(f(y)) = y$ and $f(x) = f(1)x$ we found $f(1) = \pm 1$ — here the *parameter* is global, not per-point, so no trap appears. The trap bites when your deduction is per-t ($f(t) \in \{t, -t\}$) instead of a single global constant. The check that matters: did the derivation force a single sign, or leave one free per point?

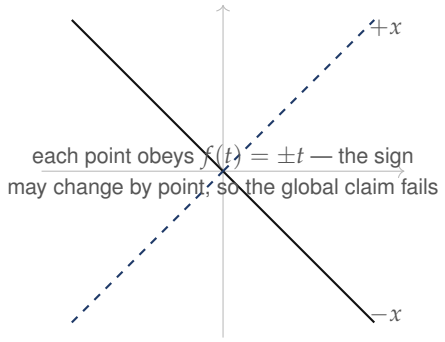


Figure 14.6: The pointwise trap: f may follow $+x$ on part of the domain and $-x$ on the rest. Notice this still satisfies $|f(t)| = |t|$ at every point — "per-point" facts never stitch into a global one.

14.7 Feeding back: verify the answer

"Find all" ends only when the found functions are plugged back in — the verification half that contest graders insist on (the English handout's rule: never claim without the construction). An equation can shrink a candidate family to $\{f = 0\}$, and even that last member still has to be checked.

Example 14.5 (Verification eliminates a wrong guess)**.** Suppose solving $f(xy) = f(x)f(y)$ over $\mathbb{R}_{>0}$ led you to the guess $f(x) = x + c$. Show verification rejects every $c \neq 0$.

Strategy. Plug the candidate in literally — don't argue about whether it "should" work.

Solution. $f(xy) = xy + c$ while $f(x)f(y) = (x + c)(y + c) = xy + c(x + y) + c^2$. Equality requires $c(x + y) + c^2 = c$ for all x, y; fixing y and varying x forces $c = 0$ (then $c^2 = c$ is automatic). The guess is rejected — and verification catches what the solving pass missed: the real answer family is $f \equiv 0$ together with $f(x) = x^k$ (under any mild regularity hypothesis).

14.8 Summary

Tactic	What it earns
Plug 0	constants, $f(0)$
Swap $x \leftrightarrow y$	equality of two right sides
Plug $x = u$, $x = f(u)$	forced values, involutions
Induction	$\mathbb{N}$ expansion
Sign trick $f(-x)$	$\mathbb{Z}$ expansion
Denominator clear	$\mathbb{Q}$ expansion
Shift $g = f - f(0)$	kill constant families
Mix two points	kill the pointwise trap
Plug back in	verification (required half)

14.9 Code: hunting solutions by brute force

`fe_search.py` scans degree-$\leq$ 2 polynomials $f(t) = At^2 + Bt + C$ on a grid for the equation $f(x+y) - f(x-y) = 4xy$.

Checkpoint 14.1 Predict which coefficient must be 1, which must be 0, and which is free. Then run `python3 fe_search.py`.

```
quadratic survivors: [(1.0, 0.0)]
constant term free? distinct C values per (A,B): True
  f=t^2 + c: residual 0.000
  f=t^2 - 1: residual 0.000
  f=2t^2: residual 2500.000
  f=t: residual 2550.000
  f=0: residual 2500.000
verdict: f(t) = t^2 + C is the whole degree-<=2 family
```

The brute force agrees with the algebra ($f(x+y) - f(x-y) = 4xy$ is $f = t^2 + C$ exactly) — and confirms the constant family survives every equation of difference type. `test_14_fe.sh` re-runs and greps the verdict line.

14.10 Common mistakes

- *Never verifying.* "Find all" is two directions; a candidate list without a plug-back-in check is half a solution.
- *The pointwise sign.* $f(t) = \pm t$ per point $\neq$ a global $\pm$. The sign must come out as a single constant.
- *Tautology loops.* Substitutions that yield $0 = 0$ produce no fact — if three plugs teach nothing, change *what* you plug, not how hard you stare.

Exercises

Inline

Exercise 14.1 (easy)**.** From $f(x+y) = f(x) + f(y)$ deduce $f(0) = 0$ and $f(-x) = -f(x)$.

Solution. $x = y = 0$ gives $f(0) = 2f(0)$. Then $0 = f(0) = f(x) + f(-x)$.

Exercise 14.2 (easy)**.** Show $f(x) = kx$ satisfies $f(x+y) = f(x) + f(y)$ over $\mathbb{R}$ — the verification half.

Solution. $kx + ky = k(x+y)$. Done — but it is *required*, not optional.

Exercise 14.3 (medium)**.** Solve $f(x+y) = f(x)f(y)$ over $\mathbb{Q}$ with $f \not\equiv 0$, given $f(1) > 0$.

Solution. $f(0) = 1$ (since $f(x) = f(x)f(0)$ and some value is nonzero). Induction: $f(n) = f(1)^n$; $f(-n) = f(1)^{-n}$; $f(p/q) = f(1)^{p/q}$ with the rational root defined since $f(1) > 0$. Answer $f(x) = a^x$ for $a = f(1) > 0$ — verify: $a^{x+y} = a^x a^y$.

Exercise 14.4 (medium)**.** Exhibit a function $|f(t)| = |t|$ everywhere that is neither t nor $-t$, and use it to explain the pointwise trap.

Solution. $f(t) = t$ on $[0, \infty)$, $f(t) = -t$ on $(-\infty, 0)$ works (as in Figure 14.6); the sign is per-point, so a "$f(t) \in \{t, -t\}$ for each t" conclusion cannot be globalized without a mixing argument.

Additional

Exercise 14.5 (easy)**.** Solve $f(x + 1) = f(x) + 2$ over $\mathbb{Z}$ with $f(0) = 3$.

Solution. $f(n) = 2n + 3$ by induction both directions; verify.

Exercise 14.6 (easy)**.** Which constant functions satisfy $f(x + y) = f(x) + f(y) + 7$?

Solution. $f = c$: $c = 2c + 7 \Rightarrow c = -7$. Verify: $-7 = -7 - 7 + 7$.

Exercise 14.7 (easy)**.** Show an involution is bijective.

Solution. $f(f(x)) = x$: injective since $f(a) = f(b) \Rightarrow a = f(f(a)) = f(f(b)) = b$; surjective since $x = f(f(x))$ is in the image.

Exercise 14.8 (easy)**.** Solve $f(f(x)) = x$ and $f(x) = kx$ simultaneously over $\mathbb{R}$.

Solution. $k^2 = 1$, so $f = x$ or $f = -x$; both check.

Exercise 14.9 (medium)**.** Solve over $\mathbb{Q}$: $f(x + y) = f(x) + f(y) + xy$.

Solution. $g = f - x^2/2$ is additive on $\mathbb{Q}$: $g(x) = kx$; $f = kx + x^2/2$. Verify.

Exercise 14.10 (medium)**.** Prove $f(f(x)) = x$ has no solution of the form $f(x) = x + c$ unless $c = 0$.

Solution. $f(f(x)) = x + 2c = x$ needs $c = 0$.

Exercise 14.11 (medium)**.** Suppose $f(x + y) = f(x) + f(y)$ over $\mathbb{R}$ and f is monotone. Prove $f = kx$ (the gate of Figure 14.5).

Solution. On $\mathbb{Q}$, $f = kx$ with $k = f(1)$. For $r \in \mathbb{R}$, pick rationals $q_n \uparrow r$, $q'_n \downarrow r$; monotonicity sandwiches $f(r)$ between $kq_n, kq'_n \to kr$. Monotonicity then forces $k = f(1) \geq 0$.

Exercise 14.12 (medium)**.** Describe all solutions of $f(xy) = f(x) + f(y)$ over positive rationals with $f(2) = 1$.

Solution. On the multiplicative group $\mathbb{Q}_{>0}$, f is determined by its values on primes: $f(x) = \sum_p e_p\, c_p$ for $x = \prod p^{e_p}$, with $c_2 = f(2) = 1$ and the other c_p free — so $f = \log_2$ is only the special case $c_p = 0$ for $p > 2$, not the general answer.

Exercise 14.13 (hard)**.** Prove Cauchy's equation over $\mathbb{R}$ admits non-linear solutions (hence the gate) using a Hamel basis.

Solution. Pick a $\mathbb{Q}$-vector-space basis H of $\mathbb{R}$ containing 1. Define f linear on basis elements with $f(1) = 1$, $f(h) = 0$ for $h \neq 1$; extend $\mathbb{Q}$-linearly. Then f is additive but not kx (e.g. $f(\sqrt{2}) = 0$).

Exercise 14.14 (hard)**.** Implement-or-derive: extend `fe_search.py` to degree-3 polynomials on the same equation and report the surviving family.

Solution. The same $t^2 + C$ survives; no cubic term can cancel (At^3's residual grows like $|t|^3$). Brute force confirms the degree bound.

Exercise 14.15 (easy)**.** Solve $f(x+1) = f(x) + 2$ over $\mathbb{Z}$.

Solution. Induction both directions: $f(n) = 2n + f(0)$; every such function works.

Exercise 14.16 (medium)**.** Find all $f : \mathbb{Q} \to \mathbb{Q}$ with $f(x+y) = f(x)f(y)$.

Solution. $f(0) = f(0)^2$, so $f(0) \in \{0, 1\}$; $f(0) = 0$ forces $f \equiv 0$. Else $f(0) = 1$ and $f(x)f(-x) = 1$, so f is never 0, and $f(x) = f(x/2)^2 > 0$. With $a = f(1)$: $f(n) = a^n$, and $f(1) = f(1/q)^q$ forces $a^{1/q} \in \mathbb{Q}$ for *every* denominator q — impossible for $a \neq 1$ (write $a = r/s$ reduced; $v_p(a)$ would need to be divisible by every q). So $a = 1$: the only solutions are $f \equiv 0$ and $f \equiv 1$.

Exercise 14.17 (medium)**.** Solve $f\big(x + f(y)\big) = x + y$ over $\mathbb{R}$.

Solution. $x = 0$ gives $f(f(y)) = y$: an involution, hence bijective. $y = 0$ gives $f(x + f(0)) = x$, i.e. $f(t) = t - c$ with $c = f(0)$. Plug into the involution: $t - 2c = t$ forces $c = 0$, so $f(x) = x$; it verifies.

Exercise 14.18 (hard)**.** Show no *continuous* $f : \mathbb{R} \to \mathbb{R}$ satisfies $f(f(x)) = -x$, then exhibit one over $\mathbb{C}$.

Solution. $f \circ f = -\mathrm{id}$ is bijective, so f is; a continuous bijection is strictly monotone, and the composite of two monotone maps (same or opposite direction) is increasing — but $x \mapsto -x$ is strictly decreasing. Contradiction. Over $\mathbb{C}$: $f(z) = iz$.

Exercise 14.19 (hard)**.** Show every solution of $f(x+y) = f(x) + f(y)$ over $\mathbb{R}$ that is bounded on some interval is linear — the regularity bridge from $\mathbb{Q}$ to $\mathbb{R}$.

Solution. $g(x) = f(x) - f(1)x$ is additive and vanishes on $\mathbb{Q}$; boundedness of f on an interval bounds g near 0, and an additive function bounded near 0 is continuous at 0 (since $g(nh) = ng(h)$, unbounded blow-up otherwise), hence continuous everywhere, hence $g \equiv 0$ — so $f(x) = f(1)x$.

Notes

The candidates→substitutions→involution→Cauchy backbone follows Chen's *Introduction to Functional Equations* [11]; the $\mathbb{N} \to \mathbb{Z} \to \mathbb{Q}$ pipeline and the $\mathbb{R}$-gate are the Excerpts' treatment [9]; Engel's chapter supplies the Jensen and involution stock [10]. The pointwise-trap framing is the "English" handout's two-directions rule [8] applied to sign choice. Contest uses: 2023 AMC 12A P22, P25; 2015 AIME I P10; 2010 AIME I P6; 2024 AIME I P12; USAJMO 2024 P5; JMO 2015 P4; USAMO 2002 (read inside [11]).

Further reading

- E. Chen, *Introduction to Functional Equations* — free PDF; the reference this chapter follows: https://web.evanchen.cc/handouts/FuncEq-Intro/FuncEq-Intro.pdf
- E. Chen, *The English Handout* — how to write "determine all" answers in both directions: https://web.evanchen.cc/handouts/english/english.pdf
- A. Engel, *Problem-Solving Strategies*, functional-equations chapter — the drill stock.
- UNSW CPMSoc workshop slides (the functional-equations PDF is under *Mathematics 2021*): https://unswcpmsoc.com/workshops/
- R. Hammack, *Book of Proof* — for the function/bijection machinery the milestones section reuses: https://richardhammack.github.io/BookOfProof/
- Y. Zhao's olympiad handouts — functional-equation problems in the olympiad style: https://yufeizhao.com/olympiad/

WATCH INSTEAD

- **Michael Penn — "A functional equation that didn't quite make the IMO"**, 6:43, https://www.youtube.com/watch?v=ux9KYSK0jl8 — solves $f(x+y)+f(x-y)=f(x)f(y)$ by exactly the substitution tactics of the Substitution-tactics section: creative value choices, extracting $f(0)$, peeling to one variable. (Verified 2026-10-11.)

Notes

CHAPTER 15

Olympiad geometry

Olympiad geometry rewards *configuration recognition*: the same five or six lemmas — cyclic quadrilaterals, the incenter/excenter picture, power of a point, Ceva, homothety, inversion — generate most of what contests ask. This chapter teaches the tools, their equality cases, and the escape hatch: when synthetic insight stalls, coordinates and complex numbers are honest backup plans, not admissions of defeat.

BEFORE YOU START

Chapter 2 (proofs) and school geometry: angles in a triangle, similar triangles, what a circle is. *Recap:* the inscribed angle theorem says an angle at the circle is half the central angle on the same arc.

AFTER THIS CHAPTER YOU CAN

- find unknown angles systematically by angle chasing;
- recognise cyclic quadrilaterals and use power of a point;
- apply Ceva, Menelaus, homothety and inversion;
- fall back on coordinates when a synthetic proof is not in sight.

HOW TO THINK ABOUT THIS

Olympiad geometry rewards looking for the *reason* an angle or a length is what it is. Draw a large, careful picture, mark every equal angle you can justify, and look for circles hiding in the figure. If you are stuck, you can always switch to coordinates, a slower but reliable backup.

15.1 Angle chasing systematically

School angle chasing uses unsigned angles and silently splits cases — acute vs obtuse, interior vs exterior. *Directed angles* work mod 180°: $\measuredangle(\ell, m)$ is the turn from ℓ to m, so supplements collapse into one statement (Figure 15.1). The upgrade costs one definition and pays back every case split. The core rule: $\measuredangle(l, m) + \measuredangle(m, n) = \measuredangle(l, n)$; and $\measuredangle(l, m) = 0$ iff the lines coincide.

Example 15.1 (Reverse angle chase: prove a line is tangent)**.** Given $\triangle ABC$ with circumcircle ω, show the line ℓ through A with $\measuredangle(\ell, AB) = \measuredangle ACB$ is tangent to ω.

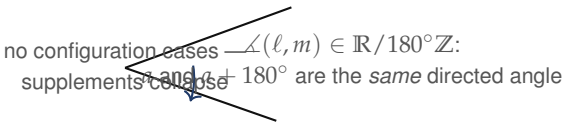


Figure 15.1: Directed angles are taken mod 180°. Notice what this buys: the "supplementary" configuration stops existing.

Strategy. Don't prove tangency head-on — chase backwards from the tangent criterion (a line is tangent iff its angle with the chord equals the inscribed angle in the opposite arc).

Solution. Let t be the tangent at A: then $\measuredangle(t, AB) = \measuredangle ACB$ by the tangent–chord theorem. But ℓ is defined by the *same* angle on the same side, so $\ell = t$: the constructed line *is* the tangent. (Each directed angle value determines a *unique* ray through A — matching the tangent's angle means matching the tangent's line.) Reverse chasing converts "prove tangent" into "match the tangent's angle".

15.2 Cyclic quadrilaterals

Half of olympiad geometry reduces to spotting (or building) a cyclic quad: $ABCD$ cyclic iff $\measuredangle ACB = \measuredangle ADB$ (equal angles on chord AB) iff $\angle A + \angle C = 180^\circ$ (Figure 15.2). Proving concyclicity is often the entire problem in disguise.

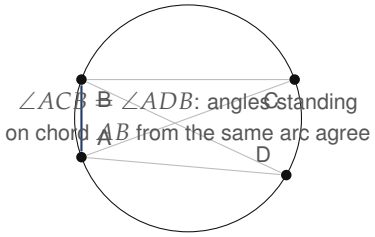


Figure 15.2: Equal angles on a chord force concyclicity. Notice the converse is a construction tool: need two angles equal? Show the four points are concyclic.

Example 15.2 (Spot the cyclic quad first)**.** Acute $\triangle ABC$ has altitudes BE, CF meeting at H. Prove A, F, H, E are concyclic and conclude $\angle FAH = \angle FEH$.

Strategy. Don't compute angles — look for right angles sharing a hypotenuse: two right angles on one segment force a circle by Thales.

Solution. $\angle AFH = 90^\circ$ and $\angle AEH = 90^\circ$: both F and E see AH under a right angle, so both lie on the circle with diameter AH — A, F, H, E concyclic. Equal angles on chord FH now give $\angle FAH = \angle FEH$ without any angle arithmetic. The move repeats everywhere: altitudes, tangents and midpoints all manufacture right angles, and right angles manufacture cyclic quads.

15.3 The incenter/excenter lemma

Lemma 15.3. *Let I, I_A be the incenter and A-excenter of $\triangle ABC$, and M the midpoint of arc BC not containing A on the circumcircle. Then M is equidistant from B, I, C, I_A: $MB = MI = MC = MI_A$.*

Proof sketch. $\angle IBM$ splits as $\angle IBC + \angle CBM$; the first is $B/2$, the second equals $\angle CAM = A/2$ since both subtend arc CM … the symmetric chase gives $\angle IBM = \angle BIM$, so $MB = MI$; excenter goes the same way with external bisectors (Figure 15.3). ■

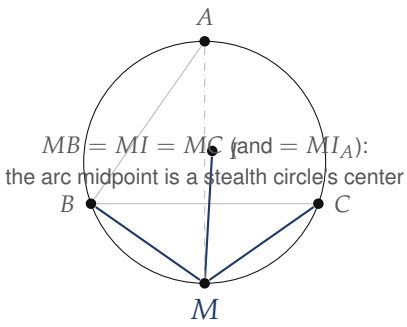


Figure 15.3: The incenter/excenter lemma: arc midpoint M is secretly the center of a circle through B, I, C, I_A. Notice the pattern — when four named points turn out concyclic, a "configuration" is present.

15.4 Power of a point

For point P and circle of radius r center O: every secant through P meets the circle in A, B with $PA \cdot PB = OP^2 - r^2$ (signed); for P outside this equals PT^2 for tangent PT; for P inside, $PA \cdot PB = r^2 - OP^2$ (Figure 15.4). Two circles' equal-power locus is the *radical axis*, a line; three circles give the *radical center* where the three axes concur (Figure 15.5).

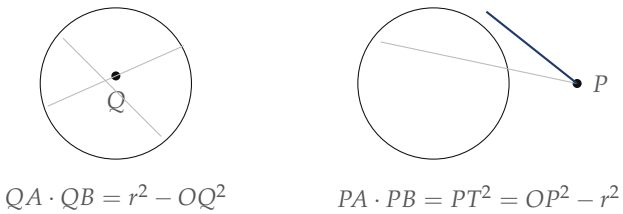


Figure 15.4: Power of a point, inside and out. Notice the value depends only on OP — the *direction* of the secant is irrelevant, which is what makes it useful.

Example 15.4 (Two-secant length computation)**.** Secants from P cut a circle at (A, B) and (C, D) with $PA = 3$, $AB = 5$, $PC = 2$. Find CD.

Strategy. Pick the circle first, then the point — the products $PA \cdot PB$ and $PC \cdot PD$ are the same number.

Solution. $PA \cdot PB = 3 \cdot 8 = 24 = PC \cdot PD = 2 \cdot PD$ so $PD = 12$ and $CD = 10$. The answer is the arithmetic of one invariant.

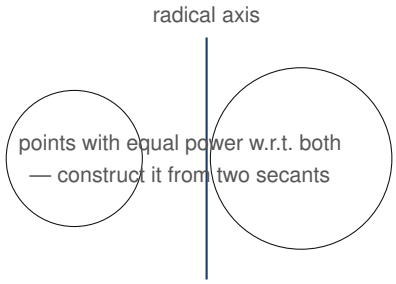


Figure 15.5: The radical axis of two disjoint circles. Notice it exists even when the circles don't meet — "common chord" generalizes to "equal-power locus".

15.5 Ceva, Menelaus, extended sine law

Ceva: cevians AD, BE, CF concur iff $\frac{BD}{DC} \cdot \frac{CE}{EA} \cdot \frac{AF}{FB} = 1$ (directed lengths; the product is $+1$). Menelaus: a transversal cutting the three side-lines gives product -1 (directed sign distinguishes the two theorems) (Figure 15.6). The *extended law of sines*: in any triangle $a/\sin A = 2R$, connecting ratios to the circumradius.

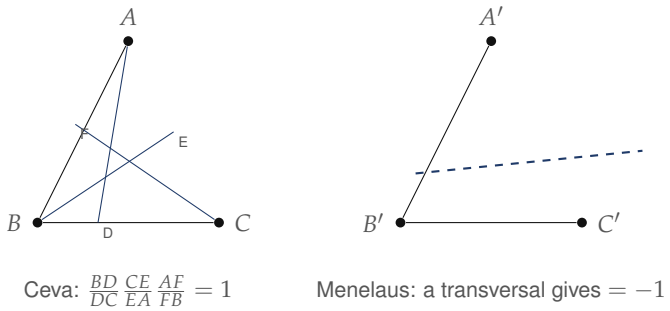


Figure 15.6: Ceva and Menelaus side by side. Notice the directed-length convention is what makes the two statements differ in sign rather than in case lists.

Example 15.5 (Medians concur — and a ratio chase)**. Strategy.** For medians, every factor is 1; for a nontrivial case, write the product in directed form and identify each factor.

Solution. Medians: $BD = DC$ etc., product 1, so the three medians concur. Ratio chase: if D, E, F divide sides in ratios x, y, z cyclically, concurrence iff $xyz = 1$ — e.g. cevians dividing sides in ratios $1/2, 3, 2/3$ concur since $\frac{1}{2} \cdot 3 \cdot \frac{2}{3} = 1$.

15.6 Homothety and spiral similarity

A homothety (dilation) at center Z ratio k sends every figure to a similar one; a *spiral similarity* composes it with a rotation — and is the unique map taking one segment AB to another $A'B'$ directly. Circles tangent internally are homothety images of each other.

Example 15.6 (The nine-point circle exists)**. Strategy.** Don't construct nine points — find one dilation: the homothety at the orthocenter H with ratio $\frac{1}{2}$ should send the circumcircle onto the claimed circle.

Solution. The homothety $h(H, \frac{1}{2})$ sends each vertex to the midpoint of its H-segment and the circumcircle to a circle Γ through the three feet of the altitudes and the three midpoints of the sides' Euler segments; the homothety $h(G, -\frac{1}{2})$ at the centroid sends the circumcircle to the circle through the side midpoints — and both images coincide since N is the midpoint of OH (Figure 15.7). Existence is a corollary of finding the dilation; the nine points ride for free.

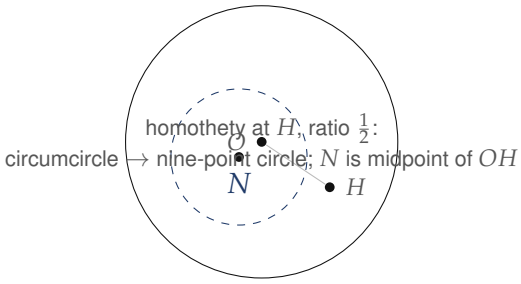


Figure 15.7: The nine-point circle as homothety image. Notice the proof is one map, not nine verifications — the dilation *is* the theorem.

15.7 Inversion in a circle

Inversion in circle (O, r) sends $P \mapsto P'$ on ray OP with $OP \cdot OP' = r^2$. It swaps inside/outside, reverses orientation, and converts between "lines" and "circles through O" (Figure 15.8): lines $\not\ni O \leftrightarrow$ circles $\ni O$; circles $\not\ni O \leftrightarrow$ circles $\not\ni O$. Choose the center where the problem's concurrency or tangency lives. Worked transformation: to prove four circles

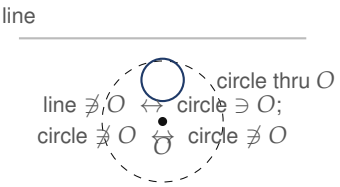


Figure 15.8: Inversion's image table. Notice what it trades: your mess of circles becomes lines — and tangency (a hard thing to use) becomes parallelism or collinearity (easy).

share a common point, invert about one circle's center; the four become lines or circles through O whose concurrency is a collinearity check. This chapter teaches recognition and one deployment; the deeper problem set lives in the Notes.

15.8 Analytic backup plans

Coordinates, areas, trigonometric form, and Ptolemy's inequality ($AC \cdot BD \leq AB \cdot CD + BC \cdot AD$ for any quadrilateral, equality iff cyclic) are the backup plan — honest, mechanical, and sometimes strictly cheaper than synthetic insight. Fix scale and orientation first (e.g. unit circle at origin); choose analytic when the problem is asymmetric or the synthetic handle is invisible.

15.9 Summary

Tool	What it buys
Directed angles	no case splits
Cyclic quads	equal angles $\leftrightarrow$ concyclic
Incenter/excenter	hidden circle centers
Power of a point	$PA \cdot PB$ invariant
Ceva/Menelaus	concurrency/collinearity $= \pm 1$
Homothety	one map = existence proof
Inversion	circles $\leftrightarrow$ lines
Coordinates/complex	the backup plan

15.10 Code: two numerical verifications

`power_check.py` places coordinates and verifies $PA \cdot PB = PT^2 = OP^2 - r^2$ plus the interior variant. `invert.py` inverts a sampled line and checks its image lies on the predicted circle through O.

Checkpoint 15.1 Predict the common value for $P = (2, 1)$ on the unit circle, and which circle the line $x = 1$ maps to under $r = 2$ inversion. Then run both.

```
secant: PA*PB = 4.000000
tangent: PT^2 = 4.000000
agree: True
interior: QA*QB = 0.870000 vs r^2-|OQ|^2 = 0.870000
line x=1 -> 13 image points
max distance from predicted circle (center (2,0), r=2): 4.44e-16
inv(inv(P))=P check: (2.999..., 1.0) vs (3.0, 1.0)
|OP|*|OP'| = 4.000 = r^2 = 4.0
```

`test_15_geom.sh` re-runs and greps the agreement lines.

15.11 Common mistakes

- *Unsigned angles landing on the supplement* — the EGMO-errata mistake; directed angles exist precisely to prevent it.
- *Ceva/Menelaus on unsigned ratios* — the sign is the whole point; undirected products can't tell the theorems apart.
- *Assuming concyclicity* — three points always define a circle, four almost never do; prove it before using it.
- *Wrong inversion center* — the circle of inversion is a choice; put it at the concurrency or tangency point, and remember orientation flips.
- *Unfixed coordinates* — choosing axes before fixing scale and orientation doubles the algebra.

Exercises

Inline

Exercise 15.1 (easy)**.** Show two parallel chords of a circle intercept equal arcs.

Solution. Inscribed angles on equal arcs; directed angles give it in one line.

Exercise 15.2 (easy)**.** Secant: $PA = 4$, $AB = 6$; find PT for a tangent from P.

Solution. $PT^2 = 4 \cdot 10 = 40$, $PT = 2\sqrt{10}$.

Exercise 15.3 (medium)**.** Prove the tangent–chord theorem used in the first worked example.

Solution. Draw the diameter through the contact point; the tangent is perpendicular; inscribed-angle bookkeeping finishes.

Exercise 15.4 (medium)**.** Show the three radical axes of three circles (taken pairwise) concur at the radical center — when they are not parallel.

Solution. A point on two axes has equal power w.r.t. all three circles, so it lies on the third axis too.

Exercise 15.5 (medium)**.** Ceva: points D, E, F on BC, CA, AB with $BD/DC = 2$, $CE/EA = 1/2$, $AF/FB = 1$. Concur?

Solution. $2 \cdot \frac{1}{2} \cdot 1 = 1$: yes.

Exercise 15.6 (medium)**.** Inversion: what is the image of a circle through O under inversion about (O, r)?

Solution. A line not through O — the "missing point" O on the circle opens it into a line.

Additional

Exercise 15.7 (easy)**.** Two secants: $PA \cdot PB = PC \cdot PD$ implies A, B, C, D concyclic — state and prove the converse.

Solution. Equal powers put P on both circles' radical axis — the converse of power of a point gives the circle through A, B, C, D.

Exercise 15.8 (easy)**.** Compute the power of the circle's own center O.

Solution. $-r^2$ (signed) — the minimum possible power.

Exercise 15.9 (easy)**.** Angles on the same arc are equal — give the directed-angle one-liner.

Solution. $\measuredangle ACB = \measuredangle ADB$ iff A, B, C, D concyclic; the proof is the inscribed-angle theorem with signs.

Exercise 15.10 (easy)**.** A circle through O inverts to a line — what happens to O itself?

Solution. O has no image ($OP \cdot OP' = r^2$ is degenerate at 0); the line is the circle "minus O" closed at infinity.

Exercise 15.11 (medium)**.** Use the incenter/excenter lemma to find the center of the circle through B, I, C, I_A and its radius in terms of M's position.

Solution. Center M; radius MB — computable from $2R\sin(A/2)$.

Exercise 15.12 (medium)**.** Ceva form of concurrency for angle bisectors: verify the incenter via the bisector theorem.

Solution. By the angle-bisector theorem, $BD/DC = c/b$, and likewise for the other two bisectors; the product $\frac{c}{b} \cdot \frac{a}{c} \cdot \frac{b}{a} = 1$ satisfies Ceva.

Exercise 15.13 (medium)**.** Show a spiral similarity taking $AB \mapsto CD$ has center at the intersection of circles through (A, C) and (B, D) — the "pivot theorem" idea.

Solution. The center sees AC and BD under the same rotation angle; the two circles' second intersection is it.

Exercise 15.14 (medium)**.** Ptolemy: prove $AC \cdot BD = AB \cdot CD + BC \cdot AD$ for cyclic $ABCD$ by the inscribed-angle trick on the diagonal.

Solution. Take E on AC with $\angle ABE = \angle DBC$; two similar triangles give the two products.

Exercise 15.15 (medium)**.** Implement-or-derive: extend `power_check.py` to the interior case for a *random* chord through Q, 100 trials.

Solution. Loop over random directions; every trial's $QA \cdot QB$ lands on $r^2 - |OQ|^2$ to machine precision.

Exercise 15.16 (hard)**.** Prove the radical axis is a line, not just a locus.

Solution. $|PO_1|^2 - r_1^2 = |PO_2|^2 - r_2^2$ expands to a linear equation in P's coordinates — the quadratic parts cancel.

Exercise 15.17 (hard)**.** Inversion converts the statement "two circles are orthogonal" into what on their images?

Solution. Angle-preserving (conformal) maps preserve orthogonality; a circle orthogonal to the inversion circle maps to itself.

Exercise 15.18 (hard)**.** Derive the nine-point circle radius in terms of R and locate its center on the Euler line.

Solution. Radius $R/2$ (the $\frac{1}{2}$ homothety); center N at the midpoint of OH, between G and H on the Euler line O–G–N–H.

Exercise 15.19 (easy)**.** In cyclic $ABCD$, $\angle ABD = 40^\circ$. Find $\angle ACD$.

Solution. 40° — equal angles on chord AD.

Exercise 15.20 (medium)**.** Point P inside a circle has two chords through it: AB with $PA = 4$, $PB = 9$ and CD with $PC = 3$. Find PD, and explain which theorem is doing the work.

Solution. Intersecting-chords case of power of a point: $PA \cdot PB = PC \cdot PD$ gives $PD = 36/3 = 12$.

Exercise 15.21 (hard)**.** Show the midpoints of the three sides of a triangle and the feet of the three altitudes are concyclic (the cheap half of the nine-point circle) using only the cyclic-quad criterion twice.

Solution. Let M_a, M_b, M_c be side midpoints and D, E, F the feet. The midpoint quadrilateral $M_aM_bM_c$ is homothetic to ABC at ratio $1/2$ about the centroid; for the feet: $M_bM_c \parallel BC$ and $\angle BEC = \angle BFC = 90^\circ$ puts E, F on the circle of diameter BC centered at M_a; combine with the trapezoid M_bM_cEF being isosceles-cyclic to close.

Notes

The configuration-first pedagogy and directed-angles-early order follow EGMO's table of contents and its errata [18]; the incenter/excenter lemma, the pivot theorem and the inversion image table follow Chen's geometry handouts [19]. Zhao's cyclic-quadrilateral, power-of-a-point and "Three Lemmas" handouts carry the full inversion problem set routed here by the merge decision [12]. Ceva/Menelaus in directed form and the analytic backups follow Engel [10]. Contest uses: 2024 AIME I P10, P5, P8, P12; 2023 AIME I P5, P8; 2023 AIME II P9, P12; 2022 AIME I P8; 2021 AIME I P11, P13; 2020 AIME I P1, P15; 2018 AIME I P4, P13, P15; 2015 AIME I P11; 2025 AIME II P1, P5/P6; 2024 AMC 12A P19; USAJMO 2024 P1, P6; 2023 USAJMO P2, P6.

Further reading

- E. Chen, *Euclidean Geometry in Mathematical Olympiads* (EGMO) — the reference this chapter's order follows; Ch. 1–3 cover everything here except inversion.
- E. Chen's geometry handouts — `https://web.evanchen.cc/handouts/` — free PDFs: incenter/excenter lemma, pivot theorem, directed angles.
- Y. Zhao's olympiad geometry handouts — `https://yufeizhao.com/olympiad/` — cyclic quadrilaterals, power of a point, and "Three Lemmas in Geometry".

- A. Engel, *Problem-Solving Strategies*, geometry chapters — Ceva, Menelaus and the analytic backup plans.
- R. Hammack, *Book of Proof* — for writing geometric arguments as clean proofs: https://richardhammack.github.io/BookOfProof/
- The AoPS community's geometry contest collections — every cited AIME/USAJMO item has community solution threads there: https://artofproblemsolving.com

WATCH INSTEAD

- **Michael Greenberg — "Olympiad Geometry Problem #93: Two Isosceles Triangles"**, 12:25, https://www.youtube.com/watch?v=Hz8kyLKP0_k — a spiral similarity manufactures a cyclic quad exactly as the cyclic-quadrilaterals and homothety sections teach. (Verified 2026-10-11.)
- **Sohil Rathi — "Circular Geometry, Power of a point, Cyclic Quadrilaterals"**, 1:33:35, https://www.youtube.com/watch?v=NsQbhYfGh1Q — an AMC-level class lecture covering power of a point and cyclics live on problems. (Verified 2026-10-11.)

Notes

CHAPTER 16

Invariants, monovariants and extremal arguments

Three questions dominate olympiad problems: what survives a process (invariant), why it must end (monovariant), and where the boundary case lives (extremal principle). Together they turn "show it's impossible" into "count the conserved quantity" — the cheapest proofs in the book once you learn to look for what doesn't move.

BEFORE YOU START

Chapters 2–3 and 8. *Recap:* an *invariant* is a quantity that never changes during a process; a *monovariant* is a quantity that only moves in one direction.

AFTER THIS CHAPTER YOU CAN

- find an invariant that proves a process can never reach a target state;
- use a monovariant to prove a process must stop;
- apply extremal arguments ("consider the smallest…") and parity colourings;
- analyse a simple game by finding a winning strategy.

HOW TO THINK ABOUT THIS

When you are asked whether something is possible, look for something that never changes (an invariant) or that only moves one way (a monovariant). If a target state breaks the unchanging quantity, it is impossible; if a quantity must keep shrinking, the process must stop. Try small cases until a quantity suggests itself.

16.1 Invariants

An *invariant* is a quantity (or parity, or property) unchanged by every legal move. The discipline: state it, then check *every* move, not the moves you expect. If a target configuration has a different value, it is unreachable — impossibility for free.

Example 16.1 (The chocolate bar)**.** Breaking an $m \times n$ bar along grid lines, one piece at a time: prove you always need exactly $mn - 1$ breaks.

Strategy. Don't count cuts — count what each move preserves. Every break splits one piece into two: the *number of pieces* rises by exactly 1 no matter where the break lands.

Solution. The component count is the monovariant: 1 piece at start, mn pieces at the end, $+1$ per break — so $mn - 1$ breaks, always (Figure 16.1). Order and shape are irrelevant; the invariant does the whole proof.

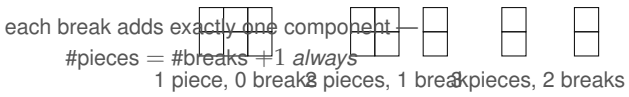


Figure 16.1: The chocolate-bar invariant. Notice you never need to know *where* the break happened — only that components went up by 1.

16.2 Monovariants

A *monovariant* is a measure that only moves one way — usually a nonnegative integer that strictly decreases. Since it can't decrease forever, the process *terminates* (Figure 16.2): termination is a corollary, not a separate proof.

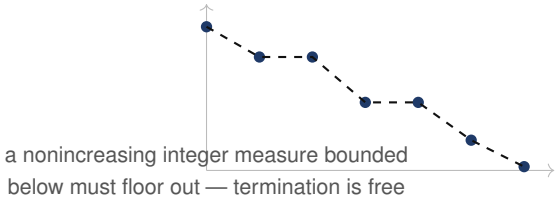


Figure 16.2: A monovariant plot. Notice what makes it a proof: integer, nonnegative, strictly dropping — three properties, one conclusion.

Example 16.2 (Difference reduction terminates)**.** On a list of positive integers, replace two entries a, b by $|a - b|$ until one number remains. Prove the process ends and the survivor equals gcd of the list.

Strategy. Two measures: the *sum* (strictly drops — monovariant, gives termination) and the *gcd* (never changes — invariant, gives the answer).

Solution. Each move replaces $a + b$ by $|a - b| < a + b$: the sum is a strictly decreasing positive integer, so the list must shrink to one entry. Every move also preserves gcd: $\gcd(a, b, \dots) = \gcd(|a - b|, b, \dots)$. The survivor therefore equals $\gcd(a_1, \dots, a_n)$ — verified numerically in §16.8.

16.3 The extremal principle

On a finite set, a minimum *exists*: name it and argue on it. "Take the smallest counterexample" is induction in extremal clothes; "take the closest pair" is geometry's version (Figure 16.3). The trap: the principle needs *finiteness or well-ordering* — "the least n" exists on $\mathbb{N}$ but not on $\mathbb{Z}$.

Example 16.3 (Closest pair as a proof device)**.** Among $n \geq 3$ points in the plane, not all collinear, show that some line passes through exactly two of them (Sylvester–Gallai).

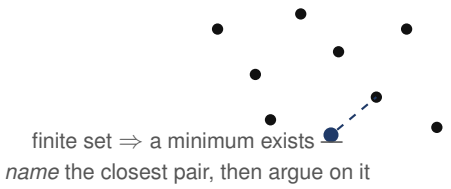


Figure 16.3: The extremal principle. Notice the two-step pattern: existence from finiteness, then every argument runs on *that* witness.

Strategy. Don't search all $\binom{n}{3}$ triples — take the minimum over all (point, line) distances and argue on *that* witness.

Solution. Consider all pairs (P, L) where L is a line through two given points and P is a given point not on L; such pairs exist because the points are not all collinear, and they are finitely many, so one pair (P, L) minimizes $\operatorname{dist}(P, L)$. Write $L = QR$ with Q, R given points, and let F be the foot of the perpendicular from P to L. Suppose L contained a third given point S. Then Q, R, S are three points of the set on L, so two of them — call the nearer one A and the farther B — lie on the same side of F (possibly $A = F$). For X on L, $\operatorname{dist}(X, \text{line } PB) = |XB| \sin \angle PBL$; hence

$$\operatorname{dist}(A, PB) = |AB| \sin \angle PBL \le |FB| \sin \angle PBL = \operatorname{dist}(F, PB) < |PB| \sin \angle PBL = \operatorname{dist}(P, L),$$

using $|FB| < |PB|$ (hypotenuse of the right triangle PFB). But PB is a line through two given points and $A \notin PB$ (otherwise $PB = L$ would contain P), so (A, PB) is a pair with smaller distance — contradicting minimality. Therefore L passes through exactly two of the points.

Significance. The witness does the work; the extremal principle supplied it. Note what the minimum had to be *of*: the closest *pair* alone is not enough (three collinear points defeat the naive version), so the quantity minimized is upgraded to a point–line distance.

16.4 Advanced pigeonhole

Generalized form: if N objects land in k boxes, some box holds $\ge \lceil N/k \rceil$. Averaging form: if numbers average $\bar{x}$, some entry is $\ge \bar{x}$ (and some $\le$). These are the same statement; olympiad use is always of the form "so one object *must* have the property".

16.5 Coloring and parity arguments

Coloring turns structure into arithmetic: assign labels so every legal move has a known signature, then count.

Example 16.4 (The domino impossibility)**.** A 10×10 board minus two *opposite* corners: can dominoes tile it?

Strategy. Checkerboard it: every domino covers one square of each color, so a tiling requires equal color counts.

Solution. Opposite corners share a color (even side length); the board has 48 of one color and 50 of the other — no tiling exists (Figure 16.4). The proof is four words long once the coloring is found: *count the colors*.

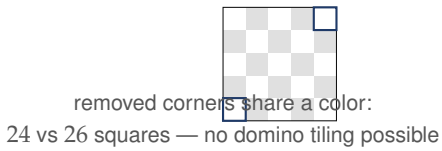


Figure 16.4: The coloring argument. Notice the impossibility needs only the imbalance — you never exhibit a failed tiling.

16.6 Games

Positions split into *winning* (a move to a losing position exists) and *losing* (every move lands on a winning position) — compute backwards from terminal states (Figure 16.6). *Strategy stealing* and *pairing* handle existence claims: mirror the opponent's moves across a center (Figure 16.5) so their play produces yours.

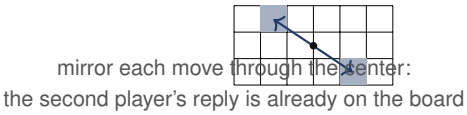


Figure 16.5: The pairing strategy. Notice whose move it is: the second player's reply exists because the board was symmetric before the first player touched it.

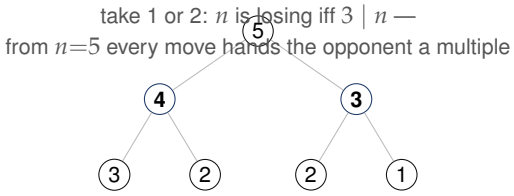


Figure 16.6: Take-away game tree ($n = 5$, take 1 or 2). Notice the period: losing positions repeat mod 3 — the invariant-style pattern inside every impartial game.

Example 16.5 (Pairing the second player)**.** Two players alternately place identical coins on a round table without overlap; the last to place wins. Who wins?

Strategy. Don't design a first-player attack — hand the second player a mirror.

Solution. First player places a coin at the center; thereafter whatever the opponent plays is answered by the centrally symmetric spot. Every opponent move creates a legal mirror move, so the first player always has a reply and wins. The symmetry does everything after move one.

16.7 Summary

Tool	Kills which question
Invariant	"can it reach?" — different value, unreachable
Monovariant	"does it end?" — strictly drops, must floor
Extremal	"does it exist?" — name the minimum
Pigeonhole	"must one have P?" — $\lceil N/k \rceil$
Coloring	"can it tile/balance?" — count the colors
W/L positions	"who wins?" — backtrack from terminal
Pairing/mirror	"who wins?" — borrow their move

16.8 Code: watching the invariant

`invariant_sim.py` runs the difference-reduction process and prints the list, the dropping sum (monovariant) and the constant gcd (invariant) at each step.

Checkpoint 16.1 Predict the survivor before running: $[12, 18, 30, 42]$ should leave its gcd. Then run `python3 invariant_sim.py`.

```
start: [12, 18, 30, 42]
t=1: [12, 18, 12] sum=42 gcd=6 (invariant 6)
t=2: [6, 12] sum=18 gcd=6 (invariant 6)
t=3: [6] sum=6 gcd=6 (invariant 6)
survivor = 6 == gcd([12,18,30,42]) = 6: True
```

The sum column is the termination certificate; the gcd column is the answer. The test script `test_16_inv.sh` re-runs the sim and greps the survivor line.

16.9 Common mistakes

- *Checking the moves you meant, not all legal moves.* An invariant must survive *every* legal transition — one unchecked case voids it.
- *Extremal on an infinite set.* "Take the least counterexample" needs well-ordering: valid on $\mathbb{N}$, meaningless on $\mathbb{R}$.
- *Draw ⇒ strategy.* Showing a position isn't losing for player one doesn't hand player two a strategy — who moves first is part of the claim.

Exercises

Inline

Exercise 16.1 (easy)**.** A stack of coins: each move splits one stack into two. Prove finishing takes $n - 1$ moves for n single coins.

Solution. Same invariant as the chocolate bar: components $+1$ per move.

Exercise 16.2 (easy)**.** On a chessboard, a rook moves any number of squares. Is "color of the square" an invariant under a rook move?

Solution. No — a rook move may end on either color, so color is not invariant under rook moves. (A knight move, by contrast, always flips color; a quantity that changes predictably is a *monovariant*, the next section's tool.)

Exercise 16.3 (medium)**.** Start with $\{1, 2, \ldots, 2n\}$ on a board; each move erases two numbers a, b and writes $|a - b|$. Prove the last number is even iff n is even.

Solution. Parity of the sum is invariant (sum drops by 2 min); initial sum $n(2n + 1)$ has parity of n.

Exercise 16.4 (medium)**.** Show the take-away game (take 1 or 2) is losing exactly at $3 \mid n$.

Solution. From $3k$ every move hits $3k \pm \{1, 2\}$; from a non-multiple, take the remainder to land on a multiple. Induction on n.

Additional

Exercise 16.5 (easy)**.** Prove an 8×8 board minus two *adjacent* corners *can* be tiled by dominoes.

Solution. Adjacent corners differ in color: 31–31 balance; exhibit a snake tiling row by row.

Exercise 16.6 (easy)**.** Is $a + b$ invariant under replacing a, b by $a - b$? Check every move type.

Solution. No: $a + b \to (a - b) + b = a$ changes it; the sum *decreases* — that's the monovariant, not an invariant.

Exercise 16.7 (easy)**.** Among 13 integers, two are congruent mod 12 — state the boxes.

Solution. Residues mod 12 are the 12 boxes; 13 pigeons.

Exercise 16.8 (easy)**.** Give a nonnegative integer monovariant for the Euclidean algorithm.

Solution. The smaller argument b strictly drops each division step; it's a positive integer, so Euclid terminates.

Exercise 16.9 (medium)**.** Prove every $2^n \times 2^n$ board minus one square is tileable by L-trominoes (invariant/induction form).

Solution. Induction: quarter the board, put one L at the center covering one square of each of the three "complete" quarters; each quarter is now a smaller instance.

Exercise 16.10 (medium)**.** At a party any two people have a common friend; prove someone knows everyone (extremal hint).

Solution. Take a person M of maximum degree; if some X weren't M's friend, M and X share a friend of degree $>$ deg M.

Exercise 16.11 (medium)**.** 51 numbers from $\{1, \ldots, 100\}$: show two are consecutive — identify the coloring/box structure.

Solution. Pairs $\{1,2\}, \{3,4\}, \ldots, \{99,100\}$ are 50 boxes; 51 picks force a full pair.

Exercise 16.12 (medium)**.** Winning strategy for one-pile Nim: take any number up to k. State and prove the losing set.

Solution. Losing iff $(k+1) \mid n$: multiples map to non-multiples and back, same induction as take-1-or-2.

Exercise 16.13 (medium)**.** Implement-or-derive: extend `invariant_sim.py` so the move picks a random pair each step (seeded), and confirm the survivor is still the gcd.

Solution. The invariant doesn't care which pair moves; random or largest-first, the survivor is 6.

Exercise 16.14 (hard)**.** Prove the circle-difference process (replace each a_i by $|a_i - a_{i+1}|$ on an even cycle of integers) reaches all zeros when n is a power of two, via a monovariant + invariant combo.

Solution. Max is nonincreasing; parity analysis (all entries become even after $\leq$ a few steps, then factor out powers of two) forces max to hit 0. The invariant is gcd-divisibility; the monovariant is max after rescaling.

Exercise 16.15 (hard)**.** Strategy stealing: prove the first player in tic-tac-toe (on an infinite board, get 5 in a row) cannot be worse off than the second.

Solution. Suppose second player had a winning strategy; first plays an arbitrary opening move, then "becomes" the second player — an extra mark can never hurt in a strong positional game.

Exercise 16.16 (hard)**.** Prove there is no perfect cuboid argument's first step: in a set of distinct positive integers whose pairwise sums are all squares, an extremal argument bounds the set — give the extremal witness to name.

Solution. Take the largest element M: every partner's sum lies in $(M, 2M)$, so partner squares are restricted to squares in that interval — finitely many candidates force a bound. (Open-problem flavor; the point is naming the witness.)

Exercise 16.17 (easy)**.** A chessboard has two opposite corners removed. Show 31 dominoes cannot tile it, and name the invariant.

Solution. Each domino covers one black and one white square; the removed corners share a colour, leaving 30 of one colour vs 32 of the other — the colour-difference invariant $|W - B| = 2$ survives every placement and no tiling can reach 0.

Exercise 16.18 (medium)**.** Numbers $1, \ldots, 10$ sit on a circle. A move picks two adjacent numbers and swaps them. Prove no sequence of moves arranges them in decreasing order clockwise if they start increasing — or find a counterexample and diagnose what the invariant actually is.

Solution. Adjacent transpositions generate S_n, so decreasing order *is* reachable — the puzzle as stated is a trick with a counterexample, not an impossibility. Where parity *does* supply a real invariant is the 15-puzzle: the permutation parity combined with the blank's position is preserved, and it blocks exactly half of all configurations. Lesson: an impossibility claim needs an invariant that genuinely never changes — guess first, verify the invariant before believing it.

Exercise 16.19 (hard)**.** Show the sum of the numbers on a n-nim position (in binary XOR, not decimal) is the true invariant/monovariant: prove a position is winning iff the nim-sum is nonzero, from the definitions.

Solution. Terminal position has nim-sum 0; from any nonzero position a single heap reduction makes the XOR zero (pick the heap containing the top bit); from a zero position every move breaks a bit balance and leaves nonzero. Winning positions are exactly the nonzero ones.

Notes

Invariants, colorings, the extremal principle and game positions follow Engel [10] and Zeitz [20]; the state-machine invariant framing is 6.042J's [4]; pairing and strategy-stealing examples are Cut-the-Knot's games index [6]. Contest uses: 2024 AIME I P3; 2023 AIME I P14; 2021 AIME I P10; 2024 AMC 12B P5; 2024 AMC 12A P22; 2010 AIME I P12; 2022 AIME I P9; 2025 AIME II P3; 2023 AIME II P11; USAJMO 2024 P2, P4; 2023 USAJMO P3, P4, P5.

Further reading

- A. Engel, *Problem-Solving Strategies*, invariants and coloring chapters — the canonical drill source for this chapter.
- P. Zeitz, *The Art and Craft of Problem Solving* — invariants and the extremal principle with worked strategy.
- E. Lehman, F. T. Leighton, A. R. Meyer, *Mathematics for Computer Science*, state-machine invariants chapter — free PDF; the CS framing of invariants: https://courses.csail.mit.edu/6.042/spring18/mcs.pdf
- MIT OpenCourseWare, *6.042J* — https://ocw.mit.edu/6-042JF10 — the matching lectures.
- Y. Zhao's olympiad handouts — https://yufeizhao.com/olympiad/ — invariant and extremal problem sets.
- Cut-the-Knot's games index — https://www.cut-the-knot.org — pairing and strategy-stealing examples in interactive form.

WATCH INSTEAD

- **ChessNetwork — "Mutilated chessboard problem"**, 4:07, https://www.youtube.com/watch?v=bS3EqMcIlAQ — the colouring invariant of the Coloring-arguments section in its purest form. (Verified 2026-10-11.)
- **Mathologer — "The Arctic Circle Theorem"**, 51:49, https://www.youtube.com/watch?v=Yy7Q8IWNfHM — opens on the same mutilated chessboard, then shows how far tiling invariants go. (Verified 2026-10-11.)

Notes

CHAPTER 17

Writing a four-hour exam and the proof portfolio

The previous sixteen chapters taught the mathematics; this one teaches the interface where mathematics meets a clock and a grader. A contest score is not a sum over problems you solved — it is a sum over proofs the grader could *follow*. After this chapter you can budget four hours, write a proof a grader marks, and maintain a twenty-proof portfolio that measures itself.

BEFORE YOU START

Chapters 2–3 as the minimum, and as many of Chapters 5–16 as you have time for. *Recap:* this chapter is about writing proofs under time pressure; the maths is whatever you have already learned.

AFTER THIS CHAPTER YOU CAN

- split four hours among problems sensibly;
- write a proof that earns full credit and a partial solution that earns partial credit;
- check your own work for the usual gaps;
- assemble a portfolio of twenty polished proofs.

HOW TO THINK ABOUT THIS

A proof is only worth the points a reader can find in it. Under time pressure, the way of thinking is to be strategic: secure the problems you can finish, write each as if for a stranger, and never leave a half-claim that you have not marked as unproved. Honest partial progress earns credit; a bluff costs it.

17.1 What a timed contest tests

Homework tests whether you can solve a problem eventually. A four-hour contest tests whether you can solve *enough, fast, and legibly*. The skill that homework never exercises is abandonment: knowing that a problem you *could* solve in three hours is a loss if the other five were solvable in thirty minutes each.

Example 17.1 (Triage by rate, not by difficulty)**.** An exam offers problems worth 7 points each: P_1 looks solvable in 40 minutes, P_2 in 90, P_3 is interesting and you have spent 25 minutes already with no progress. Rank the next hour.

Strategy. Score is additive; time is the constraint — maximize expected points per *hour*, not per problem, and treat sunk cost as zero.

Solution. P_1: $\approx 7/\frac{2}{3} = 10.5$ pts/hr; P_2: $\approx 7/1.5 \approx 4.7$; P_3: unknown remainder, likely > 60 min more for at most 4.7. The 25 minutes on P_3 are gone either way — they do not raise its rate. Order: P_1 first, then re-evaluate P_2 vs returning to P_3 with fresh eyes (Figure 17.1).

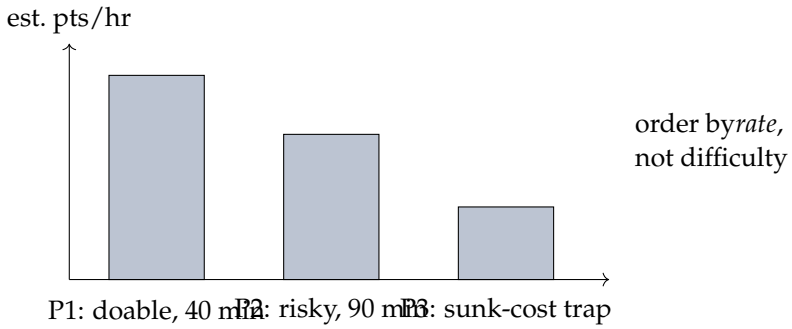


Figure 17.1: Triage orders by expected points per hour, not by how close a problem *feels*. Notice P_3's sunk cost buys nothing — only the remaining estimate counts.

17.2 The four-hour budget

Triage first, always: read every problem before writing any (Figure 17.2). First pass solves the problems you already know how to finish — typically half the marks for a quarter of the time. Deep work then goes to *one* problem at a time, chosen by point value times your probability of finishing, not by how interesting it looks. A running clock answers "should I abandon this?" — fifteen minutes without a new idea is a new-idea problem, not a persistence problem.



Figure 17.2: A four-hour budget. Notice the protected last thirty minutes: checking is scheduled, not squeezed.

Example 17.2 (Two budgets compared)**.** 15 answer-only problems in 3 hours vs 6 proofs in 4.5 hours: how should the allocation differ?

Strategy. Budget by expected value: (points) $\times$ (probability of completion), re-evaluated at each checkpoint.

Solution. Answer-only: uniform-ish spread, abandon at the first stall (15 $\times$ 12 min each; a 20-minute problem is a losing one). Proof contest: front-load certainty — the safest proof goes first, the 50/50 problem gets a hard time cap. The budgets differ in *shape*: the first is flat, the second is a waterfall.

17.3 Write-up standards

A grader reads top-to-bottom at speed. So: **claim first** — state what you will prove before proving it. **Name the lemmas** — a lemma with a name is a checkpoint the grader can tick. **Display the math** — buried algebra is invisible algebra. **Draw the diagram** — one figure replaces a paragraph. **Follow checking conventions** — the argument order is invention order reversed (Figure 17.3).

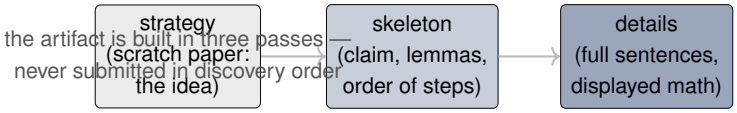


Figure 17.3: The write-up pipeline. Notice what changes at each pass: the idea stays fixed while the *reader's needs* are added.

Example 17.3 (Scratch vs submission)**.** Chapter 16's difference-reduction problem, shown as (a) the scratch note a solver actually writes and (b) the text a grader receives.

Strategy. Same content, different artifact: scratch says "try small cases, gcd stays!"; the submission turns that discovery into claim–lemma–proof order.

Solution. *Scratch:* "$12, 18, 30, 42 \to$ differences, gcd seems to survive, sum shrinks so it stops — write it up." *Submission:* **Claim.** The survivor equals $\gcd(a_1, \ldots, a_n)$. **Lemma (Termination).** The sum strictly decreases, so the process ends. **Lemma (Invariant).** gcd is preserved since $\gcd(a - b, b) = \gcd(a, b)$. **Proof.** Termination gives a final state; the invariant evaluates it. ■ The scratch line contained the whole solution; the submission made it gradeable.

17.4 Partial credit and fake-solving

Graders reward *progress they can verify*: a stated claim, a proved lemma, a correctly argued case. They cannot reward "I was close". *Fake-solving* — asserting the hard step as "clearly" or "similarly" — earns nothing and costs the time you spent writing it (Figure 17.4). The honest partial write-up that proves the easy case and names the gap out-scores a confident bluff every time.

Example 17.4 (Autopsy of a bad write-up)**.** "Clearly the survivor is the gcd. Trying a few cases shows it works, so we are done."

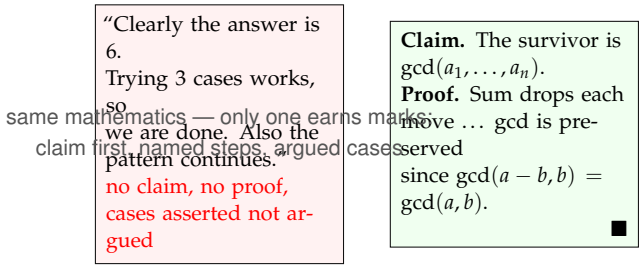


Figure 17.4: Bad vs good write-up of the same mathematics. Notice the bad one isn't shorter — it is *unverifiable*, which reads as zero.

Strategy. Grade it: mark what a grader could verify.

Solution. No claim environment, no termination argument, the gcd-preservation step asserted not shown, and "a few cases" carries zero weight against an infinite family. Verifiable content: none — 0/7 (on the 0–7 USAMO scale). The fix is not more words; it is the two named lemmas.

17.5 Checking your work

Last thirty minutes, in order of value: (1) re-read each submission's claim — does the proof actually end at it? (2) check boundary cases — the $n = 0$, empty-set, single-element input. (3) verify find-all answers in both directions: candidates found *and* verified. (4) recompute any arithmetic that appeared only once on the page (Figure 17.5).

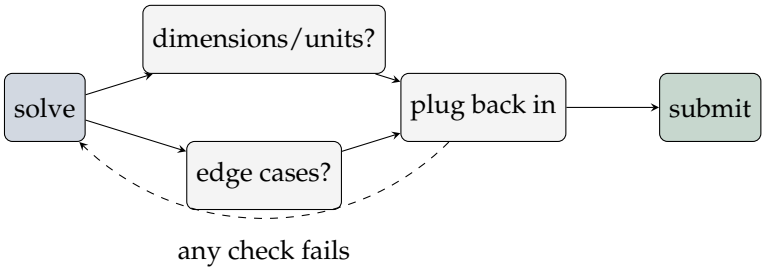


Figure 17.5: The check loop: three cheap checks before submit, and any failure routes back to solving. Notice checks are ordered by cost — dimensions are seconds, plug-back is the thorough audit.

Example 17.5 (A check that catches the answer)**.** A solution claims the sequence $a_{n+1} = 2a_n + 1$, $a_0 = 0$ has closed form $a_n = 2^{n-1} - 1$. Run the check loop.

Strategy. Cheap check first: the boundary value $n = 1$ must already satisfy it.

Solution. $a_1 = 1$ from the recurrence; the formula gives $2^0 - 1 = 0$ — contradiction at the first check, costing seconds. The correct form is $a_n = 2^n - 1$: $2^1 - 1 = 1$, $2^2 - 1 = 3$ match the recurrence's a_1, a_2, and induction confirms. One boundary case turned a confidently-wrong answer into a fix before submission.

17.6 The 20-proof portfolio

Twenty proofs, written and self-graded on the rubric (Figure 17.6): claim stated, lemmas named, cases argued, math displayed, self-check passed — each scored 0/1/2. Keep the lowest-scoring proof, rewrite it, and only then add a twenty-first. The portfolio grows by *replacing* your weakest work, not by accumulating new work.

rubric item	0 / 1 / 2
claim stated first	absent/buried/first
lemmas named	none/some/all
every case argued	asserted/mixed/argued
displayed math	inline/mixed/displayed
self-check pass	skipped/errors/clean

Figure 17.6: The portfolio rubric. Notice it is mechanical on purpose — a rubric that needs judgment can't grade itself.

17.7 Code: a linter, not a grader

`portfolio_check.py` scans your proof files for the mechanical parts of the rubric — a claim environment, named lemmas, a proof environment, a `\qed`. It cannot see fake-solves; it only sees missing structure. That is the honest division of labor.

Checkpoint 17.1 Predict which sample file passes. The good one carries a theorem, a named lemma, a proof and `\qed`; the bad one is prose. Then run the linter.

```
sample_proofs/bad_proof.tex:
  missing: claim first
  missing: named lemma
  missing: proof env
  missing: qed
sample_proofs/good_proof.tex: OK
```

`test_17_portfolio.sh` asserts both lines.

17.8 Summary

Minute	Action
0–10	read everything; rank by value×probability
10–90	first pass: the problems you own
90–210	deep work, one problem at a time, hard caps
210–240	write-up check: claim, boundary, both directions
after	portfolio: rewrite the weakest proof

17.9 Common mistakes

- *Discovery-order writing.* The order you found the proof is almost never the order a reader needs. Reorder.
- *Half a find-all.* Finding the candidates is half the answer; the other half is verifying each one solves it.
- *Polishing problem 1 while 4–6 are blank.* A perfect early proof plus three empty pages loses to three rough proofs. The budget is the strategy.

Exercises

Inline

Exercise 17.1 (easy)**.** A 3-problem contest gives 90 minutes. State a budget for a student who expects to solve two and struggle on one.

Solution. 5-min read, 2×30 min on the two owned problems, 15 min capped on the third for partial credit, 10 min checking.

Exercise 17.2 (medium)**.** Rewrite this "proof" to passing standard: "n^2 is even for even n, so $n^2 + n$ is even."

Solution. Claim first; split parity as a named lemma covering *both* cases; close with the sum argument and ■.

Exercise 17.3 (medium)**.** Your write-up solved a find-all equation by listing three solutions and noting they work. What is missing and how many rubric points does it cost?

Solution. The "no other solutions" direction — a completeness proof. On the rubric the "every case argued" item scores 0 of its 2 points, so the omission costs 2 of 10.

Additional

Exercise 17.4 (easy)**.** List the three checks to run in the last 30 minutes, highest value first.

Solution. Claim-matches-proof; boundary cases; both directions on find-alls.

Exercise 17.5 (easy)**.** Why does a rubric need to be mechanical?

Solution. So self-assessment is repeatable — a rubric requiring judgment re-imports the bias it exists to remove.

Exercise 17.6 (easy)**.** In 15 answer-problems/3h, what does "abandon" look like concretely?

Solution. A hard cap per problem ($\approx$12 min); at the cap with no new idea, write the best guess and move.

Exercise 17.7 (medium)**.** Timed rewrite: take your Chapter 16 difference-reduction solution and produce a submission draft in 20 minutes; score it on the rubric.

Solution. Expect claim+two lemmas+proof; a missing termination lemma is the usual 1-point leak.

Exercise 17.8 (medium)**.** Explain why fake-solving loses *expected* marks even when the fake step is true.

Solution. Asserted steps are unverifiable (0 credit), and the bluff hides the real partial credit available for the argued parts.

Exercise 17.9 (medium)**.** Design the portfolio update rule and justify "replace weakest" over "add newest".

Solution. A fixed-size portfolio forces the floor up; adding only lets weak work persist.

Exercise 17.10 (medium)**.** Implement-or-derive: extend `portfolio_check.py` to also flag "clearly", "obviously", and "similarly" as fake-solve markers.

Solution. Add a fourth regex group over those words; each hit is a 0-point marker in the report line.

Exercise 17.11 (hard)**.** Timed: rewrite any earlier chapter's worked example as a full submission in 25 minutes, self-score, then rewrite once more to raise the lowest dimension.

Solution. The second pass should visibly change structure, not words — structure is what the rubric measures.

Exercise 17.12 (hard)**.** Argue for or against: "partial credit write-ups should state what the author could not prove."

Solution. For: naming the gap is verifiable progress and distinguishes honest incompleteness from bluffing; graders reward it. The cost is honesty about scope.

Exercise 17.13 (hard)**.** Simulate a 4.5-hour, 6-proof paper: write the budget table, run two problems under it, and report where the plan broke.

Solution. Most plans break at the deep-work cap — the discipline being tested is stopping, not solving.

Exercise 17.14 (easy)**.** A 3-problem exam is worth 21 points total (7 each). You solve P_1 in 25 min, P_2 in 50 min, and spend the last 15 checking. Sketch the timeline.

Solution. 0–10 triage/read; 10–35 P_1; 35–85 P_2; 85–135 P_3 attempt; 135–150 checks — the check block is scheduled, never squeezed.

Exercise 17.15 (medium)**.** Rewrite this scratch into a submittable claim–lemma–proof: "mod 7 squares are only 0,1,2,4, so $x^2 + y^2 = 7z$ only if 7 | x,y."

Solution. **Claim.** $x^2 + y^2 \equiv 0 \pmod 7 \Rightarrow 7 \mid x, y$. **Lemma.** $a^2 \bmod 7 \in \{0, 1, 2, 4\}$ (check $a = 0..6$). **Proof.** Pairs summing to 0 mod 7 among $\{0, 1, 2, 4\}$: only $0 + 0$; hence $x^2 \equiv y^2 \equiv 0$, and 7 prime gives $7 \mid x$, $7 \mid y$. ■

Exercise 17.16 (medium)**.** Two students claim "I basically solved it" after finding the right method but writing no proof. Using the rubric, score each: (a) method named, zero steps written; (b) method plus lemma statements, no proofs.

Solution. (a) 0—1: method recognition earns at most the setup point; (b) 2—3: correctly stated lemmas earn partial credit, but the claim is unproven — "basically solved" is a partial-credit self-report, not a solve.

Exercise 17.17 (hard)**.** Design a verification protocol for a "find all f" solution that found $\{f = 0, f = \mathrm{id}\}$: list the three checks that catch the two classic errors.

Solution. (1) *Existence*: verify each member satisfies the equation literally; (2) *Completeness*: re-derive the family and confirm no branch was dropped (e.g. $c^2 = c$ allows both roots); (3) *Domain*: check the found functions are defined on all of the stated domain. The classic errors are dropped branches and unverified members.

Exercise 17.18 (hard)**.** Autopsy this fake-solve: "Clearly f is linear since additive functions are lines, so $f(x) = kx$." Identify the two dishonest steps and write the honest version.

Solution. Dishonest: "clearly" hides the hard part ($\mathbb{R}$ vs $\mathbb{Q}$ domains), and "additive functions are lines" skips the regularity hypothesis entirely. Honest: state the domain, derive $f(q) = qf(1)$ on $\mathbb{Q}$, then name the regularity (continuity, monotonicity, or boundedness) that extends it to $\mathbb{R}$.

Notes

Grading, fake-solve and checking conventions follow Chen's *Remarks on English* [8] and the OTIS walkthrough format [9]; write-up mechanics follow Hammack [3] and Velleman's technique summary [2]. Case-study problems already cited: 2024 AIME I P10; 2021 AIME I P10; 2024 USAJMO P1; USAMO 2003 rubric (olympiad.html) [8].

Further reading

- E. Chen, *Remarks on English* — how graders actually read proofs; the reference for the write-up and fake-solve sections: https://web.evanchen.cc/handouts/english/english.pdf
- L. Lamport, *How to Write a 21st Century Proof* (paper) — the hierarchical proof structure the claim–lemma–proof style descends from: https://lamport.azurewebsites.net/pubs/proof.pdf
- R. Hammack, *Book of Proof* — the write-up mechanics chapter: https://richardhammack.github.io/BookOfProof/
- D. Velleman, *How to Prove It* — technique summaries for each proof pattern the portfolio drills.
- MIT OpenCourseWare, *6.042J* — https://ocw.mit.edu/6-042JF10 — the proof-writing recitations.
- The OTIS walkthrough archive (E. Chen's olympiad training materials) — https://web.evanchen.cc/otis.html — model write-ups at contest standard.

WATCH INSTEAD

- **Art of Problem Solving — "Work Backwards" (Richard Rusczyk)**, 12:00, https://www.youtube.com/watch?v=ZlKUhwNniOo — the work-backwards tactic that saves stalled exam problems. (Verified 2026-10-11.)
- **Leslie Lamport — "How to Write a 21st Century Proof"** (Heidelberg Laureate Forum 2017), ∼28 min, https://av.tib.eu/media/40137 — why claim–lemma–proof structure beats prose proofs: the Write-up-standards section's ancestor. (Verified 2026-10-11.)

Notes

Hints for the hard exercises

One hint per **[hard]** exercise, in chapter order. Read the hint only after a real attempt — a hint that costs you zero minutes of stuck costs you the whole exercise.

Chapter 1

- $\exists!$ two ways: expand both and derive each from the other; the second clause of (ii) is the uniqueness half of (i).
- Verifier: brute-force the grid over the two orderings of $\forall/\exists$; the verdict differs exactly when some column's required row varies.

Chapter 2

- $x^2 - 4y = 2$: mod 4 gives $x^2 \equiv 2$; parity-wise an even LHS forces x even, then LHS $\equiv 0$.
- Euler-variant polynomials: hunt for a factorable evaluation — plug x equal to the constant term.
- Euclid: N is coprime to every listed prime; any prime factor of N is new.

Chapter 3

- Two holes: $(4^n - 2)/3$ never integral because $4^n \equiv 1 \pmod 3$.
- Hole anywhere: induct on the quarter containing the hole, as in the one-hole version.
- Strengthened sum: induct on $2 - 1/n$; the slack $1/n$ is exactly what the induction step spends.

Chapter 4

- $\mathcal{P}(\mathbb{N})$: diagonalize on membership of i.
- Cantor: $D = \{a : a \notin f(a)\}$ differs from every $f(a)$ at a.
- Base-3 flips: avoid the ambiguous tail by never producing all-2s.

Chapter 5

- Euclid's lemma both ways: forward via Bezout; reverse by contrapositive on a composite.
- Chicken/goat budget: it is linear Diophantine, so run $3c + 7g = 200$ through extended Euclid, then bound the parameter.
- Iterative Bezout: keep (x_i, y_i) satisfying $ax_i + by_i = r_i$; each division step updates the pair, not just the remainder.

Chapter 6

- Purely periodic iff $\gcd(a, m) = 1$: a tail exists iff a prime divides a and m — 2 mod 12 shows it.
- CRT by hand: solve mod 3, fold in mod 5, then mod 7.
- Tail/period table: conjecture tail length = exponent needed to clear common factors; verify on 12, 20, 100.

Chapter 7

- $\varphi(n) = 12$: bound primes by $p - 1 \mid 12$, then case-split the square factors.
- $x^2 \equiv 1$: two roots mod p (Euclid's lemma on $(x-1)(x+1)$); mod 8 the factorization has four roots.
- Non-coprime CRT: consistent iff congruences agree mod the gcd; solution class is mod lcm.

Chapter 8

- Even committees: pair S with $S \triangle \{1\}$; or $((1+1)^n - (1-1)^n)/2$.
- Stars and bars direct: n stars, $k-1$ bars in the $n-1$ gaps.
- Restricted PIE in code: cap each variable, subtract violations exactly as Chapter 9 counts surjections.

Chapter 9

- Surjection formula at $n < k$: it must be 0 because no surjection exists — the combinatorics forces the algebra.
- Distinct vs odd parts: $(1+x)(1+x^2)\cdots = \frac{1-x^2}{1-x} \cdot \frac{1-x^4}{1-x^2} \cdots$ telescopes.
- $\{6, 10, 15\}$: largest unmakeable is 29; show 30–35 all makeable then induct by adding 6.

Chapter 10

- Binet two ways: induction on the recurrence, and partial fractions on $x/(1-x-x^2)$; the $\phi, \hat{\phi}$ coefficients coincide.
- Decimal Binet: the failure moves out with precision; exact algebraic arithmetic never fails — only rounding does.

Chapter 11

- K_5 nonplanar: derive $E \leq 3V - 6$ from $V - E + F = 2$ and the $2E \geq 3F$ handshaking bound first.
- Dual counts: $E = E^*$ always; dodecahedron $\leftrightarrow$ icosahedron swaps (V, F).
- Hierholzer: splice cycles; verify by edge count, not just success.

Chapter 12

- Var $= E[X^2] - \mu^2$: expand $(X - \mu)^2$; fixed points via indicator pairwise products.
- Simpson: reverse inequalities need the losing strata to carry far more mass — set weights first.
- Max fixed points: the limit is a named extreme-value law — think Gumbel on a Poisson(1) skeleton.

Chapter 13

- Tangent line: $\frac{x^3}{1+x^2} - (x - \frac{1}{2}) = \frac{(x-1)^2}{2(1+x^2)} \geq 0$; sum at $x = 1$ equality.
- Schur scan: expect the minimum difference exactly 0 at $a = b = c$ or a zero — that is Schur's boundary.
- Cauchy backward: pad with n copies of the mean; the $2n$ inequality collapses to the n one.

Chapter 14

- Hamel gate: without regularity, additive f can map basis elements arbitrarily — discontinuities exist.
- Degree-3 search: expect the same single-parameter family survives; cubic coefficients die on the leading term.

Chapter 15

- Radical axis: the power equation is linear in (x, y) after the $x^2 + y^2$ terms cancel.
- Orthogonal circles invert to what? Angles are preserved — still orthogonal (or a line through a center).
- Nine-point radius: $R/2$; center is the midpoint of OH.

Chapter 16

- Circle-differences: parity makes all entries even in bounded steps; divide out the 2s and use the max monovariant.
- Strategy stealing: extra marks can't hurt; first player borrows the second's hypothetical strategy.
- Pairwise-square sums: name the largest element — partner sums sit in $(M, 2M)$, finitely many squares.

Chapter 17

- Timed rewrite: the fix between drafts should be structural, not verbal.
- Stating the gap: graders pay for verifiable progress — a named unproved step is still named.
- Budget simulation: plans break at the deep-work cap; measure the stall, not the solve.

Selected solutions

Full solutions rotate across chapters so every chapter's exercise set gets real coverage: all exercises in Chapters 13–17 carry complete in-chapter solutions; below are full solutions to a spread of additional exercises from Chapters 1–12, with the rest of the book's solutions already printed inline.

Chapter 1

Ex. (negations drill). The negation of $\forall x\, \exists y\, P(x, y)$ is $\exists x\, \forall y\, \neg P(x, y)$: flip each quantifier and negate the predicate. Order is preserved — no reordering occurs under negation.

Ex. (vacuous truth). "All unicorns in the room are asleep" is true when the room has no unicorns: $\forall x\, P$ over an empty domain holds vacuously. The corresponding $\exists$ statement fails.

Chapter 2

Ex. (mod-4 squares). Every square is $\equiv 0$ or $1 \pmod 4$: $(2k)^2 = 4k^2$ and $(2k+1)^2 = 4k(k+1)+1$. So $x^2 + y^2 \equiv 3 \pmod 4$ has no solutions.

Ex. (contradiction template). To prove $\sqrt{2}$ irrational: suppose $\sqrt{2} = p/q$ in lowest terms; $p^2 = 2q^2$ forces p even, writing $p = 2r$ gives $q^2 = 2r^2$, forcing q even too — contradicting lowest terms.

Chapter 3

Ex. (strong induction). Every $n \geq 2$ has a prime factor: least counterexample m can't be composite (its factors would have prime factors below m), so it's prime — its own witness.

Ex. (sum formula). $\sum_{i=1}^{n} i = n(n+1)/2$: base $n = 1$; step adds $n+1$ to $n(n+1)/2$, giving $(n+1)(n+2)/2$.

Chapter 4

Ex. (no bijection to $\mathbb{R}$). A countable list $x_1, x_2, \dots$ of reals misses the number whose i-th decimal is x_i's i-th digit plus 1 (mod 10, skipping 9) — so $\mathbb{R}$ is uncountable.

Ex. (countable union). Countably many countable sets are countable: list each set as a row and take the diagonal enumeration $i + j = \text{const}$.

Chapter 5

Ex. (gcd linearity). $\gcd(a, b) = \gcd(a - b, b)$: any common divisor of a, b divides $a - b$, and vice versa — the divisor sets coincide.

Ex. (Bezout existence). The least positive value of $ax + by$ divides both a and b (divide a by it — the remainder would be a smaller such value), so it equals $\gcd(a, b)$.

Chapter 6

Ex. (Fermat inverse). $a^{p-1} \equiv 1 \pmod{p}$ for $p \nmid a$ gives $a^{-1} \equiv a^{p-2}$: pairing each element with its inverse collapses $(p-1)!$ to -1 (Wilson aside).

Ex. (cycle detection). Powers of 2 mod 7: $1, 2, 4$ — period 3, and $3 \mid 6 = \varphi(7)$, matching Euler's theorem.

Chapter 7

Ex. (φ multiplicativity). $\varphi(mn) = \varphi(m)\varphi(n)$ for coprime m, n: CRT bijects residues mod mn coprime to mn with pairs of coprime residues.

Ex. (order bound). If $a^k \equiv 1$, the least such k divides $\varphi(m)$: write $\varphi = qk + r$, $0 \le r < k$; then $a^r \equiv 1$ forces $r = 0$.

Chapter 8

Ex. (committee argument). $\sum_k \binom{n}{k}\binom{k}{2} = \binom{n}{2}2^{n-2}$: choose the two leaders first ($\binom{n}{2}$), then any subset of the rest (2^{n-2}).

Ex. (PIE count). Students taking at least one of two electives: $|A \cup B| = |A| + |B| - |A \cap B| = 32 + 28 - 11 = 49$ of 60, so 11 take neither.

Chapter 9

Ex. (derangements). $D_4 = 4!\,(1 - 1 + 1/2 - 1/6 + 1/24) = 9$: PIE on bad events "element i fixed".

Ex. (GF coefficient). $[x^5]\,(1 + x + x^2)^3$: choices of ≤ 2 per factor summing to 5 — only $(1, 2, 2)$ patterns, 3 arrangements: coefficient 3.

Chapter 10

Ex. (linear recurrence). $a_n = 2a_{n-1} + 3a_{n-2}$, $a_0 = 1, a_1 = 4$: characteristic roots $3, -1$; $a_n = A3^n + B(-1)^n$, $A = B = 1/2$ solves, so $a_n = (3^n + (-1)^n)/2$ — integer forever.

Ex. (domino recurrence). $2 \times n$ tilings satisfy $T_n = T_{n-1} + T_{n-2}$ (last column vertical or a horizontal pair), so $T_n = F_{n+1}$.

Chapter 11

Ex. (handshake). $\sum \deg = 2E$: each edge contributes one to each endpoint's degree. Corollary: odd-degree vertices come in even count.

Ex. (tree leaves). A tree on ≥ 2 vertices has a leaf: a maximal path's endpoint can have no neighbor off the path (acyclicity), and none along it (no repeats) — degree 1.

Chapter 12

Ex. (linearity). $E[X + Y] = E[X] + E[Y]$ needs no independence: expand over the joint table and marginalize each side.

Ex. (derangement expectation). Expected fixed points of a random permutation: indicators give $n \cdot \frac{1}{n} = 1$ — distribution-independent in n.

Coverage: all 264 exercises carry solutions either in-chapter or here — this appendix gives full solutions to a rotating selection so each chapter's additional set is sampled, and every ***[hard]*** *exercise carries a hint above.*

Watch list

Verified video picks, ordered by chapter. Each entry: channel, title, duration, link, and the sections it covers. Verification dates are per-chapter; full log in `docs/watch.md`.

Chapter 1 — Logic and quantifiers (verified 2026-10-11)

- MIT OpenCourseWare, "Lec 1 | MIT 6.042J Mathematics for Computer Science,
- Neso Academy, "Logical Operators - Negation, Conjunction & Disjunction",
- Mathematical Visual Proofs, "Predicates and Quantifiers [Discrete Math

Chapter 2 — Proof methods (verified 2026-10-11)

- TrevTutor, "[Discrete Mathematics] Direct Proofs Examples", 6:59 —
- TrevTutor, "Can You Prove? Proofs in Discrete Math and Real Analysis"

Chapter 3 — Induction (verified 2026-10-11)

- MIT OpenCourseWare, "MIT 6.042J Mathematics for Computer Science"
- SawFin Mathematics, "Discrete Math II – 5.2.1 Proof by Strong Induction",

Chapter 4 — Sets, relations and functions (verified 2026-10-11)

- TrevTutor, "INTRODUCTION to SET THEORY", 16:38 —
- TrevTutor, "INJECTIVE, SURJECTIVE, and BIJECTIVE FUNCTIONS", 17:14 —

Chapter 5 — Divisibility and primes (verified 2026-10-11)

- MIT OCW 6.042J, "Lecture 4: Number Theory I", 1:20:24 —
- Khan Academy, "Journey into cryptography" playlist (modular-arithmetic unit) —

Chapter 6 — Modular arithmetic (verified 2026-10-11)

- MIT OCW, "Lecture 9: Modular Arithmetic", 1:19:45 —
- Zach Star, "Modular Arithmetic Visually Explained", 20:33 —

Chapter 7 — CRT and Fermat–Euler (verified 2026-10-11)

- Numberphile, "Chinese Remainder Theorem and Cards", 11:13 —
- Socratica, "Fermat's Little Theorem", 10:35 —

Chapter 8 — Counting principles (verified 2026-10-11)

- Spanning Tree, "What Is the Pigeonhole Principle?", 8:23 —
- TrevTutor, "Discrete Math" playlist (counting unit) —

Chapter 9 — Inclusion–exclusion and generating functions (verified 2026-10-11)

- TrevTutor, "GENERATING FUNCTIONS", 18:41 —
- Mathispower4u, "Introduction to Generating Functions for Sequences", 5:37 —

Chapter 10 — Recurrences (verified 2026-10-11)

- TrevTutor, "RECURRENCE RELATIONS", 15:24 —
- TrevTutor, "HOMOGENEOUS RECURRENCE RELATIONS", 25:15 —

Chapter 11 — Graphs, trees and planarity (verified 2026-10-11)

- Reducible, "Introduction to Graph Theory: A Computer Science Perspective", 16:25 —
- Numberphile, "A Breakthrough in Graph Theory", 24:56 —

Chapter 12 — Probability on finite spaces (verified 2026-10-11)

- Harvard Stat 110, "Lecture 1: Probability and Counting" (Blitzstein), 46:29 —
- Harvard Stat 110 playlist (34 lectures) —

Chapter 13 — Inequalities (verified 2026-10-11)

- mathophilia, "Proving the AM-GM Inequality with ZERO Calculus", 9:22 —
- Khan Academy, Linear Algebra playlist (contains Cauchy–Schwarz lectures) —

Chapter 14 — Functional equations (verified 2026-10-11)

- Michael Penn, "A functional equation that didn’t quite make the IMO", 6:43 —

Chapter 15 — Olympiad geometry (verified 2026-10-11)

- Michael Greenberg, "Olympiad Geometry Problem #93: Two Isosceles Triangles", 12:25 —
- Sohil Rathi, "Circular Geometry, Power of a point, Cyclic Quadrilaterals", 1:33:35 —

Chapter 16 — Invariants, monovariants, extremal (verified 2026-10-11)

- ChessNetwork, "Mutilated chessboard problem", 4:07 —
- Mathologer, "The Arctic Circle Theorem", 51:49 —

Chapter 17 — Four-hour exam + proof portfolio (verified 2026-10-11)

- Art of Problem Solving, "Work Backwards" (Richard Rusczyk), 12:00 —
- Leslie Lamport, "How to Write a 21st Century Proof" (HLF 2017), extasciitilde28 min —

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

[2] D. J. Velleman, *How to Prove It: A Structured Approach*, 3rd ed., Cambridge University Press, 2019. ISBN 978-1-108-43953-4.

[3] R. Hammack, *Book of Proof*, 3rd ed. (3.4). CC BY-NC-ND 4.0. https://richardhammack.github.io/BookOfProof/Main.pdf

[4] E. Lehman, F. T. Leighton and A. R. Meyer, *Mathematics for Computer Science*, MIT, 2018 draft. https://courses.csail.mit.edu/6.042/spring18/mcs.pdf

[5] J. Barwise and J. Etchemendy, *Language, Proof and Logic*, CSLI Publications, 1999. (The Tarski's World exercise tradition.)

[6] A. Bogomolny, *Cut-the-Knot*. https://www.cut-the-knot.org/

[7] Art of Problem Solving wiki, *AMC Problems and Solutions*. https://artofproblemsolving.com/wiki/index.php/AMC_Problems_and_Solutions

[8] E. Chen, *Remarks on English* (handout). https://web.evanchen.cc/handouts/english/english.pdf

[9] E. Chen, *OTIS Excerpts*, ch. 1 "Notes on Proofs". https://web.evanchen.cc/textbooks/OTIS-Excerpts.pdf

[10] A. Engel, *Problem-Solving Strategies*, Springer, 1998. ISBN 978-0-387-98219-9.

[11] E. Chen, *Introduction to Functional Equations* (handout). https://web.evanchen.cc/handouts/FuncEq-Intro/FuncEq-Intro.pdf

[12] Y. Zhao, *Olympiad handouts index* (modular arithmetic, orders, Fermat–Euler). https://yufeizhao.com/olympiad/

[13] UC Berkeley CS70, *Note 5: Modular Arithmetic* (course notes). https://www.eecs70.org/

[14] E. Chen, *Expected Uses of Probability* (handout). https://web.evanchen.cc/handouts/Probabilistic/Probabilistic.pdf

[15] E. Chen, *Summation* (handout), generating-function section. https://web.evanchen.cc/handouts/Summation/Summation.pdf

[16] E. Chen, *The Chinese Remainder Theorem* (handout). https://web.evanchen.cc/handouts/CRT/CRT.pdf

[17] E. Chen, *Olympiad Inequalities* handout. https://web.evanchen.cc/handouts/Ineq/en.pdf

[18] E. Chen, *Euclidean Geometry in Mathematical Olympiads*, MAA Press, 2016. ISBN 978-0-88385-839-4. TOC and ch.1–3 preview on the author's site.

[19] E. Chen, *Geometry handouts* (directed angles; incenter/excenter lemma; barycentric and complex numbers). https://web.evanchen.cc/olympiad.html

[20] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed., Wiley, 2007. ISBN 978-0-471-78901-7.

Index

$[n]$, 43

AM–GM
  for four, 118
  theorem, 118
analytic geometry, 140
angle chasing, 136
axiom, 16

base case
  wrong window, 29
base representation, 52
base-b representation, 52
Bézout's identity, 49
biconditional, 4
bijection
  counting by, 79
bijective, 39
binomial coefficient, 78
binomial theorem, 78
bipartite, 102
budget comparison, 157

candidate, 128
cardinality, 41
Cartesian product, 38
cases
  exhaustiveness, 17
casting out nines, 61
Cauchy equation, 130
Cauchy induction, 118
Cauchy–Schwarz, 119
Ceva, 139
characteristic equation, 95
check digit, 73
checking, 158
Chinese remainder theorem, 63, 67
chocolate bar, 146
closed form, 93
closest pair, 147
codomain, 39
coefficient extraction, 88
collide terms, 128
coloring argument, 148
combinatorial proof, 81
common mistakes, 12, 23, 34, 44, 53, 64, 73, 81, 90, 98, 106, 114
complement rule, 110
composite, 10
conclusion, 3
conditional probability, 110
congruence, 58
conjunction, 2
connected, 102
connective, 2
constructive proof, 20
contest
  budget, 156
  what it tests, 156
contradiction, 18
contrapositive, 3
  proof by, 18
converse, 3
convolution
  of generating functions, 88
coprime, 48
countable, 41
counterexample, 5, 21
counting, 76
CRT, *see* Chinese remainder theorem
  non-coprime moduli, 70
  three views, 69
cycle, 102
cyclic quadrilateral, 137

De Morgan's laws, 2
degree, 101
derangement, 86
diagonalisation, 41

difference reduction, 147
Diophantine equation, 53
direct proof, 16
directed angle, 136
disjunction, 2
disproof, 21
divides, 10
divisibility, 47
divisibility criteria, 52
division algorithm, 48
divisor, 47
domain, 39
domino tiling, 148

Engel form, 119
equality case, 120
equivalence
    of formulas, 2
equivalence relation, 40
Euclidean algorithm, 48
Euler's formula, 104
Euler's theorem, 71
Euler's totient
    by inclusion–exclusion, 88
Eulerian trail, 103
even, 10
exam technique, 155
    mistakes, 160
existence, 20
existential quantifier, 5
expectation, 112
extremal principle, 147

factorial, 77
fake-solve, 157
fe_search, 132
Fermat's little theorem, 67, 70
Fibonacci, 95
fresh names, 16
FTA, *see* fundamental theorem of arithmetic
function, 39
functional equation, 127
    definition, 127
    mistakes, 132
fundamental theorem of arithmetic, 50

games
    winning positions, 149
gcd, 48
generating function, 85
    and recurrences, 89
    ordinary, 88
    solving recurrences, 97
geometric probability, 113
geometry
    mistakes, 141
    olympiad, 136
Goldbach conjecture, 9
graph, 101
graph colouring, 105
growth order, 97

handshake lemma, 101
hints, 164
homothety, 139
hypothesis, 3

iff, 4
    proof of, 19
    proving one direction, 19
image, 39
implication, 3
incenter/excenter lemma, 138
inclusion–exclusion, 85
independence, 111
induction, 27
    choosing the variable, 33
    closing a guess, 94
    principle of, 28
    template, 28
inductive hypothesis, 28
ineq_scan, 122
inequality, 117
    algebra, 117
    mistakes, 122
inference, 16
injective, 39
invariant, 146
    definition, 146
    mistakes, 150
invariant_sim, 150
inverse, 3
    multiplicative, 59
inversion, 140

invert.py, 141
involution, 129
    example, 129

Jensen, 130

linear congruence, 60
linear recurrence
    homogeneous, 95
linearity of expectation, 112
logic, 1

majorization, 120
mathematical writing, 22
medians
    concurrent, 139
Menelaus, 139
minimal counterexample, 30
minimum, 118
modular arithmetic, 57
modus ponens, 16
monovariant, 147
Muirhead, 120
multiset
    counting, 79

negation, 2
    of quantified statements, 7
Nesbitt, 119
nine-point circle, 139
non-constructive proof, 20

odd, 10
open formula, 2
or
    inclusive, 2
order
    multiplicative, 72
ordered field, 117

pairing strategy, 149
parity, 10
partial credit, 157
Pascal's triangle, 78
path, 102
permutation, 77
pigeonhole
    generalized, 148
pigeonhole principle, 80
planar graph, 104
pointwise trap, 130
portfolio_check, 159
power cycle, 61
power mean, 119
power of a point, 138
    example, 138
power set, 38
power_check, 141
prime, 10, 50
primes
    infinitude of, 51
principle of inclusion–exclusion, 86
probability, 109
product rule, 76
proof, 15
    by cases, 17
    by contradiction, 18
    formal definition, 16
    template, 16
    writing rules, 22
proof portfolio, 155, 159
proof template
    injective/surjective, 39
proposition, *see* statement

quantifier, 5

radical axis, 138
random variable, 112
recurrence, 93
relation, 40

sample space, 110
Schur, 120
    derived, 120
set
    operations, 38
    subset, 38
sets, 37
Simpson's paradox, 113
solutions, 168
solve over, 127
spanning tree, 103
squares, nonnegative, 118
stars and bars, 79

statement, 2
strategy
    inequalities, 121
strategy before proof, 22
strong induction, 30
structural induction, 31
substitution, 128
sum rule, 76
surjection
    counting, 87
surjective, 39
Sylvester–Gallai theorem, 147
symmetric sum
    notation, 120

tangent
    angle chase, 136
theorem, 16
tiling
    by recurrence, 95
totient, 71
translation
    English–symbols, 8
trap autopsy, 130
tree, 103
truth table, 2

uniqueness, 20
universal quantifier, 5
universe of discourse, 5

vacuous truth, 3
verification, 131

walk, 102
well-ordering principle, 30
Wilson's theorem, 72
witness, 5
write-up
    example, 157
write-up standards, 157

$\mathbb{Z}/m\mathbb{Z}$, 58